

ARTIFICIAL INTELLIGENCE IN AGRICULTURE

Rajesh Singh, Anita Gehlot,
Mahesh Kumar Prajapat and Bhupendra Singh



DOI: 10-4304/9781315542643

TRÍ TUỆ NHÂN TẠO TRONG NÔNG NGHIỆP

Rajesh Singh, Anita Gehlot, Mahesh Kumar Prajapat và Bhupendra Singh

nipa (CRC) CRC Press Taylor & Francis Group

Về các Tác giả



Tiến sĩ Rajesh Singh hiện đang làm việc tại Đại học Lovely Professional với vai trò Giáo sư, với hơn mười sáu năm kinh nghiệm trong học thuật. Ông đã được trao huy chương vàng M.Tech từ RGPV, Bhopal (M.P) Ấn Độ và bằng danh dự B.E từ Đại học Dr. B.R. Ambedkar, Agra (U.P), Ấn Độ. Lĩnh vực chuyên môn của ông bao gồm hệ thống nhúng, robot, mạng cảm biến không dây và Internet vạn vật. Ông đã được vinh danh là diễn giả chính và chủ tọa phiên tại các hội nghị quốc tế/quốc gia, các chương trình phát triển chuyên môn cho giảng viên và các hội thảo. Ông sở hữu một trăm năm mươi hai bằng sáng chế. Ông đã xuất bản hơn một trăm bài báo nghiên cứu trên các tạp chí/hội nghị có phản biện và hai mươi bốn cuốn sách trong lĩnh vực Hệ thống nhúng và Internet vạn vật với các nhà xuất bản uy tín như CRC/Taylor & Francis, Narosa, GBS, IRP, NIPA, River Publishers, Bentham Science và RI publication. Ông là biên tập viên cho một số đặc biệt được xuất bản bởi loạt sách AISC, Springer vào năm 2017 & 2018 và IGI global vào năm 2019. Dưới sự hướng dẫn của ông, sinh viên đã tham gia các cuộc thi quốc gia/quốc tế bao gồm cuộc thi "Thử thách Thiết kế Sáng tạo" của Texas và DST và giải thưởng danh dự Laureate về kỹ thuật robot tại Madrid, Tây Ban Nha vào năm 2014 & 2015. Đội của ông đã chiến thắng "Smart India Hackathon-2019" phiên bản phần cứng do MHRD, Chính phủ Ấn Độ tổ chức cho bài toán của Mahindra & Mahindra. Dưới sự hướng dẫn của ông, đội sinh viên đã nhận được "Giải thưởng InSc 2019" trong chương trình dự án sinh viên. Ông đã được trao "Giải thưởng Sáng tạo Công nghệ Trẻ Gandhian (GYTI)", với tư cách là người hướng dẫn cho "Hệ thống Phân tích Dữ liệu Chẩn đoán Tích hợp - OBDAS", được tuyên dương trong "Sáng tạo Đột phá" tại Lễ hội Sáng tạo và Doanh nhân tại Rashtrapati Bahawan, Ấn Độ vào năm 2018. Ông đã được vinh danh với "Chứng nhận Xuất sắc" từ giải thưởng thương hiệu giảng viên lần thứ 3-15, do EET CRS tổ chức vì sự xuất sắc trong giáo dục chuyên nghiệp và công nghiệp, cho hạng mục "Giải thưởng Xuất sắc trong Nghiên cứu", 2015 và giải thưởng nhà nghiên cứu trẻ tại Hội nghị Quốc tế về Khoa học và Thông tin năm 2012.



Tiến sĩ Anita Gehlot hiện đang làm việc tại Đại học Lovely Professional với vai trò Phó Giáo sư, với hơn mười hai năm kinh nghiệm trong học thuật. Lĩnh vực chuyên môn của bà bao gồm hệ thống nhúng, mạng cảm biến không dây và Internet vạn vật. Bà đã được vinh danh là diễn giả chính và chủ tọa phiên tại các hội nghị quốc tế/quốc gia, các chương trình phát triển chuyên môn cho giảng viên và các hội thảo. Bà có một trăm ba mươi hai bằng sáng chế. Bà đã xuất bản hơn bảy mươi bài báo nghiên cứu trên các tạp chí/hội nghị có phản biện và hai mươi bốn cuốn sách trong lĩnh vực Hệ thống nhúng và Internet vạn vật với các nhà xuất bản uy tín như CRC/Taylor & Francis, Narosa, GBS, IRP, NIPA, River Publishers, Bentham Science và RI publication. Bà là biên tập viên cho một

số đặc biệt được xuất bản bởi loạt sách AISC, Springer vào năm 2018 và IGI global vào năm 2019. Bà đã được trao "giấy chứng nhận đánh giá cao" từ Đại học Nghiên cứu Dầu khí và Năng lượng vì công việc gương mẫu. Dưới sự hướng dẫn của bà, đội sinh viên đã nhận được "Giải thưởng InSc 2019" trong chương trình dự án sinh viên. Bà đã được trao "Giải thưởng Sáng tạo Công nghệ Trẻ Gandhian (GYTI)", với tư cách là người hướng dẫn cho "Hệ thống Phân tích Dữ liệu Chẩn đoán Tích hợp - OBDAS", được tuyên dương trong "Sáng tạo Đột phá" tại Lễ hội Sáng tạo và Doanh nhân tại Rashtrapati Bahawan, Ấn Độ vào năm 2018.



Mahesh Kumar Prajapat hiện đang theo học Thạc sĩ Khoa học Máy tính và Kỹ thuật tại Đại học Lovely Professional. Lĩnh vực nghiên cứu chính của ông là Trí tuệ nhân tạo, nhờ đó ông đã có 16 bằng sáng chế. Ông đã giành vị trí thứ 3 trong cuộc thi nghiên cứu AIU Anveshan 2020. Ông là người sáng lập startup Orphorhard, hoạt động vì sự phát triển của trẻ mồ côi và trẻ em có hoàn cảnh khó khăn trên cả nước. Ông đã hoàn thành các khóa học chuyên sâu bao gồm học sâu, khoa học dữ liệu từ Coursera và IBM. Ông luôn Mở rộng Tầm nhìn, Rất ham quan sát, nhận thức các ý thức hệ tự mình tạo ra và thực hiện chúng, và Yêu thích khám phá các lĩnh vực khác nhau với đầy nhiệt huyết.



Bhupendra Singh là giám đốc điều hành của Schematics Microelectronics và cung cấp dịch vụ thiết kế sản phẩm cũng như hỗ trợ R&D cho các ngành công nghiệp và trường đại học. Ông đã hoàn thành BCA, PGDCA, M.Sc. (CS), M.Tech và có hơn mười một năm kinh nghiệm trong lĩnh vực Mạng máy tính và Hệ thống nhúng. Ông đã xuất bản mười hai cuốn sách trong lĩnh vực Hệ thống nhúng và Internet vạn vật với các nhà xuất bản uy tín như CRC/Taylor & Francis, Narosa, GBS, IRP, NIPA, River Publisher, Cambridge Scholar, Bentham Science và RI Publication.

Taylor & Francis Taylor & Francis Group http://taylorandfrancis.com

NEW INDIA PUBLISHING AGENCY New Delhi-110 034

TRÍ TUỆ NHÂN TẠO TRONG NÔNG NGHIỆP

Rajesh Singh Đại học Lovely Professional Jalandhar, Punjab, Ấn Độ

Anita Gehlot Đại học Lovely Professional Jalandhar, Punjab, Ấn Độ

Mahesh Kumar Prajapat Đại học Lovely Professional Jalandhar, Punjab, Ấn Độ

Bhupendra Singh Schematics Microelectronics, Ấn Độ

(CRC) CRC Press Taylor & Francis Group Boca Raton London New York

CRC Press là một ấn hiệu của Taylor & Francis Group, một doanh nghiệp informa

nipa NEW INDIA PUBLISHING AGENCY New Delhi-110 034

Xuất bản lần đầu năm 2022 bởi CRC Press 2 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN và bởi CRC Press 6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

© 2022 New India Publishing Agency CRC Press là một ấn hiệu của Informa UK Limited

Quyền của Rajesh Singh và cộng sự được xác định là tác giả của tác phẩm này đã được khẳng định theo các mục 77 và 78 của Đạo luật Bản quyền, Kiểu dáng và Sáng chế năm 1988.

Bảo lưu mọi quyền. Không một phần nào của cuốn sách này được tái bản hoặc sao chép hoặc sử dụng dưới bất kỳ hình thức nào hoặc bằng bất kỳ phương tiện điện tử, cơ học hoặc phương tiện nào khác, hiện đã biết hoặc sau này được phát minh, bao gồm photocopy và ghi âm, hoặc trong bất kỳ hệ thống lưu trữ hoặc truy xuất thông tin nào, mà không có sự cho phép bằng văn bản từ nhà xuất bản.

Để được phép photocopy hoặc sử dụng tài liệu điện tử từ tác phẩm này, hãy truy cập www. copyright.com hoặc liên hệ với Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. Đối với các tác phẩm không có sẵn trên CCC, vui lòng liên hệ mpkbookspermissions@tandf.co.uk

Thông báo về nhãn hiệu: Tên sản phẩm hoặc tên công ty có thể là nhãn hiệu hoặc nhãn hiệu đã đăng ký, và chỉ được sử dụng để nhận dạng và giải thích mà không có ý định vi phạm.

Bản in không được bán ở Nam Á (Ấn Độ, Sri Lanka, Nepal, Bangladesh, Pakistan hoặc Bhutan).

Dữ liệu Biên mục Thư viện Anh Một bản ghi danh mục cho cuốn sách này có sẵn từ Thư viện Anh

Dữ liệu Biên mục Thư viện Quốc hội Hoa Kỳ Một bản ghi danh mục đã được yêu cầu

ISBN: 978-1-032-15810-5 (bìa cứng) ISBN: 978-1-003-24575-9 (sách điện tử) DOI: 10.1201/9781003245759 nipa

Lời nói đầu

Cuốn sách này là một nền tảng cho bất kỳ ai muốn khám phá Trí tuệ nhân tạo trong lĩnh vực nông nghiệp từ đầu hoặc mở rộng hiểu biết và các ứng dụng của nó. Cuốn sách này cung cấp một khám phá thực tế, thực hành về Trí tuệ nhân tạo, học máy, Học sâu, thị giác máy tính và Hệ chuyên gia với các ví dụ cụ thể để hiểu rõ. Cuốn sách này cũng bao gồm các kiến thức cơ bản về python với ví dụ để bất kỳ ai cũng có thể dễ dàng hiểu và vận dụng Trí tuệ nhân tạo trong lĩnh vực nông nghiệp.

Cuốn sách này được chia thành hai phần, trong đó phần đầu tiên nói về Trí tuệ nhân tạo và tác động của nó trong nông nghiệp với tất cả các nhánh và kiến thức cơ bản của chúng. Phần thứ hai của cuốn sách hoàn toàn là triển khai các thuật toán và sử dụng các thư viện khác nhau của học máy, học sâu và thị giác máy tính để xây dựng các dự án hữu ích và sâu sắc trong thời gian thực, có thể rất hữu ích để bạn hiểu rõ hơn về Trí tuệ nhân tạo.

Sau khi đọc cuốn sách này, bạn sẽ hiểu được Trí tuệ nhân tạo là gì, nó được áp dụng ở đâu, và các nhánh khác nhau của nó là gì, có thể hữu ích trong các kịch bản khác nhau. Bạn sẽ làm quen với quy trình làm việc tiêu chuẩn để tiếp cận và giải quyết các vấn đề học máy, và bạn sẽ biết cách giải quyết các vấn đề thường gặp. Bạn sẽ có thể sử dụng Trí tuệ nhân tạo để giải quyết các vấn đề thực tế, từ dự đoán sức khỏe cây trồng đến phân tích giám sát đồng ruộng, phân loại đến nhận dạng loài thực vật, v.v.,

Các tác giả xin cảm ơn nhà xuất bản đã hỗ trợ và khuyến khích để viết cuốn sách này.

Các tác giả

Taylor & Francis Taylor & Francis Group http://taylorandfrancis.com

Mục lục

Lời nói đầu vii

PHẦN A: Nền tảng của Trí tuệ nhân tạo

Chương 1 Trí tuệ nhân tạo

Chương 2 Học Python cho Trí tuệ nhân tạo

Chương 3 Học máy

Chương 4 Học sâu

Chương 5 Thị giác máy tính

Chương 6 Hệ chuyên gia dựa trên tri thức

PHẦN B: Triển khai Trí tuệ nhân tạo

Chương 7 Công cụ cho Trí tuệ nhân tạo

Chương 8 Các thư viện quan trọng cho AI

Chương 9 Các thuật toán Học máy

Chương 10 Phân loại và Phát hiện bệnh ở Thực vật

Chương 11 Nhận dạng loài ở Hoa

Chương 12 Nông nghiệp chính xác

PHẦN A: NỀN TẢNG CỦA TRÍ TUỆ NHÂN TẠO

Chương 1: Trí tuệ nhân tạo

Thế giới trí tuệ nhân tạo (AI) đang phát triển nhanh chóng khi nó vươn tới nhiều ngành công nghiệp khác nhau. Hiện tại, bạn sẽ thấy AI được sử dụng cho nhiều mục đích khác nhau, từ nông nghiệp đến ô tô; và theo thời gian, bạn sẽ còn thấy nó xuất hiện nhiều hơn nữa. Nông nghiệp là một trong những nhánh quan trọng nhất của ngành công nghiệp CNTT. Nông nghiệp là một ngành công nghiệp quan trọng và là một phần lớn trong nền tảng kinh tế của chúng ta. Về doanh thu hàng năm, ngành nông nghiệp đóng góp gần 17% cho nền kinh tế của chúng ta. AI đang trở thành một tiến bộ kỹ thuật giúp tăng cường và bảo vệ năng suất cây trồng, trong bối cảnh biến đổi khí hậu và dân số gia tăng. Trong chương này, các vấn đề cơ bản về Trí tuệ nhân tạo đã được thảo luận về việc nó có thể liên quan đến nông nghiệp như thế nào.

1. Giới thiệu

Trí tuệ nhân tạo đã trở nên nổi bật nhất trong thế hệ hiện nay. Mọi lĩnh vực đều đang sử dụng Trí tuệ nhân tạo để có kết quả tốt hơn và điều này cũng giúp tiết kiệm rất nhiều thời gian. Vì Trí tuệ nhân tạo đang thúc đẩy các lĩnh vực khác nhau tăng năng suất và hiệu quả và việc sử dụng nó trong các lĩnh vực khác nhau như trong lĩnh vực XÂY DỰNG, Trí tuệ nhân tạo có thể được sử dụng để phân tích kết cấu của các tòa nhà, đập, v.v., và các giải pháp Trí tuệ nhân tạo đang hỗ trợ rất nhiều cách để vượt qua những thách thức truyền thống trong mọi lĩnh vực.

I. Trí tuệ nhân tạo là gì

Trí tuệ nhân tạo là một trong những lĩnh vực nổi bật nhất của Khoa học Máy tính, cố gắng định nghĩa lại các nhiệm vụ được con người thực hiện trong cuộc sống hàng ngày với kết quả tốt hơn và độ chính xác tối đa. Trí tuệ nhân tạo thực sự có ý nghĩa là giảm bớt công việc của con người, trong đó từ "nhân tạo" (artificial) chỉ công việc được thực hiện mà không có sự can thiệp của con người và "trí tuệ" (intelligence) chỉ khả năng thực hiện nhiệm vụ giống như não người. Lĩnh vực Trí tuệ nhân tạo đã có rất nhiều định nghĩa khác nhau theo thời gian, có thể được nêu ra như sau:

1. Kỹ thuật và Khoa học kết hợp với nhau để làm cho máy móc Thông minh.
2. Làm cho một cỗ máy trở nên thông minh, có thể bắt chước hành vi của con người.
3. Nhánh Khoa học Máy tính có thể thực hiện các hoạt động giống con người, bao gồm suy nghĩ, học hỏi, v.v.
4. Một lĩnh vực của Khoa học Máy tính làm cho máy móc hành động và cư xử giống như con người.

Trí tuệ nhân tạo cho phép và tăng cường khả năng của máy móc, giúp chúng hiểu được sự thật, sự kiện, phân tích tình huống và hiểu biết để rút ra kiến thức từ môi trường xung quanh và hành động dựa trên chúng để giải quyết vấn đề, điều này có thể dẫn đến kết quả tốt hơn con người. Trọng tâm chính của Trí tuệ nhân tạo luôn là phát triển một cỗ máy có thể (mô phỏng) về mặt tâm lý và mô phỏng hành vi của con người thông qua việc xây dựng các suy nghĩ giống như con người từ bất kỳ môi trường hoặc tình huống nào.

II. Giới thiệu về Trí tuệ nhân tạo trong Nông nghiệp

Nông nghiệp là một trong những nghề lâu đời nhất trên thế giới. Nhân loại từ thời kỳ sơ khai cho đến thế hệ hiện tại đã trải qua một chặng đường dài trong việc phát triển các phương thức canh tác nông nghiệp và kỹ thuật sản xuất lương thực mà loài người tiêu thụ hàng ngày. Như đã thấy từ các thế hệ trước cho đến thế hệ hiện tại, dân số luôn gia tăng, dẫn đến tăng tiêu thụ lương thực và giảm 20% sản lượng sản xuất.

Khi sản xuất nông nghiệp ngày càng giảm sút qua các thế hệ, Trí tuệ nhân tạo có thể giúp chúng ta tự động hóa việc sản xuất cây lương thực trên các cánh đồng mà không cần sự can thiệp của con người, điều này sẽ dẫn đến tăng sản lượng lương thực. Các giải pháp Trí tuệ nhân tạo có thể thực hiện các nhiệm vụ như phân tích đồng ruộng để thu hoạch, giám sát cây trồng và điều kiện đất đai theo thời gian, dự báo thời tiết và dự đoán cho mục đích tương lai, v.v. Việc sử dụng trí tuệ nhân tạo trong nông nghiệp có thể tận dụng sự hỗ trợ cho nông dân và cải thiện kỹ thuật canh tác của họ bằng cách cung cấp dữ liệu chính xác theo yêu cầu. Sau đó, chúng ta sẽ thảo luận về các loại AI và các nhánh khác nhau của nó có thể có tác động lớn đến Nông nghiệp.

III. Các giai đoạn và loại hình của Trí tuệ nhân tạo

Trí tuệ nhân tạo có các giai đoạn khác nhau, cũng có thể gọi là các loại hình trí tuệ nhân tạo, bao gồm:

1. Trí tuệ nhân tạo hẹp (Artificial Narrow Intelligence - ANI)
2. Trí tuệ nhân tạo chung (Artificial General Intelligence - AGI)
3. Siêu trí tuệ nhân tạo (Artificial Super Intelligence - ASI)

Ba loại này có thể được coi là cách AI phát triển, và bây giờ chúng ta hãy xem xét riêng từng loại:

Trí tuệ nhân tạo hẹp (ANI): Còn được gọi là AI yếu, ANI chỉ có thể thực hiện các nhiệm vụ được xác định hẹp, không có khả năng nhận thức và suy nghĩ, đây chỉ là các chức năng được xác định trước đã được nạp vào hệ thống.

- Ví dụ: Siri, Alex, xe tự lái, Alpha Go, robot hình người Sophia, v.v., là những ANI mà chúng ta đã phát triển cho đến nay.

Trí tuệ nhân tạo chung (AGI): Còn được gọi là AI mạnh, AGI là giai đoạn thứ hai của quá trình tiến hóa Trí tuệ nhân tạo, sở hữu khả năng suy nghĩ và đưa ra quyết định giống như con người.

- Hiện tại, chúng ta chưa phát triển được hệ thống nào có thể coi là ví dụ hoàn hảo cho hệ thống Trí tuệ nhân tạo chung.
- AI mạnh cho đến nay vẫn bị nhiều nhà nghiên cứu, nhà khoa học và ngay cả những gã khổng lồ công nghệ như Google, Microsoft, Tesla, v.v. coi là mối đe dọa đối với sự tồn tại của con người.
- Sự phát triển hoàn chỉnh của một Hệ thống Trí tuệ nhân tạo có thể đặt dấu chấm hết cho nhân loại và cho đến nay, con người không có khả năng cạnh tranh với một hệ thống mạnh mẽ nhất về tư duy và trí thông minh như vậy.

Siêu trí tuệ nhân tạo (ASI): Đây có thể là giai đoạn của Trí tuệ nhân tạo mà khả năng của hệ thống AI sẽ vượt qua mọi giới hạn và có thể lấn át con người.

- ASI hiện tại chỉ là một tình huống giả định có thể xảy ra trong tương lai gần nếu nhân loại tiếp tục tiến bộ trong việc phát triển Trí tuệ nhân tạo.

Như chúng ta đã thấy các giai đoạn phát triển của trí tuệ nhân tạo, hãy xem một số hệ thống trí tuệ nhân tạo thuần túy dựa trên các chức năng mà chúng thực hiện. Chúng bao gồm:

1. **Trí tuệ nhân tạo phản ứng (Reactive Machine AI):** Loại hệ thống AI này nhận đầu vào là dữ liệu hiện tại và thực hiện nhiệm vụ dựa trên các tình huống hiện tại. Hệ thống máy AI phản ứng không thể phân tích và dự đoán nhiệm vụ tương lai hoặc hành động của chúng trong bất kỳ tình huống cụ thể nào. Các hệ thống này luôn thực hiện các nhiệm vụ trong phạm vi hẹp đã được xác định trước.
2. **AI có bộ nhớ hạn chế (Limited Memory AI):** Các hệ thống AI này chủ yếu dựa trên dữ liệu chúng đã lưu trữ và thường thực hiện nhiệm vụ bằng cách nghiên cứu các nhiệm vụ đã thực hiện trước đó, đồng thời dự đoán nhiệm vụ tương lai của máy.
3. **AI Lý thuyết về Tâm trí (Theory of Mind AI):** Đây là những hệ thống AI tiên tiến hơn, được sử dụng để suy đoán và đóng vai trò chính trong lĩnh vực tâm lý học. Các hệ thống AI này luôn thực hiện các nhiệm vụ liên quan đến Trí tuệ cảm xúc và những suy nghĩ có thể được thấu hiểu trong bất kỳ tình huống nào của một cá nhân.
4. **AI Tự nhận thức (Self-Aware AI):** Đây là hệ thống tiên tiến nhất cho đến nay và thuộc giai đoạn cuối cùng của Trí tuệ nhân tạo, nơi hệ thống AI có ý thức và tự nhận thức được mọi thứ. Trong tương lai gần, điều này có thể đạt được.

Như chúng ta đã thấy các giai đoạn và loại hình khác nhau của hệ thống Trí tuệ nhân tạo, nhưng bên cạnh đó, các hệ thống này được tạo ra bằng cách sử dụng một số công nghệ được coi là nhánh của trí tuệ nhân tạo. Chúng bao gồm:

- Học máy (Machine Learning)
- Học sâu (Deep Learning)
- Thị giác máy tính (Computer Vision)
- Khoa học dữ liệu (Data Science)
- Dữ liệu lớn (Big Data)

Các nhánh này của Trí tuệ nhân tạo đã được sử dụng rộng rãi trong lĩnh vực nông nghiệp để cải thiện sản xuất và mang lại hiệu quả trong thu hoạch. Chúng ta sẽ thảo luận ngắn gọn về những điều này trong các chương sau và việc triển khai chúng với một số dự án.

IV. Ứng dụng của AI trong Nông nghiệp

Trí tuệ nhân tạo có nhiều ứng dụng và có thể được sử dụng theo nhiều cách trong lĩnh vực nông nghiệp. Vì các cánh đồng nông nghiệp cung cấp cho chúng ta rất nhiều dữ liệu về nhiệt độ, lượng mưa, bức xạ mặt trời và tốc độ gió, những dữ liệu này có thể được phân tích và giúp chúng ta hưởng lợi về mặt thu hoạch từ các cánh đồng nông nghiệp.

Phần tốt nhất khi triển khai trí tuệ nhân tạo trong nông nghiệp là nó sẽ không lấy đi công việc của nông dân mà sẽ giúp nông dân cải thiện quy trình thu hoạch trên đồng ruộng.

Trí tuệ nhân tạo có thể có những ứng dụng tốt nhất trong nông nghiệp, một số trong đó là:

1. **Bot Nông nghiệp Trí tuệ nhân tạo:** Các bot nông nghiệp hỗ trợ AI có thể giúp nông dân tìm ra những cách hiệu quả hơn để tìm kỹ thuật thu hoạch tốt hơn trên bất kỳ loại đồng ruộng nào và hướng dẫn nông dân bảo vệ cây trồng khỏi cỏ dại. Những bot này cũng có thể giúp thu hoạch cây trồng trên các cánh đồng với khối lượng lớn hơn và tốc độ nhanh hơn bất kỳ lao động con người nào, bằng cách này nông dân có thể thu hoạch nhiều cây trồng hơn trong thời gian ngắn hơn. Bằng cách triển khai thị giác máy tính với các bot, nó cũng có thể phát hiện một số bệnh trên cây trồng và phun thuốc trừ sâu đúng thời điểm.
2. **Giám sát sức khỏe cây trồng và đất:** Trí tuệ nhân tạo có thể được sử dụng một cách hiệu quả để theo dõi và xác định các khiếm khuyết và thiếu hụt chất dinh dưỡng trong đất. Với sự trợ giúp của phương pháp nhận dạng hình ảnh của thị giác máy tính, các hệ thống có thể được phát triển để xác định các khiếm khuyết của cây trồng và sử dụng các ứng dụng học sâu để phân tích các mẫu hệ thực vật trong nông nghiệp. Loại ứng dụng hỗ trợ AI này sẽ giúp dễ dàng hiểu được các bộ phận khác nhau của cây trồng, các khiếm khuyết, sâu bệnh và bệnh tật của chúng.
3. **Hệ thống chuyên gia nông nghiệp:** Hệ thống chuyên gia có thể là tốt nhất cho nông dân trong việc cung cấp hướng dẫn phù hợp. Hệ thống chuyên gia mặc định

là hệ thống lưu trữ kiến thức cấp chuyên gia, được thu thập từ các chuyên gia khác nhau trong lĩnh vực đó và dựa trên các tình huống và thực tiễn khác nhau được thực hiện bởi các chuyên gia.

4. **Dữ liệu thời tiết dự báo:** Trí tuệ nhân tạo một cách tiên tiến có thể giúp nông dân cập nhật về những thay đổi thời tiết đột ngột, giúp họ bảo vệ cây trồng khỏi tình huống nguy hiểm. Phân tích do hệ thống thực hiện có thể giúp nông dân thực hiện các biện pháp phòng ngừa.
5. **AI với Máy bay không người lái (Drones):** Công nghệ máy bay không người lái giúp nông dân cải thiện năng suất cây trồng và giảm chi phí. Người dùng lập trình trước tuyến đường của máy bay không người lái và một khi nó được triển khai trên đồng, nó có thể sử dụng công nghệ trên không với nhiều tính năng khác nhau như chụp dữ liệu hình ảnh trực tiếp của cây trồng, phun thuốc trừ sâu khi phát hiện bệnh ở các cây trồng cụ thể trên đồng, v.v. Máy bay không người lái cũng có thể được lập trình cho bất kỳ nhiệm vụ cụ thể nào liên quan đến nông nghiệp như phân tích đồng ruộng theo thời gian và quét tìm bất kỳ cây trồng nào bị nhiễm bệnh. Máy ảnh trên máy bay không người lái được trang bị công nghệ hình ảnh có thể giúp quản lý tổng thể đồng ruộng bằng cách cung cấp thông tin thời gian thực về nhu cầu nước, phân bón, thuốc trừ sâu, v.v. của cây trồng. Học máy có thể được sử dụng để phân tích đúng tình trạng cây trồng và đất đai bằng cách cung cấp cho chúng ta cái nhìn sâu sắc về cây trồng và các yêu cầu của nó, điều này có thể làm tăng sản lượng cây trồng trên đồng.
6. **Thu hoạch trong nhà với AI:** Thu hoạch trong nhà đã trở thành một xu hướng trong thời gian gần đây, nơi mọi người có thể thu hoạch các loại cây trồng khác nhau trong một khu vực nhỏ bằng cách cung cấp ánh sáng nhân tạo phù hợp theo yêu cầu của chúng. Bằng cách sử dụng Trí tuệ nhân tạo, việc thu hoạch trong nhà sẽ trở nên đơn giản và dễ dàng hơn vì nó có thể phân tích mọi thứ trong một môi trường khép kín và tự động hóa quá trình thu hoạch, cho dù đó là thủy canh hay aquaponics.

V. Lợi ích của AI trong Nông nghiệp

Trí tuệ nhân tạo có nhiều lợi ích trong các lĩnh vực khác nhau và ở đây, chúng ta thảo luận về các lợi ích trong lĩnh vực nông nghiệp, bao gồm:

- Trí tuệ nhân tạo (AI) đã cung cấp cho chúng ta các phương pháp hiệu quả hơn để sản xuất, thu hoạch và có thể hướng dẫn chúng ta bán các loại cây trồng thiết yếu bằng cách phân tích giá trị thị trường.
- AI có thể được triển khai để kiểm tra khiếm khuyết của cây trồng và cải thiện việc thu hoạch các cây trồng khỏe mạnh.
- Sự gia tăng tiện ích công nghệ dựa trên Trí tuệ nhân tạo có thể củng cố các doanh nghiệp và thị trường dựa trên nông nghiệp hoạt động hiệu quả hơn.
- AI có thể giúp cải thiện các phương thức quản lý cây trồng và việc thu hoạch.

- Các giải pháp dựa trên AI có thể được sử dụng để giải quyết các thách thức truyền thống khác nhau mà nông dân đang phải đối mặt như biến đổi khí hậu, sự phá hoại của sâu bệnh và cỏ dại làm giảm năng suất cây lương thực.

Tóm tắt

Như đã thảo luận trong chương này về Trí tuệ nhân tạo và các loại hình của nó, cũng như các lợi ích của nó trong nông nghiệp.

Chương này cũng thảo luận rằng Trí tuệ nhân tạo có thể giúp tăng năng suất và bảo vệ cây trồng khi canh tác nông nghiệp trong thời gian thực, điều này có thể rất hữu ích cho nông dân ngày nay.

Do sự tiến bộ của công nghệ, nông dân cũng nên có sự tiến bộ trong kỹ thuật nông nghiệp để tăng sản lượng cây trồng và tiết kiệm thời gian, trong đó Trí tuệ nhân tạo có thể đóng một vai trò rất lớn.

CHƯƠNG 2 HỌC PYTHON CHO TRÍ TUỆ NHÂN TẠO

Như chúng ta đã thấy Trí tuệ Nhân tạo có thể hữu ích như thế nào trong nông nghiệp nhưng câu hỏi tiếp theo luôn xuất hiện trong đầu là chúng ta có thể bắt đầu học từ đâu, bằng cách nào và các mô hình học một trí tuệ nhân tạo là gì.

Vì Trí tuệ Nhân tạo là một phần rất lớn của Khoa học Máy tính, điều này có nghĩa là chúng ta cần một ngôn ngữ làm phương tiện để thực hiện nó và chúng ta luôn ưu tiên Python cho việc đó, một ngôn ngữ rất linh động và dễ học.

Trong chương này, chúng ta sẽ học về những kiến thức cơ bản của python để nó sẽ hữu ích cho chúng ta trong việc triển khai trí tuệ nhân tạo sau này.

I. Giới thiệu

Python là một ngôn ngữ đáng chú ý, vừa dễ học vừa mạnh mẽ. Bạn sẽ ngạc nhiên khi thấy thật đơn giản để không phải tập trung vào ngôn ngữ bạn đang lập trình, mà là tập trung vào việc giải quyết vấn đề.

Guido van Rossum là người tạo ra ngôn ngữ Python, được đặt tên theo chương trình "Monty Python's Flying Circus" của đài BBC.

Python được giới thiệu chính thức là:

> "Python là một ngôn ngữ lập trình mạnh mẽ, dễ học. Nó có các cấu trúc dữ liệu cấp cao hiệu quả và một cách tiếp cận đơn giản nhưng hiệu quả với lập trình hướng đối tượng. Cú pháp tao nhã và kiểu dữ liệu động của Python, cùng với bản chất thông dịch, làm cho nó trở thành ngôn ngữ lý tưởng cho việc viết kịch bản và phát triển ứng dụng nhanh trong nhiều lĩnh vực trên hầu hết các nền tảng."

II. Các đặc điểm của Python

- **Đơn giản:** Ngôn ngữ Python đơn giản và tinh gọn. Đọc một chương trình Python tốt gần giống như đọc tiếng Anh, nhưng thực sự rất cô đọng! Việc Python giống như mã giả là một trong những tài sản lớn nhất của nó. Điều này giúp bạn tập trung vào giải pháp cho vấn đề thay vì bản thân ngôn ngữ.
- **Dễ học:** Như bạn sẽ thấy, Python có thể được bắt đầu rất nhanh. Như đã mô tả, Python có cú pháp cực kỳ đơn giản. * **Miễn phí và mã nguồn mở:** Python là một ví dụ về FLOSS (Phần mềm Tự do và Mã nguồn mở). Bạn có thể chia sẻ các bản sao miễn phí, đọc mã nguồn của phần mềm, thực hiện các thay đổi quan trọng và sử dụng một phần của nó trong các chương trình miễn phí mới. FLOSS dựa trên ý tưởng về một mạng lưới chia sẻ trí tuệ. Đây là một trong những yếu tố Tại sao Python được phát triển và liên tục thay đổi.

- **Ngôn ngữ cấp cao:** Bạn không bao giờ phải bận tâm đến các thông tin nhỏ nhặt như quản lý việc sử dụng bộ nhớ trong chương trình của mình, v.v., khi bạn lập trình bằng Python.
- **Khả chuyển (Portable):** Python đã được điều chỉnh cho (tức là, cập nhật cho) nhiều nền tảng vì nó là mã nguồn mở. Bất kỳ hệ thống nào trong số này cũng sẽ chạy tất cả các chương trình Python của bạn mà không cần sửa đổi nếu bạn đủ cẩn thận để tránh các chức năng phụ thuộc vào hệ thống. Python cũng có thể được sử dụng cho GNU/Linux, Windows, FreeBSD, Macintosh, Solaris, $OS/2$, Amiga, AROS, AS/400, BeOS, OS/390, z/OS, Palm OS, Windows CE và PocketPC. Bạn cũng có thể tạo trò chơi cho máy tính, iPhone, iPad hoặc Android của mình với một nền tảng như Kivy.
- **Khả năng mở rộng (Extensible):** Bạn có thể lập trình một phần của chương trình bằng C hoặc $C++$ và sau đó sử dụng nó từ chương trình Python của mình nếu bạn cần một phần mã quan trọng chạy thật nhanh hoặc bạn muốn một số thuật toán không bị tiết lộ.
- **Khả năng nhúng (Embeddable):** Để cung cấp cho người dùng khả năng viết kịch bản cho phần mềm của bạn, bạn có thể tích hợp Python vào các chương trình C/C++ của mình.
- **Thông dịch (Interpreted):** Một chương trình được biên dịch như C hoặc $C++$ được dịch từ ngôn ngữ nguồn, tức là C hoặc $C++$ sang ngôn ngữ lập trình (mã nhị phân tức là 0 và 1) bằng một trình biên dịch chứa các cờ và tùy chọn khác nhau Phần mềm Linker/Loader (Trình liên kết/Trình tải) sao chép và bắt đầu chạy chương trình này từ đĩa cứng vào bộ nhớ. Mặt khác, Python không yêu cầu biên dịch sang mã nhị phân. Phần mềm được chạy trực tiếp từ mã nguồn. Python tự động chuyển đổi mã nguồn thành một loại trung gian gọi là mã byte (bytecodes), sau đó dịch nó sang ngôn ngữ gốc của máy tính rồi thực thi. Trên thực tế, sử dụng Python dễ dàng hơn nhiều vì bạn không phải lo lắng về việc biên dịch phần mềm để đảm bảo các thư viện chính xác được đính kèm và tải, v.v. Điều này làm cho chương trình Python của bạn dễ chuyển đổi hơn, vì bạn chỉ cần sao chép Python của mình sang một máy khác và nó cứ thế hoạt động!
- **Hướng đối tượng (Object Oriented):** Python hỗ trợ lập trình hướng thủ tục cũng như lập trình hướng đối tượng. Trong các ngôn ngữ hướng thủ tục, phần mềm được xây dựng xung quanh các thủ tục hoặc hàm, vốn chỉ là những đoạn chương trình chung. Trong các ngôn ngữ hướng đối tượng, phần mềm được thiết kế xung quanh các đối tượng kết hợp dữ liệu và chức năng. So với các ngôn ngữ chính như C++ hoặc Java, Python có một cách thức tốt nhưng đơn giản hóa để thực hiện OOP (Lập trình hướng đối tượng).
- **Thư viện phong phú:** Thư viện cơ bản của Python vô cùng lớn. Nó sẽ giúp bạn thực hiện nhiều việc khác nhau với biểu thức chính quy (regular expressions), tạo tài liệu, phân tích đơn vị (unit analysis), phân luồng (threading), cơ sở dữ liệu, trình duyệt web, CGI, FTP, email, XML, XML-RPC, HTML, WAV, mã hóa, GUI (Giao diện đồ họa người dùng) và các đối tượng phụ thuộc hệ thống khác. Tất cả

những điều này đều được tích hợp sẵn ở bất cứ đâu Python được cài đặt. Điều này được biết đến với triết lý "pin kèm theo" (batteries included) của Python. Bên cạnh thư viện chính, Chỉ mục Gói Python (Python Package Index) cũng cung cấp nhiều thư viện chất lượng cao khác.

III. Cài đặt

Python có bản phân phối cho một số lượng lớn các nền tảng. Bạn chỉ cần tải xuống mã nhị phân (binary code) cho ứng dụng của mình và cài đặt Python.

Nếu mã nhị phân cho nền tảng của bạn không có sẵn, bạn cần một trình biên dịch C để biên dịch mã nguồn bằng tay. Bạn có thể biên dịch mã nguồn một cách linh hoạt hơn để đưa ra lựa chọn về các tính năng cần thiết trong quá trình cài đặt.

Dưới đây là mô tả ngắn gọn về việc cài đặt Python trên các nền tảng khác nhau -

- **Cài đặt trên Unix và Linux**
  - Hãy làm theo các bước sau để cài đặt Python trên máy Unix/Linux.
  - Mở trình duyệt Web và truy cập https://www.python.org/downloads/ .
  - Làm theo liên kết để tải xuống mã nguồn đã nén có sẵn cho Unix/Linux.
  - Tải xuống và giải nén các tệp.
  - Chỉnh sửa tệp Modules/Setup nếu bạn muốn tùy chỉnh một số tùy chọn.
  - chạy tập lệnh ./configure
  - make
  - make install
  - Thao tác này sẽ cài đặt Python tại vị trí tiêu chuẩn /usr/local/bin và các thư viện của nó tại /usr/local/lib/pythonXX, trong đó XX là phiên bản của Python.
- **Cài đặt trên Windows**
  - Truy cập https://www.python.org/downloads/ và tải xuống phiên bản mới nhất. Tại thời điểm viết bài này, đó là Python 3.7 hoặc phiên bản cao hơn. Việc cài đặt cũng giống như bất kỳ phần mềm nào khác chạy trên Windows.
  - Lưu ý rằng nếu phiên bản Windows của bạn là pre-Vista (trước Vista), bạn chỉ nên tải xuống Python 3.4 vì các phiên bản mới hơn yêu cầu các phiên bản Windows mới hơn.
  - **THẬN TRỌNG:** Hãy chắc chắn rằng bạn đã chọn tùy chọn **Add Python 3.7 or greater version to PATH** (Thêm Python 3.7 hoặc phiên bản cao hơn vào PATH).
  - Để thay đổi vị trí cài đặt, hãy nhấp vào **Customize installation** (Tùy chỉnh cài đặt), sau đó nhấp vào **Next** và nhập C:\python35 (hoặc một vị trí thích hợp khác) làm vị trí cài đặt.

- Nếu trước đó bạn không chọn tùy chọn **Add Python 3.7 or greater version PATH** (Thêm Python 3.7 hoặc phiên bản cao hơn vào PATH), hãy chọn **Add Python to environment variables** (Thêm Python vào biến môi trường). Tùy chọn này thực hiện điều tương tự như **Add Python 3.7 or greater version to PATH** trên màn hình cài đặt đầu tiên.
- Bạn có thể chọn cài đặt **Launcher** (Trình khởi chạy) cho tất cả người dùng hoặc không, điều đó không quan trọng lắm. Launcher được sử dụng để chuyển đổi giữa các phiên bản Python khác nhau đã được cài đặt.
- Nếu PATH của bạn không được thiết lập đúng (bằng cách chọn tùy chọn **Add Python 3.7 or greater version Path** hoặc **Add Python to environment variables**), thì hãy làm theo các bước trong phần tiếp theo (DOS Prompt) để khắc phục. Nếu không, hãy chuyển đến phần **Running Python prompt on Windows** (Chạy dấu nhắc Python trên Windows) trong tài liệu này.
- Lưu ý: Dành cho những người đã biết lập trình, nếu bạn đã quen thuộc với Docker, hãy xem qua Python trong Docker và Docker trên Windows.

- **Cài đặt trên Mac OS**
  - Nếu bạn đang dùng Mac OS X, bạn nên sử dụng **Homebrew** để cài đặt Python 3. Đây là một trình cài đặt gói tuyệt vời cho Mac OS X và nó thực sự dễ sử dụng.
  - Nếu bạn chưa có Homebrew, bạn có thể cài đặt nó bằng lệnh sau:

```
ruby -e "(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/master/install)"
```

  - Chúng ta có thể cập nhật trình quản lý gói bằng lệnh bên dưới:

```
brew update
```

  - Bây giờ hãy chạy lệnh sau để cài đặt Python3 trên hệ thống của bạn:

```
brew install python3
```

IV. Thiết lập PATH

Các chương trình và các tệp thực thi khác phải nằm trong nhiều thư mục khác nhau để cung cấp cho hệ điều hành một đường dẫn tìm kiếm liệt kê các thư mục mà HĐH tìm kiếm.

Đường dẫn (route) được lưu trữ dưới dạng một chuỗi có tên được duy trì bởi hệ điều hành trong một biến môi trường. Biến này được cung cấp cho Shell lệnh và các chương trình khác.

Đối với Unix và Windows, thuộc tính đường dẫn được gọi là PATH (Unix phân biệt chữ hoa chữ thường; Windows thì không). Trình cài đặt sẽ quản lý thông tin theo dõi trong Mac OS. Bạn phải thêm thư mục Python vào đường dẫn của mình để gọi trình thông dịch Python từ bất kỳ thư mục cụ thể nào.

- **Thiết lập Path tại Unix/Linux**
  - Để thêm thư mục Python vào đường dẫn cho một phiên cụ thể trong Unix
  - **Trong csh shell** - gõ setenv PATH "PATH:/usr/local/bin/python" và nhấn Enter.
  - **Trong bash shell (Linux)** - gõ export ATH="PATH:/usr/local/bin/python" và nhấn Enter.
  - **Trong sh hoặc ksh shell** - gõ PATH="PATH:/usr/local/bin/python" và nhấn Enter.
  - *Lưu ý: /usr/local/bin/python là đường dẫn của thư mục Python*.
- **Thiết lập Path tại Windows**
  - Để thêm thư mục Python vào đường dẫn cho một phiên cụ thể trong Windows -
  - Tại dấu nhắc lệnh (command prompt) - gõ path %path%;C:\Python và nhấn Enter.
  - *Lưu ý: C:\Python là đường dẫn của thư mục Python*.



Fig. 2.1 Python Example in IDE

V. Chạy Python với GUI

Nếu bạn có GUI (Giao diện đồ họa người dùng) trên máy của mình hỗ trợ Python, bạn cũng có thể chạy Python từ môi trường GUI.

- **Unix:** IDLE là IDE (Môi trường phát triển tích hợp) đầu tiên trên Unix dành cho Python.
- **Windows:** PythonWin là giao diện Windows đầu tiên cho Python và là một IDE có GUI.
- **Macintosh:** Phiên bản Python dành cho Macintosh cùng với IDLE IDE có sẵn trên trang web chính, có thể tải xuống dưới dạng tệp MacBinary hoặc BinHex'd.

Bạn nên nhờ sự hỗ trợ từ người quản trị hệ thống nếu không thể thiết lập framework một cách chính xác. Đảm bảo rằng hệ thống Python được thiết kế đúng cách và chạy tốt.

Một trang web khác tên là Anaconda có thể được sử dụng. Chúng cũng là mã nguồn mở. Nó bao gồm hàng trăm gói khoa học dữ liệu phổ biến, gói conda và trình quản lý môi trường ảo cho Windows, Linux và MacOS. Bạn có thể truy cập nó từ liên kết https:/ www.anaconda.com/download/ bằng hệ điều hành của mình.

Trong cuốn sách này, tất cả các chương trình đều sử dụng Python 3.6 trên MS Windows 10.

VI. Bắt đầu với python

(a) Bước đầu tiên:

Bây giờ chúng ta sẽ xem cách chạy một chương trình 'Hello World' tiêu chuẩn trong Python. Bạn sẽ học cách viết, lưu và chạy các chương trình Python. Có hai phương pháp sử dụng Python để chạy phần mềm: dùng dấu nhắc thông dịch tương tác (interactive interpreter prompt) hoặc sử dụng tệp nguồn (source file). Bây giờ chúng ta hãy xem cách sử dụng hai cách tiếp cận này.

Bằng Dấu nhắc Thông dịch (Interpreter Prompt):

Mở terminal (cửa sổ dòng lệnh) trong hệ điều hành của bạn và sau đó mở dấu nhắc Python bằng phím python3. Phím [enter] sẽ được nhấn và mở ra.

Bạn sẽ thấy dấu >>> khi bạn bắt đầu gõ nội dung sau khi đã khởi động Python. Đây được gọi là *dấu nhắc trình thông dịch python* (python interpreter prompt).

Tại dấu nhắc trình thông dịch Python, gõ:

>>print ("Hello World")

Sau đó nhấn phím [Enter] và nó sẽ đưa ra kết quả -> Hello World.

Đây là một ví dụ về giao diện màn hình sẽ trông như thế nào trong Hình 2.1

Thoát khỏi Dấu nhắc Thông dịch

Bạn có thể thoát khỏi trình thông dịch bằng cách nhấn [ctrl + d] hoặc nhập exit() (nếu bạn đang sử dụng shell GNU/Linux hoặc OS X (lưu ý: nhớ sử dụng dấu ngoặc đơn, ())), sau đó nhấn phím [enter].

Trên dấu nhắc lệnh/trình thông dịch của Windows, nhấn [ctrl+z] rồi nhấn [enter].

Sử dụng Trình chỉnh sửa (Editor)

Trình chỉnh sửa mã (code editor) là một công cụ để viết và chỉnh sửa mã. Chúng thường dễ làm quen và có thể rất tuyệt. Tuy nhiên, khi phần mềm của bạn lớn hơn, bạn sẽ phải kiểm tra và gỡ lỗi (debug) mã của mình.

IDE (môi trường phát triển tích hợp) giúp hiểu mã của bạn tốt hơn nhiều so với một trình soạn thảo văn bản (text editor). Nó thường bao gồm các tính năng như tự động hóa cấu trúc (construct automation), bố cục ứng dụng (application layout), kiểm thử (testing) và gỡ lỗi (debugging). Nó sẽ cải thiện đáng kể việc nghiên cứu của bạn. Nhược điểm là các IDE rất khó sử dụng.

Có rất nhiều Trình chỉnh sửa như python IDE, Atom, PyCharm, Visual Studio code, v.v.. Chúng có thể được sử dụng cho mục đích lập trình và phát triển các dự án khác nhau.

(b) Những điều cơ bản:

Chỉ in ra "hello world" là không đủ, phải không? Bạn muốn làm điều gì đó khác - như thực hiện một số thay đổi, sử dụng chúng và nhận lại kết quả. Trong Python, điều này có thể được thực hiện bằng cách sử dụng hằng số và biến, và trong chương này, một số khái niệm khác cũng sẽ được thảo luận.

BÌNH LUẬN (COMMENTS)

Bình luận là văn bản nằm ở bên phải ký hiệu # và thường hữu ích cho người đọc về mặt kỹ thuật. Ví dụ:

```
>>print ('hello world') # Lưu ý rằng print là một hàm
```

hoặc:

```
# Lưu ý rằng print là một hàm
>>print ('hello world')
```

Hãy sử dụng càng nhiều bình luận hữu ích càng tốt để làm rõ các quyết định quan trọng trong chương trình của bạn, làm rõ các dữ kiện quan trọng, giải thích các vấn đề bạn đang cố gắng giải quyết, tương ứng.

Code cho bạn thấy **làm thế nào** (how), bình luận giải thích **tại sao** (why).

HẰNG SỐ CỤ THỂ (LITERAL CONSTANTS)

Một số ví dụ về hằng số cụ thể là số như 5, 1.23, hoặc một chuỗi như 'This is a string' hay 'It's a string,'. Nó được gọi là "cụ thể" (literal) vì bạn đang sử dụng đúng nghĩa của nó. Số 2 luôn là 2 và không là gì khác - nó là một hằng số, vì giá trị của nó không thể thay đổi. Tất cả những thứ này cũng được gọi là hằng số cụ thể.

CÁC CON SỐ (NUMBERS)

Hai loại số thường được sử dụng là Số nguyên (Integers) và Số thực (floats). Một ví dụ là 2, đây chỉ là một con số. Ví dụ về số thực là 3.23 và 52.3E-4 (hay gọi tắt là floats). Ký hiệu E biểu thị cho lũy thừa 10. Trong trường hợp này, 52.3E-4 là $52.3 * 10^{-4}$.

CHUỖI (STRINGS)

Chuỗi (string) là một tập hợp các ký tự. Về cơ bản, chuỗi chỉ là một loạt các từ. Gần như mọi chương trình Python bạn viết đều sẽ sử dụng chuỗi, vì vậy hãy rất cẩn thận với phần sau đây.

DẤU NHÁY ĐƠN (SINGLE QUOTE) Bạn có thể chỉ định chuỗi bằng cách sử dụng dấu nháy đơn, chẳng hạn như 'Quote me on this'. Tất cả các khoảng trắng, tức là dấu cách và tab, bên trong dấu ngoặc kép, đều được giữ nguyên.

DẤU NHÁY KÉP (DOUBLE QUOTE) Chuỗi trong dấu nháy kép hoạt động giống hệt như chuỗi trong dấu nháy đơn. Một ví dụ là "What's your name?".

DẤU NHÁY BA (TRIPLE QUOTE) Bạn có thể chỉ định chuỗi nhiều dòng bằng cách sử dụng dấu nháy ba - (""" hoặc '''). Bạn có thể tự do sử dụng dấu nháy đơn và dấu nháy kép bên trong dấu nháy ba. Một ví dụ là:

```
'''This is a multi-line string. This is the first line.
This is the second line.
"What's your name?," I asked.
He said "Bond, James Bond."
'''
```

Chuỗi là BẤT BIẾN (IMMUTABLE): Điều này có nghĩa là bạn không thể thay đổi một chuỗi sau khi đã tạo ra nó. Và ngay cả khi điều đó có vẻ là một điều tiêu cực, nó không hẳn là vậy. Chúng ta sẽ thấy các dịch vụ khác nhau mà chúng ta thấy sau này không bị giới hạn bởi điều này.

BIẾN (VARIABLES):

Chỉ sử dụng hằng số cụ thể sẽ sớm trở nên lặp đi lặp lại nếu đó là cách duy nhất để lưu trữ và thao tác dữ liệu. Đây là lúc các biến (variables) xuất hiện. Biến đúng như tên gọi của nó - giá trị của chúng có thể thay đổi, tức là, bạn có thể lưu trữ bất cứ thứ gì bằng một biến. Biến chỉ là một phần bộ nhớ của máy tính nơi thông tin được lưu trữ. Bạn cần một cách để kiểm soát các biến này và sau đó gọi chúng, trái ngược với các hằng số cụ thể.

TÊN ĐỊNH DANH (IDENTIFIERS)

Biến là ví dụ về tên định danh. Tên định danh (Identifiers) là tên được đặt để xác định một cái gì đó.

Có một số quy tắc bạn phải tuân theo khi đặt tên định danh:

- Ký tự đầu tiên của tên định danh phải là một chữ cái (ASCII viết hoa hoặc ASCII viết thường hoặc ký tự Unicode) hoặc dấu gạch dưới (_).
- Phần còn lại của tên định danh có thể bao gồm các chữ cái (ASCII viết hoa hoặc ASCII viết thường hoặc ký tự Unicode), dấu gạch dưới (_) hoặc chữ số (0-9).
- Tên định danh có phân biệt chữ hoa chữ thường. Ví dụ: myname và myName là không giống nhau. Lưu ý chữ n viết thường ở tên trước và chữ N viết hoa ở tên sau.
- Ví dụ về tên định danh hợp lệ là i, name_2_3. Ví dụ về tên định danh không hợp lệ là 2things, this is spaced out, my-name và >alb2_c3.

KIỂU DỮ LIỆU (DATA TYPES)

Các biến có thể chứa các giá trị thuộc nhiều loại khác nhau, được gọi là kiểu dữ liệu. Các dạng chính, mà chúng ta đã mô tả, là số và chuỗi. Chúng ta sẽ thảo luận trong các chương sau về cách hình thành các lớp (class) của riêng mình.

Python coi mọi thứ được sử dụng trong chương trình như một thực thể. Điều này được nói một cách chung chung. Họ nói "đối tượng" (object), thay vì "một cái gì đó". Python rất hướng đối tượng, theo nghĩa mọi thứ đều là một thực thể, bao gồm cả số, chuỗi và hàm.

Bây giờ chúng ta sẽ xem các biến có thể được sử dụng với các hằng số cụ thể như thế nào. Chạy chương trình và lưu ví dụ sau.

Cách viết một chương trình trong python:

Các bước cần tuân theo:

- Mở trình chỉnh sửa/Trình thông dịch (editor/Interpretor)
- Gõ mã chương trình như trong ví dụ
- Lưu nó dưới dạng tệp có tên với [DẤU CHẤM]py hoặc phần mở rộng ".py"
- Chạy trình thông dịch bằng lệnh python filename.py để chạy chương trình hoặc bạn có thể sử dụng nút run trên trình chỉnh sửa để chạy chương trình.
- Lưu ý: Tên tệp (Filename) có thể là bất cứ thứ gì nhưng nó phải có phần mở rộng .py

Ví dụ: Gõ và chạy chương trình sau

```
>>> # Tên tệp: var.py
i = 5
print (i)
i = i + 1
print (i)
s = '''This is a multi-line string.
This is the second line
Welcome to artificial intelligence.'''
print (s)
```

Kết quả (Output):

```
5
6
This is a multi-line string.
This is the second line
Welcome to artificial Intelligence.
```

Cách nó hoạt động (How It Works)

Đây là cách chương trình này hoạt động. Đầu tiên, chúng ta gán giá trị hằng số cụ thể 5 cho biến i bằng toán tử gán (=). Dòng này được gọi là một câu lệnh (statement) vì nó nêu rõ rằng cần phải làm gì đó và trong trường hợp này, chúng ta kết nối tên biến i với giá trị 5. Tiếp theo, chúng ta in giá trị của i bằng câu lệnh print, không ngạc nhiên, nó chỉ in giá trị của biến ra màn hình.

Sau đó, chúng ta cộng 1 vào giá trị được lưu trữ trong i và lưu nó trở lại. Sau đó, chúng ta in nó ra và như mong đợi, chúng ta nhận được giá trị 6.

Tương tự, chúng ta gán chuỗi cụ thể cho biến s và sau đó in nó ra.

THỤT LỀ (INDENTATION)

Khoảng trắng (Whitespace) trong Python là rất quan trọng. Khoảng trắng ở đầu dòng thực sự cần thiết. Đó là cái được gọi là thụt lề (indentation). Khoảng trắng hoặc tab ở đầu dòng logic (leading whitespace) được sử dụng để quyết định mức độ thụt lề của dòng logic đó, có tác dụng xác định việc nhóm các câu lệnh.

Điều đó có nghĩa là tất cả các câu lệnh nói chung sẽ có cùng một mức thụt lề. Tất cả các câu lệnh này được gọi là một *khối* (block). Trong các trang tiếp theo, chúng ta có thể thấy các ví dụ về tầm quan trọng của các khối.

Một điều bạn nên biết là việc đặt sai vị trí sẽ dẫn đến lỗi trong chương trình. Ví dụ:

```
>>> i = 5
# Lỗi bên dưới! Lưu ý có một dấu cách ở đầu dòng
 print ('Value is', i)
print ('I repeat, the value is', i)
```

Bạn sẽ nhận được lỗi sau khi chạy chương trình này:

```
  File "whitespace.py", line 3
    print ('Value is', i)
    ^
IndentationError: unexpected indent
```

Lưu ý ở đầu dòng thứ hai có một dấu cách. Lỗi của Python có nghĩa là cú pháp của chương trình không chính xác và chương trình chưa được viết đúng. Điều này đảm bảo cho bạn rằng các khối câu lệnh mới không thể được bắt đầu một cách ngẫu nhiên (ngoại trừ khối chính hiện có mà bạn đã sử dụng từ trước đến nay).

TOÁN TỬ & BIỂU THỨC (OPERATORS & EXPRESSIONS)

Phần lớn các câu lệnh bạn viết (các dòng logic) sẽ chứa các từ. Một biểu thức 2 + 3 là một ví dụ đơn giản. Một biểu thức có thể được chia thành các toán tử (operators) và toán hạng (operands).

Toán tử là các hàm có thể làm điều gì đó, các ký hiệu như + hoặc các phím đặc biệt là biểu tượng của chúng. Toán tử cần dữ liệu khác và những dữ liệu đó được gọi là toán hạng. 2 và 3 là các toán hạng trong trường hợp này.

Chúng ta sẽ thảo luận ngắn gọn về các toán tử và cách sử dụng chúng. Lưu ý rằng các biểu thức trong ví dụ có thể được kiểm tra tương tác với trình thông dịch. Ví dụ, dấu nhắc trình thông dịch Python tương tác được sử dụng để kiểm tra biểu thức 7 + 5:

```
>>>7 + 5
12
>>>3 * 8
24
>>>
```

Dưới đây là bản tóm tắt ngắn gọn về các toán tử có sẵn:

- **+ (cộng)** Cộng hai đối tượng. + 5 cho kết quả 8. 'a' + 'b' cho kết quả ‘ab’.
- **- (trừ)** Thực hiện phép trừ một số cho số khác; nếu toán hạng đầu tiên không có, nó được giả định là số không. -5.2 cho một số âm và 50 - 24 cho kết quả 26.
- *** (nhân)** Thực hiện phép nhân hai số hoặc trả về chuỗi được lặp lại nhiều lần. * 3 cho kết quả 6. 'la' * 3 cho kết quả ‘lalala’.
- **** (lũy thừa)** Trả về x mũ y. ** 4 cho kết quả 81 (tức là $3 * 3 * 3 * 3$). (Lưu ý: tài liệu gốc ghi $3 * 3 * 3 * 3 * 3$ là sai, phải là 4 lần 3)
- **/ (chia)** Chia x cho y. 3 cho kết quả 4.333333333333333.
- **// (chia và làm tròn xuống)** Chia x cho y và làm tròn câu trả lời xuống giá trị số nguyên gần nhất. Lưu ý rằng nếu một trong các giá trị là số thực (float), bạn sẽ nhận lại một số thực. 13 // 3 cho kết quả 4.
- [Tiếp tục danh sách toán tử…]
- -13 // 3 gives -5
- 9//1.81 gives 4.0
- **% (modulo)**
    - Trả về phần dư của phép chia.
    - 3 cho kết quả 1. -25.5% 2.25 cho kết quả 1.5.
- **<< (dịch trái)**
    - Dịch các bit của số sang bên trái theo số lượng bit được chỉ định. (Mỗi số được biểu diễn trong bộ nhớ bằng các bit hoặc chữ số nhị phân, tức là 0 và 1)
    - 2 << 2 cho kết quả 8. 2 được biểu diễn bằng 10 ở dạng bit. Dịch trái 2 bit sẽ cho 1000, đại diện cho số 8 trong hệ thập phân.
- **\>> (dịch phải)**
    - Dịch các bit của số sang bên phải theo số lượng bit được chỉ định.

    - 11 >> 1 cho kết quả 5. 11 được biểu diễn bằng bit là 1011, khi dịch phải 1 bit sẽ cho kết quả là 101, tức là số 5 trong hệ thập phân.
- **& (AND bit-wise)**
    - Phép toán AND bit-wise của các số.
    - 5 & 3 cho kết quả 1.
- (OR bit-wise)
    - Phép toán OR bit-wise của các số.
    - 5 | 3 cho kết quả 7.
- **^ (XOR bit-wise)**
    - Phép toán XOR bit-wise của các số.
    - ^ 3 cho kết quả 6.
- **~ (đảo bit-wise)**
    - Phép đảo bit-wise của x là $-(x+1)$.
    - ~5 cho kết quả -6.
- **< (nhỏ hơn)**
    - Trả về liệu x có nhỏ hơn y hay không. Tất cả các toán tử so sánh trả về True hoặc False. Lưu ý cách viết hoa của các tên này.
    - 5 < 3 cho kết quả False và 3 < 5 cho kết quả True. Các phép so sánh có thể được nối tùy ý: 3 < 5 < 7 cho kết quả True.
- **\> (lớn hơn)**
    - Trả về liệu x có lớn hơn y hay không.
    - 5 > 3 trả về True. Nếu cả hai toán hạng là số, chúng sẽ được chuyển đổi sang một kiểu chung. Nếu không, nó luôn trả về False.
- **<= (nhỏ hơn hoặc bằng)**
    - Trả về liệu x có nhỏ hơn hoặc bằng y hay không.
    - x=3; y=6; x<=y trả về True.
- **\>= (lớn hơn hoặc bằng)**
    - Trả về liệu x có lớn hơn hoặc bằng y hay không.
    - x=4; y=3; x>=3 trả về True.
- **== (bằng)**
    - So sánh xem các đối tượng có bằng nhau không.
    - x=2; y=2; x==y trả về True. (Tài liệu gốc ghi $x = y$)
    - x= 'str'; y=^{\circ}stR^{\prime}; x==y trả về False. (Tài liệu gốc ghi $x = y$)
    - x= 'str'; y=^{\circ}str^{\prime}; x==y trả về True. (Tài liệu gốc ghi $x = y$)
- **!= (không bằng)**
    - So sánh xem các đối tượng có không bằng nhau không.
    - x=2; y=3; x!=y trả về True.

- **not (phủ định logic NOT)**
  - Nếu x là True, nó trả về False. Nếu x là False, nó trả về True.
  - x= True; not x trả về False.
- **and (VÀ logic AND)**
  - x and y trả về False nếu x là False, ngược lại nó trả về giá trị của y.
  - x= False; y= True; x and y trả về False vì x là False. Trong trường hợp này, Python sẽ không đánh giá y vì nó biết rằng vế bên trái của biểu thức 'and' là False, điều này ngụ ý rằng toàn bộ biểu thức sẽ là False bất kể các giá trị khác. Điều này được gọi là **đánh giá ngắn mạch** (short-circuit evaluation).
- **or (HOẶC logic OR)**
  - Nếu x là True, nó trả về True, ngược lại nó trả về giá trị của y.
  - x= True; y= False; x or y trả về True. Đánh giá ngắn mạch cũng áp dụng ở đây.

LUỒNG ĐIỀU KHIỂN (CONTROL FLOW)

Trong các chương trình chúng ta đã thấy cho đến nay, các câu lệnh từ trên xuống dưới của Python cũng đã được thực hiện một cách trung thực. Sẽ thế nào nếu nhịp điệu hoạt động của nó thay đổi? Ví dụ, bạn có muốn hệ thống đưa ra một số quyết định và làm những việc nhất định tùy theo các tình huống khác nhau, như in 'Good morning' (Chào buổi sáng) hoặc 'Good evening' (Chào buổi tối) tùy thuộc vào thời điểm trong ngày không?

Như bạn có thể tưởng tượng, các câu lệnh luồng điều khiển được sử dụng để thực hiện điều này. Trong Python có ba câu lệnh luồng điều khiển—**if**, **for** và **while**.

- **Câu lệnh If (If Statement)** Nếu điều kiện là đúng, chúng ta chạy một khối (được gọi là **khối if**), ngược lại chúng ta phải xử lý một khối (được gọi là **khối else**). (Câu lặp lại trong tài liệu gốc) Phần else là tùy chọn.

Ví dụ (lưu với tên if.py):

```
>>>number = 77
guess = int (input ('Enter an integer : ')) # Nhập một số nguyên :
if guess == number:
    # New block starts here # Khối mới bắt đầu ở đây
    print ('Congratulations, you guessed it.') # Chúc mừng, bạn đã đoán đúng.
    print (' (but you do not win any prizes!)') # (nhưng bạn không thắng giải
thưởng nào!)
    # New block ends here # Khối mới kết thúc ở đây
elif guess < number:
    # Another block, in this we can use if for n number of times.
    # Khối khác, trong khối này chúng ta có thể sử dụng if cho n số lần.
    print ('No, it is a little higher than that') # Không, nó cao hơn một
chút
```

```
    # You can do whatever you want in a block... # Bạn có thể làm bất cứ điều
gì bạn muốn trong một khối...
else:
    print ('No, it is a little lower than that') # Không, nó thấp hơn một
chút
    # you must have guessed > number to reach here # bạn phải đoán > số để
đến đây
print ('Done') # Đã xong
# This last statement is always executed, # Câu lệnh cuối cùng này luôn được
thực thi,
# after the if statement is executed.: # sau khi câu lệnh if được thực thi.:
```

KẾT QUẢ:

```
 python if.py
Enter an integer: 22
No, it is a little higher than that
Done
```

Chúng ta lấy dự đoán từ người dùng trong chương trình này và kiểm tra xem nó có phải là số chúng ta có hay không. Chúng ta đã chỉ định số của biến cho bất kỳ số nguyên nào, giả sử là 23. (Lưu ý: mã nguồn ghi 77) Sau đó, chúng ta sử dụng hàm input để lấy dự đoán của người dùng. …là các đoạn chương trình có thể tái sử dụng. Chúng ta cung cấp một chuỗi cho hàm input tích hợp để in ra màn hình và đợi người dùng nhập.

Hàm input() trả về những gì chúng ta đã nhập dưới dạng một chuỗi sau khi chúng ta gõ gì đó và nhấn phím kbd: [enter]. Chuỗi này được dịch sang một số nguyên bằng int và được lưu trữ trong biến guess (dự đoán). (Tài liệu gốc ghi “conjecture”) Nói chung, int là một lớp (class), nhưng tất cả những gì bạn cần biết ngay bây giờ là bạn có thể sử dụng nó để chuyển đổi một chuỗi thành một số nguyên.

Tiếp theo, chúng ta so sánh dự đoán của người dùng với số chúng ta đã chọn. bằng cách in một thông báo thành công nếu chúng bằng nhau. Lưu ý, để cho Python biết câu lệnh nào thuộc khối nào, chúng ta sử dụng các mức thụt lề. Do đó, thụt lề trong Python rất cần thiết.

Lưu ý câu lệnh if ở cuối chứa một dấu hai chấm: chúng ta thông báo cho Python rằng sau nó là một khối các câu lệnh.

Sau đó, chúng ta kiểm tra xem giả định (guess) có nhỏ hơn con số không, và nếu có, chúng ta sẽ nói cho người tiêu dùng biết rằng con số cần đoán cao hơn một chút so với giả định. Mệnh đề elif mà chúng ta đã sử’ dụng ở đây đơn giản là kết hợp hai câu lệnh if-else tương tự thành một câu lệnh if-elif-else duy nhất. Nó cải thiện hệ thống và giảm mức độ thụt lề cần thiết.

Các câu lệnh elif và else phải luôn có dấu hai chấm ở cuối phần logic, theo sau là một chuỗi các câu lệnh (tất nhiên là được thụt lề đúng cách).

Bạn có thể có một câu lệnh if khác bên trong khối if của một câu lệnh if và cứ như thế—đây được gọi là một câu lệnh if lồng nhau (nested if).

Hãy nhớ rằng các phần elif và else là tùy chọn. Một câu lệnh if hợp lệ tối thiểu là:

```
>>>if                                                    True:
   print ('Yes, it is true') # Vâng, điều đó là đúng
```

Sau khi Python hoàn thành toàn bộ câu lệnh if cùng với các mệnh đề elif và else tương ứng, nó sẽ chuyển sang câu lệnh tiếp theo trong chuỗi sau câu lệnh if. Câu lệnh tiếp theo là câu lệnh print ('Done'). Đây là một khối lớn (nơi chương trình bắt đầu thực thi). Sau đó, Python thấy phần cuối của chương trình và kết thúc.

- **Câu lệnh For (For Statement)**

Câu lệnh for..in là một câu lệnh lặp (blowing statement - *Lưu ý: đây có thể là lỗi in ấn, "looping statement" mới là thuật ngữ đúng*), nó duyệt qua từng phần tử trong một chuỗi (sequence). Chúng ta sẽ tìm hiểu thêm về chuỗi trong các phần sau. Bây giờ bạn chỉ cần biết rằng chuỗi chỉ là một dãy các đối tượng.

Ví dụ (lưu với tên for.py):

```
for i in range (1, 5):
    print (i)
else:
    print ('The for loop is over') # Vòng lặp for đã kết thúc
```

Kết quả:

```
 python for.py
1
2
3
4
The for loop is over
```

Chúng ta in một chuỗi số trong chương trình này. Chúng ta sử dụng hàm range tích hợp để tạo ra chuỗi số này. Tất cả những gì chúng ta làm ở đây là cung cấp cho nó hai số và nó trả về một chuỗi số từ số thứ nhất đến số thứ hai. Ví dụ, range(1, 5) cung cấp chuỗi [1, 2, 3, 4].

Theo mặc định, range có bước nhảy (move number) là 1. Nếu chúng ta cung cấp số thứ ba, nó sẽ là bước nhảy trong chuỗi. Ví dụ, range(1, 5, 2)……liệt kê [1, 3]. Lưu ý rằng range mở rộng *cho đến* số thứ hai, tức là, số thứ hai không được bao gồm.

Hãy nhớ rằng, nếu bạn muốn có một bộ số hoàn chỉnh, ví dụ, list(range(5)) sẽ cho kết quả [0, 1, 2, 3, 4] (list() được giải thích trong chương cấu trúc dữ liệu). Vòng lặp for sau đó lặp qua danh sách này (list(s))—ví dụ, for i in range(1, 5) tương đương với việc gán từng

số trong chuỗi [1, 2, 3, 4] cho i. Trong trường hợp này, giá trị chỉ được in ra trong câu lệnh.

Lưu ý rằng phần else là tùy chọn. Khi được sử dụng, nó chỉ được thực hiện sau khi vòng lặp kết thúc, trừ khi gặp câu lệnh break.

Lưu ý rằng vòng lặp for..in hoạt động với mọi chuỗi. Ở đây, chúng ta có một danh sách các số được tạo bởi hàm range tích hợp, nhưng chúng ta có thể sử dụng bất kỳ loại chuỗi đối tượng nào!.

- **Câu lệnh While (While Statement)**

Câu lệnh while cho phép bạn thực thi lặp đi lặp lại một chuỗi các câu lệnh, miễn là một điều kiện còn đúng (valid). Đây là một ví dụ về cái gọi là câu lệnh lặp (looping statement). Một câu lệnh while có thể bao gồm một mệnh đề else tùy chọn.

Ví dụ (lưu với tên while.py):

```
number = 23
running = True
while running:
    guess = int (input ('Enter an integer: ')) # Nhập một số nguyên:

    if guess == number:
        print ('Congratulations, you guessed it.') # Chúc mừng, bạn đã đoán
đúng.
        running = False # điều này khiến vòng lặp while dừng lại
    elif guess < number:
        print ('No, it is a little higher than that.') # Không, nó cao hơn
một chút.
    else:
        print ('No, it is a little lower than that.') # Không, nó thấp hơn
một chút.
else:
    print ('The while loop is over.') # Vòng lặp while đã kết thúc.
# Do anything else you want to do here # Làm bất cứ điều gì khác bạn muốn làm
ở đây
print ('Done') # Đã xong
```

Kết quả:

```
 python while.py
Enter an integer: 50
No, it is a little lower than that.
Enter an integer: 22
No, it is a little higher than that.
Enter an integer: 23
```

Congratulations, you guessed it.
The while loop is over.
Done

Chúng ta vẫn chơi trò chơi đoán số trong chương trình này, nhưng lợi ích là người dùng có thể đoán cho đến khi họ nghĩ ra số đúng, không giống như phần trước—không cần phải chạy nó lặp đi lặp lại. Điều này cho thấy việc sử dụng Câu lệnh While một cách hợp lý.

Chúng ta chuyển phần nhập liệu vào bên trong vòng lặp while và đặt biến running thành True trước vòng lặp while. Đầu tiên, chúng ta kiểm tra xem biến running có phải là True không và sau đó tiếp tục thực thi (các câu lệnh bên trong). Sau khi khối này được hoàn thành, điều kiện (biến running) được kiểm tra lại. Nếu nó vẫn đúng (valid), chúng ta sẽ chạy lại khối While, nếu không, khối else tùy chọn sẽ tiếp tục được thực thi, và sau đó câu lệnh tiếp theo sẽ được thực thi.

Khối else được thực thi khi điều kiện while trở thành không đúng —ngay cả khi điều kiện được kiểm tra là sai ngay từ lần đầu tiên. Nếu có một mệnh đề else cho vòng lặp while, nó sẽ luôn hoạt động trừ khi bạn thoát ra khỏi vòng lặp bằng break.

True và False được gọi là các dạng Boolean (Boolean forms) và có thể được coi là tương đương với 1 và 0.

CÂU LỆNH BREAK (THE BREAK STATEMENT)

Câu lệnh break được sử dụng để thoát ra khỏi một câu lệnh lặp , tức là, dừng một câu lệnh lặp, ngay cả khi điều kiện của nó chưa trở thành False hoặc chuỗi các mục chưa được duyệt qua hoàn toàn.

Một điểm quan trọng là bất kỳ khối else nào đi kèm của vòng lặp sẽ không được thực thi khi bạn thoát ra khỏi vòng lặp for hoặc while bằng break.

Ví dụ (lưu với tên break.py):

```
while True:
    s = input ('Enter something : ') # Nhập gì đó :
    if s == 'quit':
        break
    print ('Length of the string is', len (s)) # Độ dài của chuỗi là
print ('Done') # Đã xong
```

Kết quả:

python break.py
Enter something : Programming is fun
Length of the string is 18
Enter something : When the work is done

Length of the string is 21
Enter something : if you wanna make your work also fun:

Length of the string is 37
Enter something : use Python!
Length of the string is 11
Enter something : quit
Done

CÂU LỆNH CONTINUE (THE CONTINUE STATEMENT)

Sử dụng câu lệnh continue, Python sẽ bỏ qua phần còn lại của các câu lệnh trong khối vòng lặp hiện tại và bắt đầu với vòng lặp (iteration) tiếp theo của vòng lặp.

Ví dụ (lưu với tên continue.py):

```
while True:
    s = input ('Enter something : ') # Nhập gì đó :
    if s == 'quit':
        break
    if len (s) < 3:
        print ('Too small') # Quá nhỏ
        continue
    print ('Input is of sufficient length') # Dữ liệu nhập có độ dài đủ
  # Do other kinds of processing here... # Thực hiện các loại xử lý khác ở đây...
```

Kết quả:

python continue.py
Enter something : a
Too small
Enter something : 12
Too small
Enter something : abc
Input is of sufficient length
Enter something : quit

HÀM (FUNCTIONS)

Hàm là những đoạn chương trình có thể tái sử dụng. Bạn có thể đặt tên cho một khối các câu lệnh, và bạn có thể sử dụng tên đó ở bất kỳ đâu và bất kỳ số lần nào để chạy khối lệnh đó trong chương trình của mình. Đây được gọi là *gọi hàm* (function calling). Chúng ta đã sử dụng một số hàm tích hợp như len và range.

Có thể đây là khối xây dựng chính cho bất kỳ ứng dụng không tầm thường nào (trong bất kỳ ngôn ngữ lập trình nào).

Các hàm được định nghĩa bằng từ khóa def. Sau từ khóa này là tên hàm, theo sau là một cặp dấu ngoặc đơn có thể chứa tên của các biến…và dấu hai chấm cuối cùng kết thúc dòng. Tiếp theo là tập hợp các khai báo (declarations) thuộc về hàm này.

Một ví dụ cho thấy điều này thực sự đơn giản:

Ví dụ (lưu với tên function1.py):

```
def say_hello ():
    # block belonging to the function # khối lệnh thuộc về hàm
    print ('Welcome to agriculture') # Chào mừng đến với nông nghiệp
# End of function # Kết thúc hàm

say_hello () # call the function # gọi hàm
say_hello () # call the function again # gọi lại hàm
```

Kết quả:

```
python function1.py
Welcome to agriculture
Welcome to agriculture
```

BIẾN CỤC BỘ (LOCAL VARIABLES)

Khi bạn khai báo các biến bên trong một định nghĩa hàm, chúng không liên quan gì đến các biến khác có cùng tên được sử dụng bên ngoài hàm - tức là, tên biến là cục bộ (local) đối với hàm. Đây được gọi là *phạm vi* (scope) của biến. Tất cả các biến có phạm vi là khối mà chúng được khai báo, bắt đầu từ điểm định nghĩa của tên.

Ví dụ (lưu với tên function_local.py):

```
x = 50

def func (x):
    print ('x is', x) # x là
    x = 2
    print ('Changed local x to', x) # Đã thay đổi x cục bộ thành

func (x)
print ('x is still', x) # x vẫn là
```

(Lưu ý: Có một lỗi logic trong mã nguồn gốc . func(x) được gọi trước khi x bên trong hàm được thay đổi. Bản dịch vẫn giữ nguyên mã, nhưng kết quả đầu ra sẽ phản ánh mã đã cho.)

Kết quả:

```
python function_local.py
x is 50
```

Changed local x to 2
x is still 50

CÂU LỆNH GLOBAL (THE GLOBAL STATEMENT)

Nếu bạn muốn gán một ý nghĩa (giá trị) cho một tên biến ở cấp cao nhất của chương trình (tức là, không nằm trong bất kỳ phạm vi nào, chẳng hạn như hàm hoặc lớp), bạn phải nói cho Python biết rằng một tên nhất định không phải là cục bộ, mà là toàn cục (global) . Chúng ta sử dụng câu lệnh global để làm điều này. Nếu không có câu lệnh global, bạn không thể gán giá trị cho một biến được định nghĩa bên ngoài một hàm.

Bạn có thể sử dụng các giá trị được định nghĩa bên ngoài hàm (mà không cần global nếu chỉ đọc). Điều này không được khuyến khích và nên tránh vì người dùng chương trình không biết ý nghĩa của biến này ở đâu. Từ câu lệnh global, có thể thấy rõ ràng rằng biến được định nghĩa trong khối bên ngoài.

Ví dụ (lưu với tên function_global.py):

```
x = 50
def func ():
    global x
    print ('x is', x) # x là
    x = 2
    print ('Changed global x to', x) # Đã thay đổi x toàn cục thành

func ()
print ('Value of x is', x) # Giá trị của x là
```

(Lưu ý: Có một lỗi nhỏ trong mã nguồn gốc, print ('Changed global x to',) thiếu biến x. Tôi đã sửa lại dựa trên ngữ cảnh là print ('Changed global x to', x) để khớp với kết quả đầu ra.)

Kết quả:

python function_global.py
x is 50
Changed global x to 2
Value of x is 2

GIÁ TRỊ THAM SỐ MẶC ĐỊNH (DEFAULT ARGUMENT VALUES)

Đối với một số hàm, bạn có thể tùy chọn hóa các tham số và sử dụng các giá trị mặc định nếu người dùng không muốn cung cấp giá trị cho chúng. Các giá trị tham số mặc định được sử dụng để làm điều đó. Bằng cách thêm toán tử gán (=) vào tên tham số trong đặc tả tham số, bạn có thể đặt giá trị tham số mặc định cho các tham số.

Hãy nhớ rằng câu lệnh mặc định là một giá trị không đổi. Cụ thể hơn—nó sẽ được thảo luận chi tiết trong các chương sau—các giá trị tham số mặc định phải là bất biến (immutable).

Ví dụ (lưu với tên function_default.py):

```
def say (message, times = 1):
    print (message * times)

say ('Hello')
say ('World', 5)
```

(Lưu ý: Mã nguồn gốc có một dấu * lạc. Dựa trên ngữ cảnh, nó có thể là lỗi định dạng. Câu lệnh đúng phải là print (message * times).)

Kết quả:

```
 python function_default.py
Hello
WorldWorldWorldWorldWorld
```

THAM SỐ TỪ KHÓA (KEYWORD ARGUMENTS)

Cú pháp đặc biệt **kwargs trong định nghĩa hàm trong python được sử dụng để truyền một danh sách tham số có độ dài thay đổi, có từ khóa. Chúng ta sử dụng tên kwargs với dấu sao kép. Lý do là vì dấu sao kép cho phép chúng ta truyền qua các tham số từ khóa (và bất kỳ số lượng nào trong số chúng).

Tham số từ khóa (keyword argument) là nơi bạn cung cấp tên cho biến khi bạn truyền nó vào hàm. Người ta có thể nghĩ kwargs như một từ điển (dictionary) ánh xạ mỗi từ khóa tới giá trị mà chúng ta truyền cùng với nó. Đó là lý do tại sao khi chúng ta lặp qua kwargs, dường như không có thứ tự nào mà chúng được in ra.

Ví dụ (lưu với tên function_keyword.py):

```
def func (a, b=5, c=10):
    print ('a is', a, 'and b is', b, 'and c is', c) # a là, và b là, và c là

func (3, 7)
func (25, c=24)
func (c=50, a=100)
```

Kết quả:

```
 python function_keyword.py
a is 3 and b is 7 and c is 10
a is 25 and b is 5 and c is 24
a is 100 and b is 5 and c is 50
```

THAM SỐ VARARGS (VARARGS PARAMETERS)

Đôi khi bạn có thể muốn định nghĩa một hàm có thể nhận bất kỳ số lượng tham số nào, tức là, số lượng đối số thay đổi (variable number of arguments), điều này có thể đạt được bằng cách sử dụng các dấu sao (lưu với tên function_varargs.py):

```
def total (a=5, *numbers, **phonebook):
    print ('a', a)

    #iterate through all the items in tuple # lặp qua tất cả các mục trong
tuple
    for single_item in numbers:
        print ('single_item', single_item)

    #iterate through all the items in dictionary # lặp qua tất cả các mục
trong từ điển
    for first_part, second_part in phonebook.items ():
        print (first_part, second_part)

total (10, 1, 2, 3, Jack=1123, John=2231, Inge=1560)
```

Kết quả:

python function_varargs.py
a 10

single_item 1
single_item 2
single_item 3
Inge 1560
John 2231
Jack 1123

CÂU LỆNH RETURN (RETURN STATEMENT)

Câu lệnh return được sử dụng để trả về (kết quả) từ một hàm, tức là, thoát ra khỏi hàm. Chúng ta cũng có thể tùy chọn trả về một giá trị từ hàm.

Ví dụ (lưu với tên function_return.py):

```
def maximum (x, y):
    if x > y:
        return x
    elif x == y:
        return 'The numbers are equal' # Các số bằng nhau
    else:
        return y

print (maximum (2, 3))
```

Kết quả:

```
 python function_return.py
3
```

CẤU TRÚC DỮ LIỆU (DATA STRUCTURES)

Về cơ bản, cấu trúc dữ liệu chính là như vậy—các cấu trúc có thể giữ tất cả dữ liệu lại với nhau. Nói cách khác, chúng dùng để xử lý một danh sách các thông tin tương tự. Trong Python có bốn cấu trúc dữ liệu tích hợp sẵn là **list** (danh sách), **tuple** (bộ), **dictionary** (từ điển) và **set** (tập hợp). Chúng ta sẽ xem cách sử dụng từng loại và cách chúng cải thiện cuộc sống của chúng ta.

LIST (DANH SÁCH): List là cấu trúc dữ liệu bao gồm một tập hợp các đối tượng có thứ tự, tức là, một chuỗi các đối tượng có thể được lưu trữ trong một danh sách. Bạn có thể dễ dàng hình dung điều này, nếu bạn có thể nghĩ về một danh sách mua sắm với một danh sách những thứ cần mua, ngoại trừ việc mỗi mục thực sự được đặt riêng trên danh sách mua sắm của bạn, trong khi danh sách Python dùng dấu phẩy.

Tập hợp các mục (items) phải được bao gồm trong dấu ngoặc vuông, để Python có thể hiểu rằng bạn đang định nghĩa một tập hợp (collection). Một khi bạn đã xây dựng một danh sách, bạn có thể thêm, xóa hoặc kiểm tra các mục trong danh sách. Vì chúng ta có thể thêm và xóa các mục, danh sách là một kiểu dữ liệu *khả biến* (mutable), tức là, nó có thể được sửa đổi.

Ví dụ (lưu với tên list.py):

```
# This is my shopping list # Đây là danh sách mua sắm của tôi
shoplist = ['apple', 'mango', 'carrot', 'banana'] # táo, xoài, cà rốt, chuối

print ('I have', len (shoplist), 'items to purchase.') # Tôi có ... món đồ
cần mua.

print ('These items are:', end=' ') # Những món đồ đó là:
for item in shoplist:
    print (item, end=' ')

print ('\nI also have to buy rice.') # Tôi cũng phải mua gạo.
shoplist.append ('rice') # thêm 'gạo'
print ('My shopping list is now', shoplist) # Danh sách mua sắm của tôi bây
giờ là

print ('I will sort my list now') # Bây giờ tôi sẽ sắp xếp danh sách của mình
shoplist.sort ()
print ('Sorted shopping list is', shoplist) # Danh sách mua sắm đã sắp xếp là

print ('The first item I will buy is', shoplist [0]) # Món đồ đầu tiên tôi sẽ
```

```
mua là
olditem = shoplist [0]
del shoplist [0]
print ('I bought the', olditem) # Tôi đã mua
print ('My shopping list is now', shoplist) # Danh sách mua sắm của tôi bây giờ là
```

Kết quả:

```
 python ds_using_list.py
I have 4 items to purchase.
These items are: apple mango carrot banana
I also have to buy rice.
My shopping list is now ['apple', 'mango', 'carrot', 'banana', 'rice']
I will sort my list now
Sorted shopping list is ['apple', 'banana', 'carrot', 'mango', 'rice']
The first item I will buy is apple
I bought the apple
My shopping list is now ['banana', 'carrot', 'mango', 'rice']
```

- **TUPLE (BỘ):** Tuples (Bộ) được sử dụng để giữ nhiều đối tượng cùng nhau. Hãy coi chúng như danh sách (lists), nhưng không có các tính năng mở rộng mà bạn nhận được từ lớp (class) danh sách. Một trong những đặc điểm quan trọng nhất của tuples là tính *bất biến* (unchangeable) của chúng, nghĩa là, bạn không thể thay đổi tuples. Tuples được xác định bằng cách chỉ định các đối tượng được phân tách bằng dấu phẩy, với dấu ngoặc đơn tùy chọn. Tuples thường được sử dụng trong trường hợp một câu lệnh hoặc hàm do người dùng định nghĩa có thể giả định một cách an toàn rằng tập hợp các giá trị không thay đổi. **Ví dụ (lưu với tên tuple.py):**

  ```
  # I would recommend always using parentheses
  # to indicate start and end of tuple
  # even though parentheses are optional.
  # Explicit is better than implicit.
  # Tôi khuyên bạn nên luôn sử dụng dấu ngoặc đơn
  # để chỉ ra điểm bắt đầu và kết thúc của tuple
  # mặc dù dấu ngoặc đơn là tùy chọn.
  # Rõ ràng (Explicit) tốt hơn là ngầm định (implicit).
  zoo = ('python', 'elephant', 'penguin')
  print ('Number of animals in the zoo is', len (zoo)) # Số lượng động vật trong sở thú là
  new_zoo = 'monkey', 'camel', zoo # parentheses not required but are a good idea # không yêu cầu dấu ngoặc đơn nhưng là một ý hay
  print ('Number of cages in the new zoo is', len (new_zoo)) # Số chuồng trong sở thú mới là
  print ('All animals in new zoo are', new_zoo) # Tất cả động vật trong sở thú mới là
  ```

```
print ('Animals brought from old zoo are', new_zoo [2]) # Động vật mang
từ sở thú cũ là
print ('Last animal brought from old zoo is', new_zoo [2] [2]) # Động
vật cuối cùng mang từ sở thú cũ là
print ('Number of animals in the new zoo is', len (new_zoo)-1+len
(new_zoo [2])) # Số lượng động vật trong sở thú mới là
```

(Lưu ý: Có vẻ như mã nguồn bị cắt xén và thiếu/lỗi. Ví dụ, new_zoo và zoo đều được coi là "chuồng" (cages) trong len(new_zoo) và "động vật" (animals) trong len(zoo). Phép tính số lượng động vật cuối cùng cũng có vẻ không chính xác. Tôi đã dịch nguyên trạng mã và kết quả.)

Kết quả:

```
 python ds_using_tuple.py
Number of animals in the zoo is 3
Number of cages in the new zoo is 3
All animals in new zoo are ('monkey', 'camel', ('python', 'elephant', 'penguin'))
Animals brought from old zoo are ('python', 'elephant', 'penguin')
Last animal brought from old zoo is penguin
Number of animals in the new zoo is 5
```

- **DICTIONARY (TỪ ĐIỂN)** Từ điển (dictionary) giống như một cuốn sổ địa chỉ, trong đó bạn có thể tìm thấy địa chỉ hoặc thông tin liên lạc của một người chỉ bằng tên của người đó. Nghĩa là, nó liên kết **khóa** (keys) (tên) với **giá trị** (values) (chi tiết). Xin nhớ rằng khóa (key) phải là duy nhất, giống như nếu bạn có hai người cùng tên, bạn không thể tìm thấy các khóa chính xác. Bạn chỉ có thể sử dụng các đối tượng *bất biến* (immutable) (như chuỗi) cho khóa của từ điển, nhưng bạn có thể sử dụng các đối tượng *khả biến* (mutable - tài liệu gốc ghi "dynamic or moving") hoặc bất biến cho các giá trị của từ điển. Điều này đơn giản có nghĩa là chỉ các đối tượng đơn giản mới có thể được sử dụng cho khóa. Các cặp khóa và giá trị được định nghĩa trong từ điển bằng ký hiệu d = {key1: value1, key2: value2}. Lưu ý rằng các cặp khóa-giá trị được phân tách bằng dấu hai chấm và các cặp được phân tách bằng dấu phẩy, tất cả được chứa trong một cặp dấu ngoặc nhọn. Lưu ý rằng các cặp khóa-giá trị không được sắp xếp theo bất kỳ cách nào trong từ điển. Nếu bạn muốn một thứ tự cụ thể, bạn phải tự sắp xếp nó trước khi sử dụng. Các từ điển bạn có thể sử dụng là các thực thể/đối tượng thuộc loại dict.

Ví dụ (lưu với tên dict.py):

```
# 'ab' is short for 'a'ddress'b'ook # 'ab' là viết tắt của 's'ổ 'đ'ịa'c'hỉ
ab = {
    'Swaroop': 'swaroop@swaroopch.com',
    'Larry': 'larry@wall.org',
    'Matsumoto': 'matz@ruby-lang.org',
    'Spammer': 'spammer@hotmail.com'
```

```
}

print ("Swaroop's address is", ab['Swaroop']) # Địa chỉ của Swaroop là

# Deleting a key-value pair # Xóa một cặp khóa-giá trị
del ab['Spammer']

print ('\nThere are {} contacts in the address-book\n'.format(len (ab))) # Có
... liên hệ trong sổ địa chỉ

for name, address in ab.items ():
    print ('Contact {} at {}'.format (name, address)) # Liên hệ ... tại ...

# Adding a key-value pair # Thêm một cặp khóa-giá trị
ab['Guido'] = 'guido@python.org'

if 'Guido' in ab:
    print ("\nGuido's address is", ab['Guido']) # Địa chỉ của Guido là
```

Kết quả:

```
 python ds_using_dict.py
Swaroop's address is swaroop@swaroopch.com

There are 3 contacts in the address-book

Contact Swaroop at swaroop@swaroopch.com
Contact Matsumoto at matz@ruby-lang.org
Contact Larry at larry@wall.org

Guido's address is guido@python.org
```

- **SET (TẬP HỢP):** Sets (Tập hợp) là các mảng (arrays) không có thứ tự của các mục đơn giản. Chúng được sử dụng khi sự hiện diện của một mục trong một tập hợp quan trọng hơn là thứ tự hoặc số lượng của nó. Bạn có thể sử dụng các tập hợp để kiểm tra tư cách thành viên (membership), xem nó có phải là tập hợp con (subset) của một tập hợp khác hay không, tìm giao (crossing) giữa hai tập hợp, v.v.. **Ví dụ (chạy nó trong trình thông dịch python hoặc trong IDE):**

  ```
  >>> bri = set (['brazil', 'russia', 'india'])
  >>> 'india' in bri
  True
  >>> 'usa' in bri
  False
  >>> bric = bri.copy ()
  >>> bric.add ('china')
  >>> bric.issuperset (bri) # bric có phải là tập hợp cha của bri không
  ```

```
True
>>> bri.remove ('russia')
>>> bri & bric # HOẶC bri.intersection (bric) (phép giao)
{'brazil', 'india'}
```

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OBJECT ORIENTED PROGRAMMING)

Trong tất cả các chương trình chúng ta đã viết cho đến nay, chúng ta đã xây dựng chương trình của mình để thao tác dữ liệu, tức là các khối câu lệnh. Đây được gọi là cách lập trình *hướng thủ tục* (process-oriented). Một cách khác để tổ chức chương trình của bạn là kết hợp dữ liệu và các tính năng (features) và gói chúng lại thành một thứ gọi là *thực thể* (entity).

Phong cách lập trình này được gọi là *hướng đối tượng* (object-oriented). Bạn thường có thể sử dụng lập trình hướng thủ tục, nhưng bạn có thể sử dụng các kỹ thuật lập trình hướng đối tượng khi viết các chương trình lớn hoặc nếu có một vấn đề phù hợp hơn với quy trình này.

Hai khía cạnh chính của lập trình hướng đối tượng là **lớp** (classes) và **đối tượng** (objects). Một lớp cung cấp một loại (kind) khác nhau, trong đó các đối tượng là các *thực thể* (instances) của lớp. Một sự tương tự là bạn có thể có các biến int, điều này cho thấy rằng các biến lưu trữ số nguyên là các thực thể (đối tượng) của lớp int.

Các đối tượng có thể lưu trữ dữ liệu bằng cách sử dụng các biến đối tượng tiêu chuẩn. Các trường (Fields) được gọi là *thuộc tính* (attribute) thuộc về một đối tượng hoặc lớp. Các đối tượng cũng có thể cung cấp các chức năng thông qua việc sử dụng các hàm thuộc sở hữu của lớp. Các hàm này được gọi là *phương thức* (methods) của lớp.

Thuật ngữ này rất quan trọng vì nó cho phép chúng ta phân biệt giữa các hàm và biến độc lập với các hàm và biến cụ thể của lớp hoặc đối tượng. Các trường và phương thức có thể được gọi chung là các *thuộc tính* (attributes) của lớp.

Các trường có hai loại—chúng có thể được phân loại vào bất kỳ thực thể/đối tượng nào hoặc chúng có thể được phân loại theo chính chúng (lớp). Chúng được gọi tương ứng là *biến thực thể* (instance variables) và *biến lớp* (class variables).

Lớp được phát triển bằng từ khóa class. Các trường và phương thức của lớp được hiển thị trong một loạt các thụt lề.

SELF (CHÍNH NÓ): Chỉ có một điểm khác biệt cụ thể duy nhất của các phương thức lớp so với các hàm thông thường—chúng yêu cầu một tên đầu tiên bổ sung phải được áp dụng vào đầu danh sách tham số, nhưng khi bạn gọi một phương thức, Python không gán ý nghĩa (giá trị) cho tham số này. Thuộc tính này tham chiếu đến chính đối tượng đó và theo mặc định được gọi là self.

Mặc dù rất nên sử dụng tên self cho tham số này—việc sử dụng bất kỳ tên nào khác chắc chắn sẽ bị phản đối. Có một số lợi ích khi sử dụng tên chung—bất kỳ người dùng nào trong phần mềm của bạn cũng có thể nhận ra nó ngay lập tức, và ngay cả các IDE tiên tiến cũng vậy.

Bạn phải tự hỏi làm thế nào Python gán ý nghĩa (giá trị) cho self và tại sao bạn không nên gán ý nghĩa (giá trị) cho nó. Điều này được làm rõ bằng một minh họa. Giả sử bạn có lớp MyClass và một đối tượng myObject của lớp đó. Khi bạn gọi một phương thức myobject.method(arg1, arg2), điều này được Python tự động dịch thành MyClass.method(myobject, arg1, arg2)—đó là toàn bộ vấn đề về self đặc biệt này .

Điều này cũng có nghĩa là ngay cả khi bạn có một quy trình (phương thức) không có đối số, luôn có một đối số—đó là "self.".

CÁC LỚP (CLASSES): Lớp đơn giản nhất có thể được hiển thị trong ví dụ sau (lưu với tên oop_simplestclass.py).

```
class Person:
    pass # An empty block (Một khối rỗng)

p = Person ()
print (p)
```

Kết quả:

```
 python oop_simplestclass.py
<__main__.Person instance at 0x10171f518>
```

Ở đây chúng ta thấy self đang hoạt động. Lưu ý rằng phương thức say_hi không nhận tham số nào nhưng vẫn có self trong định nghĩa hàm.

Phương thức init (The init Method):

Trong các lớp Python có một số tên phương thức có ý nghĩa đặc biệt. Ý nghĩa của quy trình (phương thức) __init__ bây giờ đã rõ ràng. Nếu một đối tượng của một lớp được khởi tạo (ví dụ: được tạo), phương thức __init__ sẽ được thực thi. Phương thức này hữu ích cho bất kỳ khởi tạo nào bạn muốn tạo (tức là, truyền các giá trị ban đầu cho đối tượng của bạn). Xin lưu ý dấu gạch dưới kép ở đầu và cuối của thuật ngữ.

Ví dụ (lưu với tên oop_init.py):

```
class Person:
    def __init__ (self, name):
        self.name = name
    def say_hi (self):
        print ('Hello, my name is', self.name) # Xin chào, tên tôi là
```

```
p = Person ('Mahesh')
p.say_hi ()
# The previous 2 lines can also be written as # 2 dòng trước cũng có thể được
viết là
# Person ('Swaroop').say_hi ()
```

Kết quả:

```
 python oop_init.py
Hello, my name is Mahesh
```

Ở đây, phương thức __init__ được chỉ định với tên của một tham số (bao gồm cả 'self' quen thuộc). Chúng ta chỉ đang tạo một trường (field) mới có tên là 'name' ở đây. Lưu ý rằng có hai biến vì cả hai đều có tên là 'name'. Cả hai đều riêng biệt. Ký hiệu dấu chấm self.name chỉ ra rằng thứ có tên "name" là một phần của đối tượng "self," và 'name' còn lại là một biến cục bộ. Không có sự mơ hồ nào vì chúng ta nêu rõ ràng chúng ta đang đề cập đến 'name' nào.

Khi bạn xây dựng thực thể mới p, của lớp Person, chúng ta sử dụng tên của lớp và sau đó là các đối số trong dấu ngoặc đơn: p = Person('Mahesh'). Chúng ta không gọi phương thức __init__ một cách tường minh. Đây là ý nghĩa đặc biệt của phương thức này.

Bây giờ, chúng ta có thể sử dụng trường self.name trong các phương thức của mình, điều này được minh chứng trong phương thức say_hi.

BIẾN LỚP và BIẾN ĐỐI TƯỢNG (CLASS and OBJECT VARIABLES)

Chúng ta đã đề cập đến các chức năng của các lớp và đối tượng (tức là các phương thức), và bây giờ chúng ta có thể nghe về thành phần dữ liệu . Phần dữ liệu, tức là các trường (fields), chỉ là các biến thông thường được kết nối với không gian tên (namespaces) của lớp và đối tượng. Điều này có nghĩa là những tên này chỉ tham chiếu đến các lớp và đối tượng này. Đó là lý do tại sao chúng được đặt tên là không gian tên.

Hai loại trường được phân loại tùy thuộc vào việc biến đó là thuộc tính của lớp hay đối tượng:

- **Biến lớp (Class variables)** được chia sẻ—tất cả các thực thể (instances) của lớp đó đều có thể truy cập chúng. Chỉ có 1 bản sao của biến lớp, bản sao này sẽ được sử dụng bởi tất cả các thực thể khác nếu một đối tượng thay đổi một biến lớp.
- **Biến đối tượng (Object variables)** được nắm giữ bởi mỗi đối tượng/thực thể của lớp. Trong trường hợp này, mỗi đối tượng có bản sao riêng của trường, tức là, nó không được chia sẻ trong một thực thể riêng biệt và không có cách nào kết nối với trường có cùng tên trong một thực thể khác.

Điều này sẽ dễ hiểu thông qua một ví dụ (lưu với tên oop_objvar.py):

```
class Robot:
    """Represents a robot, with a name.""" # Đại diện cho một robot, với một
```

```
cái tên.

    # A class variable, counting the number of robots
    # Một biến lớp, đếm số lượng robot
    population = 0

    def __init__ (self, name):
        """Initializes the data.""" # Khởi tạo dữ liệu.
        self.name = name
        print (" (Initializing {})".format (self.name)) # (Đang khởi tạo ...)

        # When this person is created, the robot
        # adds to the population
        # Khi người này (robot) được tạo, robot
        # được thêm vào quần thể
        Robot.population += 1

    def die (self):
        """I am dying.""" # Tôi đang chết.
        print ("{} is being destroyed!".format (self.name)) # ... đang bị phá
hủy!

        Robot.population -= 1

        if Robot.population == 0:
            print ("{} was the last one.".format (self.name)) # ... là con
robot cuối cùng.
        else:
            print ("There are still {:d} robots working.".format
(Robot.population)) # Vẫn còn ... robot đang hoạt động.

    def say_hi (self):
        """Greeting by the robot.
        Yeah, they can do that.""" # Lời chào của robot. Vâng, chúng có thể
làm điều đó.
        print ("Greetings, my masters call me {}.".format (self.name)) # Chào
mừng, chủ nhân của tôi gọi tôi là ...

    @classmethod
    def how_many (cls):
        """Prints the current population.""" # In ra số lượng hiện tại.
        print ("We have {:d} robots.".format (cls.population)) # Chúng ta có
... robot.

droid1 = Robot ("R2-D2")
droid1.say_hi ()
Robot.how_many ()

droid2 = Robot ("C-3PO")
droid2.say_hi ()
```

```
Robot.how_many ()

print ("\nRobots can do some work here.\n") # Robot có thể làm một số công
việc ở đây.

print ("Robots have finished their work. So let's destroy them.") # Robot đã
hoàn thành công việc. Vì vậy, hãy phá hủy chúng.
droid1.die ()
droid2.die ()

Robot.how_many ()
```

Kết quả:

```
 python oop_objvar.py
(Initializing R2-D2)
Greetings, my masters call me R2-D2.
We have 1 robots.
(Initializing C-3PO)
Greetings, my masters call me C-3PO.
We have 2 robots.

Robots can do some work here.

Robots have finished their work. So let's destroy them.
R2-D2 is being destroyed!
There are still 1 robots working.
C-3PO is being destroyed!
C-3PO was the last one.
We have 0 robots.
```

(Giải thích ví dụ oop_objvar.py từ Trang 29)

Đây là một ví dụ rộng (bao quát) về cách các biến lớp và biến đối tượng được minh họa. Trong trường hợp này, population (quần thể) thuộc về lớp Robot và do đó là một thuộc tính của lớp. Biến name (tên) là một biến đối tượng (biến được phân bổ bởi chính nó). Chúng ta cũng tham chiếu đến biến population là Robot.population thay vì Self.population. Chúng ta sử dụng ký hiệu self.name trong các phương thức của đối tượng để tham chiếu đến tên biến đối tượng. Hãy chú ý đến sự khác biệt cơ bản này giữa các biến của lớp và biến của đối tượng.

Cũng lưu ý rằng giá trị biến (lớp) sẽ bị che khuất bởi một đối tượng (biến đối tượng) có cùng tên!. (Lưu ý: Câu 909-910 trong tài liệu gốc có vẻ mâu thuẫn/khó hiểu) Chúng ta đáng lẽ nên sử dụng self.__class__.population thay vì Robot.population, vì bất kỳ đối tượng nào cũng được tham chiếu đến lớp của nó thông qua thuộc tính self.__class__ .

how_many là một hàm không thuộc về đối tượng mà thuộc về lớp. Điều này có nghĩa là, tùy thuộc vào việc chúng ta có cần biết chúng ta đang ở lớp nào hay không, chúng ta có thể mô tả nó là một phương thức động (dynamic) hoặc tĩnh (static). Chúng ta sử dụng dạng lớp (class form - @classmethod) vì chúng ta tham chiếu đến một thuộc tính của lớp. Chúng ta đã liệt kê nhiều cách sử dụng decorator (trình trang trí) như một công cụ của lớp. Decorators có thể được coi là các lối tắt đến một hàm được gọi bởi hàm bọc (wrapper) (ví dụ: một hàm "bọc" xung quanh một hàm để nó có thể làm điều gì đó trước hoặc sau hoạt động bên trong). Do đó, việc sử dụng decorator @classmethod cũng giống như gọi: how_many = classmethod(how_many) .

Bằng cách này, chúng ta tăng số lượng population lên 1 khi chúng ta thêm một robot khác. Hãy nhớ rằng các giá trị self.name là duy nhất cho mỗi thực thể, điều này cho thấy sự tồn tại của các biến của thực thể (biến đối tượng). Hãy nhớ rằng các biến và phương thức của cùng một đối tượng chỉ nên được tham chiếu bằng cách sử dụng self. Đây được coi là một thuộc tính tham chiếu.

Chúng ta cũng thấy trong phần mềm này việc sử dụng *docstrings* (chuỗi tài liệu) của lớp và quy trình (phương thức). Chúng ta có thể sử dụng Robot.__doc__ và phương thức docstring Robot.say_hi.__doc__. (Lưu ý: tài liệu gốc viết Robot. doc và Robot.say hi. doc) Trong phương thức die, chúng ta chỉ đơn giản là giảm số đếm Robot.population đi 1.

Mỗi thành viên của lớp đều là *public* (công khai). Một ngoại lệ là Python sử dụng cơ chế *name-mangling* (thay đổi tên) để biến một biến thành *private* (riêng tư) một cách hiệu quả bằng cách sử dụng các tên có tiền tố gạch dưới kép như __private_var. Quy tắc là mọi biến chỉ để sử dụng trong lớp hoặc đối tượng sẽ bắt đầu bằng dấu gạch dưới, và tất cả các tên khác là public và có thể được sử dụng cho tất cả các lớp hoặc đối tượng. Quy ước (practice) này được tuân theo. Hãy nhớ rằng đây chỉ là một quy ước và không bị Python thực thi (ngoại trừ tiền tố có dấu gạch dưới kép).

KẾ THỪA (INHERITANCE)

Tái sử dụng mã là một trong những lợi ích chính của lập trình hướng đối tượng và một cách để thực hiện điều này là thông qua cơ chế kế thừa (heritage - *Lưu ý: từ đúng là "inheritance"*). Kế thừa có thể được diễn giải tốt nhất như một mối quan hệ giữa các lớp thuộc loại (type) và kiểu con (subtype).

Giả sử bạn muốn viết một chương trình theo dõi giáo viên và sinh viên trong một trường đại học. Họ có một số đặc điểm chung bao gồm tên, tuổi và địa chỉ. Họ cũng có các đặc điểm riêng, chẳng hạn như tiền lương, các khóa học và học phí của giảng viên (instructor fees) và học phí của sinh viên (student charges). Bạn có thể xây dựng và xử lý hai lớp độc lập cho mỗi loại, nhưng việc thêm một đặc điểm chung sẽ (phải) thêm vào cả hai lớp độc lập này. Điều này nhanh chóng trở nên không thể quản lý được.

Sẽ là một cách tuyệt vời để xây dựng một lớp duy nhất có tên là SchoolMember và sau đó có các lớp giảng viên (instructor) và sinh viên (student) thuộc lớp này, và sau đó chúng ta có thể áp dụng các tính năng khác nhau cho các kiểu con đó. Chiến lược này có

một số lợi thế. Nếu trong SchoolMember, chúng ta thêm/sửa đổi một hàm, điều này sẽ tự động được phản ánh trong các kiểu con. Ví dụ, bằng cách chỉ cần thêm nó vào lớp SchoolMember, bạn có thể tạo một trường thẻ ID (ID card area) mới cho cả giáo viên và sinh viên. Nhưng những thay đổi trong các kiểu con không ảnh hưởng đến các kiểu con khác. Một lợi ích nữa là bạn có thể tham chiếu đến một đối tượng giáo viên hoặc sinh viên như một đối tượng SchoolMember, điều này có thể hữu ích trong một số trường hợp nhất định, chẳng hạn như đếm số lượng thành viên trong trường (schoolmembers). Đây được gọi là *tính đa hình* (polymorphism), nơi một kiểu con có thể được thay thế trong bất kỳ tình huống nào mà kiểu cha được mong đợi, tức là, đối tượng có thể được coi là một thực thể của lớp cha.

Lưu ý rằng, trong trường hợp các lớp riêng biệt, chúng ta (nay đã) tái sử dụng mã của lớp và không phải sao chép nó trong các lớp khác nhau như chúng ta đã phải làm. Trong trường hợp này, lớp Schoolmember được gọi là lớp Chính (Main) hoặc lớp Cha (Superclass). Các lớp giáo viên và sinh viên được gọi là các lớp dẫn xuất (derived grades - *Lưu ý: từ đúng là "derived classes"*).

Ví dụ (lưu với tên inheritace.py):

(Lưu ý: Tên tệp trong là inheritace.py, nhưng tên tệp trong kết quả đầu ra là oop_subclass.py. Tôi sẽ sử dụng tên inheritace.py theo phần mã.)

```
class SchoolMember:
    '''Represents any school member.''' # Đại diện cho bất kỳ thành viên nào
của trường.
    def __init__ (self, name, age):
        self.name = name
        self.age = age
        print ('(Initialized SchoolMember: {})'.format (self.name)) # (Đã
khởi tạo SchoolMember: ...)

    def tell (self):
        '''Tell my details.''' # Cho biết chi tiết của tôi.
        print ('Name:"{}" Age:"{}"'.format (self.name, self.age), end=" ") #
Tên: ... Tuổi: ...

class Teacher (SchoolMember):
    '''Represents a teacher.''' # Đại diện cho một giáo viên.
    def __init__ (self, name, age, salary):
        SchoolMember.__init__ (self, name, age) # Gọi __init__ của lớp cha
        self.salary = salary
        print ('(Initialized Teacher: {})'.format (self.name)) # (Đã khởi tạo
Teacher: ...)

    def tell (self):
        SchoolMember.tell (self) # Gọi phương thức tell của lớp cha
        print ('Salary: "{:d}"'.format (self.salary)) # Lương: ...
```

```
class Student (SchoolMember):
    '''Represents a student.''' # Đại diện cho một học sinh.
    def __init__ (self, name, age, marks):
        SchoolMember.__init__ (self, name, age) # Gọi __init__ của lớp cha
        self.marks = marks
        print ('(Initialized Student: {})'.format (self.name)) # (Đã khởi tạo
Student: ...)

    def tell (self):
        SchoolMember.tell (self) # Gọi phương thức tell của lớp cha
        print ('Marks: "{:d}"'.format (self.marks)) # Điểm: ...

t = Teacher ('Mrs. Shrividya', 40, 30000)
s = Student ('Swaroop', 25, 75)

# prints a blank line # in một dòng trống
print ()

members = [t, s]
for member in members:
    # Works for both Teachers and Students # Hoạt động cho cả Giáo viên và
Học sinh
    member.tell ()
```

(Lưu ý: Mã nguồn cho Student.__init__ có vẻ bị lỗi định dạng trong tài liệu gốc. Tôi đã diễn giải nó theo cách hợp lý nhất để khớp với ngữ cảnh của lớp Teacher.)

Kết quả:

```
 python oop_subclass.py
(Initialized SchoolMember: Mrs. Shrividya)
(Initialized Teacher: Mrs. Shrividya)
(Initialized SchoolMember: Swaroop)
(Initialized Student: Swaroop)

Name:"Mrs. Shrividya" Age:"40" Salary: "30000"
Name:"Swaroop" Age:"25" Marks: "75"
```

TỆP TIN (FILES):

Bạn có thể mở và sử dụng tệp để đọc hoặc viết bằng cách tạo một đối tượng của lớp (class) file và sử dụng các phương thức read, readline hoặc write của nó. Việc đọc hay viết tệp phụ thuộc vào chế độ (mode) bạn đã xác định khi mở tệp . Cuối cùng, khi bạn hoàn tất tệp, bạn gọi phương thức close để thông báo cho Python rằng chúng ta đã sử dụng xong kịch bản (tệp).

Ví dụ (lưu với tên file.py):

```
poem = '''Programming is fun
When the work is done
if you wanna make your work also fun:
    use Python!'''

# Open for 'w'riting # Mở để 'w' (ghi)
f = open ('poem.txt', 'w')
# Write text to file # Viết văn bản vào tệp
f.write (poem)
# Close the file # Đóng tệp
f.close ()

# If no mode is specified, # Nếu không có chế độ nào được chỉ định,
# 'r'ead mode is assumed by default # chế độ 'r' (đọc) được giả định là mặc
định
f = open ('poem.txt')
while True:
    line = f.readline ()
    # Zero length indicates EOF # Độ dài bằng 0 cho biết EOF (Kết thúc tệp)
    if len (line) == 0:
        break
    # The 'line' already has a newline # 'line' đã có một dòng mới
    # at the end of each line # ở cuối mỗi dòng
    # since it is reading from a file. # vì nó đang đọc từ một tệp.
    print (line, end='')
# close the file # đóng tệp
f.close ()
```

(Lưu ý: Tên tệp trong ví dụ đầu ra là io_using_file.py, nhưng trong phần lưu tệp là file.py. Tôi sẽ dịch theo kết quả đầu ra được cung cấp.)

Kết quả:

```
 python3 io_using_file.py
Programming is fun
When the work is done
if you wanna make your work also fun:
   use Python!
```

Lưu ý rằng bằng cách đơn giản sử dụng phương thức open, chúng ta có thể tạo một đối tượng tệp mới. Thông qua việc sử dụng hàm open tích hợp và chỉ định tên tệp cùng chế độ chúng ta muốn mở tệp, chúng ta mở (hoặc tạo nó nếu nó chưa tồn tại). Chế độ có thể bao gồm chế độ đọc ('r'), ghi ('w') hoặc nối thêm ('a'). Chúng ta cũng có thể chỉ định, liệu chúng ta đang đọc, viết hoặc nối thêm ở dạng văn bản ('t') hay nhị phân ('b'). Trên thực tế, có nhiều chế độ khác có sẵn và help(open) sẽ cung cấp cho bạn thêm thông tin.

Theo mặc định, open() coi tệp là 'tệp văn bản' (text file) và sau đó mở nó ở chế độ 'đọc'. Trong ví dụ của chúng ta, ban đầu chúng ta mở/tạo tệp ở chế độ văn bản và sử dụng quy

trình (phương thức) write của đối tượng tệp để ghi chuỗi biến poem của chúng ta và sau đó đóng tệp.

Đầu tiên (Tiếp theo), để xem, chúng ta mở lại cùng một tệp. Chúng ta không phải chỉ định chế độ vì chế độ mặc định là 'đọc tệp văn bản'. Chúng ta đọc từng dòng của tệp trong một vòng lặp bằng phương thức readline. Toàn bộ dòng, bao gồm ký tự xuống dòng ở cuối dòng, được trả về. Khi chúng ta trả về một chuỗi trống, điều đó có nghĩa là chúng ta đang ở cuối tệp và thoát ra khỏi vòng lặp.

Cuối cùng, chúng ta đóng tệp. Chúng ta có thể thấy từ kết quả readline của mình rằng chương trình này đã thực sự ghi vào và đọc từ tệp poem.txt mới của chúng ta.

Tóm tắt (Summary)

Như chúng ta đã thảo luận về các kiến thức cơ bản quan trọng của python và mọi khái niệm của ngôn ngữ lập trình python đều được thảo luận với các ví dụ phù hợp. Python sẽ được sử dụng trong các phần triển khai của chương tiếp theo và để tạo các dự án thời gian thực có thể được sử dụng trong thời gian thực cho các nghiên cứu và học thuật.

Trong việc triển khai python cho các nhánh xa hơn của trí tuệ nhân tạo, có một số Thư viện và framework liên quan sẽ được thảo luận theo từng chủ đề với việc cài đặt và sử dụng chúng với các thuật toán khác nhau.

CHƯƠNG 3: HỌC MÁY (MACHINE LEARNING)

Như chúng ta đã biết, có nhiều nhánh khác nhau của trí tuệ nhân tạo và học máy là một trong số đó, đã trở thành một nhánh tiên tiến và được sử dụng rộng rãi nhất của trí tuệ nhân tạo trong mọi lĩnh vực của thế hệ này, có thể là điện tử, xây dựng, cơ khí, kỹ thuật sinh học, y tế hoặc nông nghiệp, v.v.. Trong chương này, chúng ta sẽ nghiên cứu về học máy và các loại của nó cùng với các thuật toán học máy phổ biến nhất đang được sử dụng chủ yếu trong các tình huống khác nhau và việc triển khai chúng bằng ngôn ngữ python đã được thảo luận trong phần triển khai của cuốn sách này. Ở đây, chúng ta cũng sẽ xem học máy có thể được sử dụng ở đâu.

I. Giới thiệu

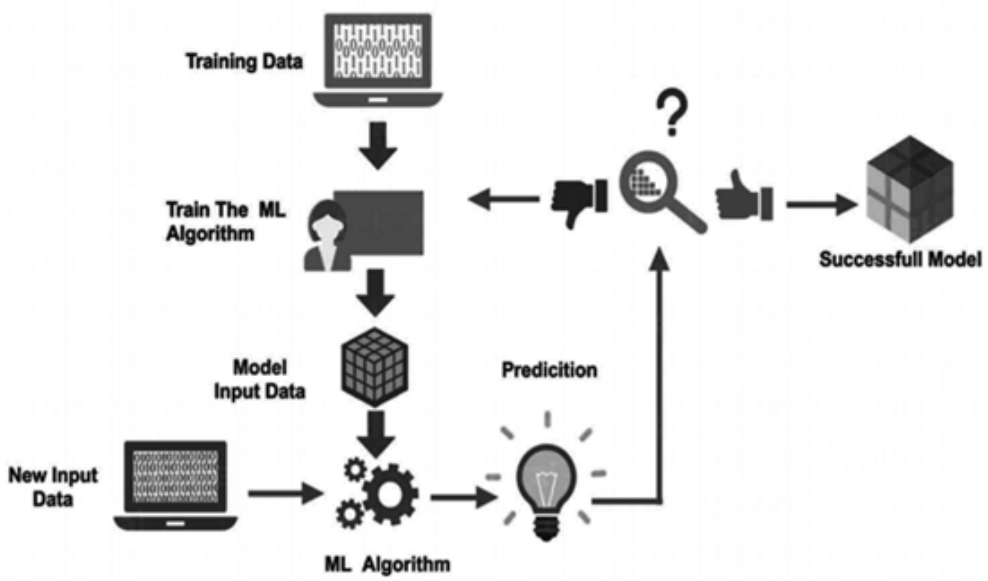
Học máy là một tập hợp con của Trí tuệ Nhân tạo và là một trong những phần quan trọng nhất của Trí tuệ Nhân tạo. Đây là một lĩnh vực của khoa học máy tính hoàn toàn khác biệt so với các phương pháp tính toán truyền thống. Trong các phương pháp truyền thống, một tập hợp các chỉ dẫn được cung cấp rõ ràng cho thuật toán để giải quyết bất kỳ vấn đề nào. Trong khi đó, Học máy là một lĩnh vực của AI, sử dụng phân tích thống kê và có thể cho phép một hệ thống luôn học hỏi từ dữ liệu của nó mà không cần bất kỳ đầu vào bên ngoài nào để đưa ra một kết quả đầu ra thiết yếu. Nhờ đó, học máy giúp máy tính tiến bộ hơn trong việc tạo ra mô hình cung cấp quyết định dựa trên dữ liệu đầu vào.

Học máy bao gồm rất nhiều thuật toán, thường dựa trên các yêu cầu và chức năng của chúng để sử dụng ở đâu và khi nào. Để tạo một mô hình học máy, chúng ta luôn cần dữ liệu để mô hình có thể được xây dựng dựa trên đó. Học máy sử dụng thuật toán liên tục học hỏi từ dữ liệu có sẵn dưới dạng dữ liệu huấn luyện. Một mô hình học máy được tạo ra sau khi thuật toán được huấn luyện từ dữ liệu. Sau khi quá trình huấn luyện hoàn tất, khi mô hình đã được huấn luyện được cung cấp dữ liệu thực tế khác, bạn có thể nhận được một kết quả thiết yếu. Ví dụ, trong hình 3.1 Mô hình Học máy, chúng ta sử dụng một thuật toán dự đoán và tạo ra một mô hình bằng cách cung cấp cho nó một số dữ liệu, và sau đó chúng ta có thể nhận được các dự đoán dựa trên dữ liệu thực tế làm đầu vào.

Trong bất kỳ lĩnh vực nào, người dùng công nghệ luôn được hưởng lợi từ Học máy. Trong trí tuệ nhân tạo, vì Học máy luôn là lĩnh vực liên tục phát triển, và thêm vào đó, luôn có những cân nhắc phải ghi nhớ khi học các thuật toán học máy và các phương pháp luận của chúng hoặc phân tích của chúng trong các lĩnh vực khác nhau.

HỌC LẶP TỪ DỮ LIỆU (ITERATIVE LEARNING FROM DATA): Vì các mô hình học máy luôn được huấn luyện từ các bộ dữ liệu khác nhau trước khi được triển khai để sử dụng. Một số mô hình trực tuyến (online) và có thể thích ứng liên tục với dữ liệu mới. Mặt khác, một số mô hình học máy ngoại tuyến (offline) được huấn luyện và một khi được triển khai ở đâu đó thì không thể thay đổi và chúng luôn đưa ra kết quả dựa trên (dữ liệu huấn luyện) đó. Các mô hình trực tuyến trong học máy thực hiện học lặp (iterative learning) vì chúng liên tục nhận được dữ liệu mới, trong đó các thuật toán tạo ra các mẫu (patterns) và liên kết nó với các yếu tố dữ liệu có trong dữ liệu. Sau khi một mô hình

được huấn luyện, nó có thể được sử dụng trong thời gian thực để học từ dữ liệu và cho chúng ta những kết quả ý nghĩa.



Hình 3.1 Mô hình Học máy

II. Phương pháp tiếp cận Học máy

Khi chúng ta nói về một số loại vấn đề được giải quyết bằng học máy, chúng liên quan đến các loại thuật toán khác nhau. Tương tự như vậy, chúng ta có các kỹ thuật khác nhau trong học máy có thể được sử dụng trong các tình huống khác nhau. Đối với các phát biểu vấn đề khác nhau, chúng ta sử dụng các kỹ thuật học máy khác nhau, có thể dựa trên loại và khối lượng dữ liệu. Học máy có các danh mục khác nhau được thảo luận ở đây:

1. Học có giám sát (Supervised Learning)

Đây là (phương pháp) Học máy được sử dụng phổ biến và rộng rãi nhất. Việc xây dựng mô hình luôn phụ thuộc vào dữ liệu có sẵn và loại đầu ra mà chúng ta yêu cầu. Trong học có giám sát, các mô hình được cung cấp dữ liệu đã được phân loại và gán nhãn sẵn cho mục đích huấn luyện hoặc để học , và sau đó bằng cách tính toán mô hình với dữ liệu thực tế khác, các kết quả được so sánh với các kết quả đã được huấn luyện giả định hoặc mong đợi. Nếu tìm thấy sự không khớp hoặc lỗi nhất định, mô hình sẽ được huấn luyện lại với nhiều dữ liệu hơn. Vì học có giám sát luôn được sử dụng để dự đoán dựa trên các mẫu được tìm thấy trong dữ liệu hoặc trên dữ liệu chưa được gán nhãn được cung cấp cho mô hình. Nói cách khác, học có giám sát có thể được cho là học có hướng dẫn hoặc học có "thầy", nơi chúng ta có một bộ dữ liệu đã được gán nhãn sẵn, hoạt động như một giáo viên để mô hình Học máy học hỏi. Nếu mô hình được huấn luyện, nó sẽ bắt đầu dự đoán hoặc xác định nếu dữ liệu mới được đưa vào.

Các chi tiết có thể được hiểu là: Giả sử chúng ta có các biến đầu vào, x và biến đầu ra, y , để tìm hiểu cách hàm ánh xạ (mapping function) có thể được học từ đầu vào đến đầu ra, chẳng hạn như: $Y = f(x)$

Bây giờ, mục tiêu chính là xấp xỉ hàm ánh xạ sao cho biến đầu ra (Y) có thể được dự đoán cho dữ liệu đó nếu chúng ta có đầu vào mới (x).

Hai loại vấn đề có thể được phân loại chủ yếu vào Học có giám sát:

- **Phân loại (Classification):** Một truy vấn phân loại được đặt tên khi chúng ta có hiệu suất được phân loại (categorized performance) như "đen", "dạy học", "không dạy học", v.v.
- **Hồi quy (Regression):** Một vấn đề hồi quy được đặt tên nếu có giá trị đầu ra thực, như "chiều rộng", "kilogram", v.v.

Các ví dụ về thuật toán học máy có giám sát là cây quyết định (decision tree), rừng ngẫu nhiên (random forest), knn, hồi quy logistic (logistic regression).

Một ứng dụng phổ biến của học có giám sát là sử dụng dữ liệu lịch sử để dự đoán thống kê các sự kiện trong tương lai. Bạn có thể dự báo các biến động sắp tới bằng cách sử dụng kiến thức thị trường chứng khoán lịch sử hoặc sử dụng (bộ lọc) thư rác để lọc (thư). Hình ảnh cây trồng hoặc cây lương thực được gắn thẻ (tagged) trong nông nghiệp có thể được sử dụng làm dữ liệu đầu vào trong huấn luyện có giám sát để phân loại các hình ảnh cây trồng và cây lương thực chưa được gắn thẻ.

2. Học không giám sát (Unsupervised Learning)

Học không giám sát là tốt nhất nếu câu hỏi liên quan đến một lượng lớn dữ liệu không được gán nhãn. Việc biết ý nghĩa đằng sau dữ liệu này bao gồm các thuật toán có thể được hiểu bằng cách phân loại dữ liệu trên cơ sở các mẫu hoặc cụm (clusters) được tìm thấy. Do đó, học không giám sát thực hiện một quy trình xử lý dữ liệu lặp đi lặp lại mà không có sự can thiệp của con người.

Các thuật toán trong học không giám sát phân đoạn dữ liệu thành các nhóm (cụm) nổi bật hoặc các nhóm đặc trưng. Dữ liệu không được gán nhãn xác định các giá trị của tham số và việc phân loại dữ liệu. Phương pháp này về cơ bản áp dụng các nhãn cho thông tin để nó trở thành có giám sát và mô hình được huấn luyện. Học không giám sát sẽ (hữu ích) khi có một lượng lớn dữ liệu. Nhà phát triển không biết bối cảnh của dữ liệu để phân tích trong trường hợp này, vì vậy tại thời điểm này, việc dán nhãn là không thể. Đó là lý do tại sao học không giám sát do đó có thể được sử dụng như bước đầu tiên trước khi dữ liệu được chuyển sang một quy trình học có kiểm soát (supervised).

Các chi tiết có thể được hiểu là: Giả sử chúng ta có một biến đầu vào x , và không có biến đầu ra nào có sẵn như trong các thuật toán học có giám sát. Nói một cách đơn giản, chúng ta có thể kết luận rằng không có câu trả lời đúng và không có người hướng dẫn

trong học không giám sát. Các thuật toán cho phép người dùng xác định các xu hướng dữ liệu hấp dẫn.

Hai dạng vấn đề sau đây có thể được chia thành các vấn đề học không giám sát:

- **Phân cụm (Clustering):** Chúng ta phải khám phá các nhóm (groupings) cơ bản của dữ liệu trong các vấn đề phân cụm. Ví dụ, phân nhóm người tiêu dùng theo hành vi mua sắm.
- **Kết hợp (Association):** Một vấn đề kết hợp được đặt tên như vậy vì những vấn đề này yêu cầu các quy tắc phải được khám phá để giải thích nhiều dữ liệu của chúng ta, ví dụ, để tìm những khách hàng mua cả x và y .

Các thuật toán học không giám sát có thể giúp hiểu được một lượng lớn nội dung mới, chưa được gán nhãn. Các thuật toán này tìm kiếm các mẫu dữ liệu tương tự như (học) có giám sát. khác biệt là dữ liệu không được biết trước. Ví dụ, Các thuật toán học máy Không giám sát là K-means để phân cụm, thuật toán Apriori để kết hợp.

Trong lĩnh vực nông nghiệp, việc thu thập một lượng lớn dữ liệu về một lĩnh vực cụ thể, ví dụ, sẽ giúp nông dân hiểu rõ hơn về các xu hướng và liên kết chúng với việc thu hoạch cây trồng và tăng năng suất của nó. Tất cả các điểm dữ liệu liên quan đến một tình trạng như sức khỏe của cây trồng sẽ mất quá nhiều thời gian để phân loại. Chiến lược học không giám sát cũng sẽ giúp đánh giá kết quả nhanh hơn so với phương pháp học có giám sát.

3. Học tăng cường (Reinforcement Learning)

Học tăng cường là một mô hình học tập hành vi. Phản hồi từ phân tích dữ liệu được nhận để hướng dẫn người dùng đạt được kết quả tốt nhất. Học tăng cường (Increased learning - *Lưu ý: từ đúng là "Reinforcement learning"*) khác với các phương pháp khác vì hệ thống không sử dụng một bộ dữ liệu mẫu để huấn luyện. Thay vào đó, hệ thống học bằng cách thử nghiệm và sai sót (testing and error). Do đó, một loạt các lựa chọn thành công sẽ "củng cố" (reinforce) quy trình vì vấn đề được giải quyết tốt nhất.

Hình ảnh hệ thống Tác nhân - Figure Agent system.

Trong Hình, phản ánh kịch bản chung của việc học tăng cường. So sánh, tác nhân (agent) không nhận được dữ liệu được chỉ định (designated) ở đây, không giống như kịch bản học có giám sát được xem xét trong phần trước. Thay vào đó, nó thu thập thông tin bằng cách giao tiếp với thế giới, thông qua một loạt các hành động. Tác nhân nhận được hai loại thông tin để phản hồi một sự kiện: (1) trạng thái của nó (state of play) trong thế giới và (2) các kịch bản thực tế (real-life scenarios), điều này là duy nhất cho nhiệm vụ và mục đích tương ứng của nó. Mục tiêu là để Tác nhân tối đa hóa phần thưởng (reward) của mình và, để đạt được mục tiêu đó, đồng ý về hướng hành động hoặc chính sách (policy) tốt nhất. Tuy nhiên, dữ liệu mà nó thu thập từ hệ thống chỉ là phần thưởng ngay lập tức cho hành động mà nó đã thực hiện. Không có phản hồi nào từ hệ thống về các ưu đãi

(incentives) trong tương lai hoặc dài hạn. Một yếu tố quan trọng của việc (học) tăng cường là tính đến các khoản thưởng hoặc phạt bị trì hoãn (late bonuses or penalties).

Tác nhân phải đối mặt với tình thế tiến thoái lưỡng nan giữa việc khám phá (exploring) các trạng thái và hành động không xác định để thu thập thêm thông tin về Hệ thống và các phần thưởng, và việc sử dụng (using - *Lưu ý: từ đúng là "exploiting" - khai thác*) thông tin đã tích lũy được để tối đa hóa sự đền bù (reparation - *Lưu ý: từ đúng là "reward" - phần thưởng*) của mình. Đây được gọi là sự đánh đổi giữa thăm dò và phát triển (exploitation - *Lưu ý: tài liệu gốc ghi "development"*) vốn có trong học Tăng cường.

Hiểu rằng có nhiều biến thể (variations) giữa kịch bản học tăng cường và học có giám sát trong một số chương trước. So sánh với học có giám sát, không có sự phân phối cố định của các thực thể trong học tăng cường, vốn xác định sự phân phối qua các quan sát. Đó là sự lựa chọn của chính sách. Trên thực tế, những thay đổi nhỏ về chính sách có thể ảnh hưởng mạnh mẽ đến phần thưởng. Ngoài ra, Hệ thống không thể được thiết lập nói chung và có thể khác nhau do hành vi được chọn bởi tác nhân. Điều này có thể thực tế hơn so với học có giám sát thông thường đối với một số vấn đề học tập. Có hai cài đặt chính: một trong đó mô hình môi trường (ambient model) được tác nhân biết đến, điều này làm giảm mục tiêu tối ưu hóa số tiền xuống thành một vấn đề lập kế hoạch (planning problem); và một trong đó mô hình môi trường không xác định và nơi tác nhân phải đối mặt với một câu hỏi học tập. Trong trường hợp trên, tác nhân phải học hỏi từ dữ liệu trạng thái và phần thưởng (recompense) thu được để có được thông tin liên quan và quyết định chiến lược tốt nhất.

Học tăng cường không phải là Học có giám sát một cách cụ thể vì không hoàn toàn dựa vào việc thu thập dữ liệu "có giám sát" (hoặc được gán nhãn) (bộ sưu tập huấn luyện). Trên thực tế, phản hồi của các hành động được thực hiện có thể được kiểm soát và tính toán dựa trên một khái niệm về "phần thưởng". Tuy nhiên, nó cũng không phải là học không giám sát vì chúng ta biết trước phần thưởng mong muốn khi chúng ta hình thành "người học" của mình.

III. Học có giám sát Vs. Không giám sát Vs. Tăng cường

Cuối cùng, vì tất cả đều đã nhận thức rõ về học có giám sát, không giám sát và Tăng cường, hãy nhìn vào khoảng cách giữa các loại hình học này. Tóm lại, học có giám sát xảy ra khi một mô hình học có hướng dẫn từ một bộ dữ liệu được xác định (defined). Và học không giám sát là nơi máy tính được huấn luyện mà không có bất kỳ hướng dẫn nào trên cơ sở dữ liệu không được gán nhãn. Học tăng cường xảy ra khi một máy tính hoặc một tác nhân giao tiếp, thực hiện các hành động và học hỏi bằng quy trình thử-và-sai (test-and-error).

Bảng sau đây:

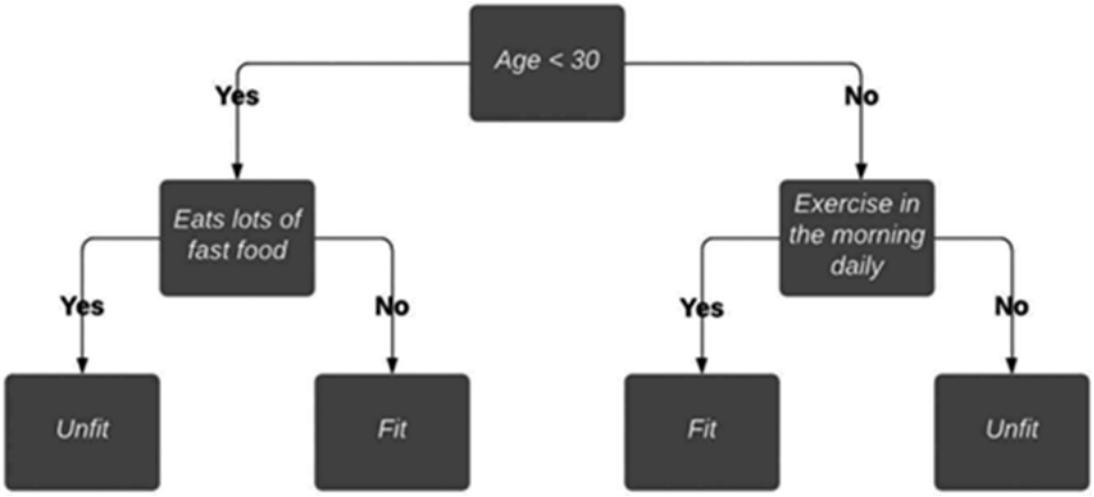
Tiêu chí (Criteria)	Học có giám sát (Supervised Learning)	Học không giám sát (Unsupervised learning)	Học tăng cường (Reinforcement Learning)
Định nghĩa	Mô hình học bằng cách sử dụng dữ liệu được gán nhãn hoặc được phân loại.	Mô hình được huấn luyện với dữ liệu không được phân loại hoặc không được gán nhãn.	Một tác nhân tương tác với mô hình bằng cách thực hiện hành động & học hỏi từ các lỗi và phần thưởng.
Loại vấn đề	Hồi quy & Phân loại.	Kết hợp & Phân cụm.	Dựa trên phần thưởng.
Loại dữ liệu	Dữ liệu được gán nhãn.	Dữ liệu không được gán nhãn.	Không có dữ liệu được xác định trước.
Huấn luyện	Giám sát bên ngoài.	Không có sự giám sát.	Không có sự giám sát.
Phương pháp tiếp cận	Ánh xạ các đầu vào được gán nhãn hoặc được phân loại tới các đầu ra đã biết.	Nhận dạng mẫu trong dữ liệu không được phân loại và khám phá ra đầu ra.	Nó tuân theo phương pháp thử và sai (trail and error).

IV. Các thuật toán học máy phổ biến nhất

- **Hồi quy tuyến tính (Linear Regression):** Đây là một trong những thuật toán thống kê và học máy phổ biến nhất. Hồi quy tuyến tính về cơ bản là một mô hình tuyến tính (linearity) giả định mối quan hệ tuyến tính giữa các biến đầu vào x và y . Nói cách khác, bạn có thể tính toán các biến đầu vào x từ một tổ hợp tuyến tính. Nó có thể được định nghĩa bằng một đường thẳng phù hợp nhất (best line) để tạo mối quan hệ giữa các biến. Hồi quy tuyến tính thường có hai loại:
    - **Hồi quy tuyến tính đơn giản (Simple Linear Regression):** Một thuật toán hồi quy tuyến tính được coi là hồi quy tuyến tính đơn giản vì chỉ có một biến độc lập.
    - **Hồi quy tuyến tính bội (Multiple Linear Regression):** Nếu có nhiều hơn một biến độc lập, một thuật toán hồi quy tuyến tính được coi là hồi quy tuyến tính bội. Hồi quy tuyến tính chủ yếu được sử dụng để ước tính các giá trị thực dựa trên (các) biến liên tục. Ví dụ, doanh số bán hàng cả ngày của một cửa hàng có thể được tính toán bằng hồi quy tuyến tính dựa trên các giá trị thực.
- **Hồi quy Logistic (Logistic Regression):** Nó còn được gọi là hồi quy logit (logit regression). Đây là một thuật toán phân loại. Hồi quy Logistic chủ yếu là một

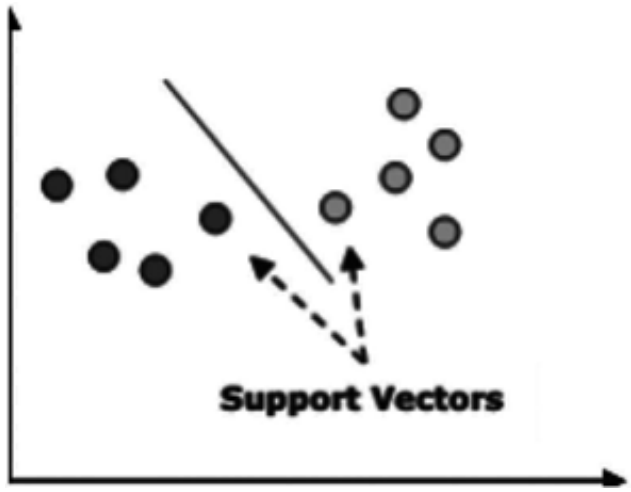
thuật toán phân loại được sử dụng để ước tính các giá trị rời rạc như 0 hoặc 1, đúng hoặc sai hoặc có hoặc không dựa trên một tập hợp riêng biệt các biến. Về lý thuyết, nó dự đoán rằng đầu ra của nó sẽ nằm trong khoảng từ 0 đến 1.

- **Cây quyết định (Decision Tree):** Đây là một thuật toán học có giám sát chủ yếu được sử dụng để phân loại vấn đề. Về cơ bản, đây là một phân vùng được phân loại (graded partition) dựa trên các biến độc lập như một phần đệ quy (recursive partitive). Cây quyết định có các nút cây có gốc (rooted tree nodes). Cây có gốc là một cây có hướng với một nút gốc. Gốc không có cạnh vào và tất cả các nút khác đều có một cạnh vào. Những nút (cuối cùng) như vậy được gọi là lá hoặc nút phán đoán (judgment).



Hình 3.2: Cây quyết định

- **Máy Vector hỗ trợ (Support Vector Machine - SVM):** Nó được sử dụng cho các bài toán phân loại và hồi quy. Tuy nhiên, nó chủ yếu được sử dụng cho các bài toán phân loại. Nguyên tắc chính của SVM là gán mỗi yếu tố dữ liệu trong không gian n-chiều làm giá trị của đặc trưng riêng lẻ. Ở đây n là các đặc trưng (features) chúng ta có. Sau đó, một biểu diễn đồ họa đơn giản để nắm bắt định nghĩa SVM từ Hình 3.3 SVM.



Hình 3.3: SVM

  - Các Vector hỗ trợ (Support Vectors) Trong Hình 3.3 SVM ở trên, chúng ta có hai đặc trưng, do đó, trước tiên chúng ta cần ánh xạ hai biến như vậy trong không gian hai chiều, nơi mỗi điểm có hai tọa độ, được gọi là các vector hỗ trợ. Đường thẳng chia dữ liệu thành hai loại được phân loại riêng biệt. Đường thẳng này là bộ phân loại (classifier) trong SVM.

- **Naïve Bayes:** Đây cũng là một kỹ thuật phân loại. Định lý Bayes là một nguyên tắc để xây dựng các bộ phân loại đằng sau kỹ thuật phân loại này. Các yếu tố dự đoán (predictors) được cho là độc lập. Nói một cách đơn giản, sự bao gồm (inclusion) của một đặc trưng cụ thể trong một lớp không liên quan đến đặc trưng của bất kỳ đặc trưng nào khác. Sau đây là phương trình của định lý Bayes:

$$P(A \mid B) = P(B \mid A)P(A)/P(B)$$

  Mô hình Naïve Bayes rất đơn giản để phát triển và đặc biệt hữu ích cho các tập dữ liệu lớn.

- **K-Nearest Neighbors (KNN) (K-Láng giềng gần nhất):** Nó được sử dụng để xác định các vấn đề và khắc phục chúng (identify problems and to rectify them - *Lưu ý: cách diễn đạt này không chuẩn, KNN dùng cho phân loại/hồi quy*). Nó thường được sử dụng để giải quyết các vấn đề với phân loại. Nguyên tắc chính của thuật toán này là nó lưu trữ tất cả các trường hợp (cases) hiện có và bằng cách bỏ phiếu đa số (plurality voting) của các láng giềng của nó để phân loại các trường hợp mới. Trường hợp này sau đó được gán cho lớp mà, theo một hàm khoảng cách, là phổ biến nhất trong số K láng giềng gần nhất của nó. Khoảng cách (The gap) có thể là Minkowski, Euclidean, và Hamming. Xem xét KNN như sau:
  - Về mặt tính toán, KNN đắt đỏ hơn các thuật toán khác được sử dụng cho các bài toán phân loại.
  - Việc chuẩn hóa các biến là cần thiết, nếu không các biến có phạm vi (range) cao hơn có thể làm sai lệch nó.
  - Tại KNN, chúng ta sẽ làm việc trên một giai đoạn như giảm nhiễu (noise reduction), tiền xử lý (pre-processing).

- **K-Means Clustering (Phân cụm K-Means):** Điều này được sử dụng để giải quyết vấn đề của các cụm (clusters), như tên gọi của nó. Đây là một loại Học không giám sát. Nguyên tắc chính của thuật toán K-Means là gán (phân chia) tập dữ liệu qua các cụm khác nhau. Hãy làm theo các bước sau để xây dựng các cụm K-means:
  - K-means chọn k số điểm cho mỗi cụm được gọi là tâm cụm (centroids).
  - Mỗi điểm dữ liệu bây giờ tạo thành một cụm, tức là k cụm, với các tâm cụm gần nhất.
  - Bây giờ, các tâm, dựa trên các thành viên hiện tại của mỗi cụm, sẽ được xác định (tính toán lại).
  - Các biện pháp (bước) này cần phải được lặp lại cho đến khi sự hội tụ (convergence) diễn ra.

- **Rừng ngẫu nhiên (Random Forest):** Đây là một thuật toán phân loại có giám sát. Thuật toán rừng ngẫu nhiên có ưu điểm là nó có thể được sử dụng cho các bài toán phân loại cũng như hồi quy. Về cơ bản, đây là tập hợp các cây ra quyết định (khu rừng) hoặc tập hợp (ensemble) các cây ra quyết định. Nguyên tắc cơ bản của rừng ngẫu nhiên là mỗi cây được phân loại (graded) và khu rừng chọn ra các phân loại (grades) tốt nhất. Những ưu điểm của thuật toán Random Forest như sau:
    - Bộ phân loại rừng ngẫu nhiên có thể được sử dụng cho các chức năng phân loại và hồi quy.
    - Các giá trị bị thiếu (missing values) có thể được quản lý.
    - Ngay cả khi chúng ta có nhiều cây hơn trong rừng, điều này sẽ không làm mô hình bị quá khớp (over suit the pattern).

V. Ứng dụng (Applications)

- **Các nhà bán lẻ** Học máy trong nông nghiệp được các nhà bán lẻ hạt giống sử dụng để biến dữ liệu thành sản lượng cây trồng tốt hơn. Thông qua việc các công ty kiểm soát dịch hại sử dụng chúng, vi khuẩn, bọ và sâu bọ (vermin) đang phát triển có thể được phát hiện.
- **Robot nông nghiệp (Agriculture Bots)** Phần lớn các công ty hiện đang chuẩn bị và chế tạo robot cho sứ mệnh nông nghiệp chính. Điều này bao gồm việc xử lý hạt giống và làm việc hiệu quả hơn nhân lực. Đây là ví dụ mới nhất về học máy trong nông nghiệp.
- **Lai tạo giống (Species Breeding)** Chọn lọc giống là một phương pháp tìm kiếm lặp đi lặp lại các gen cụ thể, quyết định hiệu quả sử dụng nước và chất dinh dưỡng, khả năng chịu đựng khí hậu, kháng bệnh, hàm lượng dinh dưỡng hoặc hương vị. Trong việc phân tích sản xuất cây trồng ở các vùng khí hậu khác nhau và phát triển các công nghệ mới, học máy, đặc biệt là các thuật toán học sâu (profound learning), đòi hỏi hàng thập kỷ dữ liệu thực địa. Dựa trên dữ liệu này, một mô hình thống kê có thể được phát triển để dự đoán gen nào có khả năng đóng góp nhất vào lợi ích (gain) của cây trồng.
- **Nhận dạng loài (Species Recognition)** Phương pháp tiếp cận thông thường của con người để phân loại thực vật là so sánh màu sắc và hình dạng của lá, nhưng bằng cách sử dụng phân tích thống kê của Học máy (Machine Leaning's) có thể cung cấp kết quả nhanh hơn và chi tiết hơn bằng cách phân tích hình thái (morphology) của lá và cung cấp thêm chi tiết về các đặc tính của lá.
- **Sản lượng (Yield Production)** Dự báo năng suất là một trong những chủ đề của nông nghiệp chính xác cao (steep-precision agriculture), vì nó xác định việc lập bản đồ và dự đoán năng suất, khớp nối nhu cầu cây trồng và quản lý cây trồng . các kỹ thuật hiện đại (state-of the-art) đã vượt xa dự đoán đơn giản dựa trên dữ liệu lịch sử, mà đã bao gồm các kỹ thuật học máy để cung cấp kiến thức liên quan

đến cây trồng, thời tiết và hoàn cảnh kinh tế và phân tích đa chiều sâu rộng để tối ưu hóa lợi nhuận cho nông dân và cộng đồng.

- **Phát hiện bệnh (Disease Detection)** Học máy có thể được sử dụng để phát hiện các bệnh lây nhiễm ở Thực vật, thường gây ra bởi dịch hạch (plague), côn trùng, bệnh tật và, trừ khi được quản lý kịp thời, chúng làm giảm năng suất trên quy mô lớn. Liên quan đến diện tích trồng trọt tính bằng mẫu Anh (acres), người trồng trọt rất tẻ nhạt (tedious) để theo dõi cây trồng hàng ngày. Bằng cách triển khai Học máy ở đây, nó cung cấp giải pháp để theo dõi khu vực canh tác một cách nhất quán và cung cấp khả năng phát hiện bệnh tự động bằng hình ảnh viễn thám.
- **Chất lượng cây trồng (Crop Quality)** Sản lượng dữ liệu (Data output) có thể được đo lường và xác định đầy đủ để nâng giá sản phẩm và giảm lãng phí. Trái ngược với các chuyên gia con người, máy móc có thể sử dụng dữ liệu và các giao diện (interfaces) dường như không liên quan để khám phá và xác định các phẩm chất mới đóng vai trò trong chất lượng tổng thể của cây trồng.
- **Phát hiện cỏ dại (Weed Detection)** Cỏ dại là mối đe dọa lớn đối với sản xuất cây trồng, ngoài các loại bệnh. Vấn đề chính trong cuộc chiến chống lại cỏ dại là chúng khó phân biệt và nhận diện (discriminate) với cây trồng. Các thuật toán thị giác máy tính (Computer vision) và ML có thể tăng cường khả năng nhận diện và phân biệt cỏ dại với chi phí thấp mà không có các vấn đề môi trường hoặc tác dụng phụ. Các công nghệ này sẽ thúc đẩy robot tiêu diệt cỏ dại và giảm nhu cầu sử dụng thuốc diệt cỏ trong tương lai.
- **Quản lý đất (Soil Management)** Đất là một nguồn tài nguyên thiên nhiên rất đa dạng với các quy trình phức tạp và các cơ chế không rõ ràng đối với các chuyên gia nông nghiệp. Nhiệt độ của chính nó có thể cung cấp cái nhìn sâu sắc về tác động của biến đổi khí hậu đối với lợi nhuận lãnh thổ (territorial returns). Các thuật toán học máy nghiên cứu quá trình bốc hơi, độ ẩm của đất và nhiệt độ để hiểu động lực của hệ sinh thái và các tác động của nông nghiệp.
- **Quản lý nước (Water Management)** Sự cân bằng thủy văn (hydrological), khí hậu (climatological) và nông nghiệp của các nguồn tài nguyên nước trong nông nghiệp có một ý nghĩa. Sử dụng các hệ thống tưới tiêu hiệu quả hơn và dự báo nhiệt độ điểm sương (dewpoint) hàng ngày, các ứng dụng ML tiên tiến nhất cho đến nay được kết nối với ước tính khí khổng (stomata) hàng ngày, hàng tuần hoặc hàng tháng, giúp phân loại thời tiết dự đoán và ước tính sự bốc hơi.
- **Chăn nuôi (Livestock Production)** Học máy cung cấp dự đoán và tính toán chính xác các thông số canh tác, phù hợp với quản lý cây trồng, để tối đa hóa hiệu quả kinh tế của các hệ thống canh tác như chăn nuôi. Để cho phép nông dân thay đổi chế độ ăn và điều kiện, ví dụ, hệ thống dự báo cân nặng sẽ ước tính trọng lượng tương lai 150 ngày trước ngày họ giết mổ.

Tóm tắt

Học máy đã được sử dụng rộng rãi trong một số lĩnh vực và nó đã trở thành một nhánh quan trọng nhất của Trí tuệ Nhân tạo. Như chúng ta đã thấy, có thể có rất nhiều ứng dụng

của Học máy trong lĩnh vực nông nghiệp, điều này có thể giúp nông dân tăng năng suất cây trồng và cứu chúng khỏi sâu bệnh và các loại côn trùng khác khi thu hoạch trên đồng. Vì vậy, chúng ta có thể thấy học máy có thể hữu ích như thế nào trong lĩnh vực nông nghiệp.

CHƯƠNG 4: HỌC SÂU

Học sâu nổi lên như một đối thủ nặng ký trong lĩnh vực này sau 10 năm phát triển bùng nổ của công nghệ tính toán. Do đó, học sâu là một dạng đặc biệt của học máy, với các thuật toán dựa trên cấu trúc và chức năng của bộ não con người.

Học Máy (Machine Learning) VS Học Sâu (Deep Learning)

Học sâu được cho là dạng hiệu quả nhất của học máy. Điều này rất quan trọng vì chúng biết cách tốt nhất để giải quyết vấn đề và cách khắc phục mọi thứ. Sau đây là mô tả về học sâu và học máy:

- **Phụ thuộc vào dữ liệu:** Điểm khác biệt đầu tiên dựa trên đầu ra của DL (Học sâu) và ML (Học máy) khi quy mô dữ liệu tăng lên. Các thuật toán học sâu hoạt động rất tốt khi dữ liệu lớn.
- **Phụ thuộc vào máy móc:** Máy móc cấu hình cao cho phép các thuật toán học sâu hoạt động một cách hoàn hảo. Mặt khác, các thuật toán học máy cũng có thể hoạt động trên các máy tính cấu hình thấp.
- **(Trích xuất đặc trưng):** Các thuật toán học sâu có thể trích xuất các đặc trưng ở cấp độ cao và tìm cách học hỏi từ chúng. Mặt khác, phần lớn các đặc trưng bắt nguồn từ học máy được xác định bởi một chuyên gia.
- **Thời gian thực thi:** Thời gian chạy phụ thuộc vào các tham số thuật toán khác nhau. Học sâu có nhiều tham số hơn các thuật toán học máy. Do đó, thời gian chạy của thuật toán DL nhiều hơn thuật toán ML, đặc biệt là thời gian huấn luyện. Tuy nhiên, thời gian xử lý (dự đoán) của thuật toán DL lại ít hơn ML.
- **Cách tiếp cận giải quyết vấn đề:** Học sâu giải quyết vấnê một cách toàn diện (end-to-end) trong khi học máy được sử dụng để giải quyết vấn đề theo cách truyền thống, tức là chia nhỏ vấn đề thành các phần.

Các thuật ngữ cơ bản trong Học Sâu

- **Neuron:** Giống như neuron hình thành đặc điểm cơ bản của bộ não chúng ta, một neuron cũng hình thành cấu trúc cơ bản của một mạng nơ-ron. Hãy nghĩ về những gì chúng ta làm khi tiếp nhận thông tin mới. Chúng ta xử lý nó và sau đó tạo ra một đầu ra. Tương tự, trong một mạng nơ-ron (như trong Hình 4.1), đầu vào của một neuron được tiếp nhận, xử lý và tạo ra một đầu ra được gửi đến các neuron khác để xử lý tiếp hoặc trở thành đầu ra cuối cùng.

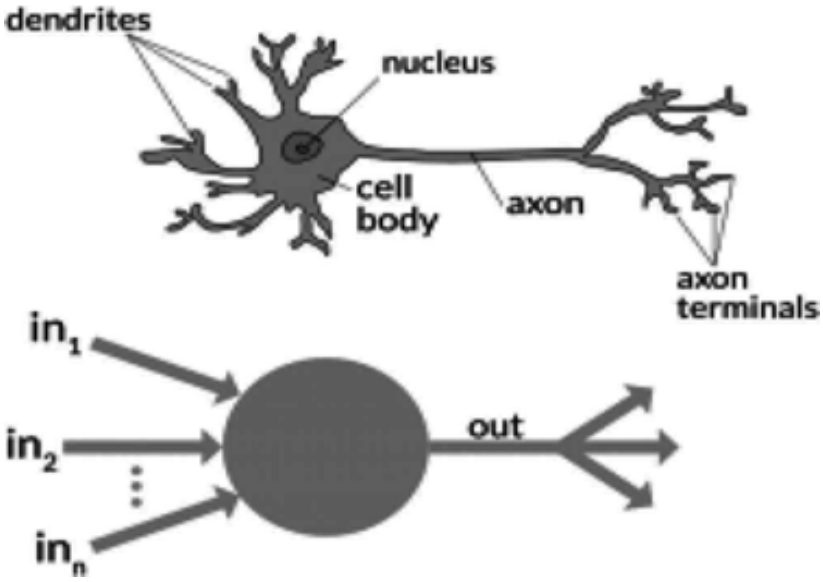


Fig. 4.1 Structure of Neuron

- **Trọng số (Weights):** Đầu vào được nhân với một trọng số khi được áp dụng cho neuron. Ví dụ, nếu một neuron có hai đầu vào, trọng số sẽ được gán cho mỗi đầu vào. Chúng ta khởi tạo trọng số một cách ngẫu nhiên và thay đổi các trọng số này trong quá trình huấn luyện mô hình. Sau khi huấn luyện, mạng nơ-ron sẽ gán trọng số lớn hơn cho đầu vào mà nó thấy quan trọng hơn. Trọng số bằng không có nghĩa là yếu tố cụ thể đó không quan trọng.
- **Độ lệch (Bias):** Ngoài các trọng số, đầu vào còn được áp dụng thêm một thành phần tuyến tính khác, gọi là độ lệch (bias). Kết quả sẽ có dạng $a * W_1 + bias$ sau khi thêm độ lệch này. Đây là thành phần tuyến tính cuối cùng của phép biến đổi dữ liệu.
- **Hàm kích hoạt (Activation Function):** Một hàm phi tuyến tính được giới thiệu sau khi thành phần tuyến tính được tính toán. Hàm kích hoạt chuyển đổi tín hiệu đầu vào thành tín hiệu đầu ra. Hàm này được áp dụng cho mối quan hệ tuyến tính. Kết quả sau khi kích hoạt sẽ có dạng $f(a * W_1 + b)$, trong đó f là hàm kích hoạt. Trong sơ đồ (Hình 4.2), các đầu vào là X_1 đến X_n và các trọng số tương ứng là Wk_1 đến Wk_n . Chúng ta có một độ lệch (bias) là bk . Đầu tiên, các trọng số được nhân với đầu vào tương ứng rồi cộng lại với nhau. Chúng ta gọi đây là u .

$$u = \sum(w * x) + b$$

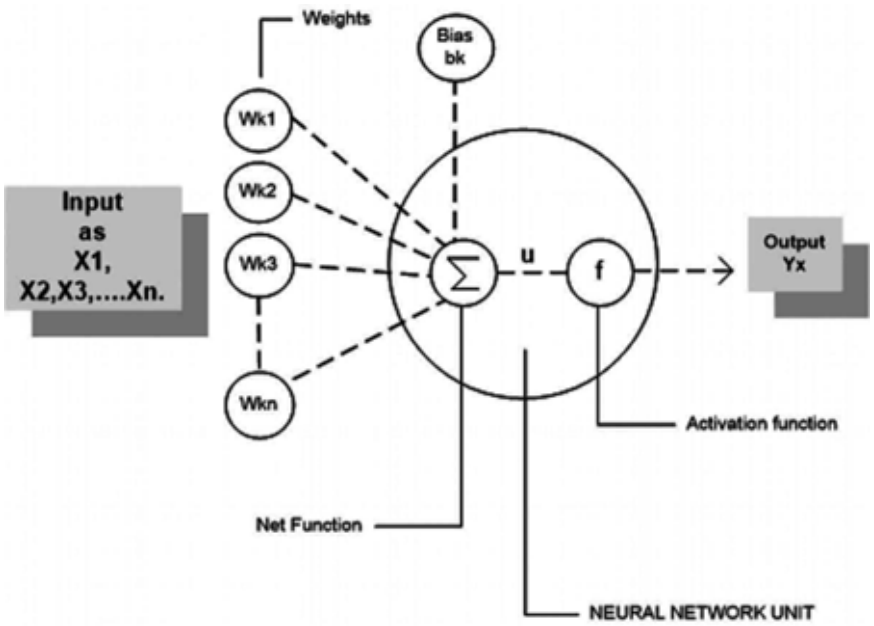


Fig. 4.2 Activation Function

Hàm kích hoạt được áp dụng cho u , tức là $f(u)$, và chúng ta nhận được đầu ra cuối cùng của nơ-ron là:

$$yk = f(u)$$

- **Các hàm kích hoạt** được sử dụng phổ biến nhất là sigmoid, ReLU và SoftMax.
- **Lớp đầu vào/Lớp đầu ra/Lớp ẩn (Input/Output/Hidden layer):** Đúng như tên gọi, lớp đầu vào là lớp đầu tiên của mạng nhận đầu vào. Lớp đầu ra là lớp xuất kết quả hoặc là lớp cuối cùng của mạng. Các lớp truyền tải của mạng là các lớp ẩn. Các lớp ẩn này thực hiện các nhiệm vụ khác nhau trên dữ liệu đầu vào và chuyển đầu ra đến lớp tiếp theo. Các lớp đầu vào và đầu ra có thể được nhìn thấy trong khi các lớp trung gian bị "che khuất" (Hình 4.3).

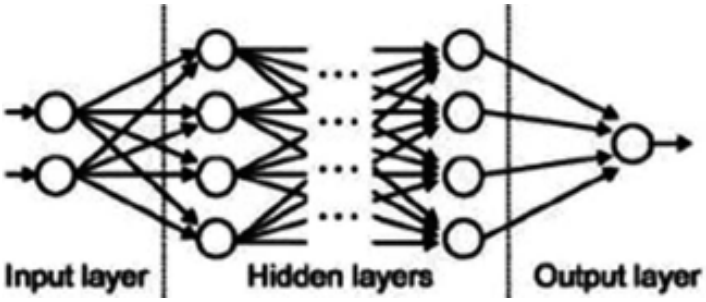


Fig. 4.3 Representation of layers

- **Perceptron đa lớp (Multilayer Perceptron):** Một nơ-ron đơn lẻ không thể hoàn thành các nhiệm vụ phức tạp. Do đó, chúng ta tạo ra các đầu ra cần thiết bằng các "ngăn xếp" nơ-ron. Trong mạng đơn giản nhất, chúng ta có một lớp đầu vào, một lớp ẩn và một lớp đầu ra. Mỗi lớp có nhiều nơ-ron, và tất cả các nơ-ron trong mỗi lớp được kết nối với các nơ-ron của lớp tiếp theo. Các mạng này cũng có thể được gọi là kết nối đầy đủ (fully connected).

- **Hàm chi phí (Cost Function):** Khi chúng ta tạo một mạng, mạng cố gắng ước tính xem hiệu suất của nó gần với giá trị thực tế đến mức nào. Chúng ta sử dụng hàm chi phí/mất mát (cost/loss function) để tính toán độ chính xác này. Nếu mạng mắc lỗi, hàm chi phí sẽ "phạt" nó. Mục tiêu của chúng ta là nâng cao khả năng dự đoán và giảm lỗi, do đó giảm chi phí. Hàm chi phí hoặc mất mát tối thiểu tương ứng với hiệu suất hiệu quả nhất. Nếu hàm chi phí được định nghĩa là lỗi toàn phương trung bình (mean squared error), nó có thể được viết là:

$$C = (1/m) * \sum(y - a)^2$$

  trong đó m là số lượng đầu vào huấn luyện, a là giá trị dự đoán và y là giá trị thực tế. Chu trình học tập chính là việc giảm chi phí.
- **Giảm theo Gradient (Gradient Descent):** Đây là một thuật toán tối ưu hóa để giảm chi phí. Để hình dung, hãy tưởng tượng bạn đi xuống một ngọn đồi, bạn sẽ đi xuống dần từng bước thay vì chạy thẳng xuống. Để tìm cực tiểu cục bộ của một hàm, chúng ta thực hiện các bước tỷ lệ thuận với giá trị âm của gradient của hàm, coi điểm dưới cùng là điểm chi phí tối thiểu.
- **Tốc độ học (Learning Rate):** Trong mỗi lần lặp, tốc độ học được định nghĩa là mức độ giảm thiểu chi phí. Tốc độ mà chúng ta đi xuống điểm tối thiểu của hàm chi phí đơn giản là tốc độ học. Chúng ta nên cẩn thận lựa chọn tốc độ học, vì nếu quá cao, giải pháp tối ưu có thể bị bỏ qua, hoặc nếu quá thấp, mạng có thể mất rất nhiều thời gian để hội tụ (Hình 4.5).

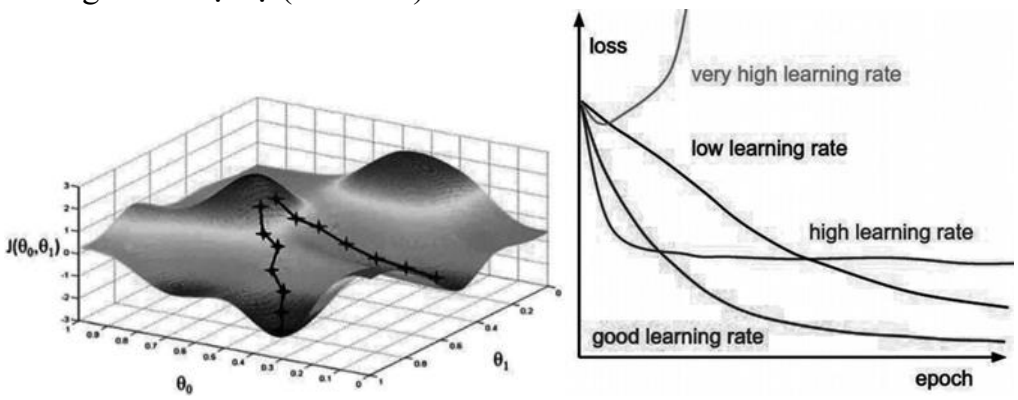


Fig. 4.5 Representation of Learning Rate

- **Lan truyền ngược (Backpropagation):** Khi xây dựng một Mạng nơ-ron, chúng ta gán trọng số và độ lệch ngẫu nhiên cho các nút. Chúng ta có thể đo lỗi của mạng sau một lần lặp. Lỗi này sau đó được lan truyền ngược lại mạng cùng với gradient của hàm chi phí để điều chỉnh các trọng số. Các trọng số này được sửa đổi để giảm lỗi trong các lần lặp tiếp theo. Sự thay đổi trọng số này được gọi là lan truyền ngược.
- **Lô (Batches):** Trong quá trình huấn luyện, chúng ta chia đầu vào thành nhiều phần (lô) có kích thước bằng nhau một cách ngẫu nhiên, thay vì gửi toàn bộ đầu vào trong một lần. Việc huấn luyện dữ liệu theo lô giúp mô hình dễ mở rộng hơn.
- **Kỷ nguyên (Epochs):** Một epoch (kỷ nguyên) được định nghĩa là một lượt lan truyền thuận và ngược của *toàn bộ* dữ liệu đầu vào (qua tất cả các lô). Bạn có thể

chọn số lượng epoch để huấn luyện mạng của mình. Nhiều epoch hơn có khả năng cho độ chính xác cao hơn, nhưng sẽ mất nhiều thời gian hơn để hội tụ.

- **Dropout:** Dropout là một kỹ thuật điều chuẩn (regularization) giúp tránh hiện tượng mạng bị quá khớp (overfitting). Đúng như tên gọi, một số nơ-ron trong lớp ẩn bị "loại bỏ" một cách ngẫu nhiên trong quá trình huấn luyện. Điều này có nghĩa là việc huấn luyện diễn ra trên nhiều kiến trúc khác nhau với các tổ hợp nơ-ron khác nhau. Bạn có thể coi dropout như một quy trình "tổ hợp" (ensemble), tạo ra kết quả cuối cùng bằng cách sử dụng đầu ra của nhiều mạng (Hình 4.6).

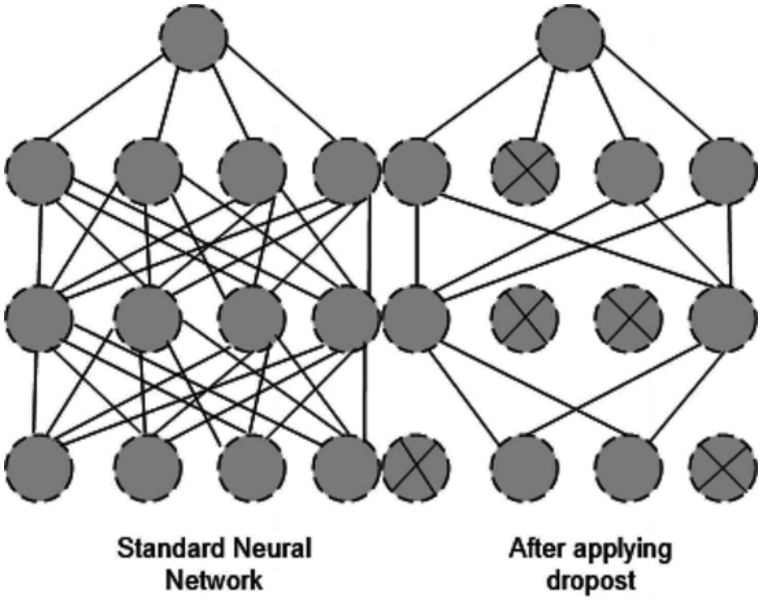


Fig. 4.6 Demonstrating working of dropout

- **Chuẩn hóa theo lô (Batch Normalization):** Chuẩn hóa theo lô có thể được xem như một quy trình giống như chúng ta đặt một điểm kiểm soát đặc biệt trên một dòng sông. Điều này được thực hiện để đảm bảo phân phối dữ liệu (distribution) giống như những gì lớp tiếp theo mong đợi. Khi huấn luyện mạng nơ-ron, các trọng số thay đổi sau mỗi pha giảm gradient. Điều này làm thay đổi dạng thức dữ liệu được gửi đến lớp tiếp theo. Tuy nhiên, lớp tiếp theo lại mong đợi phân phối dữ liệu giống như những gì nó đã thấy trước đó. Vì vậy, trước khi gửi dữ liệu sang giai đoạn tiếp theo, chúng ta chuẩn hóa (normalize) dữ liệu một cách đặc biệt.
- **Huấn luyện (Training):** Các trọng số bắt đầu bằng các giá trị ngẫu nhiên. Khi mạng nơ-ron dần làm quen với loại dữ liệu đầu vào, mạng sẽ thay đổi trọng số dựa trên bất kỳ lỗi phân loại nào phát sinh từ các trọng số trước đó. Khi mạng đã học xong, chúng ta có thể dự đoán kết quả cho đầu vào liên quan.

Mạng Nơ-ron (Neural Networks)

Mạng nơ-ron là các thiết bị tính toán song song nhằm cố gắng xây dựng một mô hình máy tính mô phỏng bộ não. Mục tiêu chính là phát triển một hệ thống cho phép thực hiện

một số tác vụ máy tính nhanh hơn các hệ thống truyền thống. Các nhiệm vụ này bao gồm nhận dạng và xác định mẫu, ước tính, tối ưu hóa và phân cụm dữ liệu.

Mạng Nơ-ron Nhân tạo (Artificial Neural Networks - ANN)

Mạng nơ-ron nhân tạo (ANN) là một hệ thống tính toán hiệu quả với chủ đề cốt lõi được lấy từ sự tương đồng với các mạng nơ-ron sinh học. ANN còn được gọi là Hệ thống Nơ-ron Nhân tạo, Hệ thống Xử lý Phân tán Song song và Hệ thống Kết nối (Connectionist Systems).

ANN có được một tập hợp lớn các đơn vị (unit) được liên kết theo một mẫu nào đó để cho phép tiếp xúc giữa chúng. Các đơn vị như vậy là các bộ xử lý đơn giản hoạt động song song, còn được gọi là các nút (node) hoặc nơ-ron (neuron). Mỗi nơ-ron được liên kết với một nơ-ron khác bằng một liên kết kết nối. Mỗi liên kết kết nối được liên kết với một trọng số mang thông tin tín hiệu đầu vào. Đây là kiến thức hữu ích nhất để nơ-ron giải quyết một vấn đề cụ thể, vì trọng số thường kích hoạt hoặc ức chế tín hiệu. Trạng thái bên trong của mỗi nơ-ron được gọi là tín hiệu kích hoạt. Sau khi kết hợp các tín hiệu đầu vào và quy tắc kích hoạt, các tín hiệu đầu ra được tạo ra có thể được truyền đến các đơn vị khác.

Mô hình Mạng Nơ-ron Nhân tạo: Sơ đồ (Hình 4.7) biểu diễn mô hình ANN tổng quát và quy trình của nó. Đối với mô hình tổng quát trên, đầu vào ròng (net input) có thể được tính như sau:

$$y_{in} = x_1.w_1 + x_2.w_2 + x_3.w_3 \ldots x_m.w_m$$

Tức là, Đầu vào ròng $y_{in} = \sum m_i x_i.w_i$

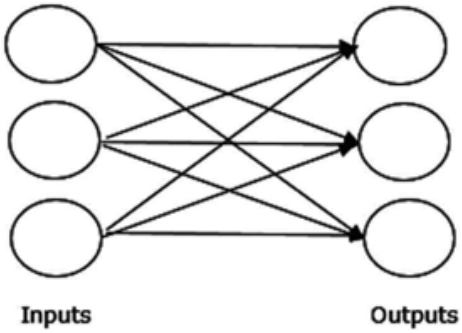


Fig. 4.7 Single Layer Feed forward Network

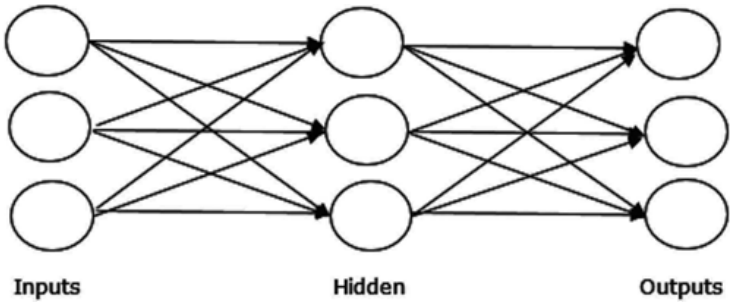


Fig. 4.8 Multiple Layer Feedforward Network

Đầu ra có thể được tính bằng cách áp dụng hàm kích hoạt lên đầu vào ròng.

$$Y = F(y_{in})$$

Quá trình xử lý của ANN phụ thuộc vào ba khối xây dựng sau đây:

1. Cấu trúc liên kết mạng (Network Topology) Sự sắp xếp của một mạng với các nút và đường liên kết của nó là cấu trúc liên kết mạng. ANN có thể được phân loại theo các dạng sau đây dựa trên cấu trúc liên kết:

- **Mạng truyền thẳng (Feedforward Network):** Đây là một mạng không có vòng lặp, nơi tín hiệu chỉ có thể truyền theo một hướng từ đầu vào đến đầu ra.
    - **Mạng truyền thẳng một lớp:** Chỉ có một lớp trọng số; lớp đầu vào được kết nối đầy đủ với lớp đầu ra.
    - **Mạng truyền thẳng đa lớp:** Có nhiều hơn một lớp trọng số, bao gồm một hoặc nhiều lớp ẩn nằm giữa lớp đầu vào và lớp đầu ra.
- **Mạng phản hồi (Feedback Network):** Mạng này có các đường dẫn phản hồi, cho thấy tín hiệu sẽ chạy qua các vòng lặp theo bất kỳ hướng nào. Điều này làm cho nó trở thành một hệ thống động phi tuyến tính.
    - **Mạng Hồi quy (Recurrent Networks):** Đây là các mạng phản hồi vòng kín.
    - **Mạng Hồi quy Đầy đủ:** Tất cả các nút đều được kết nối với tất cả các nút khác (Hình 4.9).

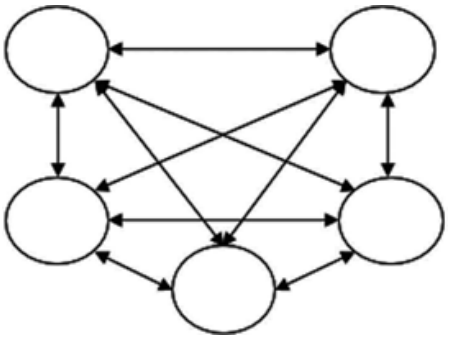


Fig. 4.9 Fully Recurrent Network

    - **Mạng Jordan:** Một mạng mạch kín nơi đầu ra được truyền đến đầu vào dưới dạng phản hồi (Hình 4.10).

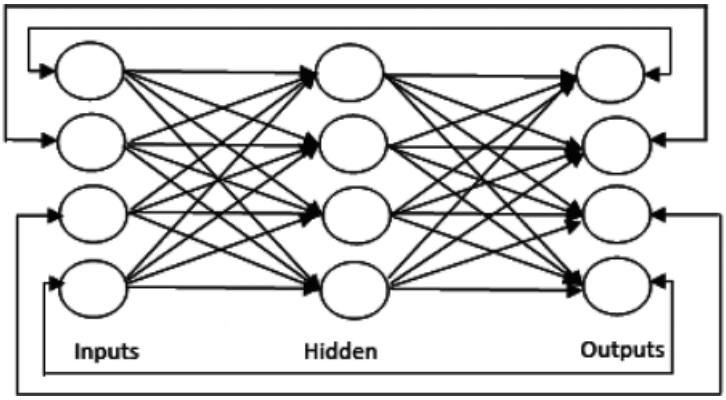


Fig. 4.10 Jordan Network

2. Điều chỉnh Trọng số hoặc Học tập Phương pháp sửa đổi trọng số của các kết nối giữa các nơ-ron được gọi là học tập. Việc học của ANN có thể được phân thành ba nhóm:

- **Học có giám sát (Supervised Learning):** Diễn ra dưới sự hướng dẫn của "giáo viên". Vector đầu vào được đưa vào mạng, tạo ra một vector đầu ra. Vector này được so sánh với vector đầu ra mong muốn. Nếu chúng khác nhau, một tín hiệu lỗi sẽ được tạo ra. Các trọng số được điều chỉnh dựa theo tín hiệu lỗi này cho đến khi đầu ra thực tế khớp với đầu ra mong muốn.

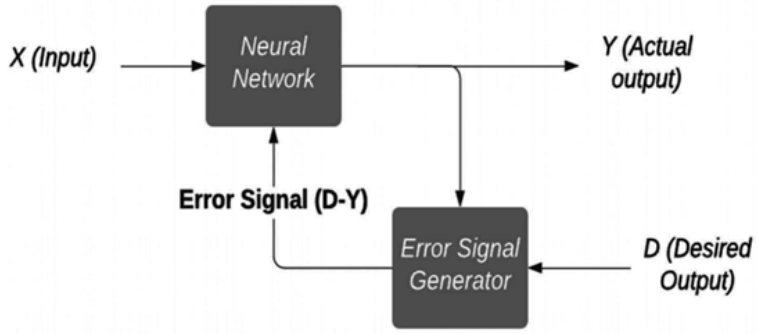


Fig. 4.11 Dependent Learning Process

- **Học không giám sát (Unsupervised Learning):** Được thực hiện mà không có sự giám sát. Các vector đầu vào có cùng dạng được gộp vào các cụm (cluster). Mạng phải tự khám phá các mẫu và đặc trưng của đầu vào.

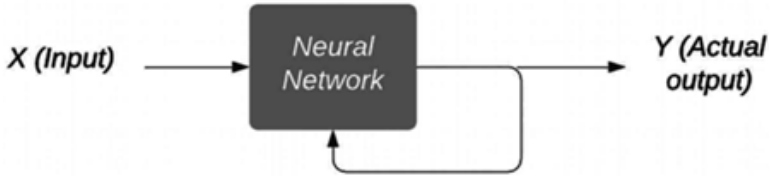


Fig. 4.12 Individual Learning Process

- **Học tăng cường (Reinforcement Learning):** Được sử dụng để củng cố hoặc cải thiện mạng thông qua dữ liệu "phê bình" (hoặc phản hồi). Quá trình này tương tự như học có giám sát, nhưng chúng ta có thể có ít thông tin hơn nhiều. Mạng nhận được một số đầu vào từ "môi trường". Phản hồi này không mang tính mô tả chi

tiết, vì không có "người hướng dẫn". Mạng sẽ thay đổi các trọng số để tối ưu hóa thông tin trong tương lai sau khi nhận được phản hồi.

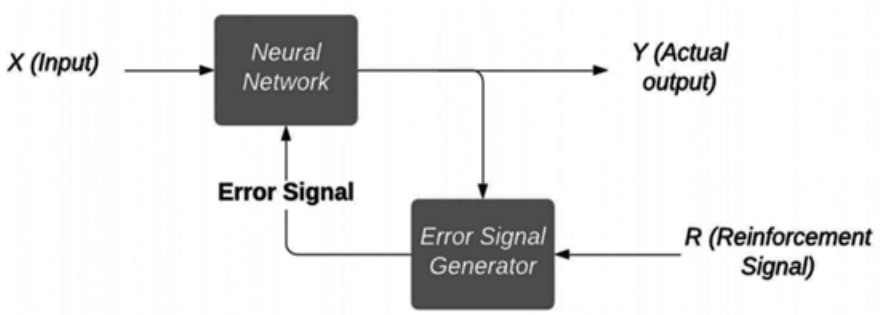


Fig. 4.13 Reinforcement Learning

3. Các hàm kích hoạt (Activation Functions)

Hàm kích hoạt có thể được định nghĩa là lực hoặc nỗ lực bổ sung tác động lên đầu vào để có được đầu ra chính xác. Dưới đây là một số hàm kích hoạt quan trọng:

- **Hàm kích hoạt tuyến tính:** Còn được gọi là hàm đồng nhất (identity function) vì nó không chỉnh sửa dữ liệu. Công thức: $F(x) = x$
- **Hàm kích hoạt Sigmoid:** Có hai loại:
  - **Hàm Sigmoid nhị phân:** Chỉnh sửa đầu vào trong khoảng từ 0 đến 1. Công thức: $F(x) = sigm(x) = 1 \div (1 + exp(-x))$
  - **Hàm Sigmoid song cực:** Điều chỉnh đầu vào trong khoảng từ -1 đến 1. Công thức: $F(x) = sigm(x) = 2 \div (1 + exp(-x)) - 1$

Thách thức trong ANN Bước đầu tiên trong việc giải quyết câu hỏi phân loại hình ảnh bằng ANN là biến đổi hình ảnh 2D thành vector 1D trước khi huấn luyện. Có hai bất tiện:

- Khi kích thước ảnh tăng, số lượng tham số có thể huấn luyện tăng lên đáng kể.
- ANN làm mất các đặc điểm không gian (spatial characteristics) của hình ảnh.
- ANN không thể ghi lại thông tin tuần tự (sequential information) để xử lý dữ liệu chuỗi.

Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN)

Mạng nơ-ron tích chập (CNN) cũng giống như mạng nơ-ron thông thường vì nó bao gồm các nơ-ron có trọng số học được và các đặc điểm độ lệch (bias). Các mạng nơ-ron thông thường bỏ qua cấu trúc dữ liệu đầu vào và chuyển đổi tất cả dữ liệu thành một mảng 1-D.

Vấn đề này được CNN giải quyết dễ dàng. Nó tính đến cấu trúc 2D của hình ảnh khi xử lý chúng, cho phép chúng trích xuất các thuộc tính dành riêng cho hình ảnh. Mục tiêu chính của CNN là chuyển từ dữ liệu hình ảnh thô ở lớp đầu vào sang lớp (class) chính xác ở lớp đầu ra.

Tổng quan kiến trúc của CNN Kiến trúc CNN được cấu trúc với các nơ-ron theo ba chiều: chiều rộng, chiều cao và chiều sâu. Mỗi nơ-ron của lớp hiện tại được liên kết với một mảng nhỏ (receptive field) của đầu ra lớp trước đó.

Các lớp được sử dụng để xây dựng CNN

- **Lớp đầu vào (Input Layer):** Nhận dữ liệu hình ảnh thô.
- **Lớp tích chập (Convolutional Layer):** Đây là thành phần trung tâm của CNN, nơi thực hiện hầu hết các phép tính. Lớp này đo lường các phép tích chập (convolutions) trên đầu vào.
- **Phần đệm (Padding):** Khi bộ lọc không khớp hoàn toàn với hình ảnh đầu vào, chúng ta có hai lựa chọn:
    - Đệm ảnh bằng các số 0 (zero padding) để vừa vặn.
    - Xóa thành phần ảnh không khớp. Đây được gọi là "đệm hợp lệ" (valid padding), chỉ giữ lại phần hợp lệ của hình ảnh.
- **Lớp ReLU (Rectified Linear Unit):** Dùng để kích hoạt đầu ra của lớp trước. Nó làm tăng tính phi tuyến tính của mạng.
- **Lớp Gộp (Pooling Layer):** Gộp giúp bảo tồn các phần thiết yếu khi chúng ta tiến sâu vào mạng. Lớp gộp hoạt động trên từng lát (slice) đầu vào một cách độc lập và thay đổi kích thước không gian của nó. Nó thường được gọi là lấy mẫu con (subsampling) hoặc lấy mẫu giảm (down sampling).
    - Các dạng gộp không gian có thể là: Gộp Cực đại (Max Pooling) , Gộp Trung bình (Average Pooling) , và Gộp Tổng (Sum Pooling).
    - Gộp cực đại lấy phần tử lớn nhất từ bản đồ đặc trưng đã được chỉnh lưu (rectified).
- **Lớp Kết nối Đầy đủ/Lớp Đầu ra (Fully Connected Layer):** Chúng ta "làm phẳng" (flatten) ma trận và đưa nó vào một lớp kết nối đầy đủ (FC layer), tương tự như một mạng nơ-ron. Các giá trị đầu ra ở lớp cuối cùng được tính toán bởi lớp này. Cuối cùng, chúng ta có một hàm kích hoạt để phân loại đầu ra, chẳng hạn như SoftMax hoặc sigmoid.

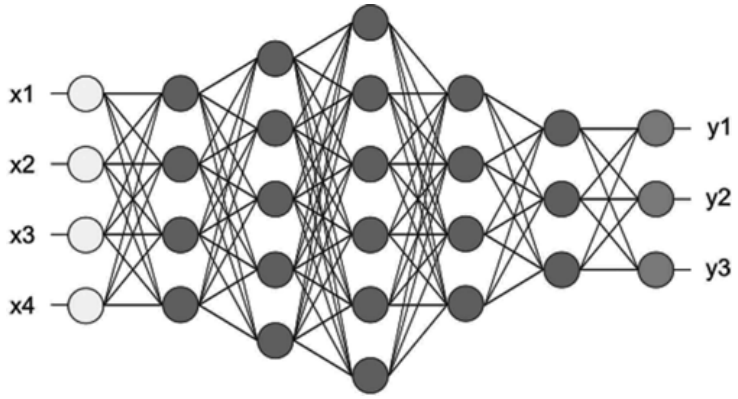


Fig. 4.14 After pooling layer, flattened as FC layer

Mạng Nơ-ron Hồi quy (Recurrent Neural Network - RNN)

Mạng nơ-ron hồi quy (RNN) là một lớp mạng nơ-ron nhân tạo, nơi các kết nối giữa các đơn vị tạo thành một đồ thị có hướng dọc theo một chuỗi (sequence). Điều này cho phép chúng thể hiện các hành vi phức tạp theo thời gian.

Bộ nhớ trong của RNN có thể được sử dụng để xử lý các chuỗi đầu vào. Điều này khiến chúng có thể áp dụng cho các tác vụ như nhận dạng chữ viết tay hoặc ngôn ngữ không phân đoạn, có liên quan.

RNN phát huy tác dụng khi mô hình yêu cầu "bối cảnh" (context) để cung cấp đầu ra dựa trên dữ liệu. Tương tự như khi bạn xem phim, bạn hiểu được diễn biến vì bạn nhớ những gì đã xem trước đó. RNN cũng "ghi nhớ" mọi thứ. Trong RNN, tất cả các đầu vào đều được kết nối. Để làm điều này, RNN tạo ra các mạng có vòng lặp (loop) cho phép thông tin tồn tại dai dẳng (persist) (Hình 4.16).

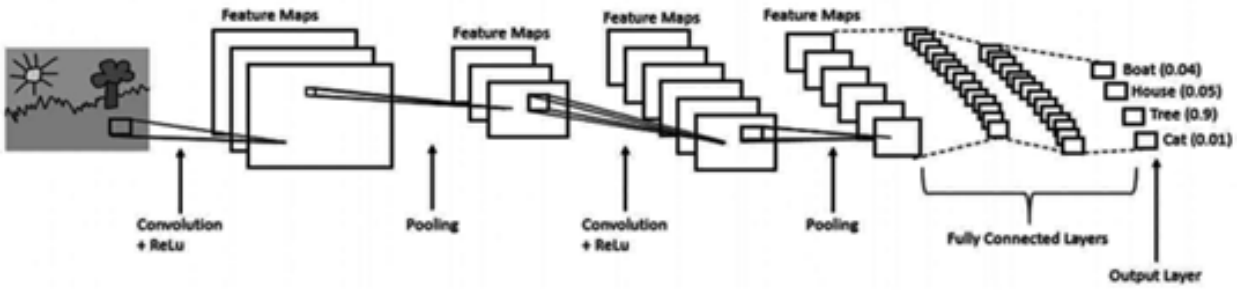


Fig. 4.15 Complete CNN architecture.

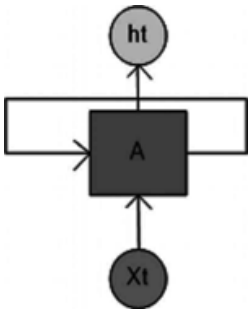


Fig. 4.16 RNN with a LOOP

Nếu "trải dài" (unrolled) cấu trúc (Hình 4.17), bạn có thể thấy: Đầu tiên, $x(0)$ được lấy từ chuỗi đầu vào và tạo ra $h(0)$. Sau đó, $h(0)$ và $x(1)$ là đầu vào cho bước tiếp theo. Tương tự, $h(1)$ cùng với $x(2)$ là đầu vào cho bước sau đó, cứ thế tiếp diễn.

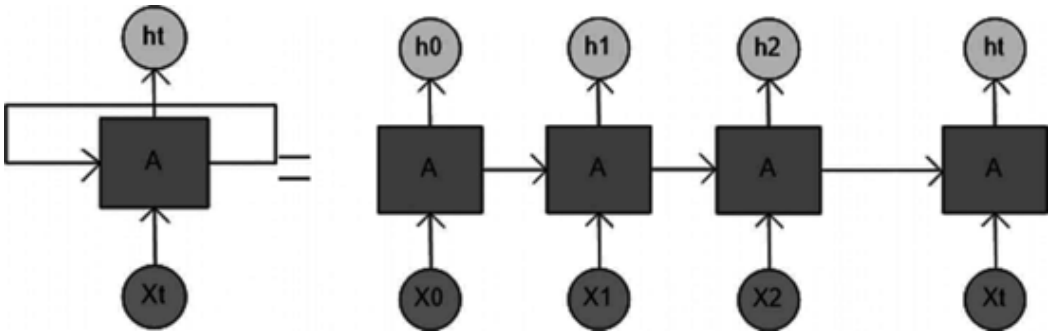


Fig. 4.17 Unrolled Structure of RNN

Các loại RNN RNN cho phép hoạt động trên các chuỗi vector: chuỗi đầu vào, chuỗi đầu ra, hoặc cả hai. (Xem Hình 4.18)

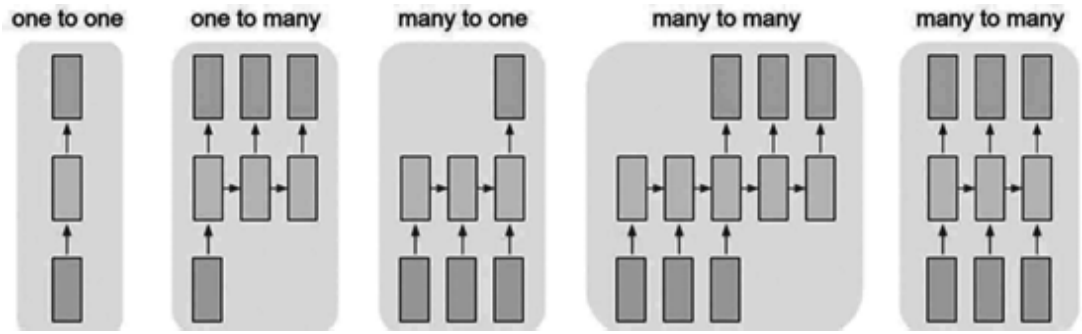


Fig. 4.18 Types Of RNN

- **Một-một (One to One):** Mạng nơ-ron thông thường. Xử lý đầu vào kích thước cố định sang đầu ra kích thước cố định. Ví dụ: Phân loại hình ảnh.
- **Một-nhiều (One to many):** Nhận đầu vào kích thước cố định và cung cấp chuỗi dữ liệu làm đầu ra. Ví dụ: Gắn thẻ ảnh (Image captioning).
- **Nhiều-một (Many to One):** Nhận chuỗi thông tin làm đầu vào và đưa ra một đầu ra cố định. Ví dụ: Phân tích cảm xúc (sentimental analysis).
- **Nhiều-nhiều (Many to Many):** Nhận một chuỗi thông tin làm đầu vào và xử lý để tạo ra một chuỗi đầu ra. Ví dụ: Dịch máy.
- **Nhiều-nhiều (đồng bộ):** Chuỗi đầu vào và đầu ra được đồng bộ hóa. Ví dụ: Phân loại video (video grading) nơi mỗi khung hình video được gán nhãn.

Thách thức của RNN Các RNN sâu (RNN có nhiều bước thời gian) thường bị ảnh hưởng bởi các vấn đề "gradient biến mất" (vanishing gradient) và "gradient bùng nổ" (exploding gradient), vốn là đặc điểm chung của tất cả các loại mạng nơ-ron.

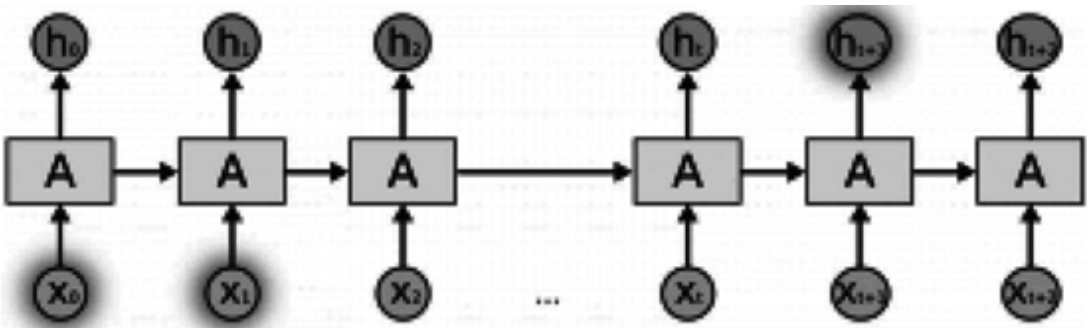


Fig. 4.19 Recurrent Neural Network

Sự khác biệt giữa ANN, CNN & RNN

Bảng sau đây mô tả sự khác biệt giữa Mạng Nơ-ron Nhân tạo, Mạng Nơ-ron Tích chập và Mạng Nơ-ron Hồi quy:

	ANN	**CNN**	**RNN**
Dữ liệu	Dữ liệu dạng bảng (Tabular)	Dữ liệu hình ảnh	Dữ liệu chuỗi (Sequence)
Kết nối Hồi quy	Không	Không	Có
Chia sẻ Tham số	Không	Có	Có
Mối quan hệ Không gian	Không	Có	Không
Gradient Biến mất/Bùng nổ	Có	Có	Có

Tóm tắt

Trong chương này, chúng ta đã nghiên cứu về những điều cơ bản của học sâu, bao gồm các thuật ngữ hoặc các khái niệm cơ bản chúng ta đề cập khi sử dụng học sâu hoặc bất kỳ thuật toán nào khác như độ lệch (bias), tốc độ học (learning rate), v.v., và chúng ta cũng đã nghiên cứu về mạng nơ-ron nhân tạo, Mạng nơ-ron Tích chập và Mạng nơ-ron Hồi quy với cấu trúc cơ bản của chúng, có thể được sử dụng trong các tình huống khác nhau tùy theo yêu cầu của vấn đề.

Trong phần Triển khai của cuốn sách này, chúng ta sẽ sử dụng CNN để phân loại và thử nhận dạng thực vật hoặc các loài của chúng.

CHƯƠNG 5: THỊ GIÁC MÁY TÍNH

Giới thiệu

Nông nghiệp đã đóng một vai trò quan trọng trong nền kinh tế toàn cầu trong những năm gần đây. Sự gia tăng hơn nữa của dân số đang dẫn đến việc giảm dần diện tích đất nông nghiệp và gây thêm căng thẳng ngày càng tăng cho hệ thống trang trại. Nhu cầu về các phương pháp sản xuất thực phẩm nông nghiệp thành công và lành mạnh đang tăng lên. Các công nghệ cảm biến và động lực sáng tạo cùng các hệ thống thông tin và truyền thông tiên tiến và Trí tuệ nhân tạo phải bổ sung cho các phương pháp quản lý nông nghiệp truyền thống để tăng tốc độ tăng trưởng năng suất trang trại một cách chính xác hơn, qua đó thúc đẩy sản xuất nông nghiệp chất lượng cao và hiệu suất cao. Hệ thống giám sát bằng thị giác máy tính đã trở thành công cụ có giá trị trong hoạt động trang trại trong nhiều thập kỷ qua, và việc sử dụng chúng đã tăng lên đáng kể.

Thị giác máy tính là gì?

Thị giác máy tính có thể được mô tả như một lĩnh vực của AI dùng để trích xuất thông tin từ hình ảnh kỹ thuật số. Loại dữ liệu thu được từ một hình ảnh có thể khác nhau, từ nhận dạng, đo lường không gian điều hướng hoặc các ứng dụng thực tế tăng cường.

Các ứng dụng của thị giác máy tính cũng có thể được mô tả. Thị giác máy tính xây dựng các thuật toán nhận dạng và sử dụng chất lượng của hình ảnh cho các mục đích khác.

Con người nhìn và trải nghiệm thế giới xung quanh một cách vật lý thông qua mắt và não bộ. Thị giác máy tính là khoa học muốn cung cấp cho máy móc hoặc thiết bị một hiệu suất tương tự, nếu không muốn nói là tốt hơn.

Mục đích của thị giác máy tính là tự động thu thập, diễn giải và hiểu dữ liệu hữu ích và có ý nghĩa từ một hình ảnh hoặc một chuỗi hình ảnh. Điều này bao gồm việc thiết lập một cơ sở lý thuyết và thuật toán cho sự hiểu biết thị giác tự động.

Trong nông nghiệp, hệ thống thị giác máy tính rõ ràng là khả thi cho việc phân loại và đánh giá các đặc tính như màu sắc, hình dạng, độ dày, khuyết tật bề mặt và sự nhiễm bẩn của các sản phẩm thực phẩm thành các cấp độ khác nhau.

Phân cấp thị giác máy tính: Thị giác máy tính được phân thành ba loại cơ bản:

- Cấp thấp: Chứa hình ảnh xử lý để trích xuất đặc trưng
- Cấp trung gian: Liên quan đến nhận dạng đối tượng và cảnh 3D
- Cấp cao: Cung cấp một cái nhìn tổng quan về khái niệm của một kịch bản sự cố, mục tiêu và hành vi

Nhiệm vụ của Thị giác máy tính

- **Nhận dạng (Recognition):** Vấn đề bình thường trong thị giác máy tính, xử lý ảnh và thị giác máy (machine vision) là liệu dữ liệu hình ảnh có chứa bất kỳ đối tượng, chức năng hoặc hoạt động cụ thể nào hay không. Thông thường, một người có thể vượt qua tình huống khó xử này một cách hiệu quả và dễ dàng, tuy nhiên trong trường hợp tổng quát, thị giác máy vẫn chưa giải quyết thỏa đáng vấn đề này: các đối tượng tùy ý trong các hoàn cảnh tùy ý. Các phương pháp có sẵn để xử lý trong lĩnh vực này, tốt nhất cũng chỉ có thể giải quyết vấn đề này cho các đối tượng cụ thể, chẳng hạn như các đối tượng hình học cơ bản (ví dụ: khối đa diện), đặc điểm con người, ký tự viết tay hoặc bản thảo hoặc phương tiện giao thông, và môi trường xung quanh trong các hoàn cảnh đặc biệt thường được mô tả rõ về điều kiện ánh sáng. Có nhiều loại Nhận dạng khác nhau được chú ý như:
    - **Nhận dạng (Recognition):** Một hoặc nhiều đối tượng hoặc lớp, thường cùng với vị trí hình ảnh 2D hoặc tư thế 3D của chúng, có thể được nhận dạng trước hoặc học được trong một cảnh nhất định. Điều này có thể là bất kỳ người nào, vật phẩm nào như trái cây, rau củ, v.v.
    - **Nhận diện (Identification):** Một trường hợp riêng biệt được nhận dạng cho một đối tượng. Ví dụ: nhận diện khuôn mặt hoặc dấu vân tay của một cá nhân cụ thể hoặc nhận diện một phương tiện cụ thể.
    - **Phát hiện (Detection):** Dữ liệu hình ảnh được sàng lọc cho một tình huống cụ thể. Ví dụ: nhận diện, trong hình ảnh nông nghiệp, các tế bào hoặc mô bất thường tiềm ẩn hoặc nhận diện thu phí đường bộ tự động. Việc phát hiện bằng phép tính tương đối đơn giản và nhanh chóng thường được sử dụng để xác định các vùng nhỏ chứa dữ liệu thú vị, mà sau đó có thể được phân tích thêm để có được sự diễn giải chính xác bằng cách sử dụng các công nghệ đòi hỏi tính toán phức tạp hơn.
- **Phân tích chuyển động (Motion Analysis):** Nhiều nhiệm vụ khác nhau liên quan đến việc ước tính chuyển động khi xử lý một chuỗi hình ảnh để đo tốc độ tại mỗi điểm trong hình ảnh hoặc cảnh hình ảnh 3D, hoặc thậm chí là của máy ảnh tạo ra hình ảnh. Các ví dụ như sau:
    - **Chuyển động tự thân (Egomotion):** Xác định chuyển động cứng 3D của máy ảnh từ chuỗi hình ảnh của máy ảnh (quay và tịnh tiến).
    - **Theo dõi (Tracking):** Theo dõi (thông thường) chuyển động của một chuỗi các điểm quan tâm hoặc đối tượng trong chuỗi hình ảnh (ví dụ: ô tô, con người hoặc các sinh vật khác).
    - **Dòng quang (Optical flow):** để đánh giá cách điểm di chuyển trong mối quan hệ với mặt phẳng hình ảnh cho mỗi điểm trong ảnh, tức là chuyển động biểu kiến của nó. Chuyển động này đại diện cho cả cách điểm 3D di chuyển trong cảnh và cách máy ảnh di chuyển so với đối tượng.
- **Tái tạo cảnh (Scene Reconstruction):** Nếu một tái tạo cảnh có một hoặc (thường là) nhiều hình ảnh của một cảnh hoặc một bức tranh, nó nhằm mục đích tính toán

một mô hình 3D của cảnh đó. Mô hình có thể là một chuỗi các điểm 3D trong trường hợp đơn giản nhất. Các phương pháp chi tiết hơn tạo ra một mô hình bề mặt 3D hoàn chỉnh. Sự ra đời của hình ảnh 3D không cần chuyển động hoặc quét và các thuật toán xử lý liên quan cho phép tiến bộ nhanh chóng trong lĩnh vực này. Cảm biến lưới 3D có thể được sử dụng trong xử lý đa góc của hình ảnh ba chiều. Một số hình ảnh 3D hiện có sẵn để được hợp nhất thành các đám mây điểm và mô hình 3D.

- **Khôi phục ảnh (Image Restoration):** Mục tiêu của khôi phục ảnh là loại bỏ nhiễu khỏi ảnh (nhiễu cảm biến, mờ do chuyển động, v.v.). Các loại bộ lọc đa dạng như bộ lọc thông thấp hoặc bộ lọc trung bình là giải pháp khả thi dễ dàng nhất để giảm nhiễu. Các phương pháp tiên tiến hơn được thiết kế để phân biệt giữa nhiễu và các cấu trúc hình ảnh cục bộ. Khi dữ liệu hình ảnh trước tiên được phân tích bởi các cấu trúc hình ảnh cục bộ, chẳng hạn như đường thẳng hoặc cạnh, và sau đó việc lọc được quản lý dựa trên thông tin cục bộ từ giai đoạn phân tích, mức độ loại bỏ nhiễu tốt hơn thường đạt được so với các phương pháp đơn giản hơn.

Các thách thức liên quan đến việc nhận dạng/phát hiện hình ảnh:

- Thay đổi góc nhìn
- Thay đổi tỷ lệ
- Biến thiên trong cùng một lớp
- Biến dạng hình ảnh
- Che khuất hình ảnh
- Ánh sáng
- Nền lộn xộn

Hệ thống thị giác máy tính

Cấu trúc hệ thống thị giác máy tính là rất quan trọng. Trong một số trường hợp, thiết bị là một hệ thống con của một thiết kế lớn hơn, ví dụ như với các hệ thống con để điều khiển cơ cấu truyền động cơ học, lập kế hoạch, kho lưu trữ thông tin, giao diện người-máy, v.v. Một số hệ thống có các triển khai riêng biệt, giải quyết một vấn đề đo lường và phát hiện cụ thể. Điều này cũng sẽ phụ thuộc vào việc chức năng của nó được chỉ định trước hay liệu một phần của chương trình thị giác máy tính có thể được cập nhật hoặc sửa đổi trong quá trình hoạt động hay không.

Tuy nhiên, nhiều hệ thống thị giác máy tính có các đặc điểm truyền thống.

- **Thu thập hình ảnh (Image Acquisition):** Chụp một hoặc nhiều hình ảnh bằng cách sử dụng các cảm biến hình ảnh kỹ thuật số được phát triển để bao gồm các cảm biến phạm vi, thiết bị chụp cắt lớp, radar, camera siêu âm, v.v., ngoài loại camera nhạy sáng cụ thể. Dữ liệu hình ảnh thu được phụ thuộc vào dạng cảm biến: một hình ảnh 2D cụ thể, một khối 3D hoặc một chuỗi hình ảnh. Các giá trị pixel thường đề cập đến cường độ ánh sáng trong một hoặc nhiều độ dài phổ (ảnh xám

hoặc ảnh màu), nhưng cũng có thể áp dụng cho các phép đo vật lý khác nhau, chẳng hạn như độ rộng, độ hấp thụ hoặc độ phản xạ của sóng âm hoặc sóng điện từ, hoặc cộng hưởng từ hạt nhân.

- **Tiền xử lý (Pre-Processing):** Để truy xuất các mẩu thông tin nhất định, một phương pháp thị giác máy tính thường có thể được sử dụng để phân tích dữ liệu nhằm đảm bảo rằng tất cả các mong đợi mà kỹ thuật đề xuất đều có thể được đáp ứng.
- **Trích xuất đặc trưng (Feature Extraction):** Dữ liệu hình ảnh trích xuất đặc trưng ở các mức độ phức tạp khác nhau. Các ví dụ điển hình là: đường thẳng, cạnh và gờ. Các điểm quan tâm cục bộ như cạnh, đốm hoặc đường thẳng. Kết cấu, hình dạng hoặc chuyển động có thể phức tạp hơn.
- **Phát hiện/Phân đoạn (Detection/Segmentation):** Có một quyết định ở một số giai đoạn trong quá trình xử lý về việc điểm hình ảnh hoặc vùng hình ảnh nào là quan trọng để xử lý thêm. Điều này có thể giống như:
    - Thứ tự của một tập hợp các điểm quan tâm cụ thể.
    - Phân đoạn một hoặc nhiều phần của hình ảnh có chứa một điểm quan tâm cụ thể.
    - Chia hình ảnh thành Kiến trúc cảnh lồng nhau bao gồm nền trước, các lớp đối tượng, các đối tượng riêng lẻ hoặc các mẩu đối tượng nổi bật (còn được gọi là hệ thống phân cấp của cảnh phân loại không gian).
    - Phân đoạn hoặc đồng phân đoạn một hoặc nhiều video thành một tập hợp các mặt nạ tiền cảnh trên mỗi khung hình trong khi vẫn duy trì tính liên tục về ngữ nghĩa thời gian của nó.
- **Xử lý cấp cao (High Level Processing):** Đầu vào thường bao gồm một tập dữ liệu nhỏ, chẳng hạn như một tập hợp các điểm hoặc một vùng hình ảnh được cho là chứa một đối tượng cụ thể. Nó giải quyết các vấn đề:
    - Kiểm tra xem dữ liệu có đáp ứng các giả định rõ ràng dựa trên mô hình và triển khai hay không.
    - Ước tính các thông số cụ thể của ứng dụng như vị trí đối tượng hoặc kích thước đối tượng.
    - Nhận dạng hình ảnh - nhóm một đối tượng được phát hiện vào các danh mục khác nhau.
    - Đăng ký hình ảnh - so sánh và tích hợp hai chế độ xem khác nhau của cùng một đối tượng.
- **Ra quyết định (Decision Making):** Đưa ra phán đoán cuối cùng cho ứng dụng,
    - Ví dụ: Ứng dụng kiểm tra tự động đạt/không đạt.
    - Khớp/không khớp trong các ứng dụng nhận dạng.
    - Gắn cờ để con người xem xét thêm trong các ứng dụng y tế, quân sự, bảo vệ và nhận dạng.

Để triển khai Thị giác máy tính, chúng ta sử dụng framework như OpenCV. Chúng ta sẽ nghiên cứu về nó trong phần tiếp theo.

📸 OPENCV

OpenCV là một gói hoặc thư viện mã nguồn mở được phát triển bởi Intel. Thư viện này là đa nền tảng, nghĩa là chúng ta có thể sử dụng nó với java, C++, python, v.v. Trong tất cả các loại xử lý và phân tích hình ảnh và video, OpenCV hoặc phần mềm Open Source Control Vision (http:/opencv.org) đều được sử dụng. Nó có thể xử lý hình ảnh và video để nhận dạng người, khuôn mặt hoặc thậm chí là chữ viết tay. Được tích hợp vào nhiều thư viện, bao gồm NumPy, Python có khả năng phân tích cấu trúc mảng của OpenCV. OpenCV chuyển đổi kiến thức thị giác máy sang không gian của đối tượng. Mô hình hình ảnh và chức năng từ không gian vector có thể được xác định và các phép toán toán học được thực hiện trên các đặc trưng này.

Cài đặt OpenCV

- **Trên Windows:** Dưới đây là danh sách các bước cần thiết để hoàn tất cài đặt.
    - **Cài đặt OpenCV:** Chạy lệnh “pip install opencv_python” trên Command prompt của Windows và OpenCV sẽ tự động được cài đặt.
- **Trên linux/ubuntu:**

Nhập OpenCV

Vào IDLE (môi trường lập trình Python) và nhập OpenCV:

```
>>> import cv2
```

Bạn cũng có thể kiểm tra phiên bản mình đang dùng:

```
>>> cv2.version '3.4.1'
```

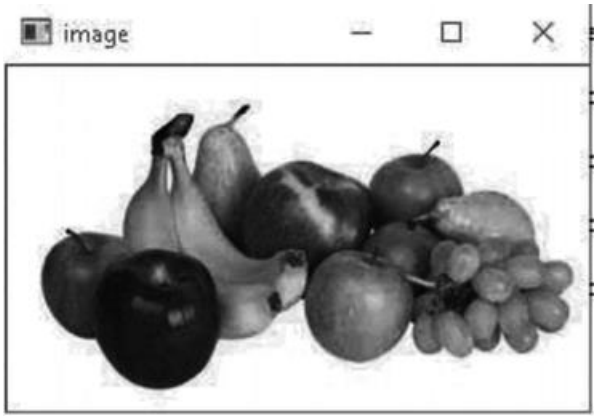
Lưu ý: Phiên bản có thể khác, tùy thuộc vào bản phát hành của nó.

Hãy thử làm gì đó với OpenCV:

Ví dụ, lưu tệp với tên opencv.py:

```
import cv2 #Thư viện OPENCV
# Tải một ảnh màu
img = cv2.imread ('download.jpg')
#Ảnh phải ở cùng thư mục nơi tệp chương trình được lưu
cv2.imshow ('image',img)
cv2.waitKey (0)
cv2.destroyAllWindows ()
```

Đầu ra: (Hiển thị một cửa sổ có tên ‘image’ chứa ảnh ‘download.jpg’)

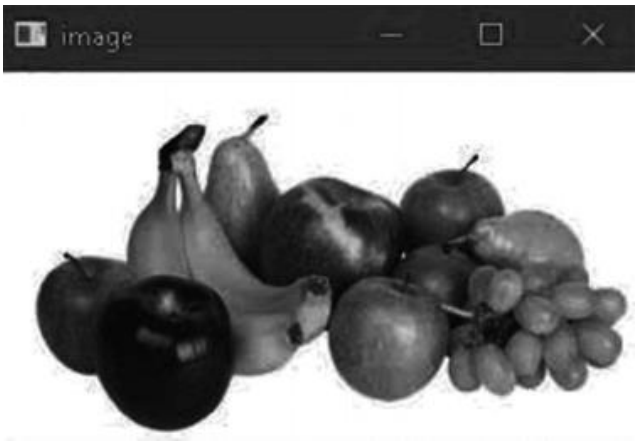


(Hình 5.1. Hình ảnh đầu ra)

Chúng ta có thể chuyển đổi hình ảnh Hình 5.1 ở trên thành thang độ xám bằng cách truyền giá trị 0 vào đối số imread:

```
import cv2 #Thư viện OPENCV
# Tải một ảnh màu
img = cv2.imread ('download.jpg',0)
#truyền 0 để chuyển nó thành thang độ xám
#Ảnh phải ở cùng thư mục nơi tệp chương trình được lưu
cv2.imshow ('image', img)
cv2.waitKey (0)
cv2.destroyAllWindows ()
```

Đầu ra: (Hiển thị ảnh tương tự nhưng ở thang độ xám)



(Hình 5.2 Ảnh xám)

Giải thích:

- **Cv2.imread():** được sử dụng để đọc một hình ảnh. Và như đã thấy bằng cách truyền đối số thứ hai là 0, chúng ta đã chuyển đổi hình ảnh sang thang độ xám (Hình 5.2). Đối số thứ hai luôn mô tả cách hình ảnh nên được đọc:
    - **cv2.IMREAD_COLOR:** Tải một hình ảnh màu. Mọi độ trong suốt của hình ảnh đều bị bỏ qua. Đây là giá trị mặc định.
    - **cv2.IMREAD_GRAYSCALE:** Tải hình ảnh ở thang độ xám.
    - **cv2.IMREAD_UNCHANGED:** Tải hình ảnh nguyên trạng bao gồm cả kênh alpha (độ trong suốt).
- **cv2.imshow():** được sử dụng để hiển thị một hình ảnh. Cửa sổ đầu ra tự động vừa với kích thước của hình ảnh. Đối số đầu tiên là một chuỗi (string) tên cửa sổ. Đối số thứ hai là hình ảnh của chúng ta. Bạn có thể tạo bao nhiêu cửa sổ tùy thích, nhưng phải có tên cửa sổ khác nhau.
- **cv2.waitKey():** Là một chức năng chờ sự kiện từ bàn phím trong OpenCV. Đối số là thời gian tính bằng mili giây. Chức năng này chờ trong khoảng thời gian đó cho bất kỳ sự kiện bàn phím nào. Nếu bạn nhấn một phím bất kỳ trong thời gian đó, chương trình sẽ tiếp tục. Nếu đối số là 0, nó sẽ chờ một phím được nhấn mãi mãi. Bạn cũng có thể thiết lập nó để phát hiện các phím bấm cụ thể (ví dụ: khi phím 'a' được nhấn, v.v.).
- **cv2.destroyAllWindows():** Phá hủy tất cả các cửa sổ đã được tạo. Nếu muốn phá hủy một cửa sổ cụ thể, chúng ta có thể sử dụng hàm cv2.destroywindow() và truyền tên cửa sổ làm đối số.

OpenCV được sử dụng rộng rãi trong rất nhiều lĩnh vực như một phần quan trọng của các công nghệ khác nhau và chúng ta sẽ sử dụng OpenCV trong các chương tiếp theo cho nhiều mục đích khác nhau và rất nhiều chủ đề khác sẽ được thảo luận. Bây giờ chúng ta hãy tạo một dự án thời gian thực với sự trợ giúp của OpenCV.

Phân loại cây trồng

Trong nông nghiệp, phân loại chất lượng cây trồng là một việc rất quan trọng, và nó có thể thực hiện đơn giản bằng cách sử dụng OpenCV.

(Phần sau đây là mã Python để phân loại hạt gạo)

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

def get_classificaton (ratio):
    ratio =round (ratio,1)
    toret=""
    if (ratio>=3):
```

```
        toret="Slender" # (Thon)
    elif (ratio>=2.1 and ratio<3):
        toret="Medium" # (Trung bình)
    elif (ratio>=1.1 and ratio<2.1):
        toret="Bold" # (Mập)
    elif (ratio<=1):
        toret="Round" # (Tròn)
    toret=" ("+toret+")"
    return toret

print ("Starting")
img = cv2.imread ('rice.png',0) #Đường dẫn nơi hình ảnh được lưu trữ
#tải ở chế độ thang độ xám

#chuyển đổi thành nhị phân (đen trắng)
ret, binary = cv2.threshold (img,160,255,cv2.THRESH_BINARY)
# (ảnh, ngưỡng 160, giá trị gán 255, loại ngưỡng)

#bộ lọc trung bình (làm mờ, giảm nhiễu)
kernel = np.ones ((5,5), np.float32)/9
dst = cv2.filter2D (binary, -1, kernel) #-1: độ sâu của ảnh đích

kernel2 = cv2.getStructuringElement (cv2.MORPH_ELLIPSE, (3,3))
#phép co (erosion - làm mỏng đối tượng)
erosion = cv2.erode (dst,kernel2, iterations = 1)
#phép giãn (dilation - làm dày đối tượng)
dilation = cv2.dilate (erosion, kernel2, iterations = 1)

#phát hiện cạnh
edges = cv2.Canny (dilation,100,200)

### Phát hiện kích thước
#Tìm các đường viền
contours, hierarchy = cv2.findContours (erosion, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

print ("No. of rice grains=",len (contours)) #In ra số hạt gạo
total_ar = 0 #Tổng tỷ lệ khung hình
for cnt in contours:
    x,y,w,h = cv2.boundingRect (cnt) #Lấy khung chữ nhật bao quanh
    aspect_ratio = float (w)/h #Tính tỷ lệ rộng/cao
    if (aspect_ratio<1):
        aspect_ratio=1/aspect_ratio #Đảm bảo tỷ lệ luôn >= 1
```

```
    total_ar+=aspect_ratio #Cộng vào tổng

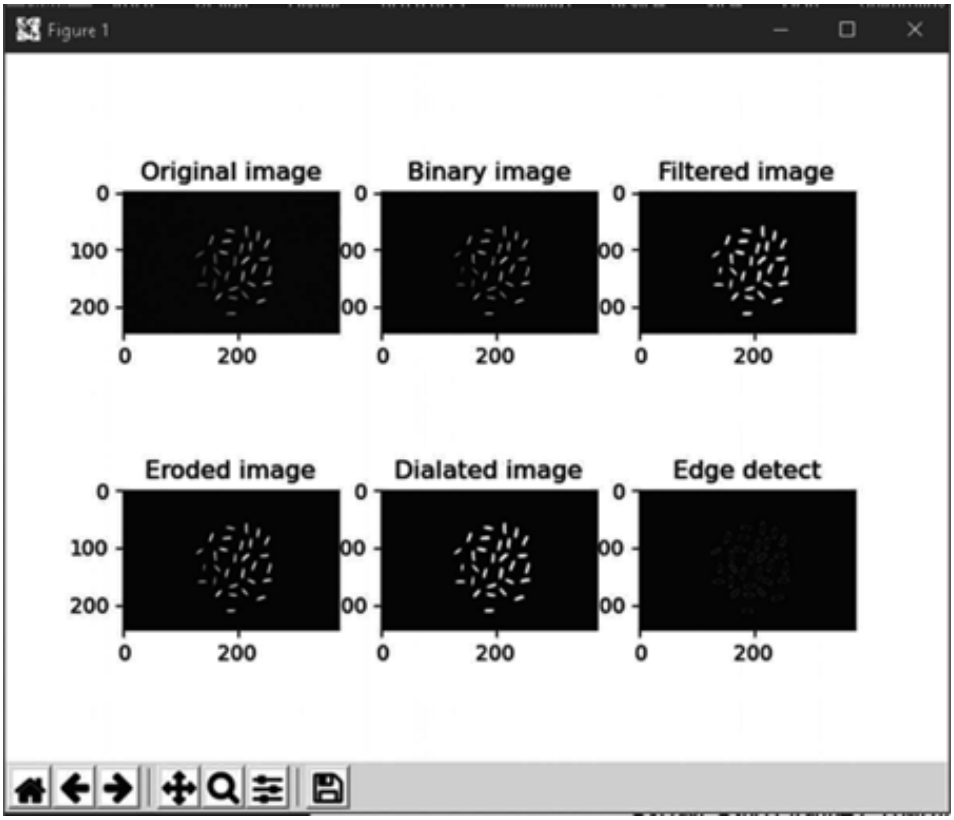
avg_ar=total_ar/len (contours) #Tính tỷ lệ trung bình
print ("Average Aspect Ratio=", round (avg_ar, 2), get_classificaton (avg_ar)) #In kết quả phân loại

#vẽ các hình ảnh (sử dụng matplotlib)
imgs_row = 2
imgs_col = 3
plt.subplot (imgs_row,imgs_col,1), plt.imshow (img,'gray')
plt.title("Original image") # (Ảnh gốc)
plt.subplot (imgs_row,imgs_col,2), plt.imshow (binary, 'gray')
plt.title("Binary image") # (Ảnh nhị phân)
plt.subplot (imgs_row,imgs_col,3), plt.imshow (dst,'gray')
plt.title("Filtered image") # (Ảnh đã lọc)
plt.subplot (imgs_row,imgs_col,4), plt.imshow (erosion, 'gray')
plt.title("Eroded image") # (Ảnh co lại)
plt.subplot (imgs_row,imgs_col,5), plt.imshow (dilation, 'gray')
plt.title("Dialated image") # (Ảnh giãn nở)
plt.subplot (imgs_row,imgs_col,6), plt.imshow (edges, 'gray')
plt.title("Edge detect") # (Phát hiện cạnh)

plt.show () #Hiển thị cửa sổ biểu đồ
```

Đầu ra (trên cửa sổ console):

```
>>>
RESTART: E:\studies\Projects\BOOK\Rice\rice-quality-analysis-master\code.py
Starting
No. of rice grains= 30
Average Aspect Ratio= 2.3 (Medium)
```



(Hình 5.3 Đầu ra của Gạo sử dụng Matplotlib) *(Hình ảnh này hiển thị 6 bước xử lý ảnh: Ảnh gốc, Ảnh nhị phân, Ảnh đã lọc, Ảnh co lại, Ảnh giãn nở, và Ảnh phát hiện cạnh)*

Như chúng ta có thể thấy trong Hình 5.3, gạo đã được phát hiện và việc phân loại đã được thực hiện bằng OpenCV, được biểu diễn bằng thư viện matplotlib (thư viện này được sử dụng để trực quan hóa đầu ra sau khi phân loại gạo).

Tóm tắt

Thị giác máy tính được coi là **đôi mắt của trí tuệ nhân tạo**, có thể giúp xử lý bất kỳ video hoặc hình ảnh nào và hiểu chúng để tạo ra dữ liệu chính xác. Trong chương này, chúng ta đã thấy tầm quan trọng của thị giác máy tính và cách nó có thể hữu ích trong nông nghiệp để nhận dạng, và với hai ví dụ nhỏ (đọc ảnh và phân loại gạo), chúng ta cũng đã minh họa việc sử dụng thị giác máy tính.

Chương 6: Hệ Chuyên Gia Dựa Trên Tri Thức

Trong chương này, chúng ta sẽ tìm hiểu về Hệ chuyên gia, một hệ thống có thể hướng dẫn và hỗ trợ một cá nhân khi được cung cấp đủ Tri thức để làm điều đó. Ở đây, chúng ta sẽ thảo luận về hệ chuyên gia và kiến trúc của nó, cùng với các bước phù hợp để xây dựng một hệ chuyên gia trong bất kỳ lĩnh vực nào bằng các công cụ khác nhau có sẵn. Đối với hệ chuyên gia, điều quan trọng nhất là Kỹ thuật Tri thức (Knowledge Engineering), tức là việc thu thập Tri thức từ một chuyên gia và "kỹ thuật hóa" nó thành một dạng mà máy móc có thể hiểu được.

Giới thiệu

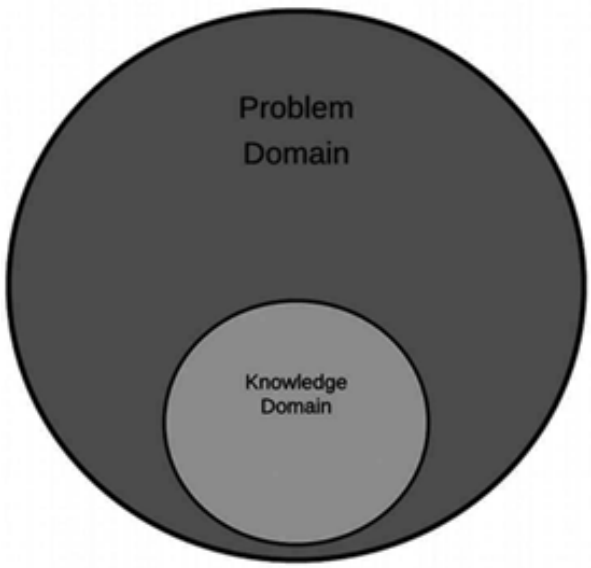
Hệ Chuyên Gia (ES) là một trong những lĩnh vực nghiên cứu tiên phong của AI (Trí tuệ Nhân tạo). Nó được đề xuất bởi các nhà nghiên cứu tại khoa khoa học máy tính của Đại học Stanford. Hệ chuyên gia là một hệ thống máy tính mô phỏng các kỹ năng ra quyết định của một chuyên gia con người.

Các hệ chuyên gia được phát triển bằng cách suy luận thông qua các cấu trúc thông tin, đại diện cho các quy tắc, thay vì thông qua mã thủ tục truyền thống, nhằm vượt qua các vấn đề phức tạp. Các Chương trình Chuyên gia sử dụng kiến thức chuyên môn một cách có hệ thống để giải quyết các vấn đề của chuyên gia con người.

Chuyên gia là một cá nhân có kinh nghiệm trong một lĩnh vực cụ thể. Chuyên gia có kiến thức chuyên môn hoặc các chứng chỉ đặc biệt mà hầu hết mọi người không học hoặc không quan tâm đến. Một chuyên gia có thể giải quyết các vấn đề hiệu quả hơn so với người bình thường. Hệ Chuyên gia thực sự mô phỏng tất cả những phẩm chất này của chuyên gia và giải quyết vấn đề một cách tốt hơn và nhanh chóng.

Ngày nay, Hệ Chuyên gia (ES) được sử dụng trong các lĩnh vực tài chính, nghiên cứu, kỹ thuật, công nghệ, y học và nhiều lĩnh vực khác nơi có một phạm vi vấn đề được xác định rõ ràng. Nguyên tắc cơ bản là một phương pháp chuyên gia cũng phải xác định các bước để minh họa lý do tại sao một vấn đề có thể được giải quyết. Trừ khi một cá nhân có thể biện minh cho lý luận của mình cho Hệ chuyên gia.

Hệ Chuyên gia luôn hoạt động dựa trên vấn đề của nó. Trong thiết kế Hệ chuyên gia, chúng ta luôn xác định miền vấn đề (problem domain) và sau đó chúng ta sẽ thu thập kiến thức liên quan đến nó. Như bạn có thể thấy trong hình bên dưới, miền tri thức (knowledge domain) của một Hệ chuyên gia luôn là một tập hợp con của miền vấn đề, như được biểu thị trong Hình 6.1. Tri thức được xây dựng luôn được thu thập từ sách, tạp chí, các bài báo hoặc từ các chuyên gia dựa trên kinh nghiệm của họ trong lĩnh vực cụ thể.



(Trang 2: Hình 6.1 Miền tri thức của Hệ Chuyên gia, cho thấy "Miền Tri thức" là một vòng tròn nhỏ hơn nằm trong "Miền Vấn đề")

Tất cả tri thức được thu thập sẽ dựa trên vấn đề đã xác định và nó sẽ cụ thể và dựa trên các sự thật (facts).

Đặc điểm của Hệ Chuyên gia

Tri thức là khía cạnh quan trọng nhất của bất kỳ chương trình chuyên gia nào. Sức mạnh của hệ chuyên gia nằm ở nhận thức chất lượng cao về các lĩnh vực nhiệm vụ. Trong các hệ chuyên gia, tri thức được tách biệt khỏi quá trình xử lý của nó, tức là cơ sở tri thức (knowledge base) và bộ máy suy luận (inference engine) được tách riêng. Để lưu trữ thông tin này, một hệ thống tiêu chuẩn là sự kết hợp giữa thông tin và cấu trúc điều khiển. Sự kết hợp này tạo ra các vấn đề trong việc hiểu và phân tích mã hệ thống, vì bất kỳ thay đổi nào trong mã đều ảnh hưởng đến thông tin và quá trình xử lý. Phương pháp chuyên gia bao gồm một cơ sở tri thức và một tập hợp các quy tắc để áp dụng cơ sở tri thức đó cho từng tình huống riêng biệt được xác định trong chương trình. Các hệ chuyên gia toàn diện có thể được cải thiện bằng cách bổ sung vào cơ sở tri thức hoặc bộ sưu tập các quy tắc. Hệ chuyên gia có thể được phát triển từ đầu hoặc được thiết kế bằng một phần phát triển phần mềm gọi là "bộ công cụ" (kit) hoặc vỏ (shell). Vỏ là một môi trường phát triển đầy đủ để tạo và duy trì ứng dụng dựa trên tri thức. Nó cung cấp một cách tiếp cận từng bước và một giao diện thân thiện với người dùng (tốt nhất là vậy) cho kỹ sư tri thức, người có thể trực tiếp thu hút các chuyên gia miền vào việc cấu trúc và mã hóa thông tin.

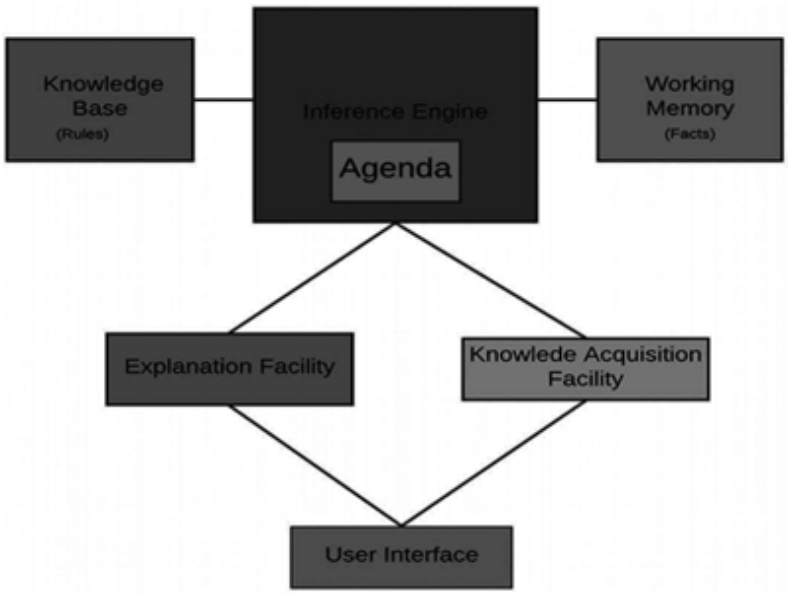
Như đã thấy, hệ chuyên gia có thể được phát triển và chúng có một số đặc điểm chung là:

- **Hiệu suất cao:** Hệ thống phải có khả năng phản hồi ở mức độ năng lực rất cao hoặc tốt hơn bất kỳ chuyên gia nào trong lĩnh vực đó. tức là, chất lượng lời khuyên do hệ thống đưa ra phải cao và tốt hơn so với chuyên gia.

- **Thời gian phản hồi phù hợp:** Hệ chuyên gia phải thực hiện hoặc phản hồi một tình huống nhanh hơn so với chuyên gia và với lời khuyên hoặc giải pháp chính xác và tốt hơn cho truy vấn.
- **Độ tin cậy cao:** Hệ chuyên gia không bị trễ (lag) và lỗi (crash), nó phải hoạt động tốt trong mọi tình huống.
- **Dễ hiểu:** Hệ chuyên gia phải có khả năng giải thích cách thức ra quyết định của mình đối với một truy vấn để người dùng có thể hiểu được.
- **Tính linh hoạt:** Hệ Chuyên gia chứa một lượng lớn dữ liệu dưới dạng tri thức liên quan đến một lĩnh vực nhất định và việc thêm, thay đổi và xóa bất kỳ tri thức nào vào hệ thống để cải thiện nó phải rất dễ dàng.

Kiến trúc của một Hệ Chuyên gia

Một công cụ hệ chuyên gia, hay còn gọi là vỏ (shell), bao gồm các thành phần thiết yếu của hệ chuyên gia. Cơ sở tri thức và Bộ máy suy luận là các thành phần chính của hệ chuyên gia. và các Yếu tố khác của Hệ Chuyên gia có thể thấy trong Hình 6.2 là:



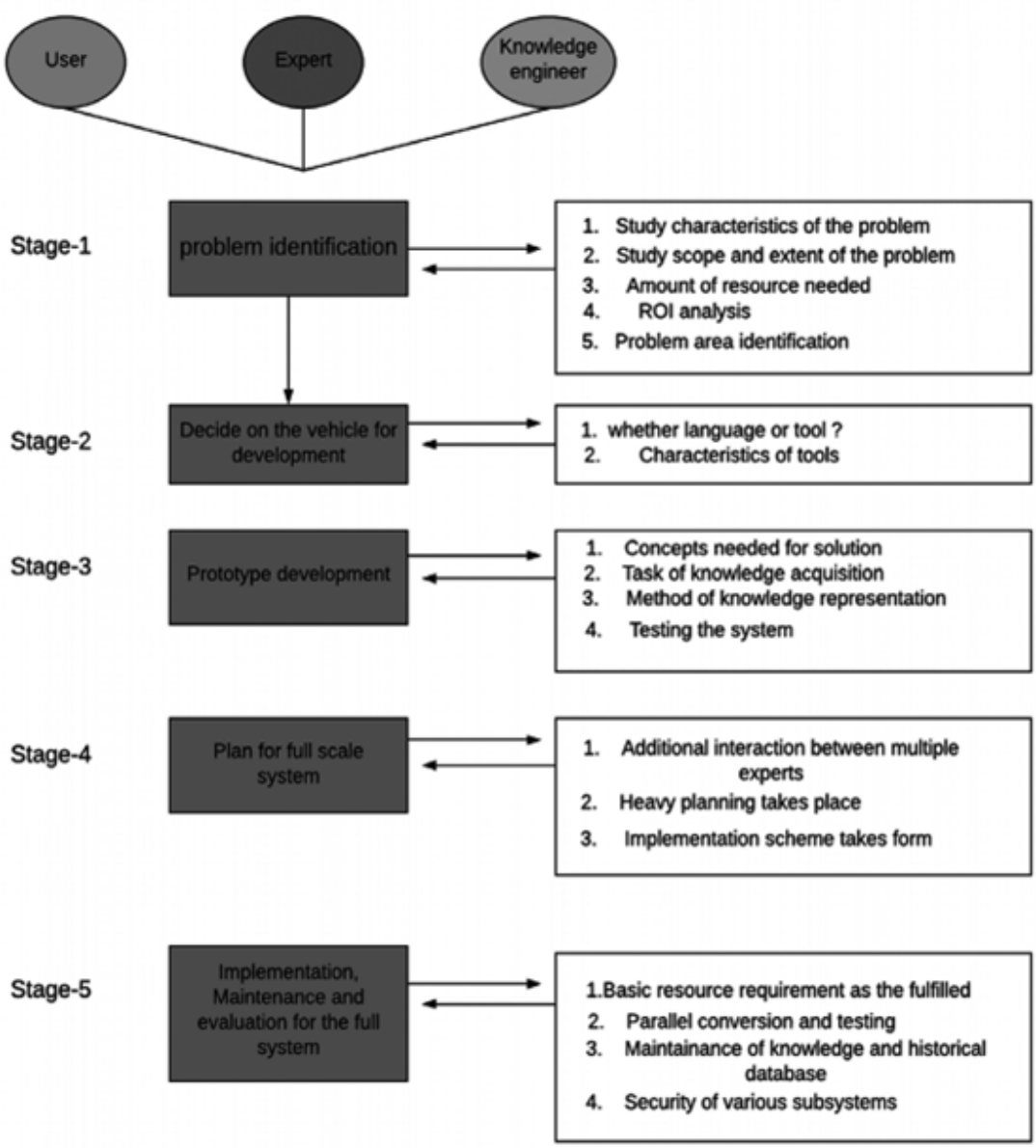
(Hình 6.2: Kiến trúc của Hệ Chuyên gia)

- **Cơ sở Tri thức (Knowledge Base):** Cơ sở tri thức bao gồm các sự thật (facts) cần được học, công thức hóa và giải quyết. Nó là một kho lưu trữ thông tin miền được thu thập từ chuyên gia con người thông qua mô-đun học tập. Nó sử dụng các quy tắc (rules), khung (frames), logic, mạng ngữ nghĩa (semantic network), v.v. để phản ánh đầu ra thông tin. Cơ sở tri thức của hệ chuyên gia cung cấp tri thức thực nghiệm (empirical) và tri thức kinh nghiệm (heuristic). Tri thức thực (True knowledge) là tri thức chung về miền nhiệm vụ, thường có trong sách giáo khoa hoặc tạp chí. Thông tin kinh nghiệm (Heuristic information) là một nhận thức về thành công ít nghiêm ngặt hơn, mang tính trải nghiệm, phán đoán và thường mang

tính cá nhân. Nó bao gồm kiến thức về các thực tiễn tốt, phán đoán tốt và logic khả thi trong lĩnh vực đó.

- **Bộ máy suy luận (Inference Engine):** Đây là bộ não của Hệ chuyên gia. Nó sử dụng hệ thống điều khiển và cung cấp một kỹ thuật suy luận. Nó phục vụ như một trình thông dịch (interpreter) phân tích và hoạt động dựa trên các quy tắc. Nó được sử dụng để phù hợp với lịch sử của các phản hồi và các quy tắc kích hoạt (firing rules) nhận được bởi người dùng. Nhiệm vụ chính của bộ máy suy luận là vạch ra con đường dẫn đến kết luận thông qua một tập hợp các quy tắc. Hai phương pháp được sử dụng ở đây, tức là, suy luận tiến (forward chaining) và suy luận lùi (rear chaining).
- **Bộ nhớ làm việc (Working Memory)**
- **Chương trình nghị sự (Agenda):** Nó chứa danh sách các nhiệm vụ hoặc truy vấn cần được thực hiện bởi hệ chuyên gia.
- **Tiện ích giải thích (Explanation facility):** Đây là một hệ thống con giải thích hành vi của hệ thống. Mô tả có thể khác nhau, từ cách đi đến các giải pháp cuối cùng hoặc trung gian đến việc giải thích sự cần thiết của các chi tiết bổ sung. Ở đây, người dùng muốn hỏi các câu hỏi đơn giản như tại sao và làm thế nào thông tin thiết bị được chia sẻ với người dùng và nó hoạt động như một gia sư.
- **Tiện ích Thu nhận Tri thức (Knowledge Acquisition Facility):** Việc thu nhận tri thức là một quá trình xây dựng, chuyển tiếp và chuyển đổi thành một hệ thống máy tính để tạo hoặc mở rộng nền tảng tri thức về chuyên môn giải quyết vấn đề từ các chuyên gia và/hoặc các nguồn thông tin được ghi lại. Nó là một hệ thống con chuyên gia để phát triển các cơ sở thông tin. Để thu nhận thông tin, các kỹ thuật phân tích quy trình, phỏng vấn và quan sát được sử dụng.
- **Giao diện Người dùng (User Interface):** Đây là một thiết bị tương tác với người dùng. Nó cung cấp các tiện ích cho cuộc hội thoại thân thiện với người dùng, chẳng hạn như menu, giao diện đồ họa (GUI), v.v. Giao diện Người dùng chịu trách nhiệm dịch các quy tắc từ biểu diễn nội bộ của nó (mà người dùng không hiểu) sang các dạng mà người dùng có thể hiểu được.

Kỹ thuật tri thức (Knowledge engineering) được biết đến để phát triển Hệ chuyên gia. Các chuyên gia miền, người dùng, kỹ sư tri thức và nhân viên bảo trì hệ thống là những người liên quan đến việc phát triển hệ chuyên gia. Các chuyên gia lĩnh vực (Field specialists) có chuyên môn, đánh giá, thực tiễn và chiến lược độc đáo để đưa ra hướng dẫn và giải quyết vấn đề. Nó cung cấp thông tin về sự thành công của công việc. Kỹ sư tri thức tham gia vào quá trình tạo ra bộ máy suy luận, khung cơ sở tri thức và giao diện người dùng. Kỹ sư Tri thức và chuyên gia về thông tin sẽ thấy trước nhu cầu của người tiêu dùng trong việc tạo ra một chương trình.



(Trang 4: Hình 6.3 Vòng đời Hệ Chuyên gia, mô tả 5 giai đoạn)

Việc phát triển một hệ chuyên gia bao gồm năm giai đoạn chính như thấy trong Hình 6.3. Mỗi giai đoạn có các đặc điểm riêng và liên kết với các giai đoạn khác.

Giai đoạn 1: Xác định vấn đề (Problem Identification): Trong bước này, chuyên gia và Kỹ sư Tri thức giao tiếp để phát hiện vấn đề. Các vấn đề chính đã thảo luận trước đây được điều tra để tìm ra các đặc điểm của vấn đề. Mức độ và bản chất [của vấn đề] được xem xét. Số lượng tài nguyên có sẵn, chẳng hạn như con người, tài nguyên máy tính, tài chính, v.v. [được xem xét]. Phân tích ROI (Tỷ suất hoàn vốn) được tiến hành. Các lĩnh vực vấn đề có thể gây ra nhiều rắc rối được xác định và thiết kế tổng thể cũng như giải pháp ý tưởng (conceptual solution) cho vấn đề đó được thiết lập.

Giai đoạn 2: Quyết định Phương thức Phát triển (Deciding Mode of Development): Giai đoạn ngay sau khi vấn đề được tìm thấy là xác định mô hình nào sẽ được xây dựng. Kỹ thuật viên phần mềm có thể sử dụng một ngôn ngữ lập trình như PROLOG và LISP hoặc một số ngôn ngữ hiện đại để phát triển khung (framework) hoặc sử dụng một vỏ phát triển (development shell) từ đầu. Trong giai đoạn này, các vỏ và công cụ khác nhau được mô tả và phân tích về sự phù hợp của chúng. Các công cụ có đặc tính phù hợp với vấn đề sẽ được đánh giá kỹ lưỡng.

Giai đoạn 3: Phát triển Nguyên mẫu (Prototype Development): Sau đây là các hoạt động bắt buộc trước khi tạo một nguyên mẫu:

- Quyết định giải pháp sẽ phải tạo ra những khái niệm (concepts) nào. Mức độ chi tiết của thông tin (granularity - độ mịn) là một yếu tố thiết yếu cần được tính toán. Việc phát triển hệ thống bắt đầu với độ mịn thô (coarse granularity) và tiến dần đến độ mịn cao (fine granularity).
- Sau đó bắt đầu quá trình thu thập thông tin. Kỹ sư tri thức và chuyên gia lĩnh vực thường xuyên trao đổi, và thông tin miền được trích xuất.
- Kỹ sư tri thức xác định hình thức biểu diễn [tri thức] cho đến khi kiến thức chuyên môn được thu nhận.
- Một hình ảnh khái niệm (conceptual image) về việc biểu diễn thông tin đáng lẽ đã xuất hiện trong quá trình xác định [vấn đề]. Quan điểm này được thực thi hoặc cập nhật trong quy trình này.
- Một nguyên mẫu được phát triển khi sơ đồ biểu diễn thông tin và (chính) thông tin đó có sẵn.
- Mỗi nguyên mẫu được đánh giá cho các vấn đề khác nhau và nguyên mẫu đó được xem xét lại. Phương pháp này dẫn đến tri thức có độ mịn cao, được mã hóa (encrypted) một cách hiệu quả trong cơ sở thông tin.

Giai đoạn 4: Lập kế hoạch Hệ thống Đầy đủ (Full System planning): Sự thành công của nguyên mẫu tạo động lực cho [việc xây dựng] hệ thống quy mô đầy đủ. Trong thiết kế nguyên mẫu, khu vực của vấn đề có thể được áp dụng một cách tương đối dễ dàng được chọn trước tiên. Trong quá trình triển khai đầy đủ, việc tạo ra một hệ thống con được giao cho các trưởng nhóm và các lịch trình được lập ra. Sơ đồ Gantt, PERT hoặc CPM sẽ được sử dụng.

Giai đoạn 5: Triển khai, Đánh giá và Bảo trì (Implementation, Evaluation and Maintenance): Đây là quy trình cuối cùng trong vòng đời của một hệ chuyên gia. Tại địa điểm [triển khai], toàn bộ hệ thống quy mô đầy đủ được tạo ra. Các tiêu chí cơ bản cho dịch vụ được đáp ứng tại địa điểm và các chiến lược chuyển đổi đồng thời (simultaneous conversion) và kiểm thử được áp dụng. Thiết bị cuối cùng được kiểm tra nghiêm ngặt và cuối cùng được chuyển giao cho khách hàng.

Đánh giá trong bất kỳ hệ thống AI nào cũng là một nhiệm vụ đầy thách thức. Như đã mô tả, các giải pháp cho các vấng đề AI chỉ ở mức "thỏa đáng" (satisfactory). Vì các yêu cầu

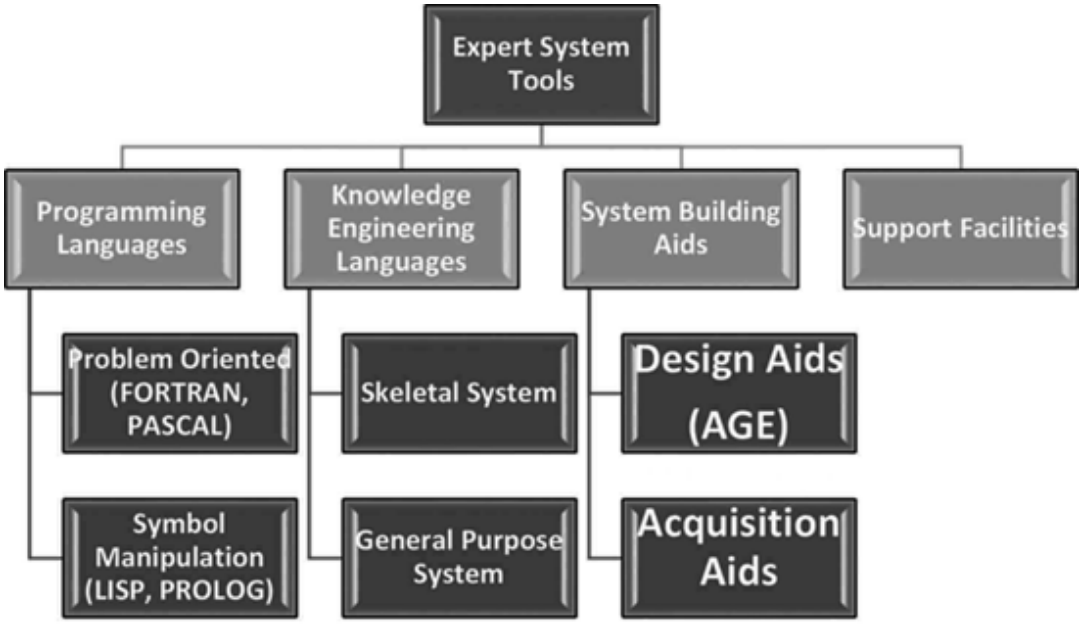
đánh giá không có sẵn, việc đánh giá rất khó khăn. Vì các yêu cầu đánh giá không có sẵn, việc đánh giá rất khó khăn. Những gì có thể làm là cung cấp cho chương trình và chuyên gia con người một loạt vấn đề và so sánh kết quả.

Việc bảo trì hệ thống đòi hỏi phải thay đổi cơ sở tri thức vì không có thông tin, môi trường hoặc vấn đề nào là cố định. Điều quan trọng là phải duy trì các hồ sơ lịch sử và theo dõi các sửa đổi nhỏ được thực hiện đối với Bộ máy suy luận. Bảo trì cũng bao gồm cả bảo mật.

Công cụ để Phát triển Hệ chuyên gia

Có một số công cụ có sẵn để phát triển một hệ chuyên gia như được biểu thị trong Hình 6.4. Các công cụ này được chia thành bốn loại:

(a) Ngôn ngữ lập trình (Programming Languages)
(b) Ngôn ngữ Kỹ thuật Tri thức (Knowledge Engineering Languages)
(c) Hỗ trợ Xây dựng Hệ thống (System Building Aid)
(d) Tiện ích Hỗ trợ (Support facilities)



(Hình 6.4 Công cụ của Hệ chuyên gia)

(a) Ngôn ngữ lập trình: Có một số ngôn ngữ nhất định có sẵn và có thể được sử dụng để phát triển một hệ chuyên gia. Nó luôn dựa trên loại vấn đề chúng ta có, như:

- **i. Ngôn ngữ Định hướng Vấn đề (Problem-Oriented languages):** Chẳng hạn như Fortran và Pascal. Chúng được thiết kế cho một loại vấn đề cụ thể. Fortran có một tính năng tiện lợi để thực hiện các phép tính đại số.

- **ii. Ngôn ngữ Thao tác Ký hiệu (Symbolic Manipulation Languages):** Chẳng hạn như LISP và PROLOG. LISP có cơ chế có thể thao tác ký hiệu có sẵn dưới dạng cấu trúc danh sách.

(b) Ngôn ngữ Kỹ thuật Tri thức: Ngôn ngữ kỹ thuật tri thức là một phương pháp tinh vi để xây dựng các hệ thống chuyên gia, bao gồm một ngôn ngữ xây dựng hệ chuyên gia trong một môi trường hỗ trợ rộng lớn.

- **i. Hệ thống Khung sườn (Skeletal System):** Một ngôn ngữ khung sườn để tạo thông tin là một hệ chuyên gia đơn giản hóa. Các miền (Domains) được loại bỏ khỏi chương trình chuyên gia, chỉ để lại Bộ máy suy luận và các tiện ích hỗ trợ. Ví dụ, MYCIN được rút gọn thành EMYCIN, vốn dễ dàng và nhanh chóng.
- **ii. Hệ thống Đa mục đích (General Purpose System):** Một ngôn ngữ Kỹ thuật Tri thức có thể xử lý nhiều lĩnh vực vấn đề và các hình thức cho một mục đích chung. Điều này cung cấp nhiều quyền kiểm soát hơn đối với việc truy cập và tìm kiếm dữ liệu so với khung sườn (skeleton framework), nhưng khó sử dụng hơn nhiều. Nó khác nhau tùy theo tính linh hoạt và tính tổng quát. Hệ thống Khung sườn và Đa mục đích thuộc danh mục hệ thống nghiên cứu.

(c) Hỗ trợ Xây dựng Hệ thống: Bao gồm các chương trình giúp phát triển kiến thức chuyên môn của chuyên gia trong ngành và các chương trình giúp thiết kế phương pháp xây dựng Hệ Chuyên gia.

- **i. Hỗ trợ Thiết kế (Designing Aid):** Giúp thiết kế hệ chuyên gia. Ví dụ: Như AGE. Nó Hỗ trợ kỹ sư tri thức thiết kế và phát triển một Hệ chuyên gia. HEARSAY, lần đầu tiên được sử dụng vào giữa những năm 1970, diễn giải dữ liệu của đối phương qua liên lạc vô tuyến HEARSAY-III HANNIBAL. Hệ thống này sử dụng thông tin vị trí dữ liệu và các đặc điểm tín hiệu để phân loại các đơn vị tổ chức và thứ tự chiến đấu của chúng. Nó cung cấp cho người tiêu dùng một tập hợp các thành phần có thể được lắp ráp như các khối xây dựng (building blocks) để tạo thành các phần của một cấu trúc chuyên gia. Một cấu trúc hệ chuyên gia như suy luận tiến, suy luận lùi, hoặc kiến trúc bảng đen (blackboards) hỗ trợ từng thành phần, một tập hợp các hàm INTERLISP.
- **ii. Hỗ trợ Thu nhận Tri thức (Knowledge Acquisition Aid):** Giúp thu nhận Tri thức cho Hệ chuyên gia. Ví dụ: TEIRSIAS. Thiết kế hệ thống của nó giúp chuyển giao thông tin đến một cơ sở tri thức từ một chuyên gia miền. Chỉ được sử dụng để quản lý cơ sở dữ liệu. Chương trình học các quy tắc mới cho lĩnh vực vấn đề thông qua một tương tác cho phép người dùng xác định các quy tắc bằng một tập hợp con cụ thể của tiếng Anh. Hệ thống này phân tích các quy tắc, xác định mức độ chính xác và rõ ràng của các quy tắc này và cho phép người dùng sửa chúng.

(d) Tiện ích Hỗ trợ: Bao gồm các công cụ giúp lập trình - như các công cụ gỡ lỗi (debugging aids), trình chỉnh sửa Cơ sở Tri thức để tăng cường khả năng của một hệ thống đã hoàn thành và các gói phần mềm bổ sung.

Các thành phần chính của Tiện ích Hỗ trợ là:

- **i. Hỗ trợ gỡ lỗi (Debugging aids):** Được chia thành nhiều loại khác nhau như:
    - **Tiện ích theo dõi (Trace facilities):** hiển thị tên hoặc số hiệu quy tắc của danh sách các quy tắc đã được kích hoạt, các chương trình con (sub-routines) được gọi.
    - **Gói ngắt (Break packages):** cho phép người dùng cho biết nơi dừng thực thi chương trình để người dùng có thể dừng nó ngay trước khi lỗi xảy ra.
    - **Kiểm thử tự động (Automated testing):** cho phép người dùng kiểm tra một chương trình tự động trên một số bài kiểm chuẩn (benchmarks).
- **i. [lỗi đánh số trong văn bản gốc, nên là ii.] Tiện ích I/O (I/O facilities):** Được chia thành nhiều loại khác nhau như:
    - **Thu nhận Tri thức thời gian chạy (Run time Knowledge Acquisition):** bản thân công cụ cho phép người dùng đối thoại với hệ thống và tìm kiếm thông tin được yêu cầu trong cơ sở tri thức.
    - **Menu:** cung cấp cho người dùng các menu để chọn đầu vào. Menu: Không yêu cầu mã cụ thể.
    - **Khả năng truy cập hệ điều hành (Accessibility of the operating system):** cho phép ES theo dõi và quản lý công việc khác trong quá trình phục vụ.
- **(e) [lỗi đánh số trong văn bản gốc] Tiện ích giải thích (Explanation facilities):** Được chia thành nhiều loại khác nhau như:
    - **Hồi cứu (Retrospective):** giải thích cách hệ thống đạt được một trạng thái cụ thể. Cách hệ thống đi đến một kết luận.
    - **Giả định (Hypothetical):** giải thích điều gì sẽ xảy ra nếu một sự thật (fact) hoặc quy tắc (rule) nào đó khác đi.
    - **Phản thực tế (Counterfactual):** tại sao một kết luận mong đợi đã không đạt được.
- **(d) [lỗi đánh số trong văn bản gốc] Trình chỉnh sửa cơ sở tri thức (Knowledge base editors):** Được chia thành nhiều loại khác nhau như:
    - **Ghi chép tự động (Automatic book keeping):** Giúp theo dõi các thay đổi do người dùng thực hiện.
    - **Kiểm tra cú pháp (Syntax checking):** Kiểm tra xem các quy tắc có được nhập đúng định dạng và phù hợp với cấu trúc ngữ pháp của ngôn ngữ ES hay không.
    - **Kiểm tra tính nhất quán (Consistency checking):** Kiểm tra ngữ nghĩa của quy tắc được nhập để xem có bất kỳ xung đột nào với cơ sở tri thức hiện có hay không.
    - **Trích xuất Tri thức (Knowledge Extraction):** Giúp thêm các quy tắc mới hoặc sửa đổi các quy tắc.

Ưu và Nhược điểm của Hệ Chuyên gia (ES)

Nó có rất nhiều tính năng hấp dẫn.

- **Tăng tính sẵn có:** Nó tăng khả năng cung cấp chuyên môn của mình một cách đại trà trên bất kỳ phần cứng máy tính phù hợp nào.
- **Giảm chi phí:** Chi phí cung cấp chuyên môn cho mọi người dùng giảm đi rất nhiều.
- **Giảm nguy hiểm:** Nó có thể được sử dụng ở bất kỳ vị trí nguy hiểm hoặc xa xôi nào.
- **Tính vĩnh viễn:** Một khi hệ chuyên gia được phát triển, nó là vĩnh viễn, trong khi chuyên gia của lĩnh vực đó có thể mất đi nhưng hệ chuyên gia sẽ tồn tại mãi mãi.
- **Đa chuyên môn:** Tri thức của một số Hệ chuyên gia có thể được làm cho hoạt động đồng thời và thường xuyên trên một vấn đề nhất định cho đến khi nó được giải quyết vào bất kỳ thời điểm nào trong ngày hay đêm. Mức độ chuyên môn của một hệ chuyên gia cao hơn nhiều so với chuyên gia con người trong lĩnh vực đó.
- **Tăng độ tin cậy:** Hệ chuyên gia ổn định, không cảm tính và luôn hoàn thành phản hồi đúng hạn.
- **Khả năng giải thích:** Hệ Chuyên gia cung cấp tất cả mô tả về cách nó đi đến kết luận cho một truy vấn hoặc vấn đề. Trong cùng trường hợp đó, chuyên gia con người không thể lúc nào cũng cung cấp lời giải thích do mệt mỏi hoặc tình trạng sức khỏe.
- **Phản hồi nhanh:** Hệ chuyên gia có khả năng phản hồi nhanh và theo thời gian thực mọi lúc.
- **Gia sư thông minh:** Hệ chuyên gia có thể trở thành một gia sư cho sinh viên để nghiên cứu, áp dụng các chương trình cơ bản và giải thích kết quả của nó.
- **Cơ sở dữ liệu thông minh:** Nó có thể được sử dụng để truy cập cơ sở dữ liệu một cách thông minh.

Nhược điểm của Hệ Chuyên gia

- Hệ chuyên gia có **Tri thức nông (Shallow Knowledge)**. Nó không có hiểu biết sâu sắc về các khái niệm và mối quan hệ của chúng cho đến khi chúng ta cung cấp. Nó không có ý thức thông thường (common-sense).
- Nó là một **thế giới khép kín (closed world)** với kiến thức cụ thể.
- Hệ chuyên gia có thể **không có hoặc không chọn phương pháp thích hợp nhất** để giải quyết một vấn đề cụ thể.
- Một số **vấn đề dễ dàng trong hệ chuyên gia lại trở nên tốn kém về mặt tính toán**.

Tóm tắt

Hệ Chuyên gia đã trở thành một nhánh rất tiên phong của Trí tuệ Nhân tạo với một số ứng dụng trong nhiều lĩnh vực. Sau khi nghiên cứu về hệ chuyên gia, chúng ta có thể thấy hệ chuyên gia là gì, các hệ chuyên gia được xây dựng như thế nào và các công cụ để tạo ra một hệ chuyên gia là gì.

Hệ chuyên gia trong lĩnh vực nông nghiệp có thể giúp đỡ nông dân theo nhiều cách bằng cách hướng dẫn họ với kiến thức phù hợp để sử dụng công nghệ tiên tiến và các loại thảo mộc khác (other herbal) [Ghi chú: "herbal" ở đây có thể là lỗi chính tả, có thể ý là "methods" - phương pháp] để tăng sản lượng cây trồng và bảo vệ nó khỏi sâu bệnh.

PHẦN B: TRIỂN KHAI TRÍ TUỆ NHÂN TẠO

CHƯƠNG 7: CÁC CÔNG CỤ CHO TRÍ TUỆ NHÂN TẠO

Trong chương này, chúng ta sẽ nói về các công cụ cần thiết để thực hiện dự án về trí tuệ nhân tạo. Tất cả các công cụ được thảo luận dưới đây đều sử dụng ngôn ngữ lập trình Python để thực hiện bất kỳ dự án nào trong lĩnh vực tương ứng. Chúng ta đã thảo luận về cách mọi người có thể cài đặt và sử dụng các công cụ để xây dựng một dự án thời gian thực trong Trí tuệ Nhân tạo.

Sử dụng Python IDE

Để tải về và cài đặt Python, hãy truy cập trang web chính thức của Python https://www.python.org/downloads/ và chọn phiên bản của bạn. Chúng tôi đã chọn phiên bản Python theo yêu cầu; tất cả các mã trong cuốn sách này đều dựa trên phiên bản Python 3.6 như bạn có thể thấy trong Hình 7.1.



Fig. 7.1 Python Download

Khi quá trình tải về hoàn tất, hãy chạy tệp .exe để cài đặt Python. Bây giờ hãy nhấp vào Cài đặt ngay (Install Now) như trong Hình 7.2.

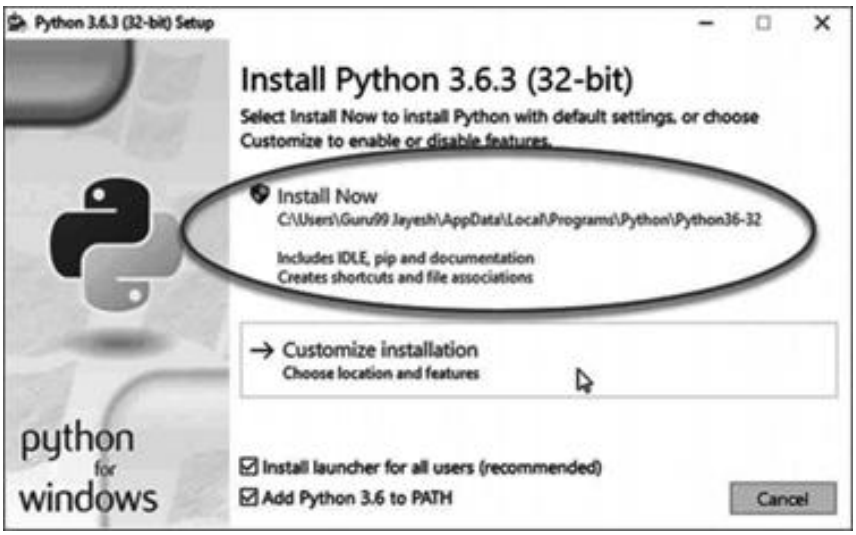


Fig. 7.2 Installation of Python

Bạn có thể thấy Python đang được cài đặt tại thời điểm này như trong Hình 7.3

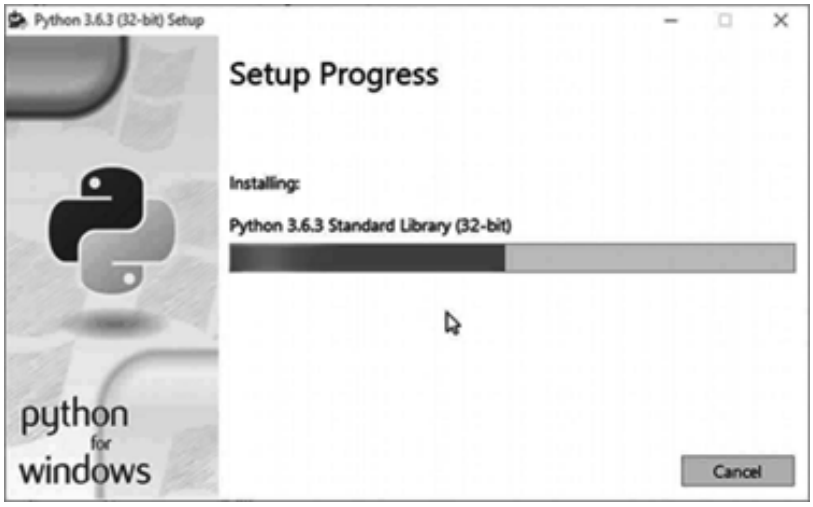


Fig. 7.3 Installation process

Khi hoàn tất, bạn có thể thấy một màn hình thông báo Cài đặt đã thành công (Setup was successful). Bây giờ hãy nhấp vào "Đóng" (Close) như trong Hình 7.4.



Fig. 7.4 Installation Complete

Sử dụng Anaconda

Trên Windows:

Đi tới liên kết sau: Anaconda.com/downloads .

Trang Tải về Anaconda sẽ trông giống như trong Hình 7.5:



Fig. 7.5 Installation Complete

Chọn bất kỳ hệ điều hành nào của bạn từ ba hệ điều hành được liệt kê như trong Hình 7.6.



Fig. 7.6 Anaconda Versions Select windows

chúng tôi đã chọn Windows. Tải về bản phát hành Python 3 gần đây nhất như trong Hình 7.7.



Fig. 7.7 Python version for Anaconda

Tại thời điểm viết bài, bản phát hành gần đây nhất mà tôi đang sử dụng là Phiên bản Python 3.6. Python 2.7 là phiên bản cũ. Đối với người giải quyết vấn đề, hãy chọn phiên bản Python 3.6. Nếu bạn không chắc máy tính của mình đang chạy phiên bản Windows 64-bit hay 32-bit, hãy chọn 64-bit vì Windows 64-bit là phổ biến nhất.

Gói tải về khá lớn nên có thể mất một lúc để Anaconda tải xong. Khi quá trình tải về hoàn tất, hãy mở và chạy trình cài đặt .exe. Khi bắt đầu cài đặt, bạn cần nhấp vào Tiếp theo (Next) để xác nhận cài đặt như trong Hình 7.8.



Fig. 7.8 Python version for Anaconda

Sau đó, đồng ý với giấy phép như trong Hình 7.9.



Fig. 7.9 License Agreement

Tại màn hình Tùy chọn Cài đặt Nâng cao (Advanced Installation Options), tôi khuyên bạn không nên chọn "Thêm Anaconda vào biến môi trường PATH của tôi" (Add Anaconda to my PATH environment variable) như trong Hình 7.10

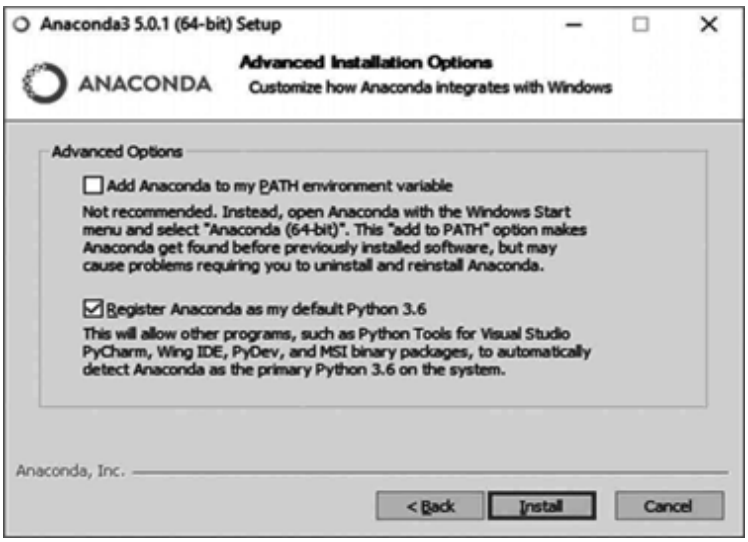


Fig. 7.10 Click Install for Anaconda

Sau khi cài đặt Anaconda hoàn tất, bạn có thể vào menu Bắt đầu (Start) của Windows và chọn Anaconda Prompt như trong Hình 7.11

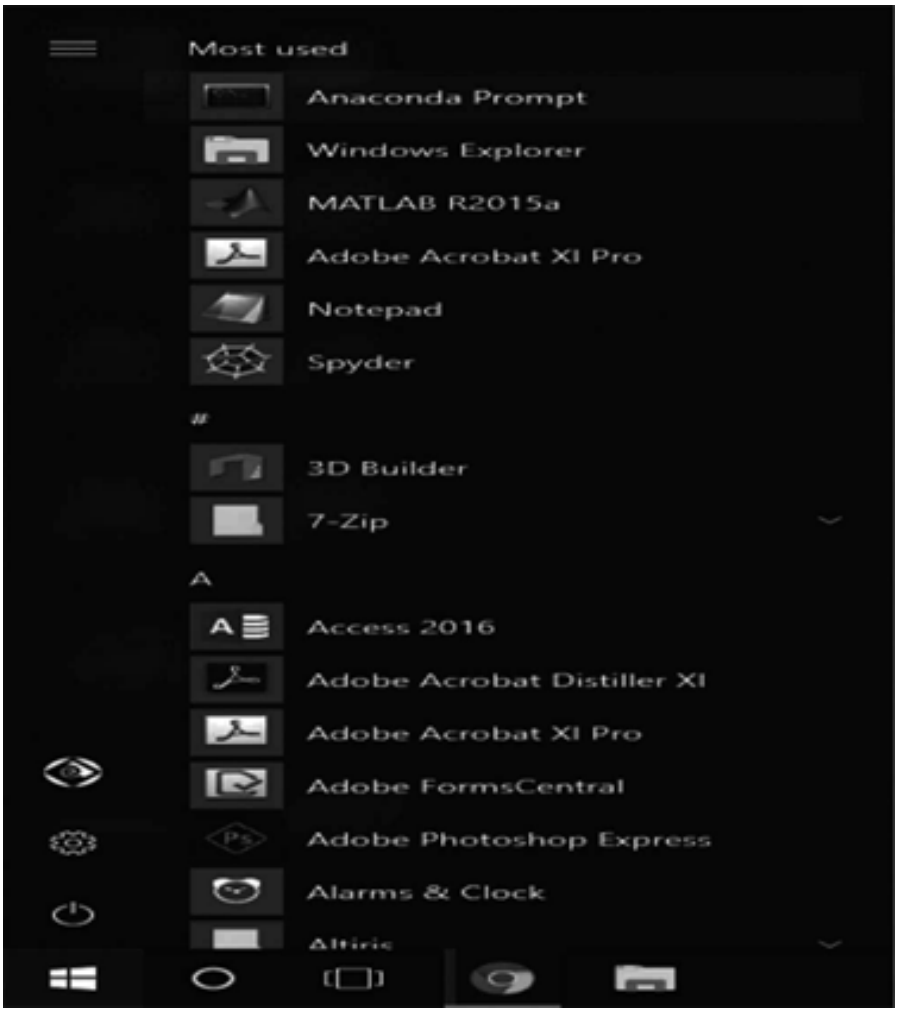


Fig. 7.11 Anaconda search in system

Thao tác này sẽ mở Anaconda Prompt. Anaconda là một bản phân phối Python và Anaconda Prompt là một trình bao dòng lệnh (command line shell) (một chương trình nơi bạn nhập lệnh thay vì sử dụng chuột). Màn hình đen và văn bản tạo nên Anaconda Prompt trông không có gì nhiều, nhưng nó thực sự hữu ích cho những người giải quyết vấn đề bằng Python.

Tại dấu nhắc Anaconda, gõ python và nhấn [Enter]. Lệnh python sẽ khởi động trình thông dịch Python (Python interpreter), còn được gọi là Python REPL (viết tắt của Read Evaluate Print Loop - Vòng lặp Đọc Đánh giá In). bạn sẽ có thể truy cập python từ cửa sổ dòng lệnh (terminal) của Anaconda prompt. Từ Anaconda prompt, bạn có thể cài đặt tất cả các thư viện của Python cần thiết để xây dựng các dự án khác nhau.

Trên MacOS

Truy cập trang tải về của Anaconda. Đi tới liên kết sau: Anaconda.com/downloads

Chọn MacOS và tải về trình cài đặt .pkg

Trong hộp hệ điều hành, chọn [MacOS]. Sau đó, tải về bản phân phối Python 3 gần đây nhất (tại thời điểm viết bài này, phiên bản gần đây nhất là Python 3.6) trình cài đặt đồ họa (graphical installer) bằng cách nhấp vào liên kết Tải về (Download). Python 2.7 là Python cũ. Đối với người giải quyết vấn đề, hãy chọn phiên bản Python 3 gần đây nhất như trong Hình 7.12.

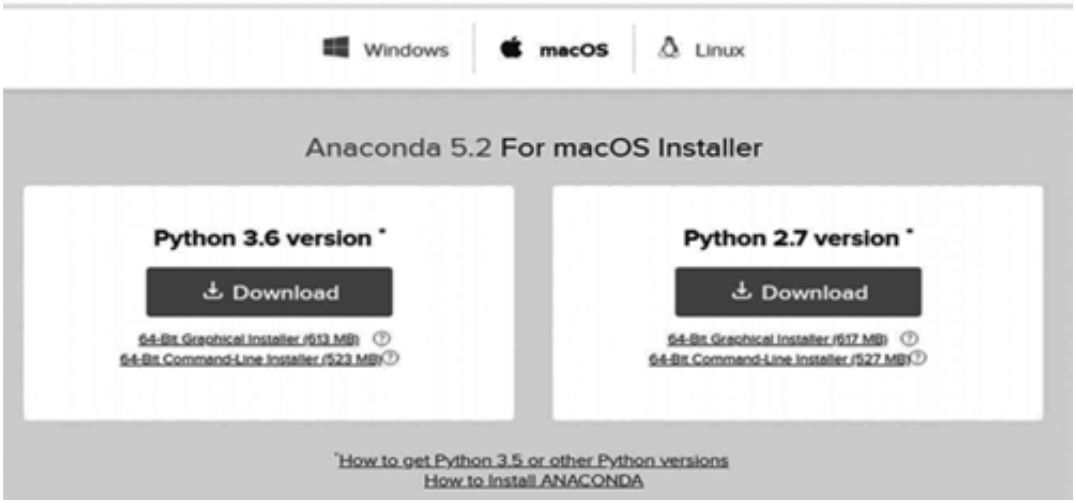


Fig. 7.12 Selecting ver- sion for Mac OS

Tiếp theo Điều hướng đến thư mục Tải về (Downloads) và nhấp đúp vào tệp trình cài đặt .pkg bạn vừa tải về. Có thể sẽ hữu ích nếu bạn sắp xếp nội dung của thư mục Tải về theo ngày để tìm tệp .pkg.

Làm theo hướng dẫn cài đặt. Bạn nên cài đặt Anaconda cho người dùng hiện tại và thêm Anaconda vào PATH của mình.

Sau khi Anaconda được cài đặt, bạn cần tải (load) các thay đổi vào biến môi trường PATH của mình trong phiên làm việc terminal hiện tại.

Mở Terminal của MacOS và gõ:

$ cd ~ $ source .bashrc

Mở Terminal của MacOS và gõ:

$ python

Bạn sẽ thấy nội dung tương tự như:

Python 3.6.3 | Anaconda Inc. |

Tại Python REPL (dấu nhắc >>> của Python), hãy thử:

>>> import this

Nếu bạn thấy “Zen of Python” (Thiền của Python), việc cài đặt đã thành công.

Thoát khỏi Python REPL bằng lệnh exit ().

Đảm bảo bao gồm cả dấu ngoặc đơn () sau lệnh exit.

>>> exit ()

Trên Linux

Đi tới liên kết sau: Anaconda.com/downloads . Trên trang tải về, chọn hệ điều hành Linux như trong Hình 7.13.



Fig. 7.13 Linux version

Trong hộp Phiên bản Python 3.6 (Python 3.6 Version*), nhấp chuột phải vào liên kết [Trình cài đặt 64-Bit (x86)] (64-Bit (x86) Installer). Chọn [sao chép địa chỉ liên kết] (copy link address) như trong Hình 7.14



Fig. 7.14 Copy the Downloading link of anaconda

Bây giờ liên kết trình cài đặt bash (tệp .sh) đã được lưu vào khay nhớ tạm (clipboard), hãy sử dụng wget để tải về tập lệnh cài đặt. Trong cửa sổ dòng lệnh (terminal), cd (di chuyển) vào thư mục chính (home) và tạo một thư mục mới có tên là tmp. cd vào tmp và sử dụng wget để tải về trình cài đặt. Mặc dù trình cài đặt là một tập lệnh bash, nó vẫn khá lớn và việc tải về sẽ không diễn ra ngay lập tức (Lưu ý liên kết bên dưới bao gồm <release>. bản phát hành cụ thể phụ thuộc vào thời điểm bạn tải về trình cài đặt).

$ cd ~ $ mkdir tmp $ cd tmp $ https://repo.continuum.io/archive/Anaconda3<release>.sh

Khi tập lệnh cài đặt bash đã được tải về, hãy chạy tập lệnh .sh để cài đặt Anaconda3.

Đảm bảo bạn đang ở trong thư mục chứa tập lệnh cài đặt đã tải về:

$ ls Anaconda3-5.2.0-Linux-x86 64.sh

Chạy tập lệnh cài đặt bằng bash.

$ bash Anaconda3-5.2.0-Linux-x86 64.sh

Chấp nhận Thỏa thuận Giấy phép và cho phép Anaconda được thêm vào PATH của bạn. Bằng cách thêm Anaconda vào PATH, bản phân phối Python của Anaconda sẽ được gọi khi bạn gõ $ python trong cửa sổ dòng lệnh.

Bây giờ Anaconda3 đã được cài đặt và Anaconda3 đã được thêm vào PATH của chúng ta, hãy source tệp .bashrc để tải biến môi trường PATH mới vào phiên làm việc terminal hiện tại. Lưu ý tệp .bashrc nằm trong thư mục chính. Bạn có thể thấy nó bằng lệnh $ ls -a.

$ cd $ source .bashrc

Để xác minh việc cài đặt hoàn tất, hãy mở Python từ dòng lệnh:

$ python

Nếu bạn thấy Python 3.6 từ Anaconda được liệt kê, việc cài đặt của bạn đã hoàn tất. Để thoát khỏi Python REPL, gõ:

>>> exit ()

Sử dụng Jupyter

Cách đơn giản nhất để cài đặt Jupyter notebooks là tải về và cài đặt bản phân phối Anaconda của Python. Bản phân phối Anaconda của Python đã bao gồm Jupyter notebook và không cần thêm bước cài đặt nào khác.

Dưới đây là các phương pháp bổ sung để cài đặt Jupyter notebooks nếu bạn không sử dụng bản phân phối Anaconda của Python.

Cài đặt Jupyter trên Windows bằng Anaconda Prompt

Để cài đặt Jupyter trên Windows, mở Anaconda Prompt và gõ:

> conda install jupyter

Gõ y (đồng ý) khi được nhắc. Khi Jupyter đã được cài đặt, gõ lệnh bên dưới vào Anaconda Prompt để mở trình duyệt tệp Jupyter notebook và bắt đầu sử dụng Jupyter notebooks.

> jupyter notebook

Cài đặt Jupyter trên MacOS

Để cài đặt Jupyter trên MacOS, mở terminal của MacOS và gõ:

$ conda install jupyter

Gõ y (đồng ý) khi được nhắc.

Nếu conda chưa được cài đặt, có thể cài đặt bản phân phối Anaconda của Python, thao tác này sẽ cài đặt conda để sử dụng trong terminal của MacOS.

Các sự cố có thể phát sinh trên MacOS khi sử dụng phiên bản Python do hệ thống MacOS cung cấp.

Các gói Python có thể không cài đặt đúng cách trên phiên bản Python của hệ thống.

Hơn nữa, các gói được cài đặt trên phiên bản Python của hệ thống có thể không chạy chính xác. Do đó, khuyến nghị người dùng MacOS nên cài đặt bản phân phối Anaconda của Python hoặc sử dụng homebrew để cài đặt một phiên bản Python riêng biệt không thuộc hệ thống.

Để cài đặt phiên bản Python không thuộc hệ thống bằng homebrew, hãy gõ lệnh sau vào terminal của MacOS. Xem tài liệu về homebrew tại https://brew.sh .

$ brew install Python

Sau khi homebrew cài đặt phiên bản Python không thuộc hệ thống, pip có thể được sử dụng để cài đặt Jupyter.

$ pip install jupyter

Cài đặt Jupyter trên Linux

Để cài đặt Jupyter trên Linux, mở cửa sổ dòng lệnh (terminal) và gõ:

$ conda install jupyter

Gõ y (đồng ý) khi được nhắc.

Ngoài ra, nếu bản phân phối Anaconda của Python không được cài đặt, người ta có thể sử dụng pip.

$ pip install jupyter

Sử dụng Google Collab

Google Collaboratory là một môi trường Jupyter notebook dựa trên đám mây trực tuyến miễn phí, cho phép chúng ta huấn luyện các mô hình học máy (machine learning) và học sâu (deep learning) trên CPU, GPU và TPU.

Collab cung cấp cho chúng ta 12 giờ thực thi liên tục. Sau đó, toàn bộ máy ảo sẽ bị xóa và chúng ta phải bắt đầu lại. Chúng ta có thể chạy nhiều phiên bản (instances) CPU, GPU và TPU đồng thời, nhưng tài nguyên của chúng ta được chia sẻ giữa các phiên bản này.

Bắt đầu với Google Collab:

Truy cập liên kết này và đăng ký hoặc đăng nhập bằng tài khoản Google của bạn https://colab.research.google.com/ như trong Hình 7.15

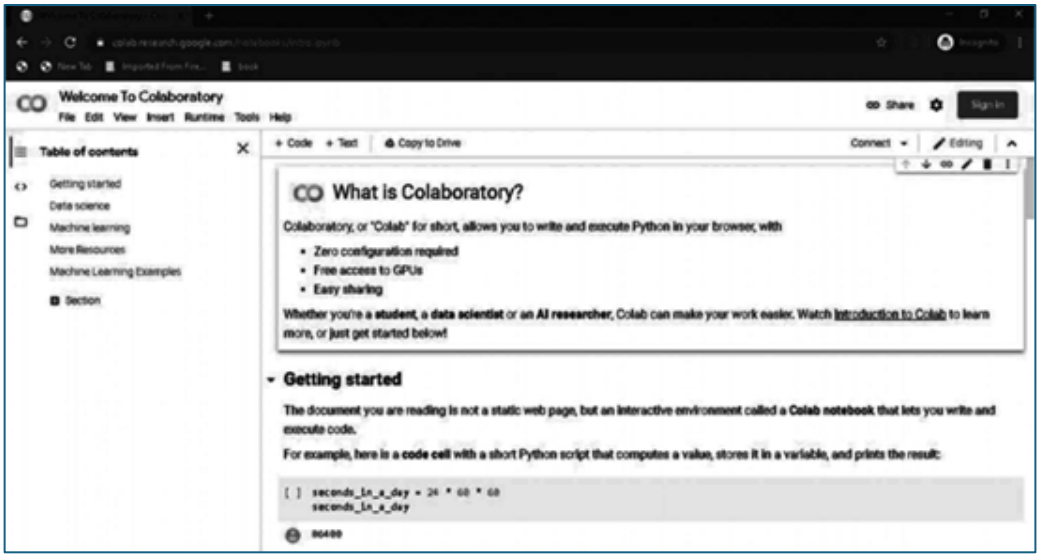


Fig. 7.15 Google Collaboratory

Sau khi đăng nhập, bạn sẽ thấy màn hình này như trong Hình 7.16

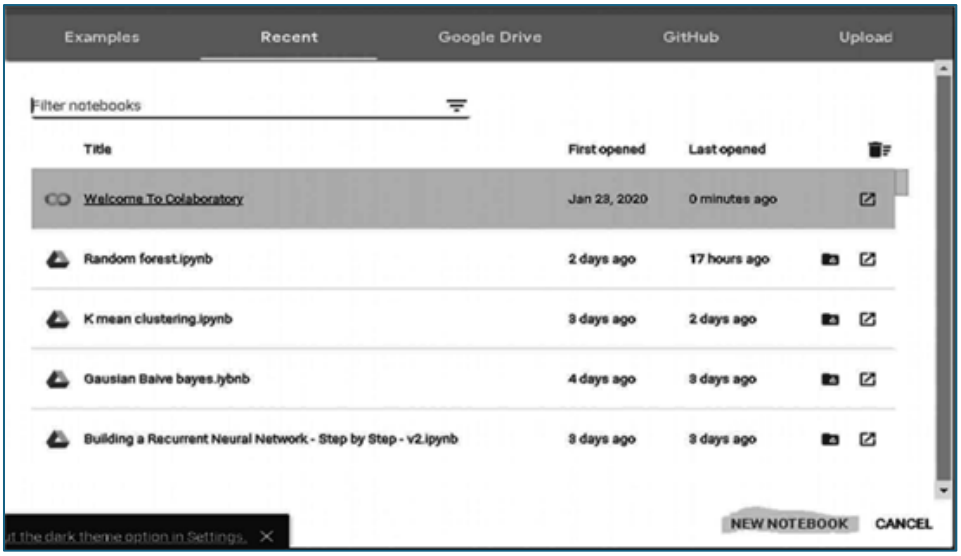


Fig. 7.16 Google Collaboratory Homepage

Nhấp vào nút SỔ TAY MỚI (NEW NOTEBOOK) để tạo một sổ tay Collab mới.

Bạn cũng có thể tải sổ tay cục bộ của mình lên Collab bằng cách nhấp vào nút tải lên (upload) như trong Hình 7.17:



Fig. 7.17 uploading Notebook

Bạn cũng có thể nhập sổ tay của mình từ Google Drive hoặc GitHub, nhưng chúng yêu cầu quá trình xác thực.

Như trong Hình 7.18, bạn có thể đổi tên sổ tay của mình bằng cách nhấp vào tên sổ tay và thay đổi nó thành bất cứ điều gì bạn muốn. Tôi thường đặt tên chúng theo dự án mà tôi đang thực hiện.

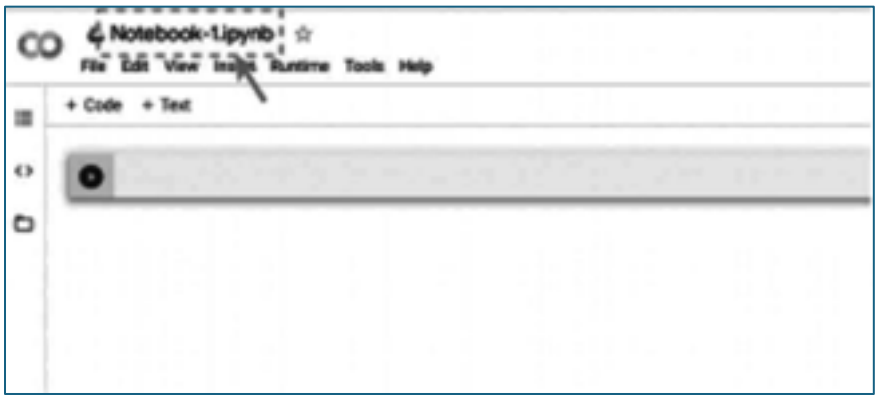


Fig. 7.18 Notebook Name change

Google Collab Runtimes - Chọn tùy chọn GPU hoặc TPU

Khả năng chọn các loại runtime (thời gian chạy) khác nhau là điều khiến Collab trở nên phổ biến và mạnh mẽ.

Dưới đây là các bước để thay đổi runtime của sổ tay của bạn:

Nhấp vào ‘Runtime’ (Thời gian chạy) trên menu trên cùng và chọn ‘Change Runtime Type’ (Thay đổi loại thời gian chạy) như trong Hình 7.19

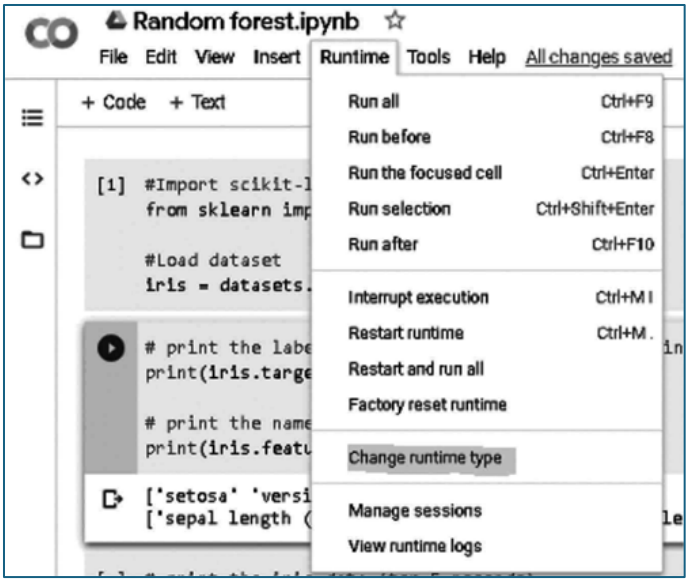
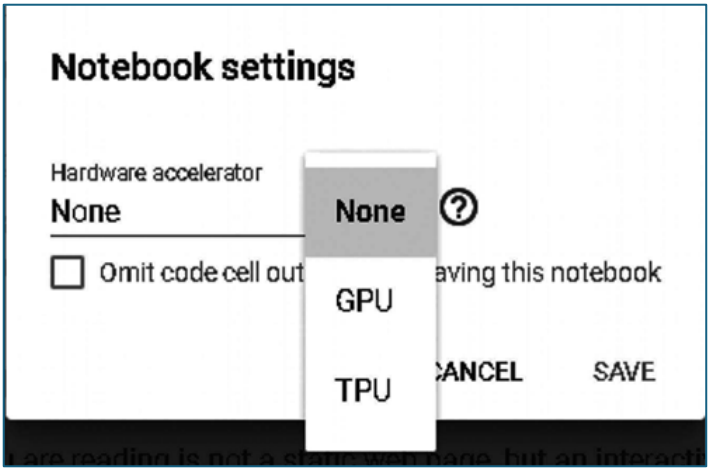


Fig. 7.19 Change Runtime Type

Tại đây bạn có thể thay đổi runtime theo nhu cầu của mình như trong Hình 7.20:



g. 7.20 select Runtime type

Tôi mong bạn tắt sổ tay của mình sau khi hoàn thành công việc để người khác có thể sử dụng các tài nguyên này vì chúng được chia sẻ giữa nhiều người dùng. Bạn có thể chấm dứt (terminate) sổ tay của mình như thế này trong Hình 7.21:

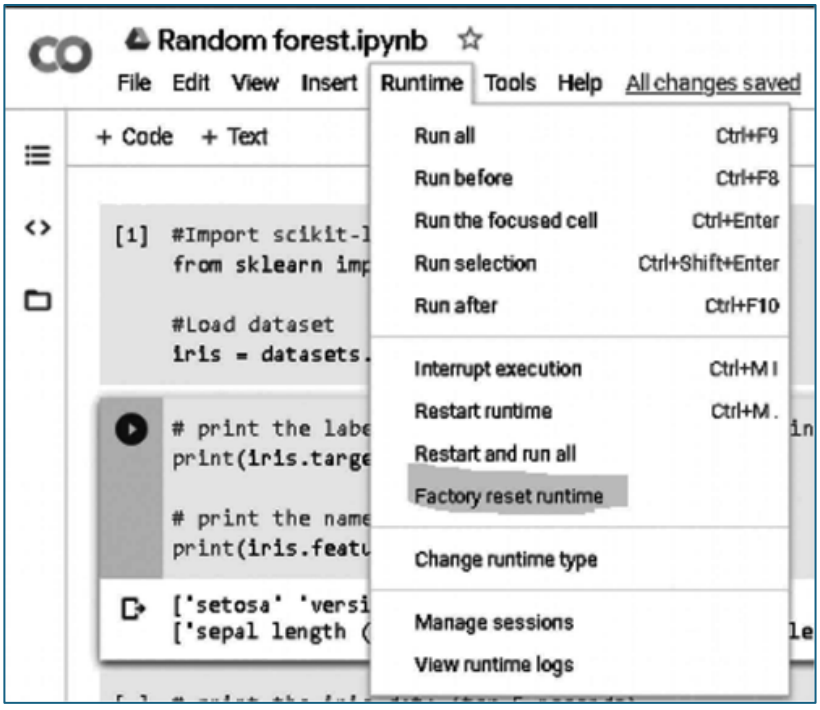


Fig. 7.21 Terminating Notebook

Sử dụng các lệnh Terminal trên Google Collab

Bạn có thể sử dụng ô (cell) của Collab để chạy các lệnh terminal. Hầu hết các thư viện phổ biến đều được cài đặt mặc định trên Google Collab. Đúng vậy, các thư viện Python như Pandas, NumPy, scikit-learn đều được cài đặt sẵn.

Nếu bạn muốn chạy một thư viện Python khác, bạn luôn có thể cài đặt nó bên trong sổ tay Collab của mình như sau:

!pip install tên_thư_viện

Khá dễ dàng, phải không? Mọi thứ đều tương tự như cách hoạt động trong một terminal thông thường. Chúng ta chỉ cần đặt dấu chấm than (!) trước khi viết mỗi lệnh như:

!ls

hoặc:

!pwd

Tải lên Tệp và Bộ dữ liệu:

Đây là một khía cạnh bắt buộc phải biết đối với bất kỳ nhà khoa học dữ liệu nào. Khả năng nhập bộ dữ liệu của bạn vào Collab là bước đầu tiên trong hành trình phân tích dữ liệu của bạn.

Cách tiếp cận cơ bản nhất là tải trực tiếp bộ dữ liệu của bạn lên Collab như trong Hình 7.22:

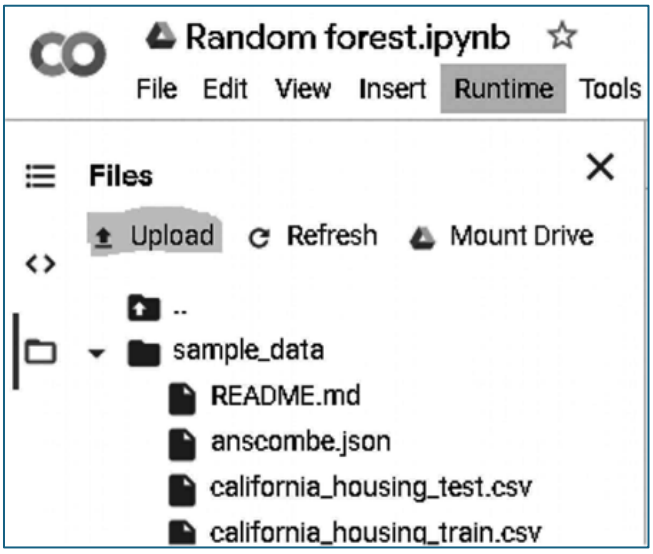


Fig. 7.22 Upload data

Bạn có thể sử dụng cách này nếu bộ dữ liệu hoặc tệp của bạn rất nhỏ vì tốc độ tải lên bằng phương pháp này khá thấp. Một cách tiếp cận khác mà tôi khuyên dùng là tải bộ dữ liệu của bạn lên Google Drive và gắn (mount) Drive của bạn trên Collab. Bạn có thể thực hiện việc này chỉ bằng một cú nhấp chuột như trong Hình 7.23:

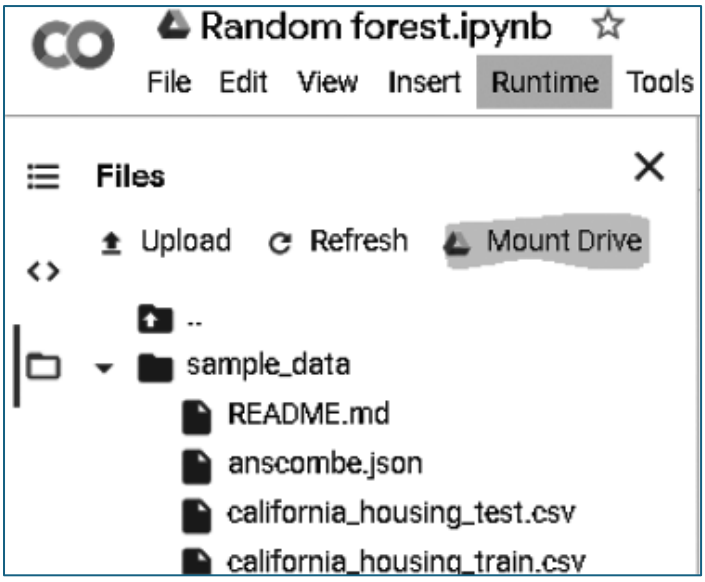


Fig. 7.23 Mounting Google drive

Bạn cũng có thể tải bộ dữ liệu của mình lên bất kỳ nền tảng nào khác và truy cập nó bằng liên kết của nó. Tôi có xu hướng sử dụng cách tiếp cận thứ hai thường xuyên hơn (khi có thể).

Lưu Sổ tay của bạn:

Tất cả các sổ tay trên Colab đều được lưu trữ trên Google Drive của bạn. Điều tuyệt vời nhất về Colab là sổ tay của bạn được tự động lưu sau một khoảng thời gian nhất định và bạn không bị mất tiến trình của mình.

Nếu muốn, từ Hình 7.24, bạn có thể xuất và lưu sổ tay của mình ở cả hai định dạng *.py và *.ipynb:

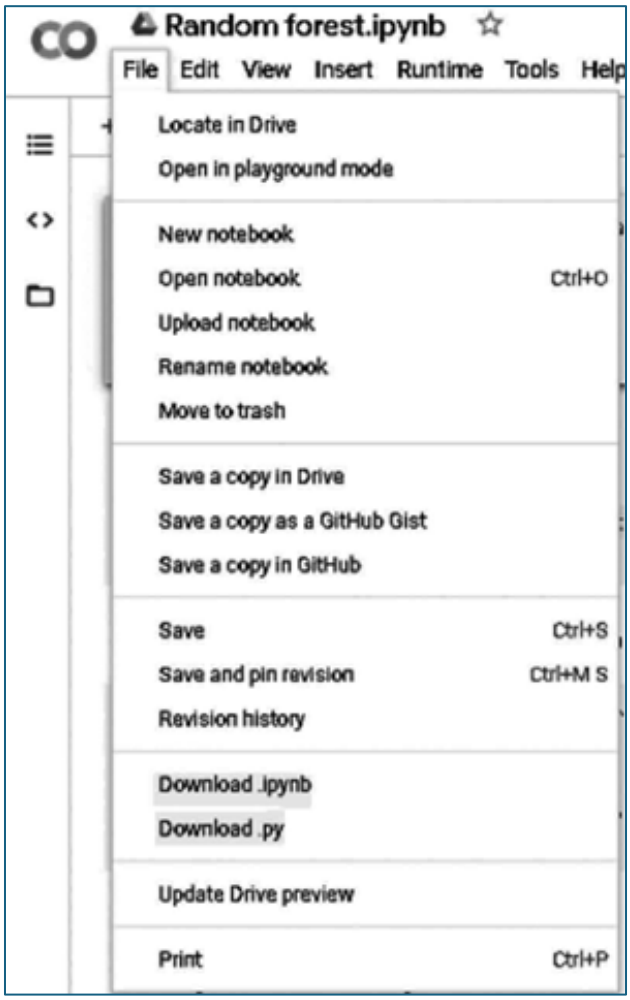


Fig. 7.24 Saving the Notebook

Không chỉ vậy, bạn cũng có thể lưu một bản sao của sổ tay trực tiếp như trong Hình 7.25 trên GitHub, hoặc bạn có thể tạo một GitHub Gist:

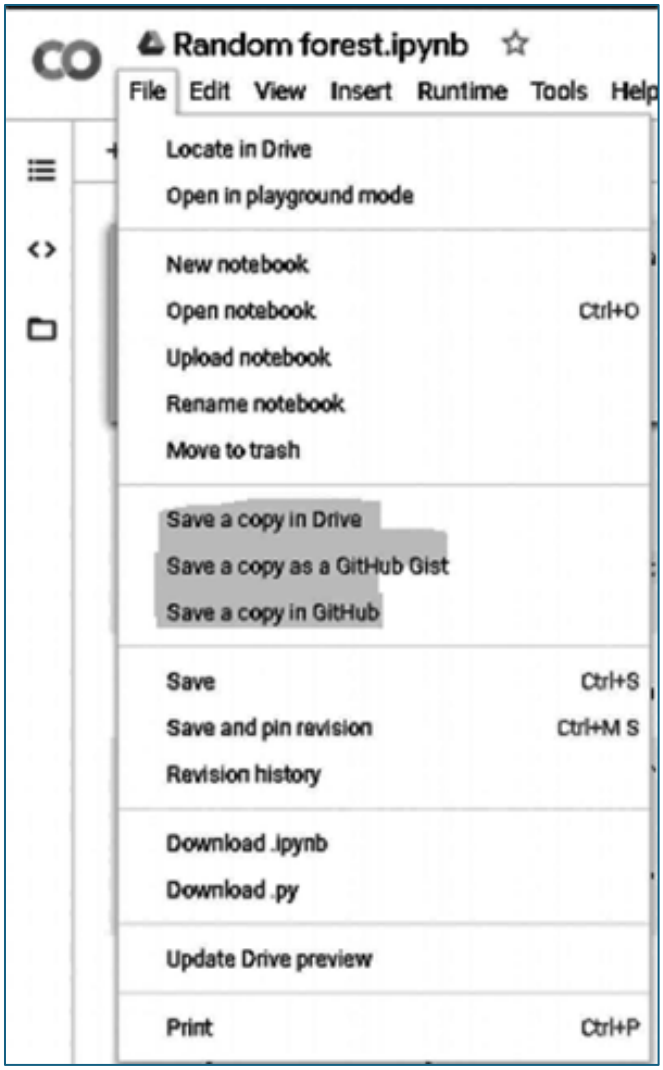


Fig. 7.25 Save Notebook

Chia sẻ Sổ tay của bạn:

Google Collab cũng cung cấp cho chúng ta một cách dễ dàng để chia sẻ công việc của mình với người khác.

Đây là một trong những điều tốt nhất về Collab như trong Hình 7.26:

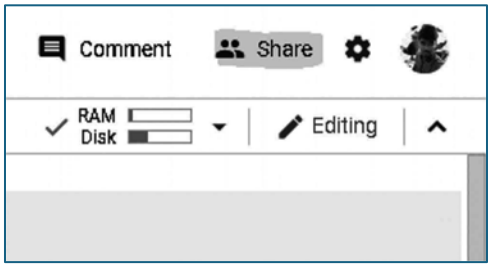


Fig. 7.26 Sharing Notebook

Chỉ cần nhấp vào nút Chia sẻ (Share) và nó cung cấp cho chúng ta tùy chọn tạo liên kết có thể chia sẻ mà chúng ta có thể chia sẻ qua bất kỳ nền tảng nào. Như trong Hình 7.27, bạn cũng có thể mời người khác bằng ID email của họ. Nó hoàn toàn giống như chia sẻ Google Doc hoặc Google Sheet. Sự phức tạp và đơn giản của hệ sinh thái Google thật đáng kinh ngạc!



Fig. 7.27: Link generating

Tóm tắt

Các công cụ để xây dựng một dự án Trí tuệ Nhân tạo như đã thảo luận có thể là python, jupyter, google Collaboratory. Bằng cách nghiên cứu tất cả các công cụ này, chúng ta có thể nói rõ ràng rằng bây giờ chúng ta có thể dễ dàng làm việc trên các môi trường này để xây dựng một dự án thời gian thực hữu ích. Thay vì tất cả những thứ này, bạn có thể chọn máy chủ Amazon EC2, nơi sẽ cung cấp cho bạn 1 năm dùng thử miễn phí để lưu trữ (host) bất kỳ mô hình hoặc dự án nào.

CHƯƠNG 8: CÁC THƯ VIỆN QUAN TRỌNG CHO AI

Trong chương này, chúng ta sẽ tìm hiểu tất cả các thư viện cơ bản và một số framework về học máy, học sâu và thị giác máy tính. Chúng ta sẽ thảo luận tất cả về cách sử dụng của chúng kèm theo ví dụ và hướng dẫn bao gồm các lệnh để cài đặt các thư viện đó nhằm xây dựng bất kỳ dự án nào trên hệ thống của chúng ta. Vậy hãy bắt đầu.

Thư viện là gì?

Thư viện là một tập hợp các tính năng và phương thức cho phép bạn thực hiện nhiều hành động mà không cần phải tự viết mã.

Ví dụ, các thư viện mà bạn phải biết nếu làm việc với dữ liệu là numpy, scipy, pandas, v.v. Chúng có các hàm xử lý dữ liệu rất đơn giản giúp bạn tiết kiệm thời gian cho các thủ thuật nhỏ.

Học máy và Python

TensorFlow: Google, cùng với nhóm Brain, đã tạo ra thư viện này. TensorFlow được sử dụng cho hầu hết mọi chương trình học máy của Google. Vì mạng nơ-ron có thể dễ dàng được biểu diễn dưới dạng đồ thị máy tính, chúng có thể được sử dụng như một phép toán Tensor bằng phương pháp TensorFlow. TensorFlow hoạt động giống như một thư viện mã để viết các thuật toán mới liên quan đến một số phép toán tensor. Hơn nữa, tensor biểu diễn dữ liệu trong các ma trận N-chiều.

Đặc điểm của TensorFlow:

- Để thực hiện các phép toán đại số tuyến tính nhanh, TensorFlow được tối ưu hóa, sử dụng các kỹ thuật như XLA.
- **Cấu trúc đáp ứng**: Với TensorFlow, mọi phần của đồ thị (điều không thể khi sử dụng NumPy hoặc SciKit) đều có thể được hiển thị trực quan một cách dễ dàng.
- **Linh hoạt**: Một trong những khía cạnh quan trọng nhất của TensorFlow là khả năng hoạt động linh hoạt, có nghĩa là nó có tính mô-đun và bạn có thể render các phần của nó một cách độc lập.
- **Dễ dàng huấn luyện**: Nó dễ dàng được huấn luyện trên CPU và GPU cho tính toán phân tán.
- **Huấn luyện mạng nơ-ron song song**: TensorFlow cung cấp khả năng xử lý song song (pipelining) để huấn luyện nhiều mạng nơ-ron và nhiều GPU, làm cho việc lập mô hình trên các hệ thống quy mô lớn trở nên cực kỳ hiệu quả.
- **Cộng đồng lớn**: Đương nhiên, một đội ngũ kỹ sư phần mềm lớn đã được Google hình thành và làm việc liên tục để cải tiến độ ổn định.
- **Nguồn mở**: Điều tuyệt vời về thư viện học máy này là nó là nguồn mở để bất kỳ ai cũng có thể sử dụng khi họ muốn chỉ bằng cách tải xuống từ internet.

Lệnh cài đặt TensorFlow Ví dụ: Pip install tensorflow

```
# Chương trình Python sử dụng TensorFlow
# để nhân hai mảng
# import tensorflow
import tensorflow as tf

# Khởi tạo hai hằng số
x1 = tf.constant ([1, 2, 3, 4])
x2 = tf.constant ([5, 6, 7, 8])

# Nhân
result = tf.multiply (x1, x2)

# Khởi tạo Session
sess = tf.Session ()

# In kết quả
print (sess.run (result))

# Đóng session
sess.close()
```

Đầu ra: [5 12 21 32]

Scikit-learn: Đây là một thư viện Python có kết nối với NumPy và SciPy. Nó được công nhận là một trong những thư viện tốt nhất để xử lý dữ liệu phức tạp. Nó được xây dựng trên nền tảng Numpy, Scipy và Matplotlib.

Thư viện này bao gồm một số cải tiến. Một cải tiến là chức năng kiểm tra chéo (cross-validation) cho phép sử dụng nhiều hơn một chỉ số đo lường. Một số kỹ thuật huấn luyện như hồi quy logistic và lân cận gần nhất đã được cải tiến.

Đặc điểm của Scikit-Learn

- **Kiểm tra chéo (Cross Validation)**: Có sẵn các kỹ thuật cụ thể để đánh giá độ chính xác của mô hình trên dữ liệu chưa thấy.
- **Thuật toán học không giám sát**: Gói này có sẵn một loạt các thuật toán từ phân cụm, phân tích nhân tố, phân tích thành phần chính đến mạng nơ-ron không giám sát.
- **Trích xuất đặc trưng**: Hữu ích cho việc trích xuất các đặc trưng của hình ảnh và văn bản (ví dụ: túi từ - bag of words).

Lệnh cài đặt Scikit-learn Pip install scikit-learn

NumPy: Nó được coi là một trong những thư viện Học máy phổ biến nhất của Python. Đối với nhiều phép toán trên tensor, TensorFlow và các thư viện khác đều sử dụng nội bộ NumPy. Tính năng tốt nhất và chính của NumPy là giao diện Mảng (Array).

Đặc điểm của NumPy:

- **Tương tác**: NumPy rất có tính tương tác và thân thiện với người dùng.
- **Toán học**: Làm cho toán học phức tạp trở nên rất đơn giản.
- **Trực quan**: Nó viết mã nhanh và các khái niệm dễ hiểu.
- **Tương tác nhiều**: Có rất nhiều sự tham gia của nguồn mở, do đó, nó được sử dụng rộng rãi.

Lệnh cài đặt NumPy - Pip install numpy

Ví dụ về NumPy: (Lưu tệp dưới tên Num1.py)

```
# Chương trình Python sử dụng NumPy
# cho một số phép toán
# cơ bản
import numpy as np

# Tạo hai mảng hạng 2
x = np.array ([[4, 5], [6, 8]])
y = np.array ([[8, 9], [10, 11]])

# Tạo hai mảng hạng 1
v = np.array ([12, 13])
w = np.array ([15, 16])

# Tích vô hướng của các vector
print (np.dot (v, w), "\n")

# Tích của ma trận và vector
print (np.dot (x, v), "\n")

# Tích của ma trận và ma trận
print (np.dot (x, y))
```

Đầu ra:

```
>>>
388
[113 176]
```

```
[[ 82 91]
 [128 142]]
>>>
```

SciPy: Đây là một thư viện học máy kỹ thuật và tạo ứng dụng. Tuy nhiên, vẫn cần phải xác định khoảng cách giữa SciPy và SciPy stack. Thư viện SciPy chứa các mô-đun tối ưu hóa, đại số tuyến tính, mô-đun tích phân và thống kê.

Đặc điểm của SciPy:

- Ưu điểm chính của thư viện SciPy là nó sử dụng NumPy để tạo mảng.
- Ngoài ra, với các cấu trúc con độc đáo của mình, SciPy cung cấp tất cả các quy trình số mạnh mẽ như tối ưu hóa, tích phân số và nhiều quy trình khác.

Lệnh cài đặt scipy

pip install scipy

Pandas: Pandas là một thư viện Học máy trong Python, cung cấp các cấu trúc dữ liệu bậc cao và một loạt các công cụ phân tích. Khả năng chuyển các phép toán phức tạp với dữ liệu bằng một hoặc hai lệnh là một trong những tính năng chính của thư viện này. Pandas có nhiều phương pháp nhóm, kết hợp dữ liệu và lọc tiêu chuẩn, cũng như các hàm cho chuỗi thời gian. Tất cả đều đi kèm với các phép đo tốc độ tuyệt vời.

Đặc điểm của pandas:

- Pandas sẽ giúp thao tác dữ liệu dễ dàng hơn.
- Các điểm nổi bật về chức năng của Pandas cung cấp hỗ trợ cho các hoạt động như lập chỉ mục lại, lặp, sắp xếp, tổng hợp, ghép nối và trực quan hóa.

Lệnh cài đặt pandas

pip install pandas

Matplotlib: Matplotlib là một thư viện trực quan hóa dữ liệu phổ biến cho Python. Nó không liên quan trực tiếp đến học máy, giống như Pandas. Nó đặc biệt hữu ích để hình dung các mẫu trong dữ liệu nếu một lập trình viên muốn.

Đây là một thư viện vẽ 2D được sử dụng để phát triển các biểu đồ 2D. Một mô-đun có tên pyplot giúp các lập trình viên vẽ biểu đồ dễ dàng kiểm soát kiểu đường, phông chữ, trục, v.v. Nó cung cấp các loại biểu đồ và đồ thị khác nhau để xem dữ liệu, ví dụ: biểu đồ tần suất, biểu đồ lỗi và các biểu đồ khác, v.v.

Lệnh cài đặt Matplotlib - Ví dụ

pip install matplotlib

```
# Chương trình Python sử dụng Matplotlib
# để tạo một biểu đồ tuyến tính
# nhập các gói và mô-đun cần thiết
import matplotlib.pyplot as plt
import numpy as np

# Chuẩn bị dữ liệu
x = np.linspace (0, 10, 100)

# Vẽ dữ liệu
plt.plot (x, x, label='linear')

# Thêm chú giải
plt.legend ()

# Hiển thị biểu đồ
plt.show()
```

Đầu ra: như Hình 8.1

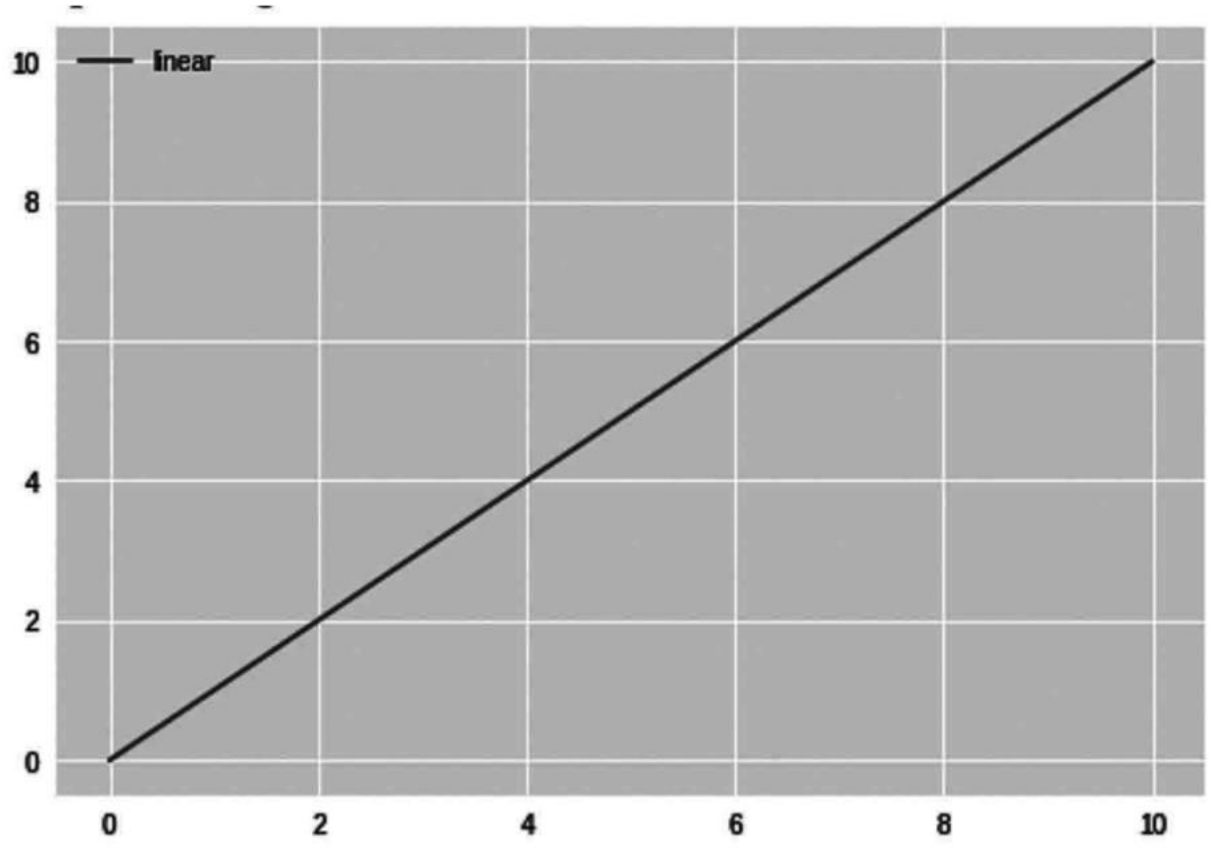


Fig. 8.1 Program output

Keras: Nó được coi là một trong những thư viện học máy tốt nhất trong Python. Nó cung cấp một cách đơn giản hơn để mô tả các mạng nơ-ron. Keras cung cấp một số công cụ tốt nhất để tạo mô hình, phân tích tập dữ liệu, trực quan hóa đồ thị và nhiều hơn nữa. Keras sử dụng Theano hoặc TensorFlow làm nền tảng (backend) bên trong. Nó cũng có thể sử dụng một số mạng nơ-ron nổi tiếng như CNTK. Khi so sánh nó với các thư viện học máy khác, Keras tương đối chậm. Vì nó sử dụng công nghệ back-end để tạo một đồ thị dữ liệu và sau đó sử dụng nó để thực hiện các phép toán. Trong Keras, tất cả các mô hình đều có thể di động (mobile).

Đặc điểm của Keras:

- Hoạt động trơn tru trên CPU và GPU.
- Keras hỗ trợ hầu như tất cả các mô hình mạng nơ-ron—kết nối đầy đủ, tích chập, gộp (pooling), hồi quy, hội tụ, v.v. Các mô hình này cũng có thể được kết hợp để tạo ra các mô hình phức tạp hơn.
- Thiết kế mô-đun của Keras vô cùng rõ ràng, linh hoạt và hoàn hảo cho công việc sáng tạo.
- Keras là một framework dựa trên Python giúp gỡ lỗi và khám phá đơn giản.

Lệnh cài đặt Keras

pip install Keras

PyTorch: Đây là thư viện học máy lớn nhất cho phép các nhà phát triển thực hiện các phép toán tensor bằng cách tăng tốc GPU, tạo các đồ thị động và tự động tính toán độ dốc (gradients). Tuy nhiên, PyTorch cungn cấp một loạt các API để giải quyết các vấn đề về thiết bị mạng nơ-ron. Thư viện Học máy này được phát triển dưới dạng Torch, một thư viện phần mềm nguồn mở bằng C với một trình bao bọc (wrapper) Lua. Được phát hành vào năm 2017, thư viện này trong Python đã trở nên phổ biến và thu hút ngày càng nhiều nhà phát triển học máy kể từ khi ra mắt.

Đặc điểm của PyTorch:

- **Hybrid Front End**: Một front-end hybrid mới cung cấp tính dễ sử dụng và linh hoạt trong chế độ “eager mode”, đồng thời chuyển tiếp trong môi trường vận hành C++ sang chế độ đồ thị để tăng tốc độ, tối ưu hóa và chức năng.
- **Huấn luyện phân tán**: Tối ưu hóa hiệu quả nghiên cứu và phát triển bằng cách hưởng lợi từ hỗ trợ gốc cho các hoạt động tập thể đồng bộ và kết nối ngang hàng (peer-to-peer) có thể truy cập từ Python và C++.

Lệnh cài đặt Pytorch - Pip install torchvision

Theano: Đây là một thư viện framework tính toán của Python để tính toán mảng đa chiều. Theano hoạt động tương tự như TensorFlow, mặc dù không mạnh bằng TensorFlow. Do không có khả năng thích ứng với môi trường sản xuất. Theano có thể được sử dụng trong các môi trường phân tán hoặc song song giống như TensorFlow.

Đặc điểm của Theano:

- **Tích hợp chặt chẽ với NumPy**: Khả năng sử dụng đầy đủ các mảng NumPy trong các hàm đã biên dịch của Theano.
- **Sử dụng GPU một cách minh bạch**: Thực hiện các phép tính với cường độ dữ liệu nhanh hơn nhiều so với CPU.
- **Vi phân biểu tượng hiệu quả**: Đối với các hàm của một hoặc nhiều đầu vào, Theano thực hiện các đạo hàm.
- **Tối ưu hóa tốc độ và độ ổn định**: Nhận được câu trả lời đúng cho $log(1 + x)$ ngay cả khi x rất nhỏ. Đây chỉ là một ví dụ về sự ổn định của Theano.
- **Tạo mã C động**: Đánh giá các biểu thức nhanh hơn bao giờ hết, do đó làm cho chúng hiệu quả hơn nhiều.
- **Kiểm tra đơn vị và tự xác minh rộng rãi**: Phát hiện và chẩn đoán nhiều loại mô hình lỗi và sự không rõ ràng.

Lệnh cài đặt Theano

pip install Theano

Học sâu

Mxnet: Đây là một framework học sâu nguồn mở được sử dụng để kết nối mạng nơ-ron và huấn luyện. Nó có năng suất cao và nhanh chóng. Nó có thể mở rộng quy mô thông qua nhiều GPU và nhiều máy.

Để cài đặt Maxnet - Tải xuống gói và cài đặt gói https://mxnet.apache.org/get_started/ ? Liên kết để biết thêm chi tiết: https://mxnet.apache.org/

Caffe: Caffe là một framework học sâu được tạo ra với mục tiêu về tính biểu cảm, tốc độ và tính mô-đun. Nó được phát triển bởi Berkeley AI Research (BAIR) và bởi những người đóng góp trong cộng đồng.

Để cài đặt Caffe- ưu tiên tài liệu trên liên kết này
https://caffe.berkeleyvision.org/installation.html Liên kết để biết thêm chi tiết:
https://caffe.berkeleyvision.org/

Fast.ai: Thư viện fastai đơn giản hóa việc huấn luyện các mạng nơ-ron nhanh và chính xác bằng cách sử dụng các phương pháp thực hành tốt nhất hiện đại. Nó dựa trên nghiên cứu về các phương pháp hay nhất trong học sâu được thực hiện tại fast.ai, bao gồm hỗ trợ "ngay lập tức" (out of the box) cho các mô hình thị giác, văn bản, dạng bảng và collab (lọc cộng tác).

Để cài đặt Fast.ai- ưu tiên tài liệu trên liên kết này https://github.com/fastai/fastai Liên kết để biết thêm chi tiết: https://docs.fast.ai/

CNTK: Microsoft Cognitive Toolkit (CNTK) là một bộ công cụ cho học sâu phân tán cấp thương mại. Mạng nơ-ron được định nghĩa thông qua một biểu đồ có hướng như một chuỗi các phép đo máy tính. CNTK giúp dễ dàng kết hợp và tích hợp các mô hình phổ biến như DNNs cấp tiến (feed-forward), CNNs và mạng nơ-ron tái phát (RNNs/LSTMs). CNTK cho phép tự động vi phân và song song hóa việc học theo gradient dốc ngẫu nhiên (SGD) trên nhiều GPU và máy chủ.

CNTK hỗ trợ hệ điều hành Linux 64-bit hoặc Windows 64-bit. Để cài đặt, bạn có thể chọn các gói nhị phân được biên dịch trước hoặc biên dịch bộ công cụ từ nguồn được cung cấp trong GitHub: https://github.com/Microsoft/CNTK Liên kết để biết thêm chi tiết: https://docs.microsoft.com/en-us/cognitive-toolkit/

Thị giác máy tính

OpenCV: OpenCV là một trong những thư viện được sử dụng rộng rãi nhất cho các ứng dụng thị giác máy tính. Python API cho OpenCV là OpenCV Python. OpenCV-Python không chỉ dễ dàng, vì bối cảnh của mã C/C++ được mã hóa và triển khai dễ dàng (là kết quả của trình bao bọc Python). Điều này làm cho việc thực thi các chương trình thị giác chuyên sâu về máy tính trở thành một lựa chọn tốt.

Lệnh cài đặt OpenCV

pip install opencv-python

Imutils: Các tác vụ xử lý hình ảnh đơn giản như dịch, xoay, thay đổi kích thước, tạo khung xương (skeletonization), hiển thị, hình ảnh Matplotlib, sắp xếp đường viền và các cạnh được đơn giản hóa với OpenCV và Python trong một loạt các tính năng tiện lợi.

Lệnh cài đặt imutils

pip install imutils

Pillow/PIL (python Imaging Library): PIL (Python Imaging Library) là một thư viện miễn phí cho ngôn ngữ lập trình Python, hỗ trợ một số định dạng tệp hình ảnh khác nhau để mở, xử lý và lưu. Tuy nhiên, với bản phát hành cuối cùng vào năm 2009, sự phát triển của nó đã bị đình trệ. Rất may, Pillow, một nhánh (fork) phổ biến của PIL, được cài đặt dễ dàng hơn, chạy trên tất cả các hệ điều hành chính và hỗ trợ Python 3. Thư viện có các tính năng xử lý hình ảnh đơn giản, chẳng hạn như các phép toán điểm, lọc bằng các hạt nhân làm mát (cooling kernels) tích hợp sẵn và chuyển đổi không gian màu.

Lệnh cài đặt pillow

pip install pillow

Scikit-image: Scikit-image là thư viện xử lý hình ảnh nguồn mở của ngôn ngữ lập trình Python. Gói này bao gồm các thuật toán phân đoạn, biến đổi hình học, thao tác không

gian màu, trực quan hóa, lọc, hình thái học, phát hiện chức năng và nhiều hơn nữa. Scikit-image là một loạt các thuật toán xử lý hình ảnh. Nó mở và miễn phí mà không có bất kỳ hạn chế nào. Điều này đòi hỏi một số triển khai các thuật toán mà OpenCV không có.

Lệnh cài đặt Scikit-image

pip install Scikit-image

Liên kết để biết thêm chi tiết: https://scikit-image.org/

SimpleCV: Một nền tảng nguồn mở khác cho các ứng dụng thị giác máy tính là SimpleCV. Nó cho phép truy cập vào các thư viện thị giác máy tính mạnh mẽ khác nhau như OpenCV mà không cần biết độ sâu bit, định dạng tệp, không gian màu, v.v. Đường cong học tập của nó nhỏ hơn một chút so với OpenCV và (như khẩu hiệu của nó nói) "máy tính rất dễ nhìn thấy."

Liên kết để biết thêm chi tiết: http://simplecv.org/

Vigra: Đây là một thư viện thị giác máy tính tập trung vào các thuật toán có thể mở rộng như là bí quyết chính của các thuật toán trong lĩnh vực này. Kết quả là, cuốn sách đã được xây dựng trong thư viện mẫu tiêu chuẩn, sử dụng lập trình chung (generic programming) do Stepanov và Musser đi tiên phong. Bạn có thể sử dụng các thuật toán VIGRA ngoài các cấu trúc dữ liệu của mình trong môi trường của mình bằng cách viết một vài bộ điều hợp (adapter) (trình lặp hình ảnh và phụ kiện). Ngoài ra, bạn có thể sử dụng các cấu trúc dữ liệu VIGRA dễ dàng thích ứng với nhiều ứng dụng khác nhau. VIGRA thực tế không có tính linh hoạt, vì kiến trúc sử dụng tính đa hình (mẫu) để biên dịch thời gian.

Liên kết để biết thêm chi tiết: https://ukoethe.github.io/vigra/

Mahotas: Đây là một thư viện khác cho thị giác máy tính và xử lý hình ảnh của Python. Nó bao gồm các chức năng xử lý hình ảnh thông thường, chẳng hạn như bộ lọc và các phép toán hình thái, cũng như các chức năng xem máy hiện đại hơn, bao gồm phát hiện điểm ưa thích và mô tả cục bộ để tính toán đối tượng. Mã Python lý tưởng để phát triển nhanh, nhưng các thuật toán được sử dụng trong C++ và được điều chỉnh tốc độ. Thư viện Mahotas nhanh với mã tối thiểu và thậm chí phụ thuộc tối thiểu.

Liên kết để biết thêm chi tiết: https://mahotas.readthedocs.io/en/latest/

Pycairo: Pycairo là một mô-đun Python cung cấp các liên kết (bindings) với thư viện. Nó hoạt động với Python 3.5+ và dựa trên cairo >= 1.13.1 . Pycairo được ủy quyền theo LGPL-2.1-only HOẶC MPL-1.1 bao gồm báo cáo này. Các hệ thống liên kết Pycairo đã được thiết kế để phù hợp với API Cairo càng chặt chẽ càng tốt, và chỉ đi chệch hướng nếu được triển khai 'python' hơn một cách rõ ràng.

Đặc điểm của pycairo:

- Cung cấp một gui đối tượng tập trung vào cairo.
- Tìm kiếm và chuyển đổi trạng thái lỗi của các tạo phẩm (artifacts) thành các ngoại lệ (exceptions).
- API C được cung cấp để các tiện ích mở rộng Python khác có thể được sử dụng.

Liên kết để biết thêm chi tiết: https://pypi.org/project/pycairo/

Pgmagick: Đối với thư viện Graphics Magick, pgmagick là một trình bao bọc dựa trên Python. Hệ thống xử lý hình ảnh Graphics Magick đôi khi được gọi là Dao quân đội Thụy Sĩ. Phạm vi hình ảnh mạnh mẽ và hiệu quả trong hơn 88 định dạng chính, bao gồm DPX, GIF, JPEG, JPEG-2000, PNG, PDF, PNM và TIFF, tạo điều kiện thuận lợi cho việc đọc, viết và chỉnh sửa ảnh.

Liên kết để biết thêm chi tiết: https://pypi.org/project/pgmagick/

Tóm tắt

Ở đây chúng ta đã thấy một số thư viện quan trọng của các lĩnh vực trí tuệ nhân tạo này và cũng đã đề cập đến ví dụ cho một số thư viện trong số chúng để chứng minh việc sử dụng chúng. Có rất nhiều thư viện cho python có thể hữu ích cho các dự án khác nhau được xây dựng trong trí tuệ nhân tạo và các lĩnh vực nghiên cứu khác cho các quan điểm và mục tiêu khác nhau. Để dùng được toàn bộ chức năng của tất cả các ứng dụng, hãy bật chế độ Hoạt động trên Các ứng dụng Gemini .

CHƯƠNG 9: CÁC THUẬT TOÁN HỌC MÁY

Trong chương này về các thuật toán học máy, chúng ta sẽ tìm hiểu các thuật toán cơ bản của học máy cùng với việc triển khai chúng bằng bộ dữ liệu và thảo luận về ưu nhược điểm của các thuật toán để hiểu rõ hơn và cách sử dụng chúng trong bất kỳ dự án nào hoặc với bất kỳ bộ dữ liệu nào để có được kết quả chính xác hữu ích cho nghiên cứu hoặc học tập.

Thành phần chính

Chúng ta đã định nghĩa một bộ dữ liệu bao gồm âm thanh, hình ảnh và nhãn nhị phân, giúp hiểu được cách chúng ta có thể huấn luyện một mô hình từ các đoạn trích dẫn đến phân loại. Loại bài toán này là một trong nhiều loại bài toán học máy khi chúng ta cố gắng dự đoán một nhãn không xác định cụ thể dựa trên các đầu vào đã biết, với điều kiện có sẵn một bộ dữ liệu các ví dụ đã biết nhãn, được gọi là **học có giám sát**.

Hãy thảo luận về các thành phần chính mà chúng ta cần để đạt được điều đó:

- Dữ liệu
- Mô hình
- Thuật toán

Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một phương pháp tuyến tính trong thống kê để mô hình hóa mối quan hệ giữa một hoặc nhiều biến độc lập và một (y) biến phụ thuộc (s). Các mối quan hệ được mô hình hóa trong hồi quy tuyến tính bằng các hàm dự đoán tuyến tính, được ước tính bằng các tham số mô hình chưa biết từ dữ liệu. Hồi quy tuyến tính là một trong những thuật toán phổ biến nhất của Học máy. Đó là vì tính đơn giản tương đối và các thuộc tính phổ biến của nó. Nó có hai loại là:

- **Hồi quy tuyến tính đơn giản (Simple Linear Regression):** Nếu bạn chỉ làm việc với một biến độc lập, hồi quy tuyến tính được gọi là đơn giản. Nó được biểu thị bằng công thức $f(x) = mx + b$.

Chúng ta sẽ sử dụng hai hàm như chi phí (cost) và gradient để tính toán và tối ưu hóa mô hình của mình với dữ liệu chúng ta cung cấp.

Hàm chi phí (Cost Function): Chúng ta tính toán độ chính xác bằng hàm chi phí lỗi bình phương trung bình (MSE) của thuật toán hồi quy tuyến tính. MSE tính toán khoảng cách bình phương trung bình giữa hiệu suất dự kiến và hiệu suất thực tế. Công thức của nó là:

$$Error(m, b) = \frac{1}{N}\Sigma_{i=1N}(\text{Đầu ra thực tế} - \text{Đầu ra dự đoán})$$

MSE rất đơn giản và có thể dễ dàng lập trình bằng Python như:

```
def cost_function(m, b, x, y):
    totalError = 0
    for i in range (0, len (x)):
        totalError += (y[i] - (m*x[i] + b))**2
    return totalError / float(len(x))
```

Tối ưu hóa: Chúng ta sẽ sử dụng gradient descent (giảm độ dốc) để tìm các hệ số giúp giảm thiểu hàm lỗi. Gradient descent là một thuật toán tối ưu hóa, lặp đi lặp lại các bước đi về phía cực tiểu cục bộ của hàm chi phí.

Để tìm đường xuống "đáy" (cực tiểu), chúng ta phải lấy đạo hàm của hàm lỗi theo m và b. Sau đó, chúng ta thực hiện một bước theo hướng âm (ngược hướng gradient).

Công thức Gradient Descent tổng quát

$$\Theta_j = \Theta_j - \alpha \frac{\partial}{\partial \Theta_j} J(\Theta_0 - \Theta_1)$$

Gradient Descent cho Hồi quy tuyến tính đơn giản.

$$\frac{\partial}{\partial m} = \frac{2}{N} \sum_{i=1}^{N} - x_i \big(y_i - (mx_i + b)\big)$$

$$\frac{\partial}{\partial b} = \frac{2}{N} \sum_{i=1}^{N} - \big(y_i - (mx_i + b)\big)$$

Việc triển khai gradient descent phức tạp hơn một chút, nhưng cũng có thể dễ dàng thực hiện bằng Python thuần túy:

```
def gradient_descent(b, m, x, y, learning_rate, num_iterations):
    N = float(len(x))
    for j in range(num_iterations):  # lặp lại cho num_iterations
        b_gradient = 0
        m_gradient = 0
        for i in range(0, len(x)):
            b_gradient += -(2/N) * (y[i] - ((m*x[i]) + b))
            m_gradient += -(2/N) * x[i] * (y[i] - ((m*x[i]) + b))
        b -= (learning_rate * b_gradient)
        m -= (learning_rate * m_gradient)
        if j % 50 == 0:
            print('error:', cost_function(m, b, x, y))
    return [b, m]
```

Triển khai Hồi quy tuyến tính

Chúng ta phải xây dựng một bộ dữ liệu và xác định một số biến ban đầu để chạy mô hình hồi quy tuyến tính của mình. Để hiển thị dữ liệu ngẫu nhiên đơn giản, chúng ta sử dụng NumPy và matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 100, 50)
delta = np.random.uniform(-10, 10, x.size)
y = 0.6*x + 5 + delta

plt.scatter(x, y)
plt.show()
```

Đầu ra: như trong Hình 9.1

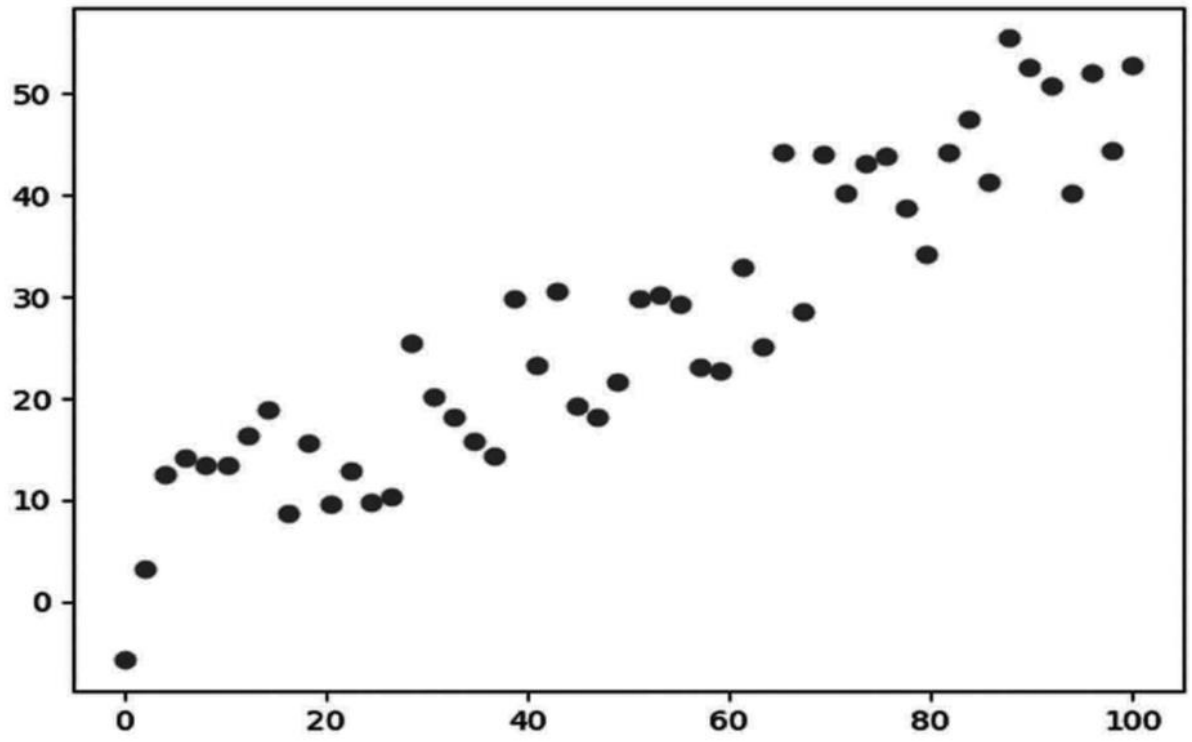


Fig. 9.1 Program Output

Bây giờ chúng ta đã có dữ liệu, chúng ta sẵn sàng sử dụng chức năng trên để huấn luyện mô hình của mình:

```
# defining some variables
learning_rate = 0.0001
```

```
initial_b = 0
initial_m = 0
num_iterations = 100

print('Initial error:', cost_function(initial_m, initial_b, x, y))
[b, m] = gradient_descent(initial_b, initial_m, x, y, learning_rate, num_iterations)
print('b:', b)
print('m:', m)
print('error:', cost_function(m, b, x, y))
plt.show()
```

Đầu ra:

```
Initial error: 1576.099063465165
error: 205.2155773301355
error: 41.73021903795362
b: 0.03851474971994741
m: 0.6744456473643745
error: 41.69053444198719
```

Bây giờ chúng ta có thể sử dụng mô hình của mình như một công cụ dự đoán, vì chúng ta đã huấn luyện nó. Chúng ta sẽ chỉ dự đoán dữ liệu huấn luyện trong ví dụ đơn giản này và sử dụng kết quả để vẽ đường thẳng phù hợp nhất bằng matplotlib.

```
predictions = [(m*x[i]) + b for i in range(len(x))]
plt.scatter(x, y)
plt.plot(x, predictions, color='r')
plt.show()
```

Đầu ra: như trong Hình 9.2

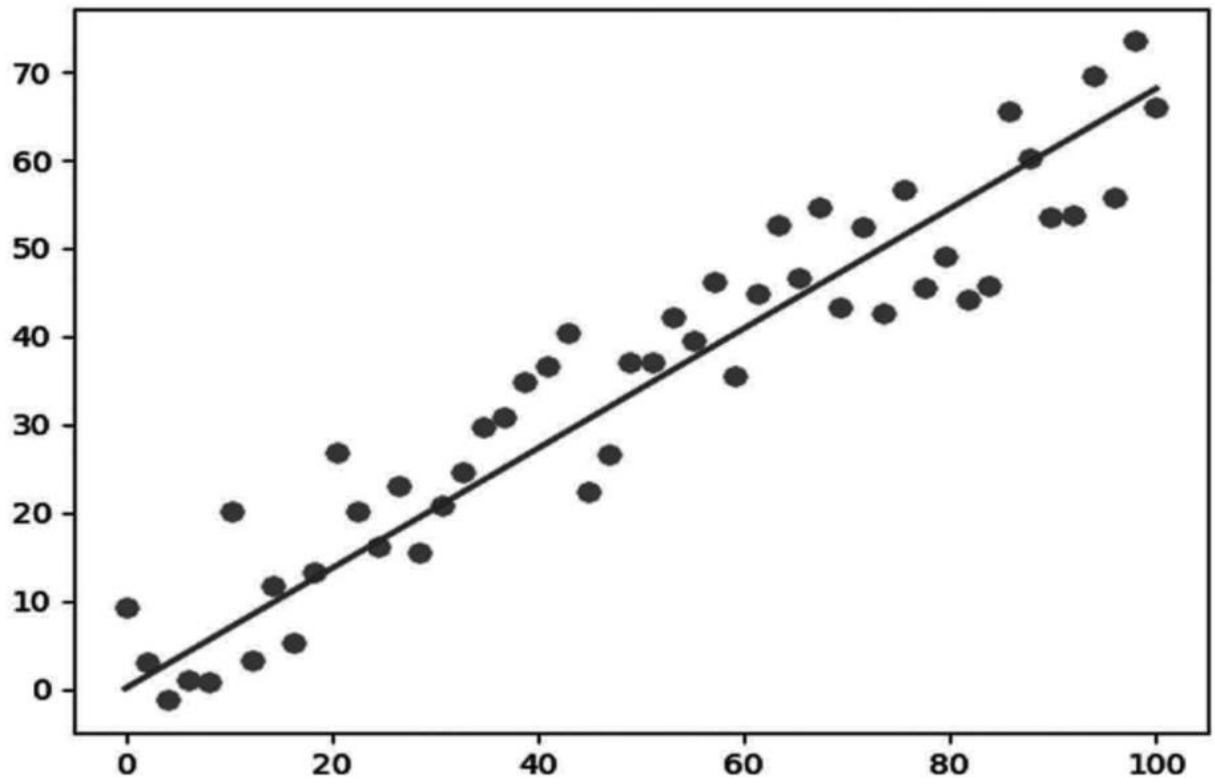


Fig. 9.2 Program Output

- **Hồi quy tuyến tính đa biến (Multiple Linear Regression):** Khi bạn xử lý ít nhất hai biến hoạt động độc lập, hồi quy tuyến tính được gọi là đa biến. Mỗi biến (còn được gọi là đặc trưng) được nhân với một trọng số mà thuật toán hồi quy tuyến tính của chúng ta đã học được. Nó được biểu thị bằng công thức:

$$f(x) = b + W_1X_1 + W_2X_2 + \cdots W_nX_n = b + \sum_{i=0}^{n} W_i X_i$$

Chúng ta sẽ sử dụng hai hàm như chi phí và gradient để tính toán và tối ưu hóa mô hình của mình với dữ liệu chúng ta cung cấp.

Hàm chi phí: Chúng ta sẽ sử dụng lỗi bình phương trung bình làm hàm mất mát, giống như chúng ta đã làm với Hồi quy tuyến tính đơn giản. Sự khác biệt duy nhất là hiệu suất dự đoán của chúng ta bây giờ đến từ một đặc trưng khác. Vì bây giờ có thể sử dụng NumPy để lập trình hàm chi phí của chúng ta với nhiều đặc trưng và trọng số.

Hồi quy Logistic (Logistic Regression)

Đây là một thuật toán phân loại, được sử dụng khi các loại của biến phản hồi thay đổi. Ý tưởng của hồi quy là tạo ra một liên kết giữa các đặc trưng và khả năng xảy ra của một kết quả cụ thể.

Ví dụ. Biến câu trả lời có hai giá trị, "đỗ" và "trượt" khi chúng ta cần dự đoán liệu một sinh viên đỗ hay trượt kỳ thi nếu số giờ học được coi là yếu tố.

Loại bài toán này được gọi là **hồi quy logistic nhị thức (binomial logistical regression)**, trong đó biến câu trả lời có hai giá trị 0 và 1, hoặc đỗ và trượt, hoặc đúng và sai.

Hồi quy Logistic đa thức (Multinomial Logistic Regression) thảo luận về các tình huống mà biến phản hồi có thể có ba hoặc nhiều giá trị tiềm năng.

Tại sao dùng Hồi quy Logistic mà không dùng Hồi quy tuyến tính?

Đặt 'x' là một hàm nào đó với phân loại nhị phân, và đặt y^s là đầu ra có thể là 0 hoặc 1. Xét đầu vào, xác suất để đầu ra là 1 là: $P(y = 1/x)$.

Chúng ta có thể nói rằng nếu chúng ta ước tính xác suất bằng hồi quy tuyến tính: $p(X) = \beta_0 + \beta_1 X$ trong đó, $p(x) = p(y = 1/x)$

Mô hình hồi quy tuyến tính có thể tạo ra xác suất dự đoán là bất kỳ số nào nằm trong khoảng từ âm đến dương, trong khi kết quả (xác suất) chỉ có thể nằm trong khoảng $0 < P(x) < 1$ như trong Hình 9.3.

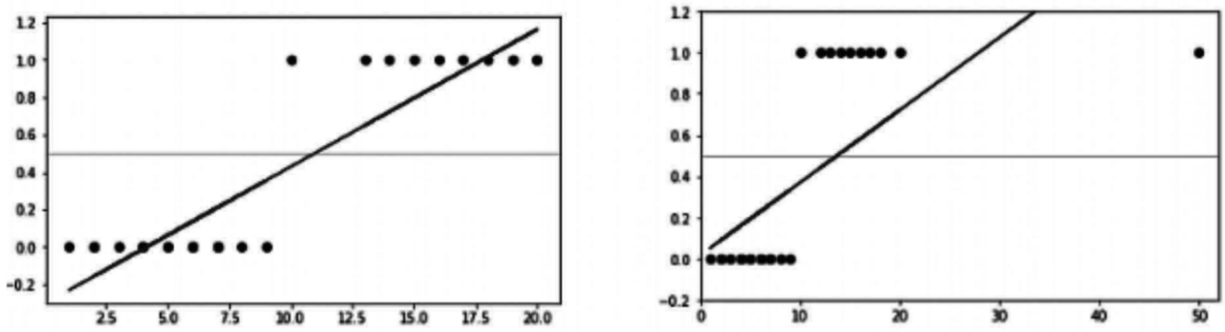


Fig. 9.3 Logistic Regression

Hồi quy tuyến tính cũng bị ảnh hưởng lớn bởi các điểm ngoại lệ (outliers). Để tránh vấn đề này, hàm log-odds hoặc hàm logistic được sử dụng.

Hàm Logistic/Logit

Hồi quy logistic có thể được biểu thị là:

$$log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X$$

Hàm logit hoặc log-odds nằm ở vế trái, và $p(x)/\big(1-p(x)\big)$ được gọi là "odds" (tỷ lệ chênh).

Tỷ lệ chênh biểu thị tỷ lệ giữa xác suất thành công và xác suất thất bại. Do đó, một tổ hợp tuyến tính của các đầu vào với log(odds) trong Hồi quy Logistic được ánh xạ - đầu ra bằng 1.

Nếu đảo ngược hàm trên, chúng ta sẽ có:

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1+e^{\beta_0+\beta_1 X}}$$

Phương trình này được gọi là **hàm Sigmoid**, là một đường cong có hình chữ S. Nó cũng cung cấp một giá trị xác suất $0 < p < 1$.

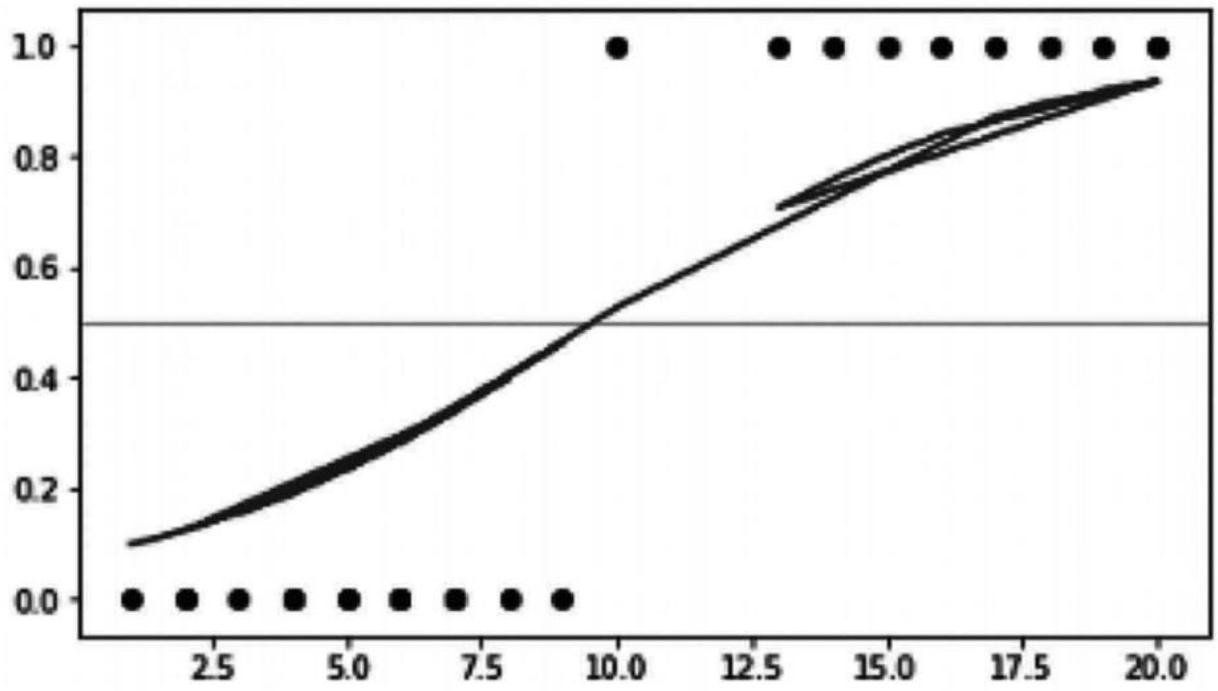


Fig. 9.4 Sigmoid Function

Ước tính các hệ số hồi quy

Chúng ta sử dụng **Ước tính Hợp lý Tối đa (Maximum Likelihood Estimation - MLE)**, không giống như hồi quy tuyến tính sử dụng Bình phương Tối thiểu Thông thường (Ordinary Least Square) để ước tính tham số.

Có thể có vô số tập hợp các hệ số hồi quy. MLE tìm ra chuỗi các hệ số hồi quy mang lại khả năng tối đa để thu được dữ liệu mà chúng ta đã quan sát.

Nếu chúng ta có kết quả nhị phân, nó là α nếu kết quả tốt và $1-\alpha$ trong trường hợp ngược lại. Do đó, chúng ta có hàm xác suất:

$$L(\beta; y) = \prod_{i=1}^{N} \left(\frac{\pi_i}{1-\pi_i}\right)^{y_i} (1-\pi_i)$$

Để đánh giá giá trị tham số, hàm log của xác suất (log-likelihood) được sử dụng vì nó không làm thay đổi các thuộc tính của hàm. Các giá trị tham số giúp tối đa hóa log-likelihood được tính toán bằng các kỹ thuật lặp như phương pháp Newton.

Hiệu suất của Mô hình Hồi quy Logistic

Độ lệch (Deviance) được sử dụng thay vì tổng bình phương để kiểm tra đầu ra của mô hình hồi quy logistic như trong bảng 9.1.

- **Độ lệch Null (Null Deviance):** thể hiện phản ứng dự kiến của mô hình chỉ có hệ số chặn (interception).
- **Độ lệch Hệ thống (System Deviance):** chỉ ra phản ứng dự kiến của hệ thống khi bổ sung các biến độc lập. Nếu phương sai của mô hình nhỏ hơn đáng kể so với độ lệch null, có thể suy ra rằng tham số hoặc tập hợp các tham số đó có ý nghĩa.

Ma trận nhầm lẫn (Confusion Matrix) là một cách khác để đánh giá độ chính xác của mô hình.

Bảng 9.1 Hiệu suất mô hình Hồi quy Logistic

	P' (Dự đoán)	n' (Dự đoán)
P (Thực tế)	True Positive	False Negative
n (Thực tế)	False Positive	True Negative

Độ chính xác (Accuracy) của mô hình được định nghĩa bởi:

$$\frac{TruePositives + TrueNegatives}{TruePositive + TrueNegatives + FalsePositives + FalseNegatives}$$

Hồi quy Logistic đa lớp (Multi-class Logistic Regression)

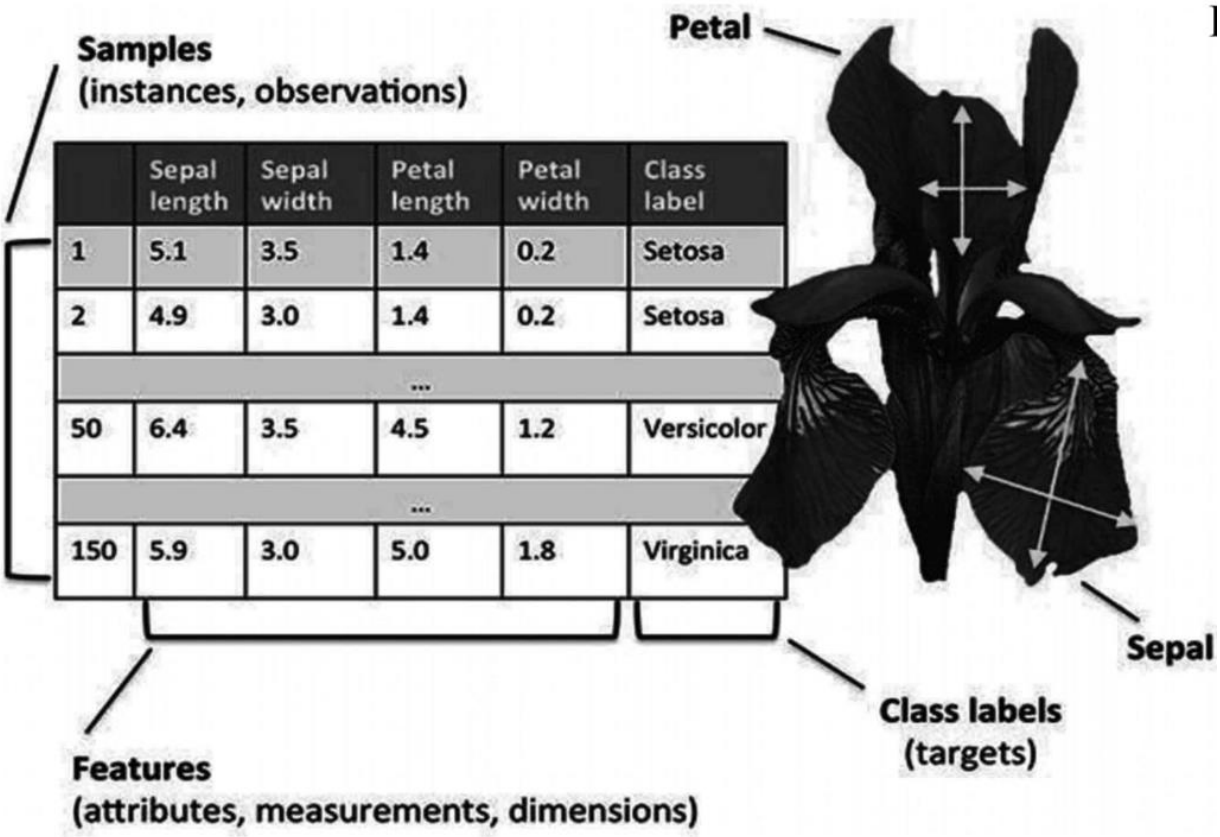
Đằng sau hồi quy đa lớp và nhị thức là cùng một trực giác cơ bản. Tuy nhiên, chúng ta theo đuổi cách tiếp cận **một-so-với-tất-cả (one-vs-rest)** cho các vấn đề đa lớp.

Ví dụ. Chúng ta xử lý Bài toán Đa lớp nếu phải dự đoán thời tiết là "nắng", "mưa" hay "có gió". Chúng ta chuyển điều này thành ba câu hỏi phân loại nhị thức, cụ thể là: (1) nắng so với không nắng, (2) mưa so với không mưa, và (3) có gió so với không có gió. Cả ba phân loại này được thực hiện độc lập. Giải pháp (lớp dự đoán cuối cùng) là lớp có giá trị xác suất lớn nhất so với các lớp khác.

Triển khai Hồi quy Logistic

Chúng ta sẽ triển khai hồi quy logistic một cách đơn giản bằng thư viện Scikit-learn. Chúng ta sẽ sử dụng **Bộ dữ liệu Iris**.

Bộ dữ liệu Iris có lẽ là một trong những bộ dữ liệu học máy đơn giản nhất. Nó mô tả các loài hoa iris (hoa diên vĩ) ở dạng số, ghi lại các phép đo chiều dài/chiều rộng của đài hoa (sepal) và cánh hoa (petal). Chúng ta sẽ cố gắng dự đoán các loài hoa dựa trên các phép đo này.



	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Fig. 9.5 Dataset Description

Bộ dữ liệu này có các cột: sepal length cm, sepal width cm, petal length cm, petal width cm, và species. Bốn cột đầu tiên là đặc trưng và cột cuối cùng là mục tiêu.

Bạn có thể chạy mã này trong bất kỳ môi trường Python nào.

Bước 1: chúng ta sẽ nhập tất cả các thư viện cần thiết

```
# import dependencies
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
# Note: The symbol # represent the comment for the program
```

Bước 2: tải dữ liệu

```
from sklearn import datasets
data = datasets.load_iris()
# here as we have imported the data through sckit's package,
# we will get the data in form of arrays,
# so here our labes is stored in
data.data
```

Đầu ra:

```
array([[5.1, 3.5, 1.4, 0.2], [4.9, 3. , 1.4, 0.2], [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2], [5. , 3.6, 1.4, 0.2], [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3], [5. , 3.4, 1.5, 0.2], [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1], [5.4, 3.7, 1.5, 0.2], [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1], [4.3, 3. , 1.1, 0.1], [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4], [5.4, 3.9, 1.3, 0.4], [5.1, 3.5, 1.4, 0.3],
       [5.7, 3.8, 1.7, 0.3],...so on...
```

```
# Targets for labels
data.target
```

Đầu ra:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Bước 3: Bây giờ chúng ta sẽ chia dữ liệu

```
# we will divide the dataset into data and target for the model
X = data.data[:, :2]  # X is the features in our dataset
y = (data.target != 0) * 1  # y is the Labels in our dataset
```

Sau khi tách, chúng ta sẽ trực quan hóa nó trên biểu đồ bằng matplotlib:

```
# here we have only taken the x line as SepalLengthCm and y line as SepalWidthCm
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='g', label='0')
```

```
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()
```

Đầu ra: Như trong Hình 9.6

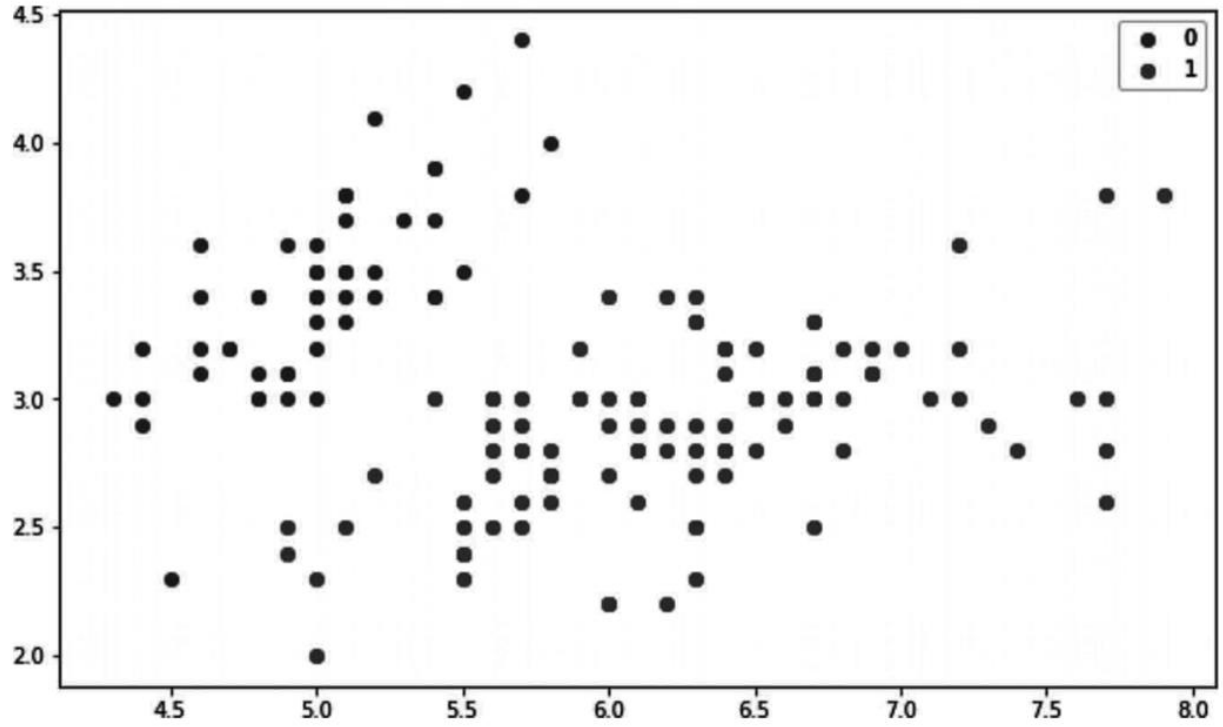


Fig. 9.6 Program Output

Bước 4: Sử dụng mô hình hồi quy logistic của scikit-learn để huấn luyện và dự đoán.

```
# import library with model
from sklearn.linear_model import LogisticRegression
# loading the model which c as parameter which can be given to
# strengthen the model with predefined values like learning rate, iteration etc.,
model = LogisticRegression(C=1e20)
# now we will train the model
model.fit(X, y)
```

Đầu ra:

```
LogisticRegression(C=1e+20, class_weight=None, dual=False,
                   fit_intercept=True, intercept_scaling=1, l1_ratio=None,
                   max_iter=100, multi_class='auto', n_jobs=None, penalty='l2',
                   random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                   warm_start=False)
```

```
# Prediction from model
preds = model.predict(X)
(preds == y).mean()
```

Đầu ra:

```
1.0
```

Ưu điểm:

- Đơn giản và hiệu quả.
- Phương sai thấp.
- Cung cấp điểm xác suất cho các quan sát.
- Cũng có thể được sử dụng để trích xuất đặc trưng.

Nhược điểm:

- Không xử lý tốt một số đặc trưng/thuộc tính dạng danh mục.
- Cần biến đổi các đặc trưng phi tuyến tính.
- Không đủ mạnh mẽ để xử lý các mối quan hệ phức tạp hơn.

Cây quyết định (Decision Tree)

Thuật toán cây quyết định thuộc danh mục học có giám sát. Cây quyết định có thể được sử dụng để biểu diễn các quyết định và việc ra quyết định một cách trực quan và rõ ràng. Như tên gọi, thuật toán này sử dụng một mẫu quyết định giống như cây. Mặc dù là một kỹ thuật khai thác dữ liệu thường được sử dụng cho một chiến lược để đạt được mục tiêu nhất định, nó cũng thường được sử dụng rộng rãi trong học máy.

Cây quyết định được thiết kế bằng cách sử dụng các thuật toán nhận dạng các cách để chia một tập hợp dữ liệu dựa trên các tiêu chí khác nhau. **Cây phân loại (Classification trees)** là các mô hình cây trong đó biến mục tiêu sẽ nhận một tập hợp các giá trị riêng biệt. **Cây hồi quy (Regression trees)** là cây quyết định trong đó biến mục tiêu có thể nhận các giá trị liên tục (thường là số thực). Thuật ngữ chung cho loại này là **Cây Phân loại và Hồi quy (CART)**.

Thuật ngữ quan trọng trong Cây quyết định

Hãy thảo luận về các thuật ngữ cơ bản được sử dụng trong cây quyết định có thể thấy trong Hình 9.7.

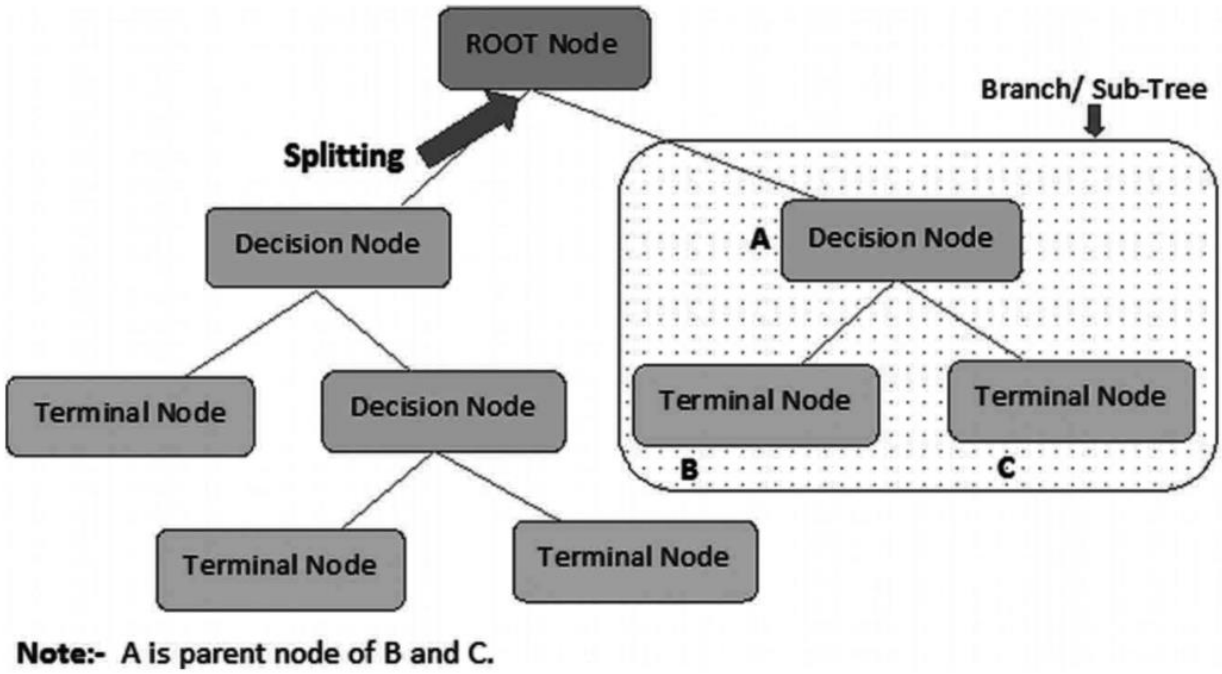


Fig. 9.7 Decision tree representation

Ghi chú: - A là nút cha của B và C.

- **Nút gốc (Root node):** Đại diện cho toàn bộ tập hợp hoặc mẫu và được chia nhỏ thành hai hoặc nhiều mẫu đồng nhất hơn.
- **Phân chia (Splitting):** Đây là cơ chế một nút được chia thành hai hoặc nhiều nút con.
- **Nút quyết định (Decision Node):** Nếu một nút con chia thành các nút con khác, nó được gọi là nút quyết định.
- **Nút cuối/Lá (Terminal Node/Leaf):** Các nút không có con (không phân chia thêm) được gọi là nút Lá và nút Cuối.
- **Cắt tỉa (Pruning):** khi chúng ta giảm kích thước của cây quyết định bằng cách loại bỏ các nút (ngược lại với phân chia).
- **Nhánh/Cây con (Branch/Sub-Tree):** Được gọi là một phần của cây quyết định.
- **Nút Cha và Nút Con (Parent and Child Node):** Một nút được chia thành các nút con được gọi là nút cha của các nút con, trong đó nút con là con của một nút cha.

Phương pháp xây dựng Cây quyết định

Thách thức lớn nhất trong Cây quyết định là chọn thuộc tính cho nút gốc và cho mỗi cấp độ. Phương pháp này được gọi là **lựa chọn thuộc tính**. Chúng ta có hai thước đo lựa chọn thuộc tính phổ biến:

- **Lợi ích thông tin (Information Gain):** Khi chúng ta sử dụng một nút trong cây quyết định, các mẫu huấn luyện được chia thành các tập con có entropy nhỏ hơn.

Sự thay đổi entropy này được xác định bằng lợi ích thông tin. Chúng ta đặt nhiều loại câu hỏi khác nhau ở mỗi nút của cây trong quá trình ra quyết định. Trên cơ sở câu hỏi được đặt ra, chúng ta sẽ đo lường lợi ích dữ liệu.

- **Định nghĩa:** Giả sử S là một tập hợp các mẫu, A là một thuộc tính, S_v là tập con của S với $A = v$, và Values(A) là tập hợp tất cả các giá trị có thể có của A, thì:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{| S_v |}{| S |} Entropy(S_v)$$

- **Entropy:** Entropy là yếu tố không chắc chắn (hoặc độ hỗn loạn) của một biến ngẫu nhiên và nó mô tả sự không tinh khiết của các mẫu tùy ý. Entropy càng lớn, giá trị của thông tin (cần để giảm sự không chắc chắn) càng lớn.
- **Chỉ số Gini (Gini Index):** Chỉ số Gini là thước đo tần suất một phần tử được chọn ngẫu nhiên từ tập hợp sẽ bị phân loại sai. Điều này có nghĩa là bạn sẽ chọn một thuộc tính có chỉ số Gini thấp hơn (ít hỗn loạn hơn). Sklearn mặc định sử dụng chỉ số "Gini".
  - Công thức dưới đây được đưa ra để tính Chỉ số Gini:

$$GiniIndex = 1 - \sum_j p_{j2}$$

(trong đó p_j là tỷ lệ của các mẫu thuộc lớp j)

Triển khai Cây quyết định

Hãy xem cách triển khai phân loại cây quyết định trong Python.

Bước 1: chúng ta nhập các thư viện.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.tree import export_graphviz
from sklearn.externals.six import StringIO
from IPython.display import Image
from pydot import graph_from_dot_data
import pandas as pd
import numpy as np
```

Bước 2: Tải bộ dữ liệu

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Categorical.from_codes(iris.target, iris.target_names)
X.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Bước 3: Phân chia dữ liệu

```
y = pd.get_dummies(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)
```

Bước 4: Sử dụng bộ phân loại cây quyết định để phân loại hoa

```
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
```

Đầu ra:

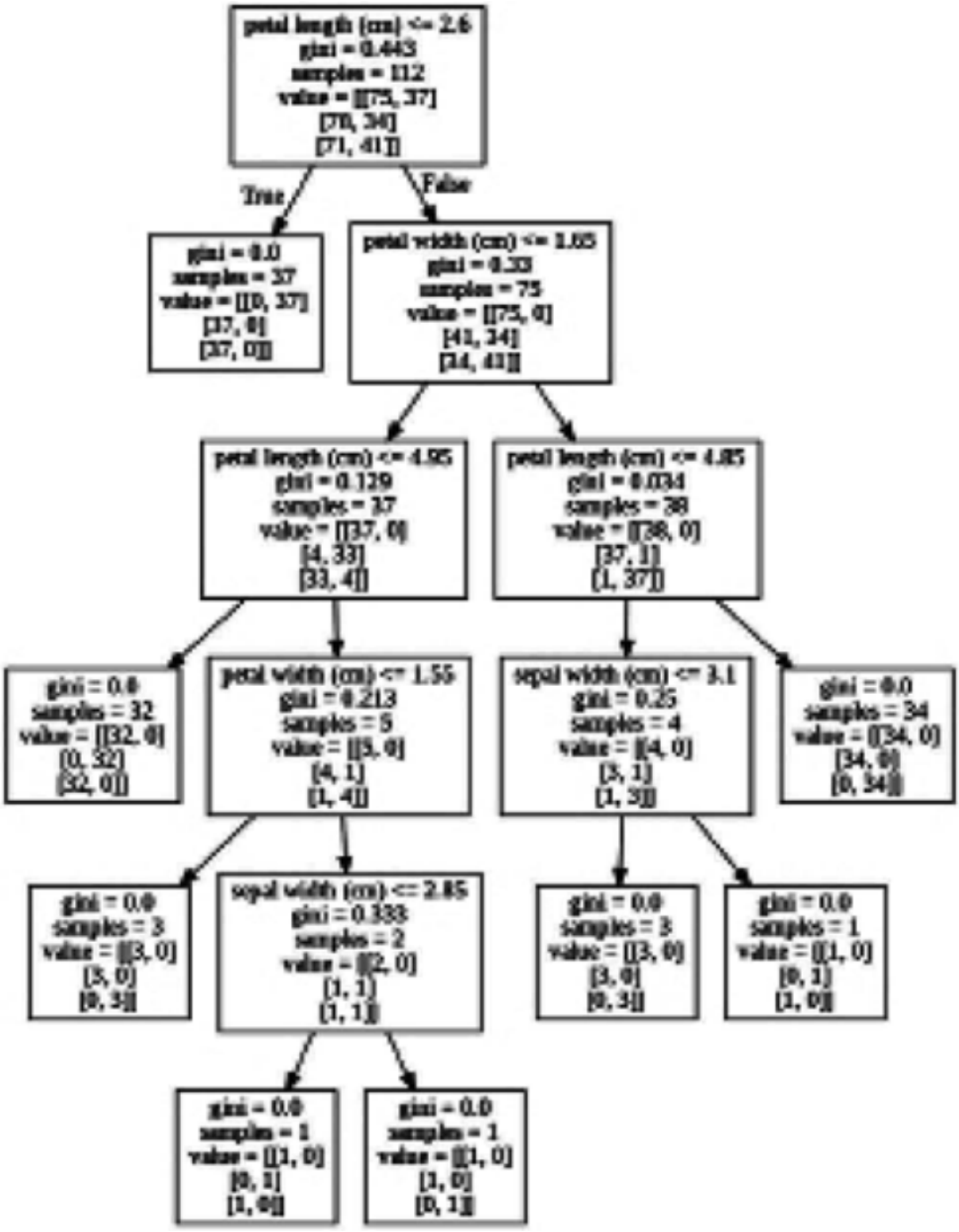
```
DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='gini',
                       max_depth=None, max_features=None, max_leaf_nodes=None,
                       min_impurity_decrease=0.0, min_impurity_split=None,
                       min_samples_leaf=1, min_samples_split=2,
                       min_weight_fraction_leaf=0.0, presort='deprecated',
                       random_state=None, splitter='best')
```

Bước 5: Biên dịch và trực quan hóa cây quyết định

```
dot_data = StringIO()
export_graphviz(dt, out_file=dot_data,
                feature_names=iris.feature_names)
(graph,) = graph_from_dot_data(dot_data.getvalue())
Image(graph.create_png())
```

Lưu ý cách nó bao gồm độ không tinh khiết Gini, tổng số mẫu, tiêu chí phân loại và số lượng mẫu bên trái/phải.

Đầu ra:



Bước 6: Dự đoán bằng ma trận nhầm lẫn

```
y_pred = dt.predict(X_test)
species = np.array(y_test).argmax(axis=1)
predictions = np.array(y_pred).argmax(axis=1)
confusion_matrix(species, predictions)
```

Đầu ra:

```
array([[13,  0,  0],
       [ 0, 15,  1],
       [ 0,  0,  9]])
```

Chúng ta có thể thấy rằng cây quyết định của chúng ta đã phân loại chính xác 37/38 cây (13+15+9=37).

Máy Vector Hỗ trợ (Support Vector Machine - SVM)

Máy Vector Hỗ trợ (SVM) là một phương pháp học máy dùng cho phân loại và phân tích hồi quy. Nó dựa trên các mô hình học có giám sát và huấn luyện bằng thuật toán. Lượng lớn dữ liệu được phân tích để tìm ra các xu hướng.

Một SVM tạo ra hai đường song song để phân vùng song song. Đối với mỗi danh mục dữ liệu, nó sử dụng hầu hết mọi thuộc tính trong một không gian nhiều chiều. Nó phân chia không gian thành các phân vùng phẳng và tuyến tính trong một lần duy nhất. Mục tiêu là chia 2 nhóm với khoảng cách rõ ràng nhất có thể. Điều này được thực hiện bằng một mặt phẳng gọi là **siêu phẳng (hyperplane)**.

Một SVM tạo ra siêu phẳng với **lề (margin)** lớn nhất để chia dữ liệu thành các lớp trong một không gian nhiều chiều.

Khoảng cách giữa hai lớp là khoảng cách lớn nhất giữa các điểm dữ liệu gần nhất của các lớp đó. Lề càng lớn, lỗi tổng quát hóa phân loại của bộ phân loại càng thấp (như trong Hình 9.8).

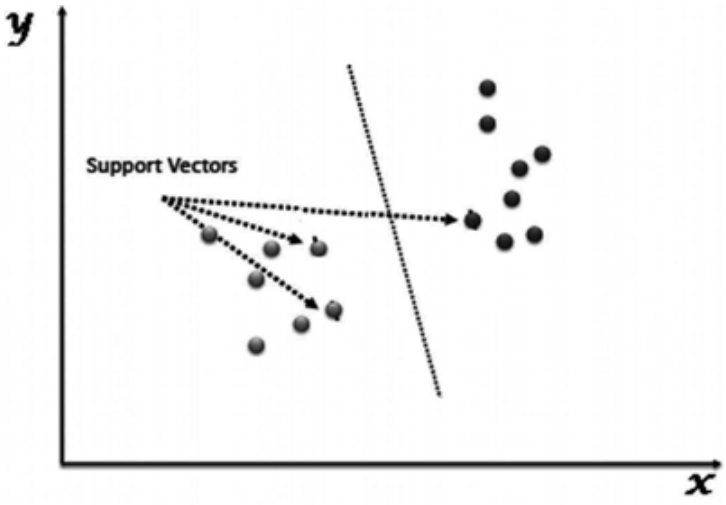


Fig. 9.8 SVM Representation

Cách thức hoạt động?

Hãy xem xét hai điều kiện để nắm bắt thuật toán SVM:

- **Trường hợp tách biệt (Separate case):** Vô số ranh giới có thể được dùng để chia hai nhóm.

- **Trường hợp không tách biệt (Non-separable case):** Không có sự phân chia rõ ràng giữa hai nhóm, mà có sự chồng chéo.

Triển khai SVM

Bước 1: Nhập các thư viện

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

Bước 2: Nhập và sử dụng bộ dữ liệu từ Google Drive

```
from google.colab import drive
drive.mount('/gdrive')
cd /gdrive
```

Đầu ra:

```
Go to this URL in a browser:
https://accounts.google.com/o/oauth2/auth?client_id=94.....
Enter your authorization code:
Mounted at /gdrive
```

Đây là mã để gắn (mount) drive của bạn với Google Research Collaboratory, nó sẽ yêu cầu bạn xác thực bằng tài khoản Gmail của mình bằng cách làm theo các bước đơn giản.

```
# nếu bạn đang sử dụng trình biên tập python, chỉ cần cung cấp đường dẫn tệp
# nơi bạn đã lưu trữ nó trên hệ thống của mình
# Nhập bộ dữ liệu bằng thư viện Seaborn
iris = pd.read_csv('/gdrive/My Drive/Colab Notebooks/IRIS.csv') #cung cấp đường dẫn cho bộ dữ liệu
# Kiểm tra bộ dữ liệu
iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Bước 3: Trực quan hóa sự tương đồng trong bộ dữ liệu bằng pair plots

```
# Tạo một pairplot để trực quan hóa sự tương đồng và đặc biệt là
# sự khác biệt giữa các loài
sns.pairplot(data=iris, hue='species', palette='Set2')
```

Đầu ra: như trong Hình 9.9

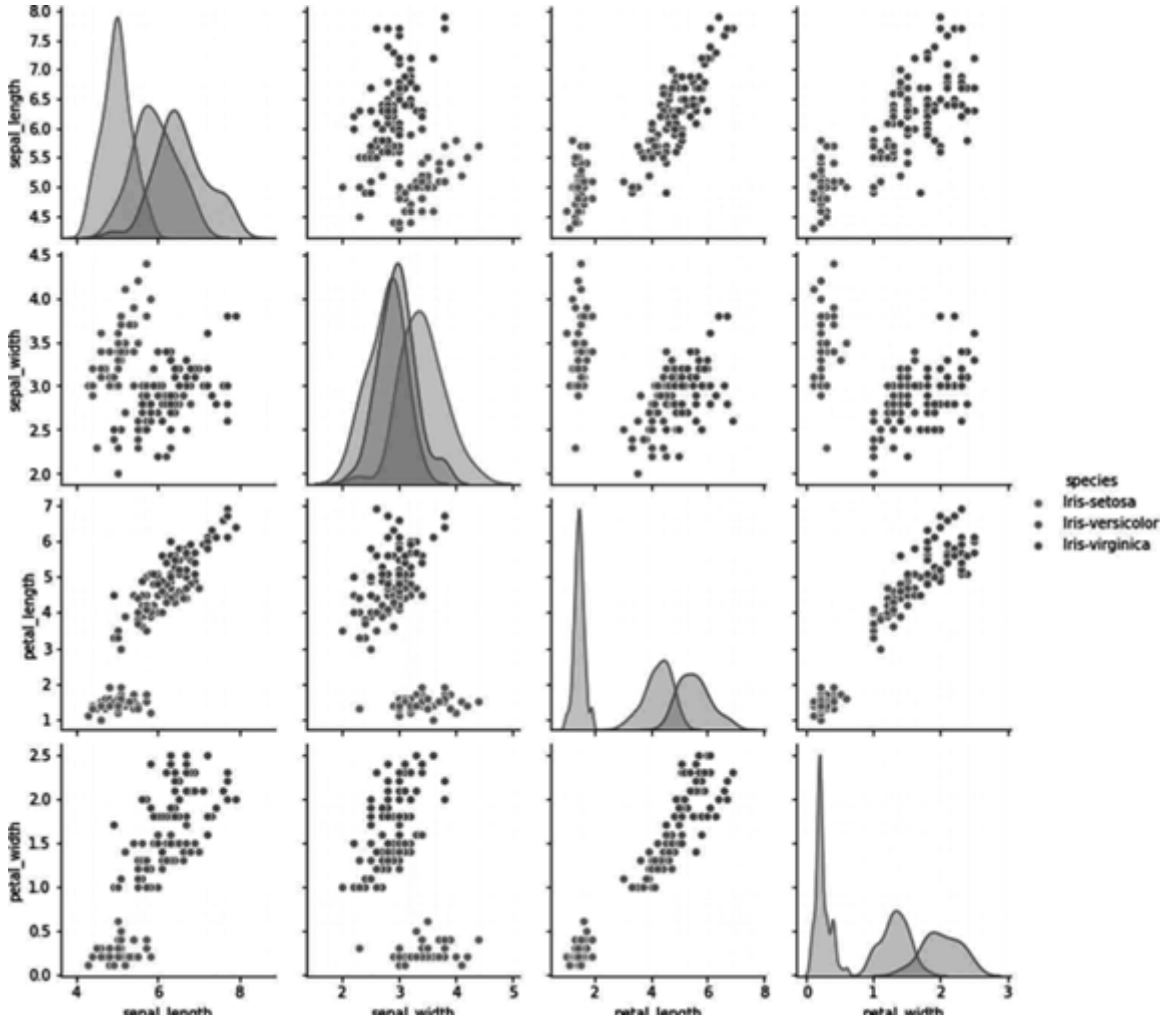


Fig. 9.9 Program output

Bước 4: Phân chia dữ liệu kiểm tra (test) và huấn luyện (train)

```
# Tách các biến độc lập khỏi các biến phụ thuộc
x = iris.iloc[:, :-1]
y = iris.iloc[:, 4]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.30)
```

Bước 5: Tải mô hình và huấn luyện

```
model = SVC()
model.fit(x_train, y_train)
```

Đầu ra:

```
SVC(C=1.0, break_ties=False, cache_size=200,
    class_weight=None, coef0=0.0, decision_function_shape='ovr',
    degree=3, gamma='scale', kernel='rbf', max_iter=-1,
```

probability=False, random_state=None, shrinking=True,
tol=0.001, verbose=False)

Bước 6: Kết quả dự đoán

```
pred = model.predict(x_test)
# Nhập báo cáo phân loại và ma trận nhầm lẫn
print(confusion_matrix(y_test, pred))
```

Đầu ra:

```
[[15  0  0]
 [ 0 14  0]
 [ 0  2 14]]
```

```
# tạo Báo cáo Phân loại
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.88	1.00	0.93	14
Iris-virginica	1.00	0.88	0.93	16
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

Naïve Bayes

Thuật toán Naive Bayes có thể được mô tả như một thuật toán Phân loại có giám sát dựa trên **định lý Bayes** với giả định **độc lập giữa các đặc trưng**. Nó cũng là một kỹ thuật phân loại.

Định lý Bayes là một nguyên tắc để xây dựng các bộ phân loại đằng sau kỹ thuật phân loại này. Các yếu tố dự đoán (predictors) được cho là độc lập. Nói một cách đơn giản, việc một đặc trưng cụ thể thuộc về một lớp không liên quan đến bất kỳ đặc trưng nào khác. Sau đây là phương trình của định lý Bayes:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Trong đó:

- $P(A \mid B)$ là xác suất của giả thuyết A khi biết dữ liệu B. Đây được gọi là **xác suất hậu nghiệm (posterior probability)**.
- $P(B \mid A)$ là xác suất của dữ liệu B khi biết giả thuyết A là đúng.
- $P(A)$ là xác suất giả thuyết A là đúng (bất kể dữ liệu). Đây được gọi là **xác suất tiền nghiệm (prior probability)** của A.
- $P(B)$ là xác suất của dữ liệu (bất kể giả thuyết).

Triển khai

Các thư viện Python cung cấp ba loại bộ phân loại Naïve Bayes:

- **Gaussian:** Giả định rằng các đặc trưng được phân phối chuẩn (Gaussian).
- **Multinomial:** Dành cho các đếm rời rạc.
- **Bernoulli:** Hữu ích nếu các vector đặc trưng là nhị phân (ví dụ: vector 0 và 1).

Trong phần triển khai, chúng ta sẽ sử dụng bộ phân loại Gaussian Naïve Bayes và bộ dữ liệu iris. Chúng ta đã biết về bộ dữ liệu iris vì chúng ta đã nghiên cứu nó trong các thuật toán trước.

Tất nhiên, trong ví dụ này chỉ liệt kê một và hai lớp. Tuy nhiên, quy trình cũng tương tự với một vector có nhiều đặc trưng và nhóm (ngoại lệ duy nhất là việc sử dụng hàm Argmax để trả về kết quả có xác suất cao nhất).

Bước 1: Nhập tất cả thư viện

```
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
# nhập các gói cần thiết
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

Bước 2: Tải và Phân chia bộ dữ liệu

```
# tải bộ dữ liệu iris, chia nó thành tập huấn luyện và
# tập xác thực (validation set)
iris = datasets.load_iris()
class_names = iris.target_names
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target
iris_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

iris_df.describe()

Đầu ra:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

Chúng ta có 4 đặc trưng và một mục tiêu dạng danh mục vì chúng ta có 3 cấp độ (Setosa, Versicolor và Virginica, tương ứng 0, 1 và 2). Giờ chúng ta có thể xây dựng bộ phân loại của mình:

```
# Bây giờ Tách dữ liệu thành kiểm tra và huấn luyện
X_train, X_test, y_train, y_test = train_test_split(
    iris_df[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']],
    iris_df['target'],
    random_state=0
)
```

Bước 3: Tải mô hình và biên dịch

```
NB = GaussianNB()
NB.fit(X_train, y_train)
y_predict = NB.predict(X_test)
print("Accuracy Naive Bayes: {:.2f}".format(NB.score(X_test, y_test)))
```

Đầu ra:

Accuracy Naive Bayes: 1.00

Như bạn có thể thấy, tất cả dữ liệu kiểm tra đều được thuật toán của chúng ta phân loại chính xác. Đây là một hiệu suất tốt, tốt nhất mà chúng ta từng đạt được. Nhưng hãy xem xét kích thước bộ dữ liệu yếu của chúng ta (chỉ 150 quan sát). Độ chính xác tiêu chuẩn 100% trên dữ liệu thực tế rất khó đạt được. Ngoài ra, lập luận về lỗi bằng không cũng có thể phản tác dụng. Thật vậy, chúng ta thích có một thuật toán có thể **tổng quát hóa** và không tuân thủ hoàn hảo dữ liệu kiểm tra, bởi vì mục tiêu cuối cùng của bất kỳ thuật toán ML nào là đưa ra dự đoán chính xác về dữ liệu mới chưa biết. Nguy cơ **overfitting** (quá khớp) sẽ dẫn đến một mô hình không bền vững nếu không thể sử dụng dữ liệu mới (sử dụng quá nhiều tham số để tạo ra một mô hình hoàn toàn phù hợp với dữ liệu kiểm tra).

Ưu điểm:

- Dễ hiểu.
- Có thể được huấn luyện trên các bộ dữ liệu nhỏ.

Nhược điểm:

- Đối với các đặc trưng có tần suất bằng 0, tổng khả năng (likelihood) là 0. Có nhiều hiệu chỉnh mẫu để giải quyết vấn đề này, chẳng hạn như "hiệu chỉnh Laplace".
- Một nhược điểm khác là nó giả định rất mạnh về tính độc lập của các đặc trưng trong lớp. Những bộ dữ liệu này trong đời thực gần như khó tìm thấy.

K-Nearest Neighbors (KNN)

Nó được sử dụng để xác định các vấn đề và khắc phục chúng. Nó thường được sử dụng để giải quyết các vấn đề phân loại. Nguyên tắc chính của thuật toán này là nó lưu trữ tất cả các trường hợp hiện có và phân loại các trường hợp mới bằng cách **bỏ phiếu theo đa số (plurality voting)** của các láng giềng. Trường hợp (dữ liệu mới) sau đó được gán cho lớp phổ biến nhất trong số **K láng giềng gần nhất** của nó, theo một hàm khoảng cách. Khoảng cách có thể là Minkowski, Manhattan, Euclidean, và Hamming.

Hãy xem xét KNN như sau:

- Về mặt tính toán, KNN tốn kém hơn các thuật toán khác được sử dụng cho các bài toán phân loại.
- Cần chuẩn hóa các biến, nếu không các biến có phạm vi (range) cao hơn có thể làm sai lệch nó.
- Tại KNN, chúng ta sẽ làm việc trên một giai đoạn như giảm nhiễu (noise reduction), tiền xử lý.

Triển khai

Bước 1: Nhập thư viện

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

Bước 2: Tải và trực quan hóa dữ liệu

```
names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
# Liên kết tải bộ dữ liệu: https://archive.ics.uci.edu/ml/machine-learning-databases/iris/
df = pd.read_csv('C:/Users/EETRO/Desktop/Book Code/iris.data.txt', header=None,
names=names) # đường dẫn lưu tệp
df.head()
```

Đầu ra:

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
setosa = df[df['class'] == 'Iris-setosa']
versicolor = df[df['class'] == 'Iris-versicolor']
virginica = df[df['class'] == 'Iris-virginica']

plt.plot(setosa['sepal_length'], setosa['sepal_width'], 'ro', label='setosa')
plt.plot(versicolor['sepal_length'], versicolor['sepal_width'], 'bo', label='versicolor')
plt.plot(virginica['sepal_length'], virginica['sepal_width'], 'go', label='virginica')
plt.xlabel('sepal_length')
plt.ylabel('sepal_width')
plt.legend()
plt.show()
```

Đầu ra: như trong Hình 9.10

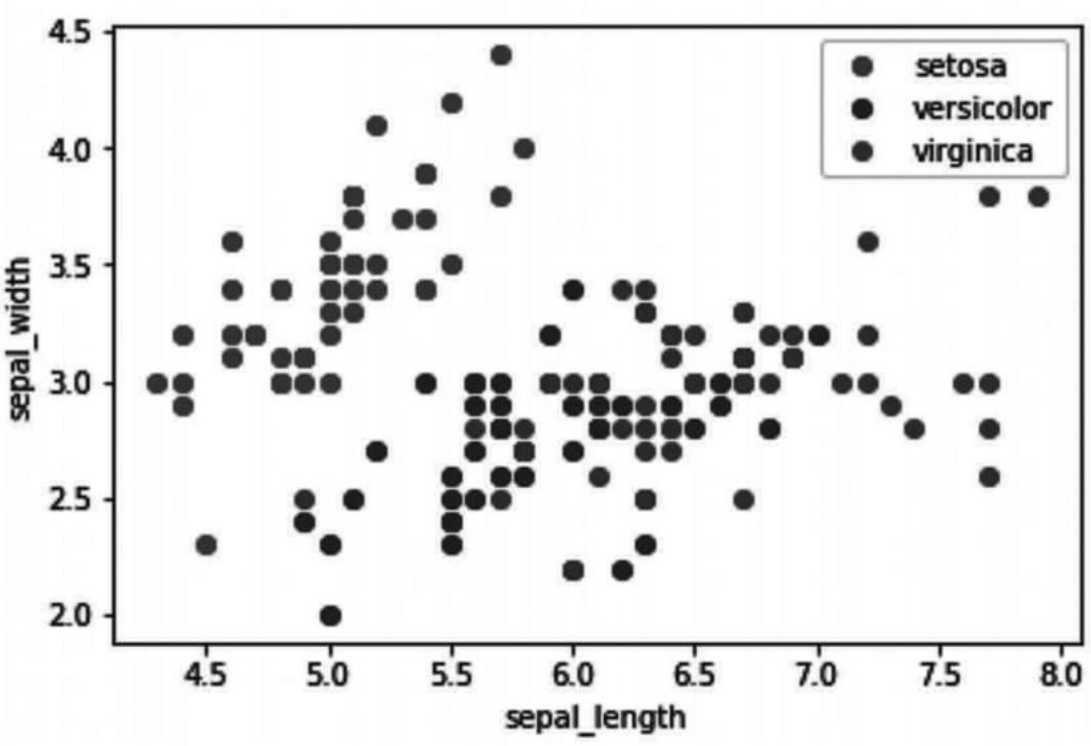


Fig. 9.10 Program Outcome

```
plt.plot(setosa['petal_length'], setosa['petal_width'], 'ro', label='setosa')
plt.plot(versicolor['petal_length'], versicolor['petal_width'], 'bo', label='versicolor')
plt.plot(virginica['petal_length'], virginica['petal_width'], 'go', label='virginica')
plt.xlabel('petal_length')
plt.ylabel('petal_width')
plt.legend()
plt.show()
```

Đầu ra: như trong Hình 9.11

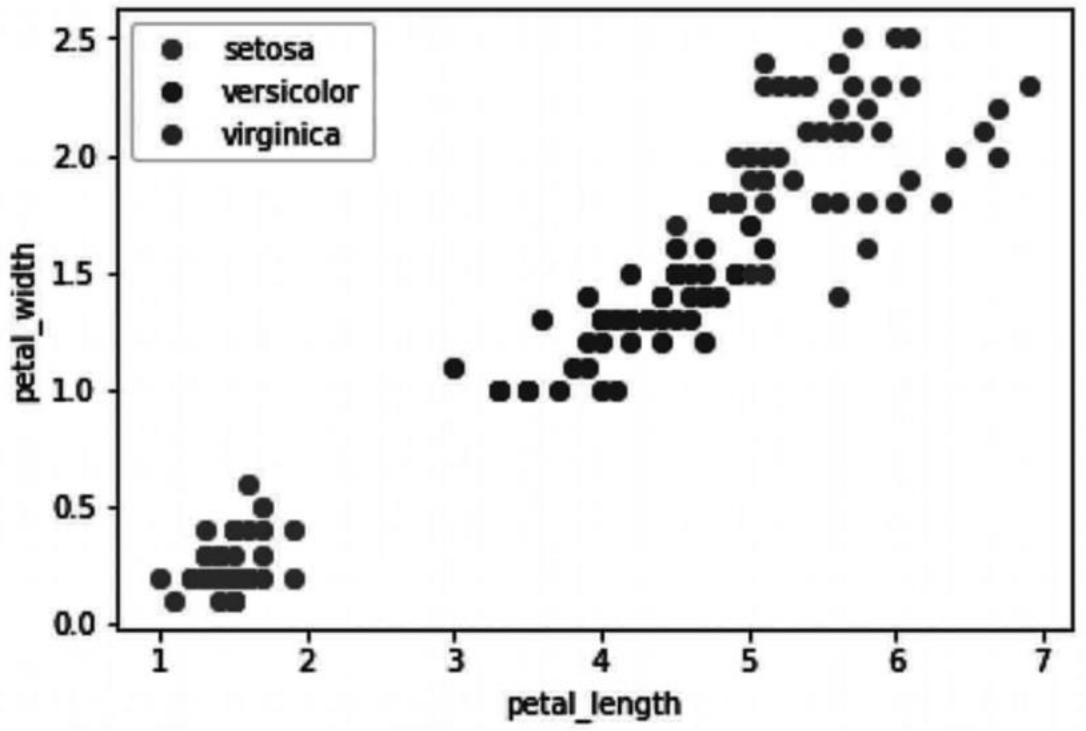


Fig. 9.11 Program outcome

Bước 3: Chia dữ liệu thành tập huấn luyện và kiểm tra với tỷ lệ 3:1

	sepal_length	sepal_width	petal_length	petal_width	class	is_train
0	5.1	3.5	1.4	0.2	Iris-setosa	True
1	4.9	3.0	1.4	0.2	Iris-setosa	True
2	4.7	3.2	1.3	0.2	Iris-setosa	False
3	4.6	3.1	1.5	0.2	Iris-setosa	True
4	5.0	3.6	1.4	0.2	Iris-setosa	True

```
df['is_train'] = np.random.uniform(0, 1, len(df)) <= .75
train, test = df[df['is_train'] == True], df[df['is_train'] == False]
print('length of train dataset: %d' % (len(train)))
print('length of test dataset: %d' % (len(test)))
```

Đầu ra:

```
length of train dataset: 111
length of test dataset: 39
```

```
train_x = train[train.columns[:len(train.columns) - 2]]
train_y = train['class']
```

```
test_x = test[test.columns[:len(test.columns) - 2]]
test_y = test['class']
```

Bước 4: Tính toán khoảng cách bằng phương pháp hàm (Euclidean Distance và Manhattan Distance)

$$EuclideanDistance = \sqrt{\sum_{i=1}^{k}(x_i - y_i)^2}$$

$$ManhattanDistance = \sum_{i=1}^{k} | x_i - y_i |$$

```
# Hàm Khoảng cách Euclidean
def euclidean_distance(data1, data2):
    distance = 0
    for i in range(data2.shape[0]):
        distance += np.square(data1[i] - data2[i])
    return np.sqrt(distance)

# Hàm Khoảng cách Manhattan
def manhattan_distance(data1, data2):
    distance = 0
    for i in range(data2.shape[0]):
        distance += abs(data2[i] - data1[i])
    return distance
```

Bước 5: Xây dựng hàm KNN

```
# Hàm KNN nhận đầu vào là
# train_x: mẫu huấn luyện,
# train_y: nhãn tương ứng,
# dis_func: hàm tính khoảng cách
# sample: một mẫu kiểm tra
# k: số lượng mẫu huấn luyện để xem xét
def knn(train_x, train_y, dis_func, sample, k):
    distances = {}
    for i in range(len(train_x)):
        d = dis_func(sample, train_x.iloc[i])
        distances[i] = d
    sorted_dist = sorted(distances.items(), key=lambda x: (x[1], x[0]))

    # lấy k láng giềng gần nhất
```

```
    neighbors = []
    for i in range(k):
        neighbors.append(sorted_dist[i][0])

    # chuyển đổi chỉ số (indices) thành các lớp
    classes = [train_y.iloc[c] for c in neighbors]

    # đếm mỗi lớp trong top k
    counts = Counter(classes)

    # bỏ phiếu cho lớp có số lượng mẫu nhiều nhất
    list_values = list(counts.values())
    list_keys = list(counts.keys())
    cl = list_keys[list_values.index(max(list_values))]
    return cl # trả về lớp của mẫu
```

Bước 6: Lấy độ chính xác của các mô hình

```
def get_accuracy(test_x, test_y, train_x, train_y, k):
    correct = 0
    for i in range(len(test_x)):
        sample = test_x.iloc[i]
        true_label = test_y.iloc[i]
        predicted_label_euclidean = knn(train_x, train_y, euclidean_distance, sample, k)
        if predicted_label_euclidean == true_label:
            correct += 1
    accuracy_euclidean = (correct / len(test_x)) * 100

    correct = 0 # reset giá trị correct về 0
    for i in range(len(test_x)):
        sample = test_x.iloc[i]
        true_label = test_y.iloc[i]
        predicted_label_manhattan = knn(train_x, train_y, manhattan_distance, sample, k)
        if predicted_label_manhattan == true_label:
            correct += 1
    accuracy_manhattan = (correct / len(test_x)) * 100

    print("model accuracy with euclidean is %0.2f" % (accuracy_euclidean))
    print("model accuracy with manhattan is %0.2f" % (accuracy_manhattan))

get_accuracy(test_x, test_y, train_x, train_y, 5)
```

Đầu ra:

model accuracy with euclidean is 94.87
model accuracy with manhattan is 94.87

Độ chính xác là tỷ lệ giữa số lượng mẫu được xác định chính xác và tổng số lượng mẫu. Ở đây, chúng ta có thể thấy rằng việc sử dụng khoảng cách Euclidean và Manhattan làm thước đo độ tương tự đều cho độ chính xác 94.87%.

Ưu điểm:

- Không có giả định về dữ liệu.
- Thuật toán đơn giản để hiểu.
- Có thể được sử dụng cho hồi quy và phân loại.

Nhược điểm:

- Yêu cầu bộ nhớ cao - Tất cả dữ liệu huấn luyện phải nằm trong bộ nhớ để đo lường K láng giềng gần nhất.
- Nhạy cảm với các đặc trưng liên quan.
- Nhạy cảm với kích thước của dữ liệu vì phải tính toán khoảng cách đến K điểm gần nhất.

Phân cụm K-Means (K-means Clustering)

Đúng như tên gọi, nó được sử dụng để giải quyết vấn đề phân cụm. Đây là một loại **Học không giám sát**. Nguyên tắc chính của thuật toán K-Means là gán bộ dữ liệu vào các cụm khác nhau. Thuật toán phân cụm K-means được sử dụng để xác định và tìm ra các mẫu, đồng thời đưa ra quyết định tốt hơn cho các nhóm không được gắn nhãn rõ ràng trong dữ liệu. Dữ liệu mới có thể dễ dàng được phân bổ vào nhóm thích hợp nhất sau khi thuật toán được thực hiện và các nhóm được xác định.

Để bắt đầu với k-means, bạn phải khởi tạo các **tâm cụm (K centroids)** một cách ngẫu nhiên. K-means là một thuật toán lặp, gồm hai bước (như trong Hình 9.12):

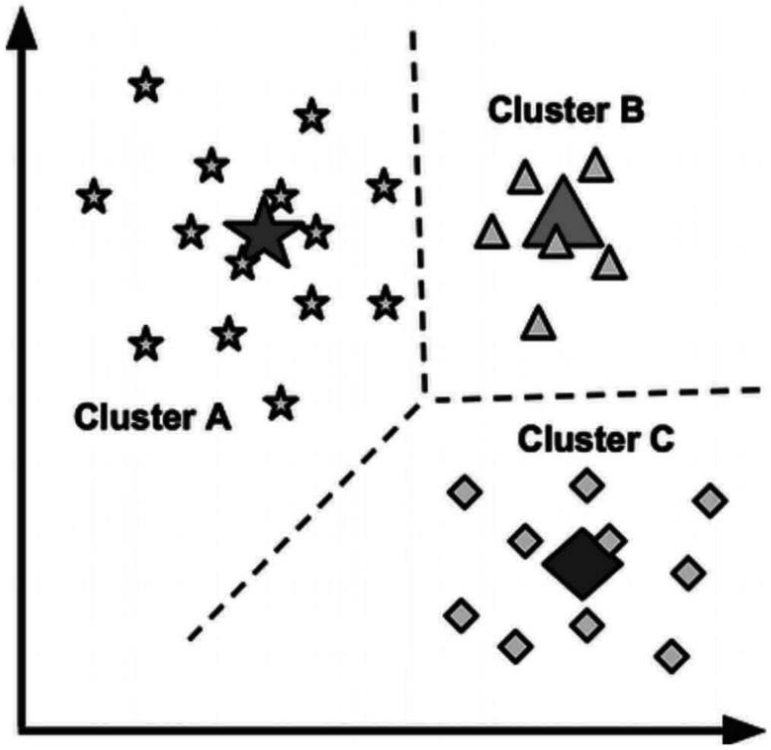


Fig. 9.12 Data Points Clustering

1. **Gán cụm (Cluster Assignment):** Thuật toán duyệt qua mọi điểm dữ liệu và gán điểm dữ liệu đó cho một trong các tâm cụm, tùy thuộc vào cụm nào gần hơn.
2. **Di chuyển tâm cụm (Move to centroid):** Các tâm cụm được di chuyển đến giá trị trung bình của các cụm được hình thành sau khi gán cụm.

Và sau đó nó lặp lại chu kỳ. Các tâm cụm không thể di chuyển nữa sau một số bước nhất định và sau đó các lần lặp có thể kết thúc.

Triển khai

Bước 1: Nhập thư viện

```
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans
```

Bước 2: Tải dữ liệu

```
iris = datasets.load_iris()
```

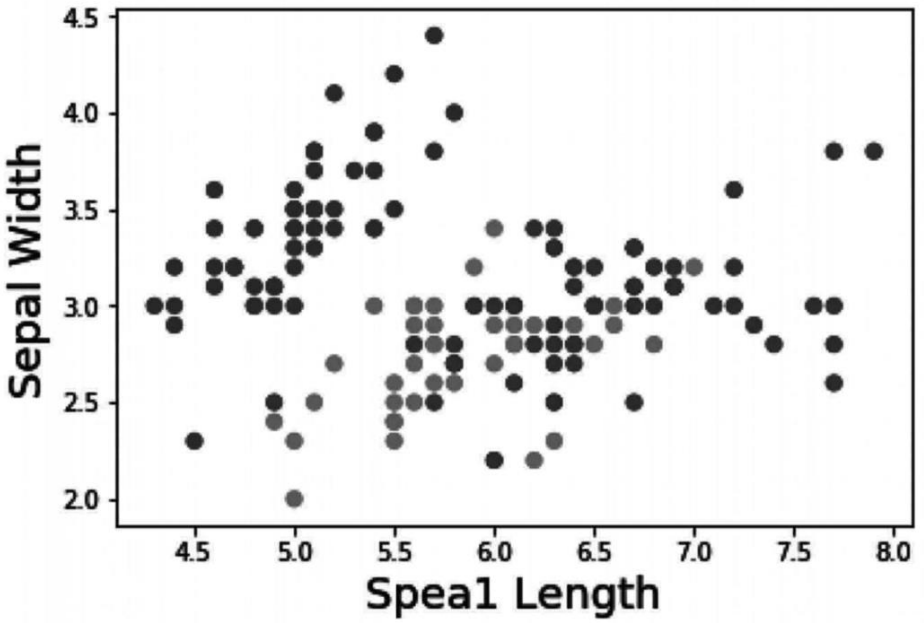
Bước 3: Xác định mục tiêu và các yếu tố dự đoán (Predictors)

```
X = iris.data[:, :2]
y = iris.target
```

Bước 4: Trực quan hóa dữ liệu

```
plt.scatter(X[:,0], X[:,1], c=y, cmap='gist_rainbow')
plt.xlabel('Speal Length', fontsize=18)
plt.ylabel('Sepal Width', fontsize=18)
```

Đầu ra: như trong Hình 9.13



Hình 9.13 Program outcome

Bước 5: Tải và Biên dịch mô hình

```
km = KMeans(n_clusters=3, n_jobs=4, random_state=21)
km.fit(X)
```

Đầu ra:

```
KMeans(algorithm='auto', copy_x=True, init='k-means++',
       max_iter=300, n_clusters=3, n_init=10, n_jobs=4,
       precompute_distances='auto', random_state=21, tol=0.0001,
       verbose=0)
```

Bước 6: Lấy Tâm cụm (Centroid)

```
centers = km.cluster_centers_
print(centers)
```

Đầu ra:

```
[[5.77358491 2.69245283]
 [5.006      3.428     ]
 [6.81276596 3.07446809]]
```

Bước 7: So sánh dữ liệu gốc và dữ liệu đã phân cụm

```
# điều này sẽ cho chúng ta biết các quan sát dữ liệu thuộc về cụm nào.
new_labels = km.labels_
# Vẽ các cụm đã xác định và so sánh với câu trả lời
fig, axes = plt.subplots(1, 2, figsize=(16,8))
axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap='gist_rainbow', edgecolor='k', s=150)
axes[1].scatter(X[:, 0], X[:, 1], c=new_labels, cmap='jet', edgecolor='k', s=150)
axes[0].set_xlabel('Sepal length', fontsize=18)
axes[0].set_ylabel('Sepal width', fontsize=18)
axes[1].set_xlabel('Sepal length', fontsize=18)
axes[1].set_ylabel('Sepal width', fontsize=18)
axes[0].tick_params(direction='in', length=10, width=5, colors='k', labelsize=20)
axes[1].tick_params(direction='in', length=10, width=5, colors='k', labelsize=20)
axes[0].set_title('Actual', fontsize=18)
axes[1].set_title('Predicted', fontsize=18)
```

Đầu ra: như trong Hình 9.14

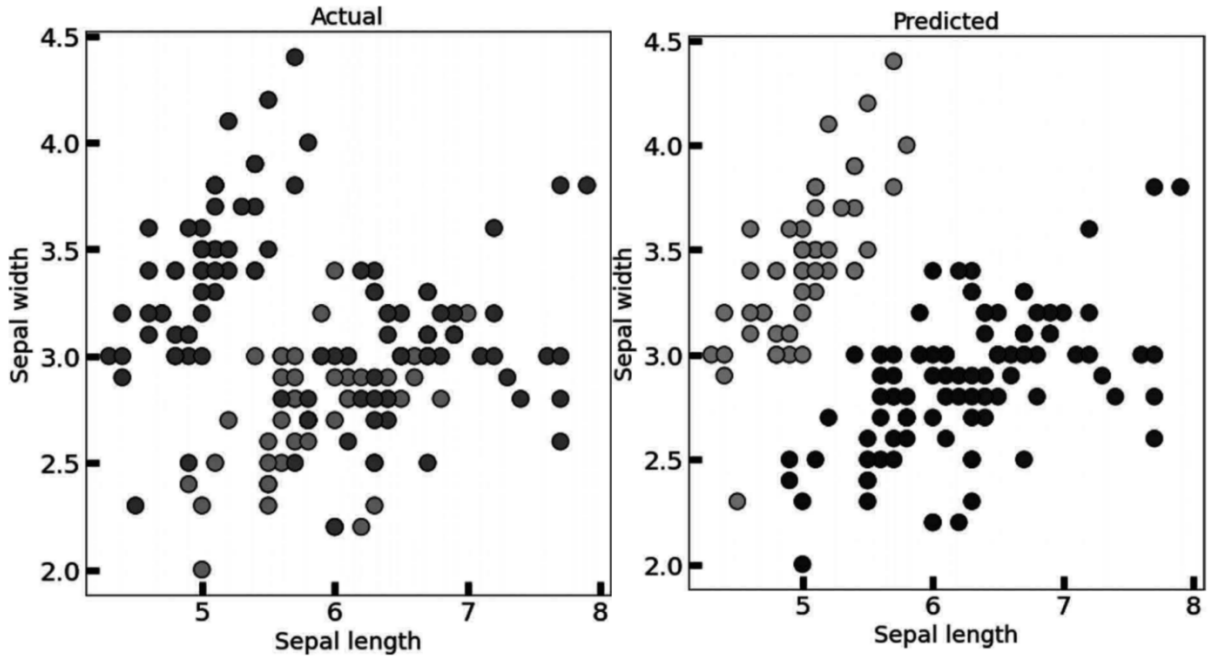


Fig. 9.14 Program outcome

Ưu điểm:

- K-means đơn giản và hiệu quả về mặt máy tính.
- Nó rất trực quan và nhanh chóng để hình dung các ảnh hưởng.

Nhược điểm:

- K-means phụ thuộc nhiều vào quy mô (scale) và không được thiết kế cho dữ liệu có mật độ và hình dạng khác nhau.
- Việc ước tính hiệu suất chủ quan hơn. Nó cần nhiều đánh giá của con người hơn là các chỉ số tin cậy.

Random Forest (Rừng ngẫu nhiên)

Đây là một thuật toán phân loại có giám sát. Thuật toán random forest có ưu điểm là có thể được sử dụng cho các bài toán phân loại cũng như hồi quy.

Về cơ bản, đây là tập hợp các cây quyết định (khu rừng) hoặc một **tập hợp (ensemble)** của các cây quyết định.

Nguyên tắc cơ bản của random forest là mỗi cây đều được "chấm điểm" (graded) và "khu rừng" sẽ chọn ra những "điểm" tốt nhất (lấy kết quả bỏ phiếu đa số).

Sự khác biệt giữa thuật toán random forest và cây quyết định là quá trình xác định nút gốc và phân chia các nút chức năng diễn ra ngẫu nhiên trong random forest.

Triển khai

Bước 1: Nhập tất cả các thư viện cần thiết

```
# Nhập Mô hình Random Forest
from sklearn.ensemble import RandomForestClassifier
# Nhập thư viện dataset của scikit-learn
from sklearn.model_selection import train_test_split
from sklearn import datasets
import pandas as pd
# Nhập mô-đun metrics của scikit-learn để tính độ chính xác
from sklearn import metrics
```

Bước 2: Tải và Trực quan hóa dữ liệu

```
# Tải bộ dữ liệu
iris = datasets.load_iris()
# in tên các loài (setosa, versicolor, virginica)
print(iris.target_names)
# in tên của bốn đặc trưng
print(iris.feature_names)
```

Đầu ra:

```
['setosa' 'versicolor' 'virginica']
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

```
# in dữ liệu iris (5 bản ghi đầu tiên)
print(iris.data[0:5])
# in nhãn iris (0:setosa, 1:versicolor, 2:virginica)
print(iris.target)
```

Đầu ra:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]
```

```
# Tạo một DataFrame từ bộ dữ liệu iris đã cho.
data = pd.DataFrame({
    'sepal length': iris.data[:,0],
    'sepal width': iris.data[:,1],
    'petal length': iris.data[:,2],
    'petal width': iris.data[:,3],
    'species': iris.target
})
data.head()
```

Đầu ra:

	sepal length	sepal width	petal length	petal width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Bước 3: Phân chia dữ liệu huấn luyện và kiểm tra

```
# Nhập hàm train_test_split
X = data[['sepal length', 'sepal width', 'petal length', 'petal width']] # Đặc trưng
y = data['species'] # Nhãn
# Chia bộ dữ liệu thành tập huấn luyện và tập kiểm tra
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3) # 70% huấn luyện và 30% kiểm tra
```

Bước 4: Nhập mô hình và biên dịch

```
# Tạo một bộ phân loại Gaussian (ở đây là Random Forest)
clf = RandomForestClassifier(n_estimators=100)
# Huấn luyện mô hình bằng cách sử dụng các tập huấn luyện
clf.fit(X_train, y_train)
```

Bước 5: Xem độ chính xác và dự đoán của mô hình

```
y_pred = clf.predict(X_test)
# Độ chính xác của mô hình, bộ phân loại đúng bao nhiêu lần?
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Đầu ra: (Lưu ý: Độ chính xác có thể thay đổi do train_test_split ngẫu nhiên, văn bản gốc ra 1.0)

Accuracy: 1.0

```
# Dự đoán một mẫu mới
prediction = clf.predict([[3, 5, 4, 2]])
species_idx = prediction[0]
print(iris.target_names[species_idx])
```

Đầu ra:

'versicolor'

Ưu điểm:

- Bộ phân loại random forest có thể được sử dụng cho các chức năng phân loại và hồi quy.
- Có thể quản lý các giá trị bị thiếu.
- Ngay cả khi chúng ta có nhiều cây hơn trong rừng, điều này sẽ không làm mô hình bị overfit (quá khớp).

Nhược điểm:

- Về bản chất, một mô hình ensemble (tổ hợp) ít diễn giải hơn một cây quyết định.
- Một số lượng lớn các cây sâu có thể tốn kém chi phí (nhưng có thể chạy song song) và sử dụng nhiều bộ nhớ.
- Dự đoán chậm hơn, điều này có thể gây ra các vấn đề khi triển khai.

Tóm tắt

Các thuật toán học máy đang được sử dụng cho rất nhiều mục đích và giúp chúng ta có được kết quả hữu ích bằng cách phân tích dữ liệu có sẵn từ một số tài nguyên. Ở đây chúng ta có các thuật toán khác nhau của học máy như hồi quy tuyến tính, hồi quy logistic, cây quyết định, v.v., đã được thảo luận trong chương này với việc triển khai bằng cách sử dụng bộ dữ liệu thời gian thực tên là Bộ dữ liệu Iris, đây là bộ dữ liệu nổi tiếng nhất bao gồm dữ liệu về các loài hoa. Từ phần triển khai của thuật toán, chúng ta có thể hiểu cách các thuật toán này có thể được triển khai trong thời gian thực. Ưu điểm và nhược điểm của các thuật toán cũng đã được nêu để hiểu rõ hơn.

CHƯƠNG 10 PHÂN LOẠI VÀ PHÁT HIỆN BỆNH Ở THỰC VẬT

Giới thiệu Trong nông nghiệp, bệnh cây trồng luôn là một vấn đề lớn, vì chúng làm giảm chất lượng cây trồng, dẫn đến giảm năng suất. Bệnh cây trồng có nhiều ảnh hưởng khác nhau, từ các triệu chứng nhỏ đến thiệt hại nghiêm trọng trên toàn bộ diện tích cây trồng, dẫn đến chi phí tài chính cao và ảnh hưởng đáng kể đến nền kinh tế nông nghiệp, đặc biệt là ở các nước đang phát triển phụ thuộc vào một hoặc một vài loại cây trồng.

Nhiều phương pháp khác nhau đã được phát triển để phát hiện bệnh nhằm tránh những tổn thất đáng kể. Các tác nhân gây bệnh đã được xác định chính xác bằng các phương pháp phát triển trong sinh học phân tử và miễn dịch học. Tuy nhiên, đối với nhiều nông dân, những kỹ thuật này khó tiếp cận và đòi hỏi kiến thức chuyên sâu về lĩnh vực hoặc tốn nhiều thời gian và công sức để sử dụng.

Theo Tổ chức Lương thực và Nông nghiệp Liên Hợp Quốc, hầu hết các trang trại trên thế giới là nhỏ và thuộc sở hữu gia đình ở các nước phát triển. Những gia đình như vậy cung cấp lương thực cho một tỷ lệ đáng kể dân số hành tinh. Tuy nhiên, đói nghèo và mất an ninh lương thực không hiếm gặp và khả năng tiếp cận thị trường cũng như dịch vụ còn hạn chế.

Vì những lý do trên, rất nhiều công trình đã được thực hiện nhằm thiết lập các phương pháp đủ tin cậy và sẵn có cho hầu hết nông dân. Trong đó, bệnh cây trồng là một yếu tố thiết yếu góp phần làm suy giảm nghiêm trọng chất lượng và số lượng sản xuất cây trồng. Nhận dạng và phân loại bệnh cây trồng đóng vai trò quan trọng trong việc cải thiện sản xuất cây trồng và phát triển kinh tế.

Khi nhận thấy bệnh cây trồng đã trở thành một mối quan tâm lớn trong nông nghiệp như thế nào, nhiều công nghệ đã được sử dụng để phát hiện bệnh một cách dễ dàng và trí tuệ nhân tạo là một trong số đó. Ví dụ, để phát hiện và phân loại bệnh cây trồng, ở đây bằng cách sử dụng học sâu (deep learning) của Trí tuệ nhân tạo, một mô hình sẽ được tạo ra và huấn luyện dựa trên bộ dữ liệu từ Kaggle là plant village (làng thực vật). Nó có thể hữu ích cho việc xác định bệnh cây trồng theo thời gian thực. Trong Dự án, chúng ta sẽ tạo một mô hình CNN (Mạng nơ-ron tích chập) được sử dụng để phân loại và phát hiện bệnh ở khoai tây dựa trên dữ liệu mà chúng ta huấn luyện nó.

Cài đặt các Gói (Packages)

Để phát triển một mô hình hoặc dự án về phát hiện bệnh cây trồng, chúng ta sẽ yêu cầu một số thư viện của Trí tuệ nhân tạo và Python. Ở đây chúng ta đã thảo luận từng bước cài đặt và mô tả của chúng để hiểu rõ hơn.

Để cài đặt tất cả các gói thư viện, chúng ta sẽ sử dụng PIP trong Windows. Các thư viện được sử dụng để phát triển dự án này là:

- Keras: Lệnh cài đặt trên Windows: pip install keras
- NumPy: Lệnh cài đặt trên Windows: pip install keras

- Math: Lệnh cài đặt trên Windows: pip install math
- Matplotlib: Lệnh cài đặt trên Windows: pip install matplotlib
- TensorFlow: Lệnh cài đặt trên Windows: pip install tensorflow

Bộ dữ liệu (Dataset)

Ở đây chúng ta lấy một Bộ dữ liệu tên là Plant village từ Kaggle, có 15 danh mục dựa trên các bệnh cây trồng khác nhau. Chúng ta đã chọn phát hiện bệnh ở cây khoai tây, trong đó chúng ta có 3 danh mục như Bệnh cháy sớm (Early Blight), Bệnh cháy muộn (Late Blight) và khoai tây khỏe mạnh (healthy potato). Đối với mỗi lớp (class) này, chúng ta sẽ có hình ảnh làm đầu vào để huấn luyện mô hình dự đoán. Bộ dữ liệu có thể được tải về từ đây. https://www.kaggle.com/emmarex/plantdisease

Các bước thực hiện

Trong bước đầu tiên của việc phát hiện bệnh, trước hết chúng ta sẽ nhập (import) tất cả các thư viện cần thiết cho việc phát triển dự án. Trong quá trình phát triển dự án này, Google Research Collaboratory đã được sử dụng và có thể truy cập bằng liên kết này http://colab.research.google.com/ . Bạn cũng có thể sử dụng Anaconda để chạy dự án này nhưng vui lòng chỉ định tất cả các đường dẫn liên quan đến bộ dữ liệu và hình ảnh kiểm tra vào dự án.

Bước 1: Đầu tiên chúng ta sẽ nhập các thư viện cơ bản cần thiết để phát triển dự án phát hiện bệnh này:

```
import numpy as np
import math
import keras
import matplotlib.pyplot as plt
from tensorflow.keras.layers import *
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from tensorflow.keras import models
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.optimizers import *
from tensorflow.keras.callbacks import LearningRateScheduler
```

Output: Using TensorFlow backend. Đây là tất cả các thư viện sẽ được yêu cầu và xử lý trong suốt dự án cho các mục đích khác nhau.

Bước 2: Có hai phương pháp mà chúng ta có thể nhập dữ liệu cho dự án Phương pháp thứ nhất:

```
!pip install kaggle
!mkdir .kaggle
```

(Phần này nằm ở trang 3)

```
import json
token = {"username":"","key":""}
with open (1/content/.kaggle/kaggle.json', 'w') as file:
 json.dump (token, file)
!chmod 600 /content/.kaggle/kaggle.json
!cp/content/.kaggle/kaggle.json /root/.kaggle/kaggle.json
 !kaggle datasets download -d xabdallahali/plantvillage-dataset/
 color -p/content
```

Output: Downloading plantvillage-dataset.zip to /content

99% 2.02G/2.04G [00:44<00:00, 0.6MB/s]

100% 2.04G/2.04G [00:44<00:00, 1.6MB/s]

Ở đây chúng ta tải trực tiếp bộ dữ liệu từ Kaggle, bạn có thể sử dụng nó như trong tệp của mình hoặc bạn có thể sử dụng phương pháp thứ hai.

Phương pháp thứ hai: Nếu bạn đang sử dụng Google Research Collaboratory thì bạn có thể tải bộ dữ liệu từ liên kết này https://www.kaggle.com/emmarex/plantdisease và tải nó lên thư mục "Colab Notebooks" trên Google Drive của bạn, sau đó bạn có thể liên kết nó bằng cách sử dụng mã này

```
from google.colab import drive
drive.mount ('/gdrive')
```

Output:

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client _ id=94…….) Enter your authorization code:

Mounted at /gdrive

Đây là mã để gắn (mount) drive của bạn với Google Research Collaboratory, nó sẽ yêu cầu bạn xác thực mà bạn có thể thực hiện bằng tài khoản Gmail của mình chỉ bằng cách làm theo các bước đơn giản. Như bạn có thể thấy trong output, Collaboratory sẽ yêu cầu bạn xác minh bằng cách nhấp vào liên kết đã cho, sau đó nó sẽ được gắn bằng cách cấp quyền xác thực cho nó.

```
with open ('/gdrive/My Drive/foo.txt', 'w') as f:
f.write ('Hello Google Drive!')
!cat '/gdrive/My Drive/foo.txt'
```

Output:

Hello Google Drive!

Đây là mã sẽ giúp bạn kiểm tra xem drive của bạn đã được gắn với Collaboratory hay chưa.

Sau khi thực hiện việc này, chúng ta sẽ truy cập bộ dữ liệu đã được tải lên trong drive trực tiếp và giải nén nó để truy cập các tệp mà chúng ta yêu cầu trong dự án này.

```
from zipfile import ZipFile
# specifying the zip file name
file name = "plantvillage-dataset.zip" #path for location of
 dataset
# opening the zip file in READ mode
with ZipFile (file name, 'r') as zi:
# extracting all the files
print ('Extracting all the files now...')
zi.extractall ()
print ('Done!')
```

ouput:

Extracting all the files now…

Done!

Ở đây chúng ta chỉ đơn giản là giải nén tất cả dữ liệu từ thư mục ZIP của PlantVillage để sử dụng sau này.

#splitting data into train and test (chia dữ liệu thành tập huấn luyện và tập kiểm tra)

```
!pip install split-folders tqdm
```

Output:

Collecting split-folders Downloading https://files.pythonhosted.org/packages/20/67/29dda743e6d23aclea3d16704d8bbb48d65faf3f1b1eaf53153b3da56c56/split_folders-0.3.1-py3-none-any.whl Requirement already satisfied: tqdm in /usr/local/lib/python3.6/dist-packages (4.38.0) Installing collected packages: split-folders Successfully installed split-folders-0.3.1

```
!mkdir potato
```

```
! cp -r '/content/plantvillage dataset/segmented/Potato _ _ _ Early _ blight' 'potato/early blight'
```

```
! cp -r '/content/plantvillage dataset/segmented/Potato _ _ _ Late _ blight' 'potato/late blight'
```

```
! cp -r '/content/plantvillage dataset/segmented/Potato _ _ _ healthy' 'potato/healthy'
```

Ở đây chúng ta đang tạo các thư mục để phát hiện bệnh khoai tây trong thư mục đã được giải nén từ bộ dữ liệu plant village.

```
import split folders
# Split with a ratio. (Chia theo tỷ lệ)
```

To only split into training and validation set, set a tuple
to 'ratio', ie, ' (.8, .2)'. (Để chỉ chia thành tập huấn luyện và tập xác thực, hãy đặt một bộ giá trị cho 'ratio', ví dụ: ' (.8, .2)')
#split folders.ratio ('color', output="dataset", seed $=1337$,
ratio= (.8, 1, 1)) # default values (giá trị mặc định)
split folders.ratio ('potato', output="dataset", seed $l=133^{-}$,
ratio (.8, 0.2)) # default values (giá trị mặc định)

Output:

Copying files: 2152 files [00:00, 9225.64 files/s]

Bước 3: Xây dựng mô hình CNN Đầu tiên, chúng ta sẽ xây dựng Mô hình CNN của mình, mã nguồn như sau.

```
def model _ cnn ():

   model = Sequential ([

     BatchNormalization (axis=1, input _ shape= (256,256,3)),

     Convolution2D (32,3,3, activation='relu'), BatchNormalization
     (axis=1), MaxPooling2D ( (3,3)),

     Convolution2D (64,3,3, activation='relu'), BatchNormalization
     (axis=1), MaxPooling2D ((3,3)),

     Flatten (),

     Dense (200, activation='relu'),

     Dropout (0.2), BatchNormalization (),

     Dense (3, activation='softmax')

    ])

return model
```

Loại mô hình mà chúng ta đang sử dụng là cơ bản và dễ dàng để tạo bất kỳ mô hình CNN nào, đó là Sequential. Ở đây chúng ta đã định nghĩa toàn bộ mô hình thành một hàm để có thể gọi nó bất cứ khi nào cần thiết. Và Sequential là phương pháp dễ nhất để xây dựng nó bằng Keras. Nó cho phép chúng ta xây dựng mô hình CNN từng Lớp (Layer by layer). Trong Mô hình, Lớp đầu tiên của chúng ta là Convolution2D (32,3,3, Activation=‘relu’), Nó xử lý các hình ảnh đầu vào và đối số 32 Mô tả số lượng nút (nodes) trong lớp có thể thay đổi theo thời gian dựa trên kích thước bộ dữ liệu. Trong đó 3,3 trong đối số mô tả kích thước hạt nhân (kernel size) là kích thước của ma trận bộ lọc cho lớp tích chập của chúng ta trong mô hình. Vì vậy, kích thước hạt nhân là 3 có nghĩa là chúng ta có ma trận bộ lọc 3X3 cho lớp mô hình của mình. Đối số Activation là hàm kích hoạt cho lớp, là ReLU hoặc Rectified Liner Activation (Kích hoạt Tuyến tính Chỉnh lưu) hoạt động tốt cho các mạng nơ-ron.

Sau đó, chúng ta sẽ sử dụng Batch Normalization (axis=1) để chuẩn hóa và tái tỷ lệ dữ liệu, trong đó đối số axis = 1 đại diện cho kênh (channel). Sau đó, chúng ta sẽ sử dụng max pooling 2D để chọn các đặc trưng tối đa được bao phủ bởi các bộ lọc. Trong lớp Tiếp theo Convolution2D (64,3,3, Activation='relu'), trong lớp này, chúng ta sẽ có 64 nút để xử lý hình ảnh đầu vào và tiếp tục chúng ta sẽ lại có cả chuẩn hóa hàng loạt (batch normalization) và gộp tối đa (max pooling). Giữa các lớp, chúng ta có lớp "flatten ()" luôn đóng vai trò là kết nối giữa lớp tích chập (convolution) và lớp dày đặc (dense). Trong mô hình, chúng ta sẽ có 2 lớp đầu ra, trong đó lớp đầu tiên là lớp đầu ra Dense (200, activation='reli'). Dense là một loại tiêu chuẩn trong mạng nơ-ron có thể được sử dụng cho nhiều mục đích. Ở đây, đối số 200 trong lớp dense có nghĩa là chúng ta sẽ có 200 nút trong lớp đầu ra của mình, một nút cho mỗi kết quả có thể có từ 0-199. Sau đó, chúng ta sử dụng dropout (0.2) được dùng để loại bỏ các đơn vị nhất định khỏi mô hình CNN, điều này sẽ giúp chúng ta ngăn chặn hiện tượng quá khớp (over fitting) trong mô hình và là một cách tiếp cận để điều chuẩn (regularization) trong mạng, giúp giảm sự phụ thuộc lẫn nhau giữa các nút của mạng. Sau đó, chúng ta sẽ sử dụng Batch Normalization trong lớp để chuẩn hóa và tái tỷ lệ dữ liệu. Và sau đó chúng ta có lớp đầu ra cuối cùng và thứ 2 là Dense (3, activation='softmax') có 3 chế độ cho mọi đầu ra từ 0-2 và hàm kích hoạt là softmax. Chúng ta cũng có thể lập trình mô hình tương tự bằng cách sử dụng hàm add, được dùng để lập trình từng lớp của mô hình CNN. Mã thay thế cho mô hình được xây dựng ở trên bằng hàm add:

Bước 4: Bây giờ chúng ta sẽ tạo dữ liệu cho mô hình và sau đó thực hiện huấn luyện.

```
train _ datagen = ImageDataGenerator (rescale=1./255)

validation _ datagen = ImageDataGenerator (rescale=1./255) image _ size = 256

# Change the batchsize according to your system RAM train _ batchsize = 20

val _ batchsize = 20

train _ generator = train _ datagen.flow _ from _ directory ( 'dataset/train',

   target _ size= (image _ size, image _ size), batch _ size=train _
   batchsize, class _ mode='categorical')

   validation _ generator = validation _ datagen.flow_from_directory (

   'dataset/val',

   target _ size= (image _ size, image _ size), batch _ size=val _
   batchsize,

   class _ mode='categorical', shuffle=True)
```

Output:

Found 1721 images belonging to 3 classes.

Found 431 images belonging to 3 classes

Ở đây chúng ta đang tạo bộ dữ liệu liên quan đến khoai tây và chia chúng thành các tập huấn luyện (training) và kiểm tra (testing) cho mô hình CNN của mình. Như bạn có thể thấy trong mã, chúng ta đã khai báo kích thước hình ảnh (image size) và kích thước lô (batch size) có thể thay đổi tùy theo nhu cầu của bộ dữ liệu và hệ thống.

Ở đây chúng ta đang tạo một hàm có tốc độ học (learning rate) sẽ thay đổi dựa trên các thay đổi của epoch (lượt huấn luyện) để chúng ta có thể nhận được kết quả tốt hơn từ mô hình đã huấn luyện. Lưu ý: Chúng ta cũng có thể giữ một tốc độ học không đổi cho tất cả các epoch. Đó hoàn toàn là mong muốn của chúng ta. Đôi khi nếu chúng ta không nhận được độ chính xác yêu cầu khi kiểm tra mô hình, chúng ta sẽ thay đổi tốc độ học.

```
def step decay (epoch):
 if epoch<=6:
return le-3
elif epoch< $<=20$:
return le-4
else:
return 1e-5
lrate
LearningRateScheduler (step_decay)
```

Ở đây chúng ta sẽ gọi lại mô hình của mình và bắt đầu huấn luyện mô hình bằng cách đặt các giá trị tốc độ học và trình tối ưu hóa (optimizers) để có kết quả tốt hơn mà bạn có thể thấy trong các output.

```
#define model (định nghĩa mô hình)
model model cnn ()
#calling the learning rate (gọi tốc độ học)
callbacks = [lrate]
# Compile the model (Biên dịch mô hình)
model.compile (loss='categorical
crossentropy',
optimizer=Adam $(lr=0.0001$, beta $1=0.9$, beta $2=0.999$,
epsilon-le-08, decay $=0.0?$, metrics=['acc'])
history model.fit generator (
train generator,
steps per epoch train generator.samples/train generator.
batch
size
validation
data=validation
generator,
generator.batch size,
validation_steps=validation generator.samples/validation_
```

epochs $=23$,
callbacks = callbacks)

Output:

(Phần output của quá trình huấn luyện Epoch 1-23 được tóm tắt) Epoch 1/23… … Epoch 6/23… … val loss: 0.0892 … Epoch 16/23… … (Bảng tóm tắt Epoch 17-23) Epoch 17/23… Epoch 18/23… Epoch 19/23… Epoch 20/23… Epoch 21/23… Epoch 22/23… Epoch 23/23…

Như bạn thấy, chúng ta đã huấn luyện mô hình của mình và đạt được độ chính xác (accuracy) là 99.42% và độ chính xác xác thực (validation accuracy) là 97.68%. Sau đó, chúng ta sẽ vẽ biểu đồ kết quả của mình bằng matplotlib.

Bước 5: Tổng kết Mô hình sau khi huấn luyện

summarize history for accuracy (tổng kết lịch sử về độ chính xác)
plt.plot (history.history['acc'])
plt.plot (history.history['val
acc'])
plt.title('model accuracy')
plt.ylabel ('accuracy')
plt.xlabel ('epoch')
plt.legend ('train', 'test'], loc='upper left')
plt.show ()

Output: as in Fig. 10.1 (như trong Hình 10.1)

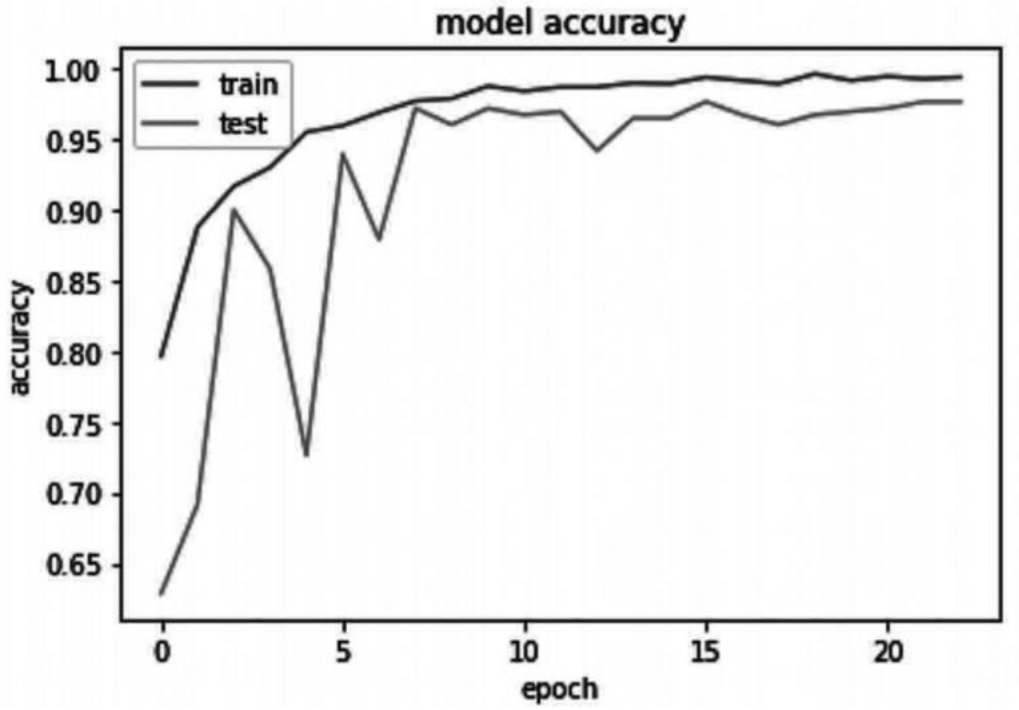


Fig. 10.1 Accuracy Representation of model

```
# summarize history for loss (tổng kết lịch sử về mất mát)
plt.plot (history.history['loss'])
plt.plot (history.history 'val
plt.title('model loss')
plt.ylabel ('loss')
plt.xlabel ('epoch')
loss'])
plt.legend ( 'train', 'test'], loc='upper left')
plt.show ()
```

Output: as in Fig. 10.2 (như trong Hình 10.2)

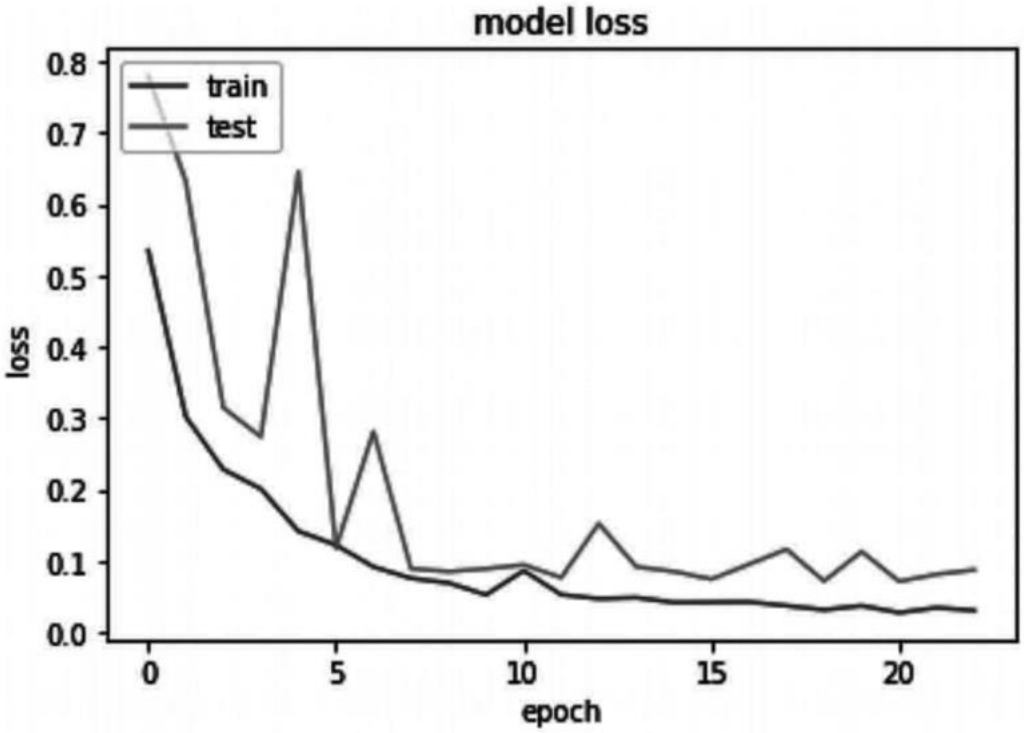


Fig. 10.2 Loss Representation of model

Ở đây chúng ta đã vẽ biểu đồ kết quả giữa độ chính xác của mô hình và mất mát của mô hình. Bạn có thể thấy trong các biểu đồ.

Bước 6: Lưu mô hình

```
model json model.to json ()
with open ("model.json",
json file.write (model
"w") as json file:
json)
```

```
# serialize weights to HDF5 (tuần tự hóa trọng số sang HDF5)
model.save weights ("model.h5")
print ("Model Saved")
```

output: Model Saved Trong bước này, chúng ta đang lưu mô hình đã huấn luyện để có thể sử dụng sau này. Bằng cách này, chúng ta không cần phải huấn luyện lại mô hình của mình nhiều lần cho bất kỳ dữ liệu đầu vào mới nào để kiểm tra kết quả của chúng.

Bước 7: Kiểm tra mô hình Ở đây chúng ta sẽ kiểm tra mô hình mà chúng ta đã lưu ở bước 5 trước đó. #importing the model (nhập mô hình)

```
from tensorflow.keras.models import model from json
json file
=
loaded
json
open ('model.json', 'r')
model json json file.read ()
file.close()
loaded model model from json (loaded model json)
# load weights into new model (tải trọng số vào mô hình mới)
loaded model.load weights ('model.h5')
```

Đầu tiên, chúng ta tải mô hình đã huấn luyện của mình, mô hình này sẵn sàng để sử dụng cho việc phát hiện bệnh ở khoai tây và tình trạng khỏe mạnh của nó.

```
class dict train generator.class
indices
class list = list (class dict.keys ())
class list
```

ở đây chúng ta đang tạo một danh sách cho tình trạng bệnh của cây liên quan đến các danh mục của nó class list=[‘early blight’, ‘healthy’, ‘late blight’] Ở đây chúng ta đang định nghĩa danh sách với tên bệnh để dự đoán khi cung cấp đầu vào.

```
 import cv2
def preprocess (image):
image cv2.cvtColor (image, cv2.COLOR
image cv2.resize (image (256,256))
7
BGR2RGB)
# create a simple mask image th the original one (tạo một hình ảnh mặt nạ đơn giản với hình ảnh gốc)
mask np.zeros (image.shape[:2], np.uint8)
# specify the background and foreground model (chỉ định mô hình nền và tiền cảnh)
```

```
backgroundModel = np.zeros( (1,
65), np.float64)
foregroundModel = np.zeros( (1,
65), np.float64)
# define the Region of Interest (ROI) as the coordinates of the
 rectangle (xác định Vùng quan tâm (ROI) là tọa độ của hình chữ nhật)
rectangle = (20, 20, 220, 220)
cv2.grabCut (image, mask, rectangle,
backgroundModel, foregroundModel,
1,cv2.GC INIT WITH RECT)
mask2
image
image
np.where ( (mask == 2)| (mask == 0), 0, 1).astype ('uint8')
*
image mask2[:, :, np.newaxis]
image/255
return image
```

Ở đây chúng ta đã sử dụng Thị giác máy tính (Computer Vision) để cung cấp đầu vào cho mô hình đã huấn luyện.

```
 def predict class (model,image path):
original image cv2.imread (image path)
image preprocess (original
tab = []
tab.append (image)
tab = np.array (tab)
h = np.expand
image)
dims (tab, axis=2)
h=model.predict (tab)
predicted class = np.argmax (h[0])
confidence = np.max (h[0])
# plot resutls (vẽ kết quả)
plt.figure (figsize $=(4,4))$
plt.imshow (image)
plt.axis ('off')
plt.title('prediction:
conf '+str (confidence))
plt.show ()
'+class list [predicted class]+' with
```

ở đây chúng ta đang vẽ kết quả cho các hình ảnh dữ liệu đầu vào vào mô hình.

predict class (loaded model,'test/lateb.jpg') #image path is
provided for prediction (đường dẫn hình ảnh được cung cấp để dự đoán)
predict_class (loaded
Output:
model, 'test/lateb2.jpg')
prediction late blight with conf 0.9999802
prediction late blight with conf 0.99996245

(Các hình ảnh đầu ra được hiển thị trong tài liệu gốc)

predict_class (loaded_model,'test/earlyb.jpg')
predict_class (loaded_model,'test/earlybl.jpg')

Output: prediction late blight with conf 0.9517815 prediction late blight with conf 9.5648595

predict_class (loaded_model, 'test/healthy.jpg')
predict_class (loaded_model,'test/healthy1.jpg')

Output: prediction late blight with conf 0.5035014

Như bạn đã thấy trong dự án, cách một mô hình CNN được tạo ra để phát hiện bệnh cây trồng và cách chúng ta đã lưu mô hình để sử dụng trong tương lai. Bạn có thể tải xuống mô hình đã lưu từ liên kết này và có thể sử dụng trực tiếp để dự đoán. #link is

Tóm tắt

Phân loại bệnh cây trồng đã rất nổi bật trong nông nghiệp, điều này làm cho nó trở nên hữu ích hơn cho nông dân trong việc xác định và ngăn chặn bệnh lây lan giữa các cây trồng để có thể ngăn ngừa thiệt hại cho cây trồng. Như trong chương này, chúng ta đã sử dụng bộ dữ liệu plant village để xác định bệnh ở cây trồng bằng cách sử dụng các triệu chứng của nó trên lá cây bằng cách sử dụng học máy (machine learning), CNN và thị giác máy tính, là các lĩnh vực của trí tuệ nhân tạo. Dự án hiện tại đã được thực hiện trong chương này có thể được sử dụng trong thời gian thực để xác định bệnh ở cây khoai tây. Tương tự như vậy, chúng ta có thể thiết kế một hệ thống nhận dạng và xác định bệnh ở thực vật cho mọi loại cây trồng, điều này có thể mang lại lợi ích cho nông dân trong quá trình canh tác của họ.

CHƯƠNG 11: NHẬN DẠNG LOÀI HOA

Giới thiệu

Cách tiếp cận truyền thống của con người để phân loại thực vật là so sánh màu sắc và hình dạng của lá, nhưng bằng cách sử dụng phân tích Trí tuệ nhân tạo, ta có thể thu được kết quả nhanh hơn và chi tiết hơn thông qua việc phân tích hình thái của lá và cung cấp thêm chi tiết về các đặc tính của lá. Nhận dạng loài là một phần rất quan trọng của nông nghiệp và có hàng ngàn loài thực vật mà một người không thể nhận dạng hết được, và ngay cả khi làm vậy, cũng sẽ mất rất nhiều thời gian.

Trong chương này, chúng ta sẽ thảo luận và xây dựng một dự án liên quan đến việc nhận dạng các loài hoa bằng hình ảnh. Trong dự án này, chúng ta sẽ sử dụng các nhánh khác nhau của Trí tuệ nhân tạo như học máy (machine learning), học sâu (deep learning) và thị giác máy tính (computer vision) để xây dựng một dự án tuyệt vời như vậy. Nào, chúng ta hãy bắt đầu

Cài đặt các Gói (Packages)

Trong dự án Nhận dạng loài này, chúng ta đã sử dụng rất nhiều thư viện cần được cài đặt trước khi triển khai. Lệnh để cài đặt các thư viện này là:

- Warnings- pip install pytest-warnings/warnings
- NumPy- pip install Numpy
- Pandas- pip install pandas
- Matplotlib- pip install matplotlib
- Seaborn- pip install seaborn
- Tqdm- pip install tqdm
- Pillow/PIL- pip install pillow
- TensorFlow- pip install TensorFlow
- Keras- pip install keras

Đối với IDE Python: Nhập các lệnh này trên command prompt trong Windows. Đối với Anaconda: Nhập các lệnh này trên Anaconda Prompt

Nếu bạn đang sử dụng bất kỳ Môi trường hoặc IDE nào khác, bạn phải chỉ định đường dẫn cho python và bạn có thể thực hiện cài đặt.

Dự án này được triển khai trong Google Research Collaboratory. Và bạn có thể tải xuống bản sao của mã nguồn từ liên kết này

Bộ dữ liệu (Dataset)

Trong dự án này, chúng tôi sử dụng bộ dữ liệu từ Kaggle có 5 loại hoa là cúc họa mi (daisy), bồ công anh (dandelion), hoa hồng (rose), hoa hướng dương (sunflower) và hoa

tulip, tổng cộng bao gồm 4242 hình ảnh với kích thước 320x240 pixel. Các hình ảnh trong bộ dữ liệu được lấy từ dữ liệu của Flickr, Google Images và Yandex Images. Những hình ảnh này có thể được sử dụng để nhận dạng các loài thực vật liên quan.

Bạn có thể tải xuống bộ dữ liệu tương tự từ liên kết này

Triển khai:

Lưu ý: Dự án này hoàn toàn được triển khai trên Google Research Collaboratory mà bạn có thể truy cập bằng liên kết này và bạn có thể tự do sử dụng bất kỳ IDE nào khác để triển khai dự án.

Bước 1: Nhập (Import) tất cả các Thư viện

Lưu ý: Ký hiệu # đại diện cho chú thích (comment) trong dự án, giúp bạn hiểu rõ hơn về mã nguồn.

```
# Bỏ qua các cảnh báo
import warnings
warnings.filterwarnings ('always')
warnings.filterwarnings ('ignore')

# trực quan hóa và thao tác dữ liệu
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import style
import seaborn as sns

# cấu hình
# đặt matplotlib ở chế độ nội tuyến (inline) và hiển thị biểu đồ bên dưới ô tương ứng.
% matplotlib inline
style.use ('fivethirtyeight')
sns.set (style='whitegrid',color codes=True)

# lựa chọn mô hình
from sklearn.model selection import train test split
from sklearn.model selection import KFold
from sklearn.metrics import accuracy score, precision
score, recall score,confusion matrix,roc curve,
roc auc score
from sklearn.model selection import GridSearchCV
 from sklearn.preprocessing import LabelEncoder
```

```
# tiền xử lý.
from keras.preprocessing.image import ImageDataGenerator

#Đây là các thư viện học sâu có trong Keras. Nếu bạn đã cài đặt Keras, tất cả chúng sẽ được cài đặt tự động.
from keras import backend as K
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam, SGD, Adagrad, Adadelta, RMSprop
from keras.utils import to_categorical

# chúng ta sử dụng các thư viện này đặc biệt để xây dựng Mạng nơ-ron tích chập (CNN) sẽ được dùng để nhận dạng các loài hoa.
from keras.layers import Dropout, Flatten, Activation
from keras.layers import Conv2D, MaxPooling2D,
BatchNormalization

import tensorflow as tf
import random as rn

# ở đây chúng ta sử dụng thư viện OpenCV để cung cấp đầu vào cho mô hình đã huấn luyện
# và đặc biệt là để thao tác các hình ảnh đã nén (zipped) và lấy các mảng NumPy chứa giá trị pixel của hình ảnh.
import cv2
import numpy as np
from tqdm import tqdm
import os
from random import shuffle
from zipfile import ZipFile
from PIL import Image
```

Bước 2: Gắn (Mount) drive để lấy bộ dữ liệu

Nếu bạn đang sử dụng Google Research Collaboratory, bạn có thể tải xuống bộ dữ liệu từ liên kết này và tải nó lên thư mục “Colab Notebooks” trong Google Drive của bạn, sau đó bạn có thể liên kết nó bằng mã này

```
from google.colab import drive
drive.mount ('/gdrive')
```

Kết quả: Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_ id=94. .....y Enter your authorization code: (Nhập mã ủy quyền của bạn:)

Mounted at /gdrive (Đã gắn tại /gdrive)

Đây là mã để gắn drive của bạn với Google Research Collaboratory, nó sẽ yêu cầu bạn xác thực. Bạn có thể làm điều này bằng tài khoản Gmail của mình chỉ bằng cách làm theo các bước đơn giản. Như bạn có thể thấy trong kết quả, Collaboratory sẽ yêu cầu bạn xác minh bằng cách nhấp vào liên kết đã cho, sau đó nó sẽ được gắn sau khi bạn cấp quyền xác thực.

```
with open ('/gdrive/My Drive/foo.txt', 'w') as f:
  f.write ('Hello Google Drive!')
!cat '/gdrive/My Drive/foo.txt'
```

Kết quả: Hello Google Drive!

Đây là mã giúp bạn kiểm tra xem drive của mình đã được gắn với Collaboratory hay chưa.

Sau khi làm điều này, chúng ta sẽ truy cập trực tiếp vào bộ dữ liệu đã được tải lên drive và giải nén nó để truy cập các tệp mà chúng ta cần trong dự án này.

Bước 3: Chuẩn bị dữ liệu sẵn sàng cho Mô hình

Ở đây, chúng ta đang tạo một mảng hình ảnh X và Z sẽ lưu trữ tất cả các hình ảnh dưới dạng hình ảnh huấn luyện và xác thực để mô hình CNN huấn luyện và được kiểm tra với dữ liệu thực tế.

```
X=[]
Z=[]
IMG_SIZE = 150 #gán kích thước hình ảnh không đổi cho dự án
FLOWER_DAISY_DIR='drive/Colab Notebooks/flower recog/flowers/
daisy'
FLOWER_SUNFLOWER_DIR='drive/Colab Notebooks/flower recog/
flowers/sunflower'
FLOWER_TULIP_DIR='drive/Colab Notebooks/flower recog/flowers/
tulip'
FLOWER_DANDI_DIR='drive/Colab Notebooks/flower recog/flowers/
dandelion'
FLOWER_ROSE_DIR='drive/Colab Notebooks/flower recog/flowers/
rose'
```

Tạo một hàm để gán nhãn cho các hình ảnh trong dữ liệu, sẽ được sử dụng tiếp cho mục đích huấn luyện.

```
def assign_label (img, flower_type):
  return flower_type
```

ở đây sau khi gán nhãn, chúng ta sẽ tạo dữ liệu huấn luyện riêng biệt từ mỗi loại hoa

```
def make_train_data (flower_type, DIR):
    for img in tqdm (os.listdir (DIR)):
        label = assign_label (img, flower_type)

        path = os.path.join (DIR,img)
        img = cv2.imread (path,cv2.IMREAD_COLOR)
        img = cv2.resize (img, (IMG_SIZE, IMG_SIZE))

        X.append (np.array (img))
        Z.append (str (label))
```

```
# Dữ liệu huấn luyện cho hoa cúc họa mi (Daisy)
make_train_data ('Daisy', FLOWER_DAISY_DIR)
print (len (X))
```

Kết quả: 100% 769/769 [00:07<00:00, 101.93it/s] 769

```
#Dữ liệu huấn luyện cho hoa hướng dương (Sunflower)
make_train_data ('Sunflower', FLOWER_SUNFLOWER_DIR)
print (len (X))
```

Kết quả: 100% 766/766 [00:07<00:00, 176.83it/s] 1535

```
#Dữ liệu huấn luyện cho hoa Tulip
make_train_data ('Tulip', FLOWER_TULIP_DIR)
print (len (X))
```

Kết quả: 100% 991/991 [00:08<00:00, 116.01it/s] 2526

```
#Dữ liệu huấn luyện cho bồ công anh (Dandelion)
make_train_data ('Dandelion', FLOWER_DANDI_DIR)
print (len (X))
```

Kết quả: 100% 1052/1052 [00:09<00:00, 114.67it/s] 3578

```
#Dữ liệu huấn luyện cho hoa hồng (Rose)
make_train_data ('Rose', FLOWER_ROSE_DIR)
print (len (X))
```

Kết quả: 100% 784/784 [00:06<00:00, 116.67it/s] 4362

Đây là cách dữ liệu huấn luyện cho tất cả các loài hoa đã sẵn sàng để mô hình được huấn luyện.

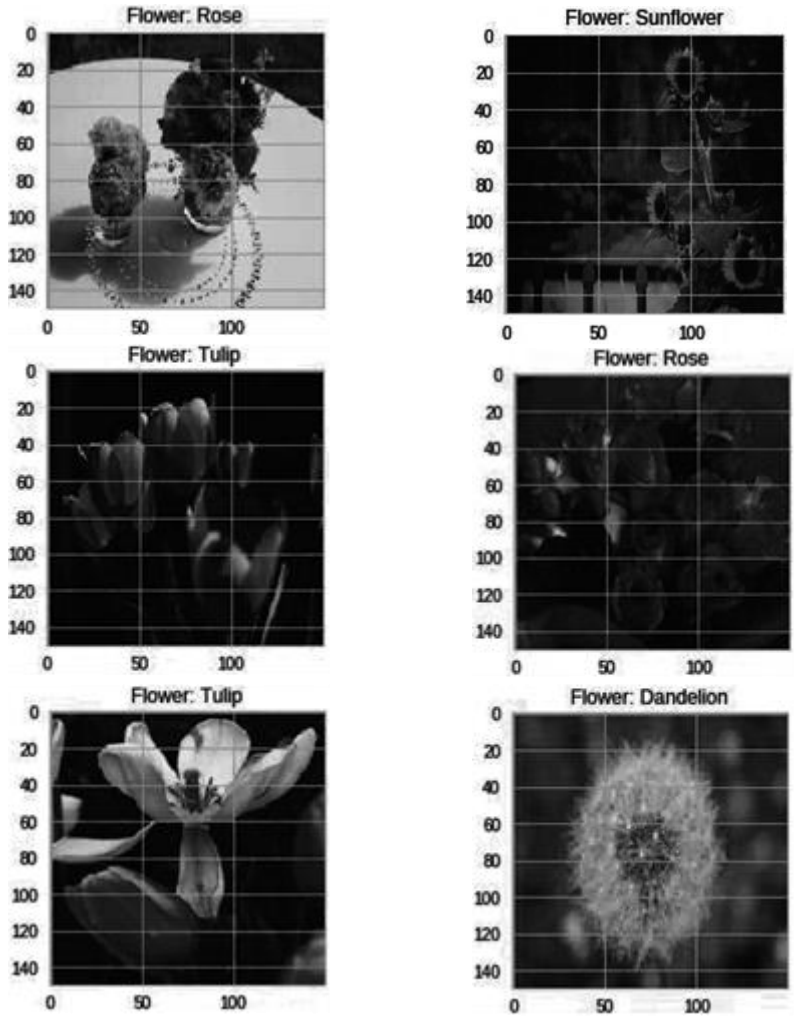
Bước 4: Trực quan hóa dữ liệu

Sau khi chuẩn bị dữ liệu để huấn luyện, chúng ta sẽ trực quan hóa dữ liệu đã chuẩn bị bằng Matplotlib và OpenCV.

```
fig,ax=plt.subplots (5,2)
fig.set_size_inches (15,15)
for i in range (5):
    for j in range (2):
        l=rn.randint (0,len (Z))
        ax[i,j] .imshow (X[l])
        ax[i,j] .set_title ('Flower: '+Z[l])

plt.tight_layout ()
```

Kết quả: như trong Hình 11.1



Bước 5: Gán nhãn dữ liệu và chia thành dữ liệu huấn luyện (training) và dữ liệu xác thực (validation)

```
#gán nhãn
le = LabelEncoder ()
Y = le.fit_transform (Z)
Y = to_categorical (Y,5)
X = np.array (X)
X = X/255

#Sau khi gán nhãn, chúng ta sẽ chia dữ liệu thành các Tập Huấn luyện và Xác thực
x_train,x_test,y_train,y_test=train_test_split (X, Y, test_size=0.25, random_state=42)

#Đặt các mầm ngẫu nhiên (random seeds)
np.random.seed (42)
rn.seed (42)
tf.set_random_seed (42)
```

Bước 6: Xây dựng Mô hình

Để xây dựng mô hình, tôi đã sử dụng Keras Sequential, với nó, việc xây dựng bất kỳ mô hình nào có đầu vào là nhiều hình ảnh để trích xuất đặc trưng và phân loại chúng trở nên rất dễ dàng.

Trong quá trình tạo mô hình, tôi đã tạo bốn lớp tích chập (convolutional layer). Ở lớp đầu tiên, chúng ta có 32 bộ lọc (filters), ma trận kernel 5x5 với đệm (padding) và hàm kích hoạt (activation function) ReLU cùng với kích thước đầu vào của hình ảnh.

Ở lớp thứ hai, chúng ta có 64 bộ lọc, ma trận kernel 3x3 với đệm và hàm kích hoạt ReLU cùng với kích thước đầu vào của hình ảnh.

Ở lớp thứ ba và thứ tư, chúng ta có 96 bộ lọc, ma trận kernel 3x3 với đệm và hàm kích hoạt ReLU cùng với kích thước đầu vào của hình ảnh.

Theo sau tất cả các lớp tích chập này, chúng ta sẽ có một lớp gộp cực đại (max pooling layer) giúp chúng ta lấy được các đặc trưng tối đa từ các bộ lọc.

Giữa các lớp, chúng ta có lớp “flatten ()” (làm phẳng) luôn đóng vai trò là kết nối giữa lớp tích chập và lớp kết nối đầy đủ (dense layer).

Và cuối cùng, chúng ta có các lớp đầu ra kết nối đầy đủ, tức là các lớp dense với một hàm kích hoạt.

```
# bắt đầu lập mô hình bằng CNN.
model = Sequential ()
model.add (Conv2D (filters = 32, kernel_size = (5,5), padding = 'Same', activation =
'relu', input_shape = (150,150,3)))
```

```
model.add (MaxPooling2D (pool_size= (2,2)))

model.add (Conv2D (filters = 64, kernel_size = (3,3),padding = 'Same', activation =
'relu'))
model.add (MaxPooling2D (pool_size = (2,2), strides = (2,2)))

model.add (Conv2D (filters = 96, kernel_size = (3,3), padding = 'Same',activation
='relu'))
model.add (MaxPooling2D (pool_size = (2,2), strides= (2,2)))

model.add (Conv2D (filters = 96, kernel_size = (3,3),padding = 'Same', activation
='relu'))
model.add (MaxPooling2D (pool_size = (2,2), strides = (2,20)))

model.add (Flatten ())
model.add (Dense (512))
model.add (Activation ('relu'))
model.add (Dense (5, activation = "softmax"))

#sau khi xây dựng mô hình, chúng ta sẽ khai báo số lượng epochs và kích thước lô (batch
size)
# và chúng ta đang sử dụng một LR annealer (bộ giảm tốc độ học) để cố định tốc độ học
(learning rate) của chúng ta
# sau một khoảng thời gian nhất định trong khi biên dịch mô hình.

#sử dụng một LR Annealer
batch_size = 128
epochs = 50

from keras.callbacks import ReduceLROnPlateau
red_lr = ReduceLROnPlateau (monitor='val_acc',
patience = 3,verbose = 1,factor = 0.1)

#chúng ta đang thực hiện Tăng cường Dữ liệu (Data Augmentation) ở đây để ngăn chặn
việc Overfitting (học vẹt)
datagen = ImageDataGenerator (
    featurewise_center=False, # đặt trung bình đầu vào về 0 trên toàn bộ bộ dữ liệu
    samplewise_center=False, # đặt trung bình của mỗi mẫu về 0
    featurewise_std_normalization=False, # chia đầu vào cho độ lệch chuẩn (std) của bộ
dữ liệu
    samplewise_std_normalization=False, # chia mỗi đầu vào cho độ lệch chuẩn của nó
    zca_whitening=False, # áp dụng làm trắng ZCA (ZCA whitening)
    rotation_range = 10, # xoay ảnh ngẫu nhiên trong phạm vi (độ, 0 đến 180)
    zoom_range = 0.1, # Thu phóng hình ảnh ngẫu nhiên
```

```
    width_shift_range = 0.2, # dịch chuyển ảnh ngẫu nhiên theo chiều ngang (phần trăm của tổng chiều rộng)
    height_shift_range = 0.2, # dịch chuyển ảnh ngẫu nhiên theo chiều dọc (phần trăm của tổng chiều cao)
    horizontal_flip=True, # lật ảnh ngẫu nhiên theo chiều ngang
    vertical_flip=False) # lật ảnh ngẫu nhiên theo chiều dọc

datagen.fit (x_train)

# bây giờ chúng ta sẽ Biên dịch (Compiling) Mô hình & Xem Tóm tắt (Summary) của Mô hình
model.compile (optimizer=Adam (lr=0.001), loss='categorical_crossentropy', metrics=['accuracy'])
model.summary ()
```

Kết quả:

```
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 150, 150, 32)      2432
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 75, 75, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 75, 75, 64)        18496
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 37, 37, 64)        0
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 37, 37, 96)        55392
_________________________________________________________________
max_pooling2d_3 (MaxPooling2 (None, 18, 18, 96)        0
_________________________________________________________________
conv2d_4 (Conv2D)            (None, 18, 18, 96)        83040
_________________________________________________________________
max_pooling2d_4 (MaxPooling2 (None, 9, 9, 96)          0
_________________________________________________________________
flatten_1 (Flatten)          (None, 7776)              0
_________________________________________________________________
dense_1 (Dense)              (None, 512)               3981824
_________________________________________________________________
activation_1 (Activation)    (None, 512)               0
_________________________________________________________________
dense_2 (Dense)              (None, 5)                 2565
=================================================================
```

==
Total params: 4,143,749
Trainable params: 4,143,749
Non-trainable params: 0

Bước 7: Huấn luyện Mô hình với bộ dữ liệu

Sau khi biên dịch mô hình, chúng ta sẽ huấn luyện mô hình của mình với dữ liệu huấn luyện (training data) mà chúng ta đã gán nhãn và tách ra trước đó.

```
History = model.fit_generator (datagen.flow (x_train,y_train,
batch_size=batch_size),
epochs = epochs, validation_data = (x_test,y_test),
verbose = 1, steps_per_epoch = x_train.shape[0] // batch_size)

# model.fit (x_train,y_train, epochs=epochs,batch_size=batch_size, validation_data =
(x_test,y_test))
```

Kết quả: Epoch 1/50 25/25 [==============================] - 23s 903ms/step - loss: 1.4743 - acc: 0.3588 - val_loss: 1.2462 - val_acc: 0.5069 Epoch 2/50 25/25 [==============================] - 20s 808ms/step - loss: 1.1573 - acc: 0.5212 - val_loss: 1.1553 - val_acc: 0.5252 Epoch 3/50 25/25 [==============================] - 20s 812ms/step - loss: 1.0605 - acc: 0.5745 - val_loss: 1.0670 - val_acc: 0.6013 Epoch 4/50 25/25 [==============================] - 21s 820ms/step - loss: 1.0024 - acc: 0.5956 - val_loss: 0.9759 - val_acc: 0.6114 Epoch 5/50 25/25 [==============================] - 20s 795ms/step - loss: 0.9267 - acc: 0.6437 - val_loss: 0.9823 - val_acc: 0.6205 Epoch 6/50 25/25 [==============================] - 20s 784ms/step - loss: 0.9096 - acc: 0.6477 - val_loss: 0.9942 - val_acc: 0.6004 Epoch 7/50 25/25 [==============================] - 20s 807ms/step - loss: 0.8594 - acc: 0.6677 - val_loss: 0.9092 - val_acc: 0.6581 Epoch 8/50 25/25 [==============================] - 20s 812ms/step - loss: 0.8468 - acc: 0.6779 - val_loss: 0.8853 - val_acc: 0.6783 Epoch 9/50 25/25 [==============================] - 20s 810ms/step - loss: 0.7789 - acc: 0.7019 - val_loss: 0.8087 - val_acc: 0.6856 Epoch 10/50 25/25 [==============================] - 20s 798ms/step - loss: 0.7613 - acc: 0.7089 - val_loss: 0.8019 - val_acc: 0.6939 Epoch 11/50 25/25 [==============================] - 20s 795ms/step - loss: 0.7660 - acc: 0.6997 - val_loss: 0.8029 - val_acc: 0.7030 Epoch 12/50 25/25 [==============================] - 20s 794ms/step - loss: 0.7742 - acc: 0.7022 - val_loss: 0.9206 - val_acc: 0.6480 Epoch 13/50 25/25 [==============================] - 20s 794ms/step - loss: 0.7367 - acc: 0.7154

- val_loss: 0.7945 - val_acc: 0.6929 Epoch 14/50 25/25
[==============================] - 19s 778ms/step - loss: 0.7020 - acc: 0.7354
- val_loss: 0.7705 - val_acc: 0.6948 Epoch 15/50 25/25
[==============================] - 20s 807ms/step - loss: 0.6742 - acc: 0.7325
- val_loss: 0.7577 - val_acc: 0.7140

Epoch 16/50 25/25 [==============================] - 20s 781ms/step - loss:
0.6758 - acc: 0.7418 - val_loss: 0.7766 - val_acc: 0.7131 Epoch 17/50 25/25
[==============================] - 20s 810ms/step - loss: 0.6605 - acc: 0.7466
- val_loss: 0.7200 - val_acc: 0.7269 Epoch 18/50 25/25
[==============================] - 20s 793ms/step - loss: 0.6453 - acc: 0.7500
- val_loss: 0.7373 - val_acc: 0.7186 Epoch 19/50 25/25
[==============================] - 20s 808ms/step - loss: 0.6381 - acc: 0.7585
- val_loss: 0.7292 - val_acc: 0.7269 Epoch 20/50 25/25
[==============================] - 20s 795ms/step - loss: 0.6110 - acc: 0.7673
- val_loss: 0.7906 - val_acc: 0.7131 Epoch 21/50 25/25
[==============================] - 20s 795ms/step - loss: 0.6187 - acc: 0.7629
- val_loss: 0.7009 - val_acc: 0.7296 Epoch 22/50 25/25
[==============================] - 20s 797ms/step - loss: 0.5574 - acc: 0.7865
- val_loss: 0.6847 - val_acc: 0.7599 Epoch 23/50 25/25
[==============================] - 20s 799ms/step - loss: 0.5884 - acc: 0.7788
- val_loss: 0.6892 - val_acc: 0.7415 Epoch 24/50 25/25
[==============================] - 20s 796ms/step - loss: 0.5553 - acc: 0.7869
- val_loss: 0.7059 - val_acc: 0.7360 Epoch 25/50 25/25
[==============================] - 20s 797ms/step - loss: 0.5731 - acc: 0.7867
- val_loss: 0.6628 - val_acc: 0.7479 Epoch 26/50 25/25
[==============================] - 20s 799ms/step - loss: 0.5141 - acc: 0.7989
- val_loss: 0.6582 - val_acc: 0.7681 Epoch 27/50 25/25
[==============================] - 20s 800ms/step - loss: 0.4982 - acc: 0.8122
- val_loss: 0.8243 - val_acc: 0.7232 Epoch 28/50 25/25
[==============================] - 20s 801ms/step - loss: 0.5484 - acc: 0.7898
- val_loss: 0.6767 - val_acc: 0.7562 Epoch 29/50 25/25
[==============================] - 20s 800ms/step - loss: 0.4928 - acc: 0.8129
- val_loss: 0.7491 - val_acc: 0.7498 Epoch 30/50 25/25
[==============================] - 20s 799ms/step - loss: 0.4768 - acc: 0.8238
- val_loss: 0.6758 - val_acc: 0.7544 Epoch 31/50 25/25
[==============================] - 20s 795ms/step - loss: 0.4678 - acc: 0.8219
- val_loss: 0.7470 - val_acc: 0.7415

Epoch 32/50 25/25 [==============================] - 20s 798ms/step - loss: 0.4604 - acc: 0.8260 - val_loss: 0.6842 - val_acc: 0.7489 Epoch 33/50 25/25 [==============================] - 20s 796ms/step - loss: 0.4755 - acc: 0.8263 - val_loss: 0.7710 - val_acc: 0.7589 Epoch 34/50 25/25 [==============================] - 20s 800ms/step - loss: 0.4409 - acc: 0.8345 - val_loss: 0.6872 - val_acc: 0.7782 Epoch 35/50 25/25 [==============================] - 20s 807ms/step - loss: 0.4372 - acc: 0.8378 - val_loss: 0.6577 - val_acc: 0.7626 Epoch 36/50 25/25 [==============================] - 20s 810ms/step - loss: 0.3909 - acc: 0.8534 - val_loss: 0.6526 - val_acc: 0.7910 Epoch 37/50 25/25 [==============================] - 20s 784ms/step - loss: 0.3928 - acc: 0.8558 - val_loss: 0.7110 - val_acc: 0.7663 Epoch 38/50 25/25 [==============================] - 20s 793ms/step - loss: 0.3955 - acc: 0.8530 - val_loss: 0.7136 - val_acc: 0.7773 Epoch 39/50 25/25 [==============================] - 20s 808ms/step - loss: 0.3658 - acc: 0.8659 - val_loss: 0.6593 - val_acc: 0.7773 Epoch 40/50 25/25 [==============================] - 20s 798ms/step - loss: 0.3916 - acc: 0.8588 - val_loss: 0.6378 - val_acc: 0.7782 Epoch 41/50 25/25 [==============================] - 20s 797ms/step - loss: 0.3722 - acc: 0.8565 - val_loss: 0.7199 - val_acc: 0.7736 Epoch 42/50 25/25 [==============================] - 20s 782ms/step - loss: 0.3746 - acc: 0.8533 - val_loss: 0.6931 - val_acc: 0.7791 Epoch 43/50 25/25 [==============================] - 20s 795ms/step - loss: 0.3255 - acc: 0.8787 - val_loss: 0.6562 - val_acc: 0.8029 Epoch 44/50 25/25 [==============================] - 20s 809ms/step - loss: 0.3306 - acc: 0.8738 - val_loss: 0.7102 - val_acc: 0.7635 Epoch 45/50 25/25 [==============================] - 20s 780ms/step - loss: 0.2981 - acc: 0.8809 - val_loss: 0.7033 - val_acc: 0.7929 Epoch 46/50 25/25 [==============================] - 20s 809ms/step - loss: 0.3009 - acc: 0.8903 - val_loss: 0.6488 - val_acc: 0.8066 Epoch 47/50 25/25 [==============================] - 20s 783ms/step - loss: 0.3078 - acc: 0.8829 - val_loss: 0.6670 - val_acc: 0.7965

Epoch 48/50 25/25 [==============================] - 20s 809ms/step - loss: 0.2886 - acc: 0.8906 - val_loss: 0.7584 - val_acc: 0.7828 Epoch 49/50 25/25 [==============================] - 20s 797ms/step - loss: 0.3103 - acc: 0.8860 - val_loss: 0.6943 - val_acc: 0.7809 Epoch 50/50 25/25 [==============================] - 20s 808ms/step - loss: 0.3007 - acc: 0.8898 - val_loss: 0.7433 - val_acc: 0.7754

Sau khi biên dịch, chúng ta có thể thấy kết quả của mô hình là độ chính xác (accuracy) đạt 88.98% và Độ chính xác xác thực (Val. Acc) là 77.54%, kết quả này có thể được tăng

lên nếu cung cấp thêm dữ liệu để huấn luyện hoặc thực hiện một số thay đổi với mô hình bằng cách thay đổi trình tối ưu hóa (optimizers), hàm kích hoạt hoặc tốc độ học (learning rate).

Bước 8: Xem xét hiệu suất của mô hình

Bây giờ chúng ta sẽ vẽ biểu đồ kết quả mô hình đã huấn luyện của mình với sự trợ giúp của matplotlib.

```
plt.plot (History.history['loss'])
plt.plot (History.history['val_loss'])
plt.title('Model Loss')
plt.ylabel ('Loss')
plt.xlabel ('Epochs')
plt.legend (['train', 'test'])
plt.show()
```

Kết quả: như trong Hình 11.2 (Biểu đồ thể hiện Lỗi (Loss) của Mô hình)

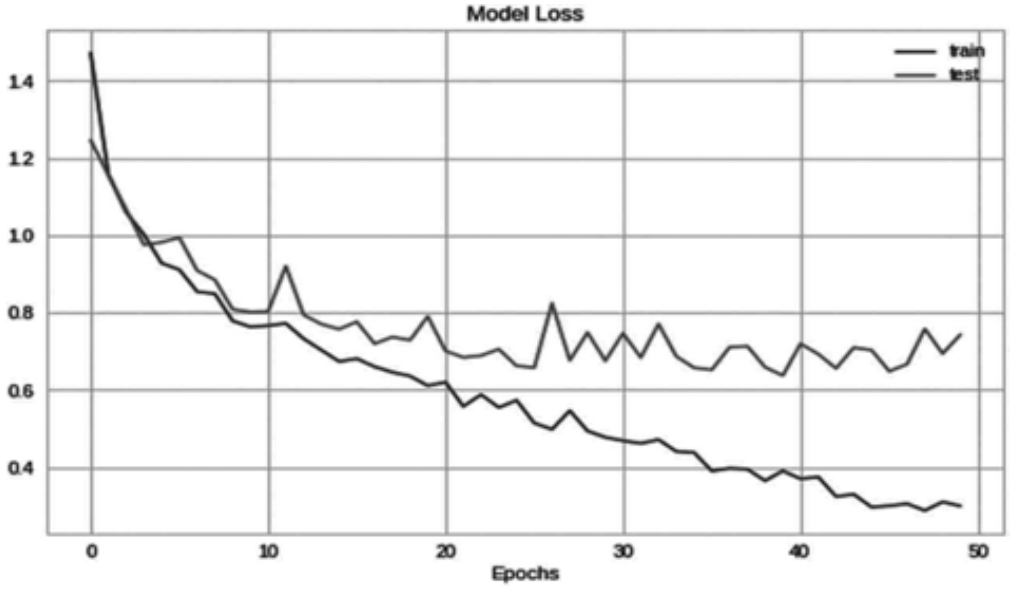


Fig. 11.2 Loss Representation of Model

```
plt.plot (History.history['acc'])
plt.plot (History.history['val_acc'])
plt.title('Model Accuracy')
plt.ylabel ('Accuracy')
plt.xlabel ('Epochs')
plt.legend (['train', 'test'])
plt.show ()
```

Kết quả: như trong Hình 11.3 (Biểu đồ thể hiện Độ chính xác (Accuracy) của Mô hình)

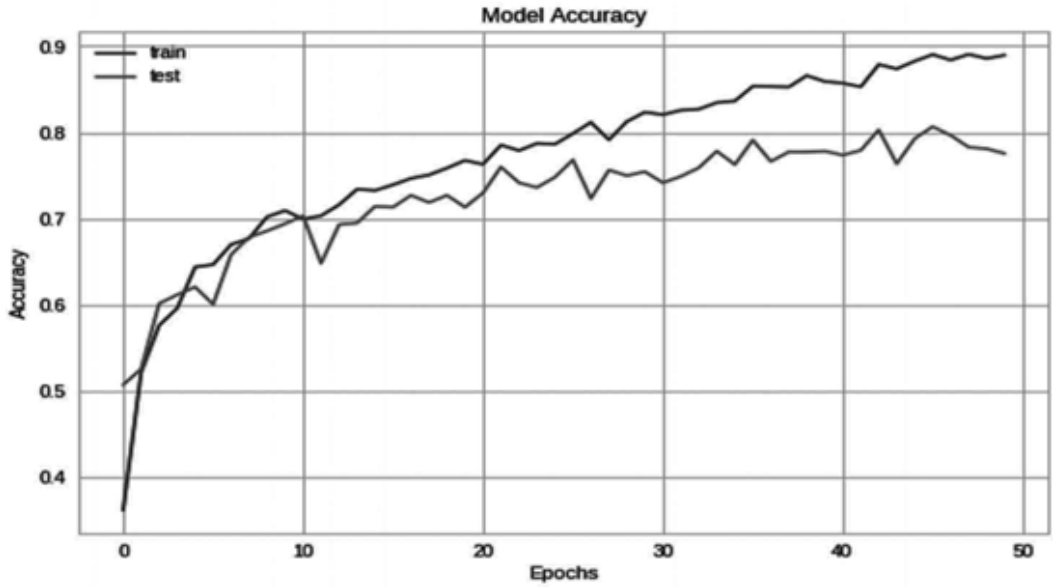


Fig. 11.3 Accuracy Representation of Model

Bước 9: Dự đoán trên dữ liệu xác thực (validation data)

Tại đây, chúng ta sẽ yêu cầu mô hình đã huấn luyện của mình dự đoán các hình ảnh bằng dữ liệu xác thực mà chúng ta đã tách ra trong khi gán nhãn và chia dữ liệu.

```
# lấy các dự đoán trên tập val (xác thực).
pred = model.predict (x_test)
pred_digits=np.argmax (pred,axis=1)

# bây giờ lưu trữ một số chỉ mục (index) được phân loại đúng cũng như phân loại sai.
i=0
prop_class=[]
mis_class=[]

# Ở đây chúng ta đang nhập các hình ảnh từ dữ liệu xác thực để kiểm tra dự đoán của mô hình.
for i in range (len (y_test)):
    if (np.argmax (y_test[i])==pred_digits[i]):
        prop_class.append (i)
    if (len (prop_class)==8):
        break

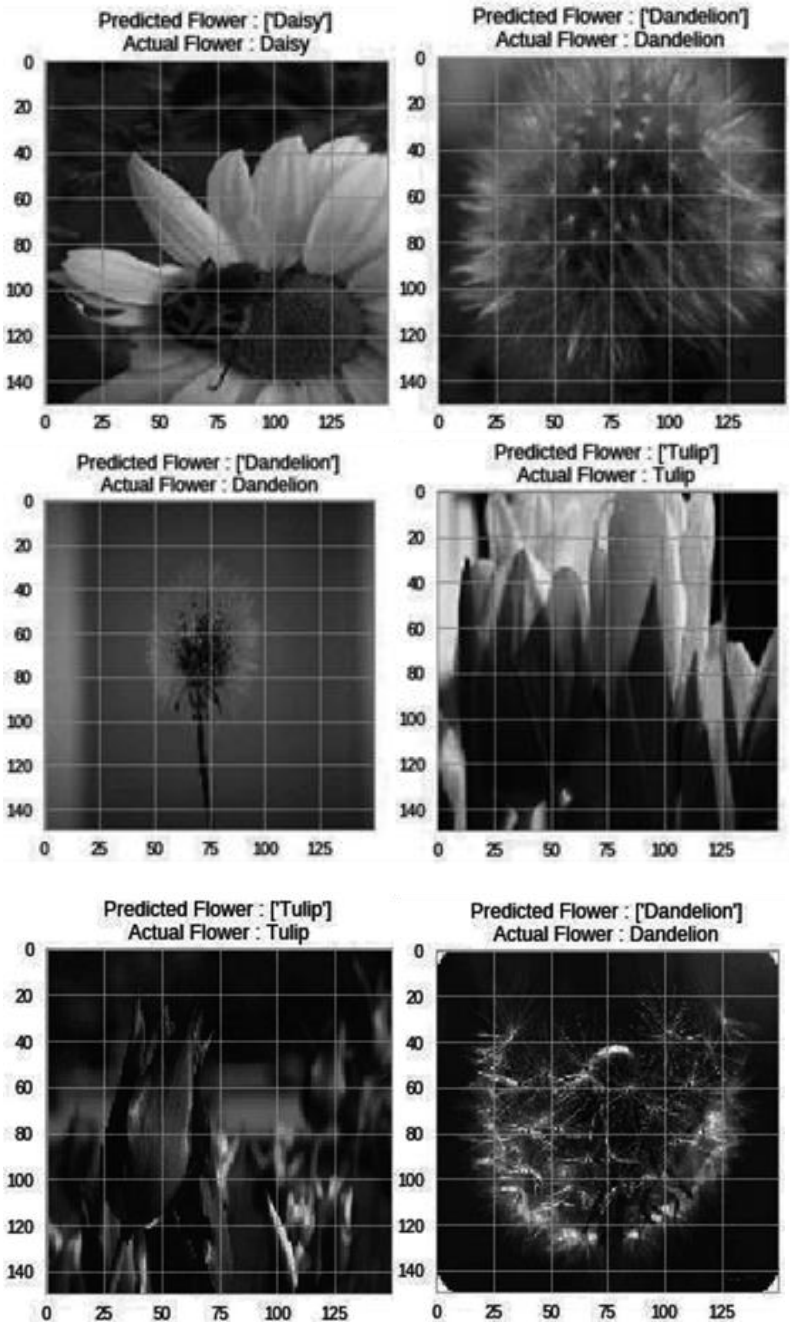
i=0
for i in range (len (y_test)):
    if (not np.argmax (y_test [i])==pred_digits[i]):
```

```
        mis_class.append (i)
    if (len (mis_class)==8):
        break

warnings.filterwarnings ('always')
warnings.filterwarnings ('ignore')

count = 0
fig,ax=plt.subplots (4,2)
fig.set_size_inches (15,15)
for i in range (3):
    for j in range (2):
        ax[i,j] .imshow (x_test [prop_class [count]])
        ax[i,j] .set_title ("Predicted Flower: "+str (le.inverse_
transform ((pred_digits [prop_class [count]])) + "\n" + "Actual
Flower: "+str (le.inverse_
transform (np.argmax ([y_test[prop_class [count]]])))
        plt.tight_layout ()
        count+=1
```

Kết quả: như trong Hình 11.4 (Dự đoán các loài hoa)



Hình 11.4 Dự đoán các loài hoa

Bước 10: Lưu mô hình

Đây là cách để lưu mô hình mà chúng ta đã huấn luyện

```
model_json = model.to_json ()
with open ("model.json", "w") as json_file:
    json_file.write (model_json)
# tuần tự hóa (serialize) trọng số sang HDF5
```

```
model.save_weights ("model.h5")
print ("Model Saved")
```

Sau khi lưu mô hình này, chúng ta có thể dự đoán các giá trị thuộc các danh mục này chỉ bằng cách sử dụng hình ảnh của các loài đó.

Tóm tắt

Nhận dạng loài cũng đóng một vai trò quan trọng trong lĩnh vực nông nghiệp, có thể giúp ích rất nhiều cho mọi người trong việc nhận dạng các loài thực vật bằng kỹ thuật số và nó sẽ tồn tại lâu dài với sự đảm bảo rằng hệ thống của chúng ta là chính xác để nhận dạng một loài thực vật.

Trong chương này, chúng ta đã thực hiện thêm một dự án liên quan đến nhận dạng loài và chúng ta có thể hoàn toàn hiểu cách sử dụng bộ dữ liệu với học máy, học sâu và thị giác máy tính để hiểu rõ hơn và ứng dụng trí tuệ nhân tạo trong thời gian thực.

Trong phần Nhận dạng loài, chúng ta đã sử dụng bộ dữ liệu hoa để huấn luyện mô hình của mình phân loại và nhận dạng các loài hoa trong thời gian thực.

Dự án này có thể được sử dụng trong thời gian thực để nhận dạng các loài hoa bằng kỹ thuật số chỉ từ hình ảnh của hoa.

Tài liệu tham khảo (Bibliography)

Rajesh Singh, Anita Gehlot, Sushabhan Choudhury, Bhupendra Singh, “Embedded System based on ATmega Microcontroller-Simulation, Interfacing and Projects”, Narosa Publishing House, 2017, ISBN: 978-81-8487-5720.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Arduino-Based Embedded Systems: Interfacing, Simulation, and LabVIEW GUI”- CRC Press (Taylor & Francis), 2017, ISBN 9781138060784.

Anita Gehlot, Rajesh Singh, Devender Kumar Saini, Monika Yadav, Bhupendra Singh, “IoT Enabled Smart Microgrid…..Rapid Prototyping”, GBS Publication, 2018, ISBN: 978-93-87374-41-6.

Anita Gehlot, Rajesh Singh, Mamta Mittal, Bhupendra Singh, Ravindra Sharma, Vikas Garg, “Hands on approach on Applications of Internet of Things in various Domain of Engineering”, International Research Publication House, 2018, ISBN: 978-93-87385-25-3.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Internet of Things Enabled Automation in Agriculture”, New India Publishing Agency, 2018, ISBN: 9789387973053.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Bikarama Prasad Yadav, “IoT Enabled Fire Safety and Security Devices for Building”, Pen2Print, EduPedia Publications, 2018, ISBN:9789386647979.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Piyush Kuchhal, “Wireless Methods and Devices for Home Automation” International Research Publication House, 2018, ISBN: 978-93-87388-27-7.

Rajesh Singh, Anita Gehlot, Raghuveer Chimata, Bhupendra Singh, P.S.Ranjith, “Internet of Things in Automotive Industries and Road Safety” River Publishers, 2018, ISBN:9788770220101&e-ISBN:9788770220095

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Arduino meets MATLAB. Interfacing, Programs and Simulink”, Bentham Science, 2018, ISBN: 978-1-68108-728-3 & e-ISBN: 978-1-68108-727-6

Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhunpendra Singh, and Priyanka Tyagi, “Getting Started for Internet of Things with Launch Pad and ESP8266”, River Publishers, 2019, ISBN: 9788770220682, e-ISBN: 9788770220675

Rajesh Singh, Anita Gehlot, Bhunpendra Singh, “Arduino and Scilab based Projects”, Bentham Science, 2019, ISBN: 9789811410918 & e-ISBN: 9789811410925

Anita Gehlot, Rajesh Singh, Lovi Raj Gupta, Bhupendra Singh, “Cook Book for Mobile Robotic Platform Control with Internet of Things and Ti Launch Pad”, BPB Publisher, 2019, ISBN: 978-93-8851-167-4

Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhupendra Singh, Mahindra Swain, “Internet of Things with Raspberry Pi and Arduino”, CRC press/Taylor & Francis, 2019, ISBN: 9780367248215

Anita Gehlot, Rajesh Singh, Gaurav Sethi, Bhupendra Singh, “The Robotic Platform Control with Atmega328 and NuttyFi Based on the Internet of Things”, Nova Science Publisher, 2020, ISBN:9781536174724

Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Navjot Rathour, Mahendra Swain, Bhupendra Singh, “IoT based projects”, BPB publication, 2020, ISBN: 9789389328523

CHƯƠNG 12: NÔNG NGHIỆP CHÍNH XÁC

Trong Chương này, nông nghiệp chính xác được thảo luận cùng với các bước triển khai. Chương này cũng bao gồm vai trò của trí tuệ nhân tạo trong việc thực hiện nông nghiệp chính xác theo nhiều cách khác nhau, cùng với phạm vi, hạn chế và thách thức của nó.

Giới thiệu

Nguồn sinh kế chính ở Ấn Độ là nông nghiệp, chiếm hai phần ba dân số Ấn Độ, phần còn lại là dịch vụ và khu vực tư nhân. Đất nông nghiệp chiếm khoảng 43% diện tích địa lý của Ấn Độ. Trước đây, Ấn Độ phụ thuộc phần lớn vào nhập khẩu lương thực, nhưng qua nhiều năm, sản xuất ngũ cốc và hạt giống của đất nước đã tự cung tự cấp, và nỗ lực này đã dẫn đến sự hình thành của cuộc Cách mạng Xanh. Trên thực tế, đã có nỗ lực mạnh mẽ để đạt được tự chủ về lương thực. Xu hướng này vẫn tiếp tục cho đến nay và đã có những cải tiến liên tục. Nông nghiệp chính xác (Precision agriculture hay precision farming) có nghĩa là làm đúng việc, đúng cách, đúng nơi và đúng thời điểm.

Nông nghiệp chính xác được kỳ vọng sẽ phù hợp với các hoạt động khí hậu nông nghiệp để cải thiện độ chính xác trong ứng dụng. Đất canh tác đã giảm một chút trong 40 năm qua nhưng số lượng nông dân lại tăng lên. Theo Tổng điều tra Nông nghiệp 2010-11, ước tính có tổng cộng 138,35 triệu hộ hoạt động (nông dân đơn lẻ) và 159,59 triệu ha được đưa vào hoạt động. Nông nghiệp chính xác là một kỹ thuật quản lý nhằm thu thập, xử lý và phân tích dữ liệu tạm thời, không gian và cá nhân, đồng thời tích hợp dữ liệu này với thông tin khác để hỗ trợ các quyết định quản lý dựa trên sự biến đổi ước tính nhằm nâng cao hiệu quả sử dụng các nguồn lực sản xuất nông nghiệp, năng suất, chất lượng, lợi nhuận và tính bền vững. Nông nghiệp chính xác quản lý mọi đầu vào của sản xuất cây trồng (phân bón, đá vôi, thuốc diệt cỏ, hạt giống, thuốc trừ sâu, v.v.). Giảm lãng phí, tăng thu nhập và duy trì hiệu quả. Nông nghiệp chính xác điều chỉnh việc quản lý đất và cây trồng một cách cẩn thận để phù hợp với các điều kiện đồng ruộng khác nhau.

Khái niệm Nông nghiệp chính xác

PFS (Hệ thống Nông nghiệp Chính xác) tập trung vào việc nhận diện sự biến đổi theo không gian và thời gian trong quá trình phát triển của cây trồng. Trong quản lý trang trại, sự biến đổi này được tính đến để tăng năng suất và giảm rủi ro môi trường. Ở các nước phát triển, các trang trại thường lớn và bao gồm nhiều khu vực. Do đó, sự biến đổi không gian trong các trang trại lớn có hai thành phần: biến đổi nội tại và biến đổi giao diện. Chương trình nông nghiệp chính xác trong một cánh đồng thường được gọi là quản lý canh tác theo địa điểm cụ thể (SSCM).

SSCM đề cập đến một hệ thống quản lý nông nghiệp được phát triển nhằm thúc đẩy các thực hành canh tác biến đổi dựa trên địa điểm hoặc đất cụ thể.

Tuy nhiên, theo Batte và VanBuren (1999), SSCM không phải là một công nghệ đơn lẻ, mà là một tập hợp các công nghệ cho phép:

- Thu thập dữ liệu ở quy mô phù hợp vào thời điểm thích hợp;
- Diễn giải và phân tích dữ liệu để hỗ trợ một loạt các lựa chọn quản lý;
- Áp dụng một phản ứng quản lý ở mức độ thích hợp vào đúng thời điểm.

Segarra (2002) trong một nghiên cứu về PFS ở các nước phát triển nhấn mạnh những lợi ích sau đây cho nông dân:

- **Tăng tổng sản lượng:** việc lựa chọn cây trồng chính xác, áp dụng đúng loại và liều lượng phân bón, thuốc diệt cỏ cũng như tưới tiêu thích hợp sẽ đáp ứng các yêu cầu của cây trồng để tăng trưởng và phát triển tối ưu. Điều này dẫn đến tăng năng suất, đặc biệt là ở những khu vực hoặc cánh đồng có các biện pháp quản lý cây trồng ổn định trong lịch sử.
- **Cải thiện hiệu quả:** Máy móc, công cụ và thông tin tiên tiến hỗ trợ nông dân cải thiện hiệu quả lao động, đất đai và thời gian trong nông nghiệp. Tại Hoa Kỳ, 1 ha lúa mì hoặc ngô sẽ được trồng chỉ trong 2 giờ.
- **Giảm chi phí sản xuất:** Việc áp dụng số lượng chính xác vào đúng thời điểm sẽ làm giảm chi phí đầu vào hóa chất nông nghiệp trong sản xuất cây trồng. Hơn nữa, lợi nhuận chung cao làm giảm chi phí trên mỗi đơn vị sản lượng.
- **Ra quyết định trong quản lý nông nghiệp:** Máy móc, thiết bị và dụng cụ nông nghiệp được nông dân sử dụng để thu thập thông tin đáng tin cậy; thông tin này được xử lý và đánh giá để đưa ra các quyết định thích hợp khi trồng, gieo hạt, bón phân, phun thuốc trừ sâu và thuốc diệt cỏ, tưới tiêu, thoát nước và sau sản xuất.
- **Giảm tác động đến khí hậu:** Việc áp dụng hóa chất nông nghiệp kịp thời với tỷ lệ thích hợp sẽ tránh được dư lượng quá mức trong đất và nước, đồng thời giảm ô nhiễm trong khu vực tự nhiên.
- **Tích lũy nhận thức của nông dân để cải thiện quản lý:** Tất cả các hoạt động của PFS trên đồng ruộng đều mang lại thông tin hữu ích về đồng ruộng và công tác quản lý; dữ liệu này được lưu giữ trong các thiết bị và máy tính.

Các bước trong Nông nghiệp chính xác

Các bước cơ bản trong nông nghiệp chính xác là:

1. Đánh giá sự biến đổi

Bước quan trọng đầu tiên trong nông nghiệp chính xác là đánh giá sự biến đổi. Bởi vì rõ ràng là bạn không thể quản lý những gì bạn không biết. Về mặt năng suất, các yếu tố và quy trình điều chỉnh hoặc ảnh hưởng đến sản lượng cây trồng thay đổi theo thời gian. Thách thức của nông nghiệp chính xác là định lượng sự biến đổi của các yếu tố và quy trình này, đồng thời quyết định xem sự thay đổi theo không gian và thời gian của năng suất cây trồng chịu trách nhiệm cho các kết hợp khác nhau khi nào và ở đâu. Các phương pháp đo lường sự biến đổi không gian luôn có sẵn và thường được sử dụng trong nông nghiệp chính xác. Phần quan trọng nhất của nông nghiệp chính xác nằm ở việc đánh giá sự biến đổi không gian.

Cũng có những phương pháp để đánh giá sự biến đổi theo thời gian, nhưng sự biến đổi không gian và thời gian hiếm khi được ghi lại đồng thời. Thống kê không gian và thời gian rất quan trọng đối với chúng ta. Chúng ta có thể thấy sự biến đổi về năng suất cây trồng trong không gian, nhưng chúng ta đang phát triển trong suốt mùa sinh trưởng, đó không gì khác chính là sự thay đổi theo thời gian. Do đó, chúng ta cần cả thống kê không-thời gian và thống kê nông nghiệp chính xác. Khi nói đến các yếu tố này, Trí tuệ nhân tạo có thể đóng một vai trò to lớn trong việc thu thập tất cả các yếu tố này với độ chính xác và độ chuẩn xác cao hơn. Trí tuệ nhân tạo có nhiều nhánh khác nhau như học máy, học sâu, thị giác máy tính và các công nghệ khác như khoa học dữ liệu và dữ liệu lớn (bigdata) có thể đóng vai trò quan trọng trong việc thu thập dữ liệu từ đất nông nghiệp để thực hiện nông nghiệp chính xác một cách dễ dàng.

2. Quản lý sự biến đổi

Khi sự biến đổi đã được phân tích đúng cách, nông dân phải điều chỉnh các yếu tố đầu vào nông nghiệp cho phù hợp với các điều kiện đã được thiết lập và các đề xuất về quản lý. Chúng phải theo đặc thù của từng địa điểm và sử dụng thiết bị kiểm soát chính xác cho các ứng dụng. Chúng ta nên cho phép sử dụng công nghệ này nhiều nhất. Điều chỉnh sự biến đổi theo địa điểm cụ thể. Chúng ta nên sử dụng hệ thống Trí tuệ nhân tạo để nâng cao độ chính xác của địa điểm và quản lý đơn giản. Chúng ta sẽ ghi nhớ tọa độ của địa điểm lấy mẫu và sử dụng chúng để quản lý khi lấy mẫu đất và thực vật. Điều này góp phần sử dụng hiệu quả các nguồn lực và tránh lãng phí, đó là điều chúng ta muốn. Ngoài ra, với sự trợ giúp của Trí tuệ nhân tạo, chúng ta có thể triển khai nhiều loại bot khác nhau trên cánh đồng lớn cho mục đích quản lý và giám sát đồng ruộng.

Tiềm năng cải thiện độ chính xác trong quản lý độ phì nhiêu của đất và tăng cường kiểm soát ứng dụng chính xác khiến việc quản lý độ phì nhiêu của đất trở thành một giải pháp thay thế thú vị nhưng phần lớn chưa được chứng minh cho quản lý đồng ruộng tiêu chuẩn hóa, vốn có thể được cải thiện với sự trợ giúp của trí tuệ nhân tạo. Định nghĩa quản lý độ phì nhiêu của đất chính xác bao gồm sự biến đổi trong đồng ruộng và được định nghĩa chính xác cũng như diễn giải chuẩn xác để ảnh hưởng đến sự tăng trưởng của cây trồng, chất lượng cây trồng và khí hậu. Các yếu tố đầu vào cũng có thể được áp dụng một cách chính xác. Sự phụ thuộc vào không gian của một loại đất bền vững càng lớn thì tiềm năng và tầm quan trọng tiềm ẩn của nó đối với quản lý chính xác càng lớn.

3. Đánh giá

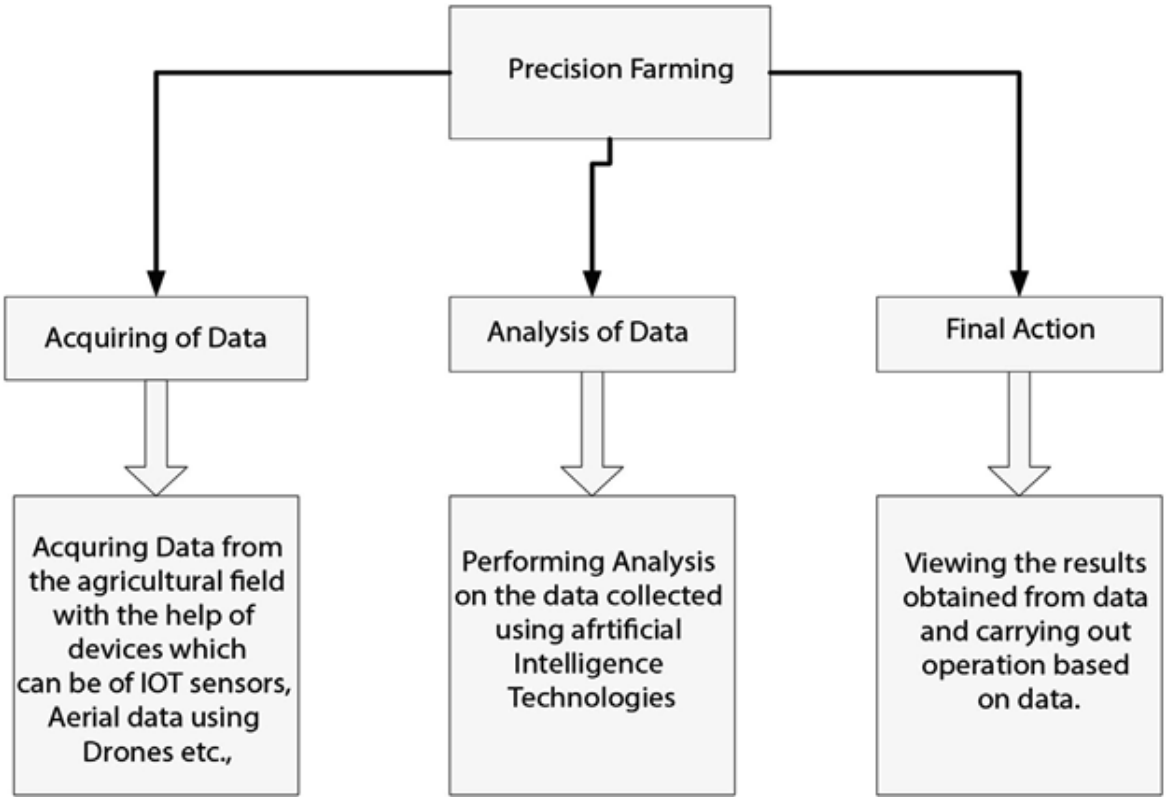
Khía cạnh quan trọng nhất của việc nghiên cứu lợi nhuận của nông nghiệp chính xác là lợi ích thường có được từ việc áp dụng dữ liệu, chứ không phải qua việc sử dụng công nghệ. [Một yếu tố khác là] những thay đổi trong tương lai đối với chất lượng môi trường. Các lợi ích tiềm năng đối với môi trường thường được kể đến là giảm sử dụng hóa chất nông nghiệp, tăng hiệu quả sử dụng chất dinh dưỡng, tăng cường kiểm soát đầu vào - đầu ra và tăng sản lượng đất nhờ giảm thoái hóa. Các công nghệ cho phép nông nghiệp chính xác có thể trở nên khả thi, các nguyên tắc nông nghiệp và các quy tắc chính sách có thể làm cho nó trở nên thiết thực và tăng hiệu quả sản xuất hoặc các loại giá trị khác.

Thuật ngữ chuyển giao công nghệ có thể có nghĩa là nông nghiệp chính xác xảy ra khi các cá nhân hoặc công ty thực sự có được và sử dụng công nghệ hỗ trợ nó. Mặc dù nông nghiệp chính xác bao gồm việc sử dụng công nghệ để kiểm soát sự biến đổi theo không gian và thời gian cũng như các khái niệm nông học, từ khóa chính vẫn là quản lý. Trong cái được gọi là chuyển giao công nghệ, phần lớn sự chú ý đã tập trung vào cách người nông dân có thể tương tác. Các vấn đề như vậy liên quan đến năng lực quản lý của người vận hành, việc cung cấp cơ sở hạ tầng không gian và kết nối giữa các trang trại khác nhau sẽ thay đổi cơ bản khi nông nghiệp chính xác tiếp tục phát triển.

Trí tuệ nhân tạo trong Nông nghiệp chính xác

Như chúng ta đã thảo luận về nông nghiệp chính xác và tầm quan trọng của nó trong lĩnh vực nông nghiệp, bây giờ chúng ta hãy xem Trí tuệ nhân tạo có thể được sử dụng ở các giai đoạn khác nhau để thực hiện nông nghiệp chính xác trên đất nông nghiệp như thế nào.

(Sơ đồ trong tài liệu mô tả quy trình 3 bước: 1. Thu thập dữ liệu (từ cảm biến IOT, Drone) -> 2. Phân tích dữ liệu (sử dụng Công nghệ Trí tuệ nhân tạo) -> 3. Hành động cuối cùng (xem kết quả và thực hiện hoạt động dựa trên dữ liệu).)



Triển khai Trí tuệ nhân tạo trong Nông nghiệp chính xác

Nó có thể được thực hiện qua 3 bước:

1. Thu thập dữ liệu

Thu thập dữ liệu bằng các công nghệ khác nhau như IOT, Drone, Hình ảnh vệ tinh, v.v.

Chúng ta hãy xem xét các công nghệ này:

- **IOT trong nông nghiệp:** Internet vạn vật là một mạng lưới kết nối điện tử của các đối tượng vật lý để thu thập và tổng hợp dữ liệu. IoT là một phần của quy trình tạo ra các cảm biến và phần mềm nông nghiệp. Ví dụ, đối với phân lỏng, nông dân có thể sử dụng phương pháp quang phổ để tính toán một mẫu nitơ, phốt pho và kali về cơ bản là không nhất quán. Sau đó, họ có thể quét cánh đồng để phát hiện nước tiểu của bò và chỉ bón phân vào những điểm cần thiết. Điều này làm giảm việc sử dụng phân bón lên đến 30%. Cảm biến độ ẩm của đất giúp xác định thời điểm tốt nhất để tưới cây từ xa.
- **Công nghệ Drone và Hình ảnh Vệ tinh:** Tiến bộ trong công nghệ drone và vệ tinh mang lại lợi ích cho nông nghiệp chính xác, vì drone chụp được hình ảnh chất lượng cao, trong khi vệ tinh chụp được bức tranh toàn cảnh rộng lớn hơn. Người điều khiển bay có thể kết hợp hình ảnh từ trên không với dữ liệu vệ tinh để dự báo năng suất trong tương lai dựa trên mức sinh khối hiện tại của đồng ruộng. Hình ảnh tổng hợp có thể tạo ra bản đồ đường đồng mức để theo dõi dòng chảy của nước, đánh giá tỷ lệ gieo hạt và tạo bản đồ năng suất cho các khu vực hiệu quả hơn hoặc kém hơn.
- **Robot:** Đã có máy kéo tự quản trong một thời gian, thiết bị này hoạt động giống như máy bay tự lái. Máy kéo hoạt động tốt nhất, với người nông dân [có mặt] trong trường hợp khẩn cấp. Công nghệ đang phát triển thành máy móc không người lái được lập trình bằng GPS để rải phân bón hoặc cày đất. Những đổi mới bổ sung là các máy chạy bằng năng lượng mặt trời có thể xác định cỏ dại và tiêu diệt chúng một cách chính xác bằng một liều thuốc diệt cỏ hoặc laser. Đã có robot nông nghiệp, còn được gọi là Bot nông nghiệp (Agricultural Bots), nhưng các robot thu hoạch tiên tiến đang được chế tạo có sử dụng Trí tuệ nhân tạo bên trong để phát hiện trái cây chín, điều chỉnh theo hình dạng và kích thước của chúng và hái cẩn thận khỏi cành.

2. Phân tích dữ liệu

Sau khi thu thập dữ liệu từ đất nông nghiệp tại một cơ sở dữ liệu cụ thể, chúng ta có thể thực hiện nhiều phân tích khác nhau trên dữ liệu bằng cách sử dụng các công nghệ như Trí tuệ nhân tạo, Học máy (Machine learning), Học sâu (Deep learning), Thị giác máy tính (Computer vision), v.v.

Từ phân tích, chúng ta có thể nhận được nhiều yếu tố có giá trị, hướng dẫn chúng ta thực hiện một số tác vụ một cách chính xác để tiến hành canh tác trên cánh đồng.

3. Hành động cuối cùng

Bước này liên quan đến việc sử dụng các phân tích và dữ liệu khác nhau đã thu thập được để tạo ra các ứng dụng dựa trên chúng, giúp thực hiện nông nghiệp chính xác trên đất nông nghiệp một cách dễ dàng.

Ngày nay, trí tuệ nhân tạo đang tham gia nhiều hơn vào lĩnh vực này để tạo ra rất nhiều ứng dụng có thể giúp bất kỳ người nông dân nào thực hiện nông nghiệp chính xác một cách dễ dàng bằng cách phân tích cây trồng của họ trên đồng ruộng.

Các ứng dụng có thể được thực hiện trên cơ sở Trí tuệ nhân tạo là:

- **Bot nông nghiệp** Phần lớn các công ty hiện đang chuẩn bị và chế tạo robot cho sứ mệnh nông nghiệp chính. Điều này bao gồm việc xử lý hạt giống và làm việc hiệu quả hơn so với nhân lực. Đây là ví dụ mới nhất về học máy trong nông nghiệp.
- **Chọn lọc giống** Chọn lọc giống là một phương pháp tìm kiếm lặp đi lặp lại các gen cụ thể, quyết định hiệu quả sử dụng nước và chất dinh dưỡng, khả năng chịu khí hậu, kháng bệnh, hàm lượng dinh dưỡng hoặc hương vị. Trong phân tích sản xuất cây trồng ở các vùng khí hậu khác nhau và phát triển các công nghệ mới, đặc biệt là học máy, các thuật toán học sâu đòi hỏi hàng thập kỷ dữ liệu thực địa. Dựa trên dữ liệu này, một mô hình thống kê có thể được phát triển để dự đoán gen nào có khả năng đóng góp nhiều nhất vào lợi ích của cây trồng.
- **Nhận dạng loài** Cách tiếp cận thông thường của con người để phân loại thực vật là so sánh màu sắc và hình dạng của lá, nhưng bằng cách sử dụng phân tích thống kê của Học máy, có thể cung cấp kết quả nhanh hơn và chi tiết hơn bằng cách phân tích hình thái của lá và cung cấp thêm chi tiết về các đặc tính của lá.
- **Dự đoán Năng suất** Dự báo năng suất là một trong những chủ đề của nông nghiệp siêu chính xác, vì nó xác định việc lập bản đồ và dự đoán năng suất, khớp nối nhu cầu cây trồng và quản lý cây trồng. Các kỹ thuật tiên tiến đã vượt xa dự đoán đơn giản dựa trên dữ liệu lịch sử, mà còn liên quan đến các kỹ thuật học máy để cung cấp kiến thức liên quan đến cây trồng, thời tiết và hoàn cảnh kinh tế cũng như phân tích đa chiều mở rộng để tối ưu hóa lợi nhuận cho nông dân và cộng đồng.
- **Phát hiện bệnh hoặc dự báo bệnh** Học máy có thể được sử dụng để phát hiện các bệnh nhiễm trùng ở thực vật thường gây ra bởi dịch hạch, côn trùng, bệnh tật và, trừ khi được quản lý kịp thời, chúng làm giảm năng suất trên quy mô lớn. Đối với diện tích trồng trọt tính bằng mẫu Anh (acres), người trồng trọt rất tốn công theo dõi cây trồng hàng ngày. Việc triển khai Học máy ở đây cung cấp giải pháp giám sát nhất quán khu vực canh tác và cung cấp khả năng phát hiện bệnh tự động bằng hình ảnh viễn thám.
- **Chất lượng cây trồng** Dữ liệu đầu ra có thể được đo lường và xác định đầy đủ để nâng giá sản phẩm và giảm lãng phí. Trái ngược với các chuyên gia con người, máy móc có thể sử dụng dữ liệu và các giao diện có vẻ không liên quan để khám

phá và xác định các phẩm chất mới đóng vai trò trong chất lượng tổng thể của cây trồng.

- **Phát hiện cỏ dại** Cỏ dại là mối đe dọa lớn đối với sản xuất cây trồng, ngoài sâu bệnh. Vấn đề chính trong cuộc chiến chống cỏ dại là chúng khó phân biệt và nhận dạng so với cây trồng. Các thuật toán thị giác máy tính và Học máy (ML) có thể tăng cường khả năng nhận dạng và phân biệt cỏ dại với chi phí thấp mà không có các vấn đề về môi trường hoặc tác dụng phụ. Các công nghệ này sẽ thúc đẩy robot tiêu diệt cỏ dại và giảm nhu cầu sử dụng thuốc diệt cỏ trong tương lai.
- **Quản lý đất đai** Đất là một nguồn tài nguyên thiên nhiên rất đa dạng với các quy trình phức tạp và các cơ chế không rõ ràng đối với các chuyên gia nông nghiệp. Nhiệt độ của chính nó có thể cung cấp những hiểu biết sâu sắc về tác động của biến đổi khí hậu đối với lợi nhuận lãnh thổ. Các thuật toán của học máy nghiên cứu quá trình bốc hơi, độ ẩm và nhiệt độ của đất để hiểu động thái của hệ sinh thái và tác động của nông nghiệp.
- **Quản lý nước** Sự cân bằng về thủy văn, khí hậu và nông nghiệp của tài nguyên nước trong nông nghiệp có một ý nghĩa quan trọng. Sử dụng các hệ thống tưới tiêu hiệu quả hơn và dự báo nhiệt độ điểm sương hàng ngày, các ứng dụng ML tiên tiến nhất cho đến nay được kết nối với ước tính khí khổng hàng ngày, hàng tuần hoặc hàng tháng, giúp phân loại thời tiết dự đoán và ước tính sự bốc hơi.
- **Chăn nuôi** Học máy cung cấp dự đoán và tính toán chính xác các thông số canh tác, phù hợp với quản lý cây trồng, để tối đa hóa hiệu quả kinh tế của các hệ thống canh tác như chăn nuôi. Để cho phép nông dân thay đổi chế độ ăn và điều kiện, ví dụ, hệ thống dự báo trọng lượng sẽ ước tính trọng lượng tương lai 150 ngày trước ngày họ giết mổ.

Phạm vi và Hạn chế trong việc áp dụng Nông nghiệp chính xác ở Ấn Độ

Các nguyên tắc Nông nghiệp chính xác có thể áp dụng cho tất cả các lĩnh vực nông nghiệp bao gồm chăn nuôi, thủy sản và lâm nghiệp. Nông nghiệp chính xác (PA) được chia thành hai nhóm—PA Mềm (Soft PA) và PA Cứng (Hard PA). PA mềm là các đánh giá Quản lý, dựa trên quan sát và trực giác, thay vì nghiên cứu thống kê hoặc thực nghiệm. Trong khi đó, tất cả các công nghệ mới nhất như IOT, drone, GPS, GIS, VRT (Công nghệ Tỷ lệ Biến đổi), v.v. được sử dụng làm PA Cứng. 96 triệu trang trại ở Ấn Độ có ít hơn bốn mẫu Anh (ha) đất trong tổng số 105,3 triệu trang trại. Mặc dù chỉ canh tác trên đất đai manh mún, Ấn Độ hiện sản xuất gần 200 triệu tấn lương thực. Năng suất cây trồng trên mỗi ha phải kinh tế và không gây tổn hại đến môi trường để cạnh tranh với sự tăng trưởng của thế giới. Ở Ấn Độ, tổng lượng phân bón tiêu thụ là 84,3 Kg/ha và phải được giảm bớt theo hướng dẫn từ [ngành] phân bón bằng cách kiểm tra đất có hệ thống và phát triển bản đồ dinh dưỡng. Kiểm soát sâu bệnh, quản lý dịch bệnh và cỏ dại, cùng với các vùng dinh dưỡng, cũng đóng một vai trò quan trọng trong năng suất cây trồng cao. Có thể theo dõi và quản lý dịch bệnh và sâu bệnh với chi phí thấp hơn bằng cách sử dụng các công nghệ tiên tiến. Một số tiểu bang, chẳng hạn như Punjab, Haryana, sử dụng phân bón và thuốc trừ sâu liều lượng lớn. Ví dụ, bang Punjab chiếm 1,5% tổng diện tích

địa lý của Ấn Độ, nhưng sử dụng 1,38 triệu tấn phân bón NPK (gần 10% tổng lượng tiêu thụ của toàn Ấn Độ) và 60% thuốc diệt cỏ được sử dụng ở Ấn Độ. Các vấn đề điển hình ở những khu vực này là suy thoái toàn bộ đất đai và sử dụng không hiệu quả các sản phẩm nông nghiệp.

Một lĩnh vực khác mà Nông nghiệp chính xác sẽ cho phép nông dân Ấn Độ lập kế hoạch tưới tiêu có lợi hơn bằng cách điều chỉnh thời gian, số lượng và vị trí đặt nước. Cơ giới hóa nông nghiệp giúp nông dân giảm chi phí lao động và tăng độ chính xác trong trang trại của họ, bao gồm lựa chọn hạt giống chất lượng, làm cỏ, thuốc trừ sâu và bón phân, thu hoạch và phân loại cây trồng theo chất lượng của chúng. Ở các nước đang phát triển nói chung và ở Ấn Độ nói riêng, có nhiều hạn chế đối với việc áp dụng nông nghiệp chính xác. Một số giới hạn này tương tự như ở các khu vực khác; tuy nhiên, điều kiện của Ấn Độ là đặc thù:

- Xã hội và quan điểm của người dùng,
- Quy mô đồng ruộng nhỏ,
- Không có câu chuyện thành công,
- Tính không đồng nhất và sự không hoàn hảo của các hệ thống cây trồng,
- Sở hữu đất đai, cơ sở vật chất và các giới hạn về thể chế,
- Thiếu kỹ năng kỹ thuật địa phương,
- Khả năng tiếp cận, chất lượng và chi phí dữ liệu.

Những thách thức

Mặc dù trong những thập kỷ gần đây, Nông nghiệp chính xác (PA) đã giúp tăng năng suất cây trồng, với chi phí và công sức của con người giảm đi, tuy nhiên, do những lý do hoặc thách thức sau đây, việc áp dụng các công nghệ mới này của nông dân vẫn còn rất hạn chế.

- **Chi phí phần cứng:** Nông nghiệp chính xác chủ yếu liên quan đến việc đo lường nhiều thông số trong thời gian thực bằng các thiết bị bao gồm cảm biến, bộ định tuyến không dây, drone, cảm biến hình ảnh quang phổ, v.v. Các cảm biến này có nhiều nhược điểm, bao gồm chi phí phát triển, bảo trì và vận hành cao. Nhiều hệ thống Nông nghiệp chính xác không tốn kém và có thể thích ứng với diện tích đất canh tác hạn chế, ví dụ như hệ thống tưới tiêu thông minh đòi hỏi phần cứng và cảm biến chi phí thấp. Các chương trình giám sát an toàn cây trồng dựa trên drone cho các vùng đất canh tác lớn là không khả thi do chi phí triển khai cao.
- **Thay đổi thời tiết:** Sự biến đổi của môi trường là một trong những mối đe dọa lớn nhất đối với tính chính xác của dữ liệu cảm biến. Các nút cảm biến tại hiện trường nhạy cảm với các biến đổi môi trường, chẳng hạn như mưa, dao động nhiệt độ, tốc độ gió, ánh sáng mặt trời, v.v. Sự nhiễu loạn do các xáo trộn khí quyển trong các kênh không dây sẽ làm gián đoạn liên lạc giữa nút không dây và đám mây. Các

nền tảng vệ tinh và drone trên không cũng nhạy cảm với những thay đổi của thời tiết. Ô nhiễm không khí và các sol khí tự nhiên khác tác động đến hình ảnh thu được từ các nền tảng này. Việc phát triển các công nghệ tiên tiến về hiệu chỉnh khí quyển, phát hiện đám mây và nội suy âm thanh là một thách thức lớn đòi hỏi cộng đồng nghiên cứu phải nỗ lực mạnh mẽ.

- **Quản lý dữ liệu dư thừa:** Dữ liệu luôn được tạo ra bởi các cảm biến trong Nông nghiệp chính xác. Một số biện pháp bảo mật dữ liệu phải được đưa ra để đảm bảo tính toàn vẹn của dữ liệu, do đó làm tăng chi phí hệ thống. Các kết quả đọc của cảm biến phải chính xác để thực hiện hành động thích hợp khi cần thiết. Các kết quả đọc có thể bị kẻ tấn công làm hỏng và máy sẽ giảm hiệu suất của nó. Các hệ thống Nông nghiệp chính xác tạo ra lượng dữ liệu khổng lồ, đòi hỏi đủ nguồn lực để tiến hành phân tích dữ liệu. Dữ liệu thời gian thực thu được từ các cảm biến được triển khai trong vài phút và hình ảnh quang phổ ở độ cao lớn/độ cao thấp tạo ra một lượng lớn dữ liệu làm tăng yêu cầu lưu trữ và xử lý. Các yêu cầu đặt ra là các nền tảng phần mềm mới và các cơ sở quản lý có thể mở rộng cho các nguồn dữ liệu lớn. Trong bối cảnh này, việc tạo ra giải pháp phần mềm tập trung vào việc kết hợp quản lý dữ liệu với các nền tảng điện toán đám mây toàn diện của IoT.
- **Tỷ lệ biết chữ:** Trình độ học vấn là một yếu tố chính ảnh hưởng đến tỷ lệ chấp nhận Nông nghiệp chính xác (PA). Nông dân trồng trọt dựa trên kinh nghiệm ở các nước đang phát triển có tỷ lệ biết chữ cao. (Lưu ý: câu này trong bản gốc có vẻ mâu thuẫn, “developing countries” thường không có “high literacy rates”, nhưng tôi dịch đúng theo văn bản). Họ không sử dụng các công nghệ nông nghiệp tiên tiến, dẫn đến giảm sút sản lượng. Để hiểu công nghệ, nông dân cần được giáo dục hoặc tin tưởng vào hỗ trợ kỹ thuật từ bên thứ ba. Ở các nước kém phát triển, Nông nghiệp chính xác cũng không phổ biến do hạn chế về nguồn lực và giáo dục, nơi tỷ lệ biết chữ không cao.
- **Kết nối:** Mạng 5G thế hệ tiếp theo có thể nhanh hơn 100 lần so với mạng 4G, giúp giao tiếp giữa máy chủ và thiết bị nhanh hơn nhiều. Hơn nữa, 5G có thể truyền tải nhiều thông tin hơn các mạng khác, khiến nó trở thành công nghệ lý tưởng cho dữ liệu được truyền tải bởi các cảm biến từ xa và drone, những công cụ chủ chốt được thử nghiệm trong môi trường PA. Trong các ứng dụng hiện tại, việc giới thiệu các mạng truyền thông 5G hiện đại là điều bắt buộc, nơi truyền dữ liệu an toàn và nhanh chóng là cần thiết để cho phép xử lý dữ liệu thời gian thực và hỗ trợ ra quyết định.
- **Khả năng tương tác:** Khả năng tương tác của các thiết bị do các tiêu chuẩn kỹ thuật số khác nhau là một trong những thách thức lớn nhất mà Nông nghiệp chính xác phải đối mặt. Sự thiếu khả năng tương tác này không chỉ cản trở việc áp dụng và làm chậm các công nghệ IoT mới mà còn cản trở việc tăng năng suất của các ứng dụng thông minh trong nông nghiệp. Kịch bản Nông nghiệp chính xác hiện tại cung cấp cơ sở cho các phương pháp và giao thức mới để kết hợp các yêu cầu giao tiếp máy móc khác nhau nhằm khám phá tiềm năng của giao tiếp máy-với-máy và chia sẻ dữ liệu hiệu quả.

Tóm tắt

Nông nghiệp chính xác là một cách tiếp cận hiện đại để tăng năng suất cây trồng thông qua việc sử dụng các công nghệ cập nhật nhất, tức là IoT, điện toán đám mây, AI và Học máy (ML), Học sâu (DL), Thị giác máy tính, Công nghệ Drone, v.v. Nông nghiệp chính xác nhằm mục đích cung cấp các hệ thống hỗ trợ quyết định tập trung vào một số thông số cây trồng, tức là chất dinh dưỡng trong đất, mực nước trong đất, tốc độ gió, cường độ ánh sáng mặt trời, nhiệt độ, độ ẩm, hàm lượng chất diệp lục và các yếu tố khác. Tuy nhiên, giai đoạn phát triển và triển khai các hệ thống này liên quan đến một số thách thức. Vì Nông nghiệp chính xác có mục tiêu chính là tối ưu hóa các nguồn tài nguyên như nước, thuốc trừ sâu, phân bón, v.v., để tạo ra năng suất vượt trội nhằm tối ưu hóa nguồn lực, để cho phép trí tuệ nhân tạo định lượng các nguồn tài nguyên cần thiết cho cây trồng khỏe mạnh ở bất kỳ giai đoạn tăng trưởng cụ thể nào.